

Instituto Federal Fluminense
Campus Campos dos Goytacazes

Controladores Lógicos Programáveis

Material didático de automação industrial

M.Sc. Leonardo de Oliveira Tavares
D.Sc. William da Silva Vianna
M.Sc. Natália Nogueira Monteiro

Revisão técnica: M.Sc. Karina Terra de Souza

Engenharia de Controle e Automação
Curso Técnico em Automação Industrial

3ª edição – Rev. 06
Setembro de 2026

Campos dos Goytacazes – RJ

Instituto Federal Fluminense

Campus Campos dos Goytacazes

Controladores Lógicos Programáveis

Material didático elaborado para o curso de Engenharia de Controle e Automação e para o Curso Técnico em Automação Industrial.

Autores:

M.Sc. Leonardo de Oliveira Tavares

D.Sc. William da Silva Vianna

M.Sc. Natália Nogueira Monteiro

Revisão técnica:

M.Sc. Karina Terra de Souza

3ª edição — Rev. 06

Setembro de 2026

Licença de uso. Esta obra é distribuída sob a licença *Creative Commons Atribuição–CompartilhaIgual 4.0 Internacional (CC BY-SA 4.0)*. É permitido copiar, redistribuir, adaptar e usar comercialmente o material, desde que seja atribuído o crédito aos autores e as obras derivadas sejam distribuídas sob a mesma licença. Texto completo da licença: <https://creativecommons.org/licenses/by-sa/4.0/legalcode.pt>.

Material didático auxiliar

Playlist de vídeos — Controladores Lógicos Programáveis



Aponte a câmera do celular para o código, ou toque no endereço para abrir a playlist no navegador:

<https://www.youtube.com/playlist?list=PLUTnDfgcyd8>

(o código e o endereço acima são clicáveis no PDF)

Sumário

I	Fundamentos e projeto	15
1	Introdução e importância da automação	16
1.1	O que é automação industrial	16
1.2	Por que automatizar	16
1.3	Onde o CLP é aplicado	18
1.4	O CLP no centro do sistema	18
1.5	Como ler esta apostila	18
2	Evolução dos CLPs	20
2.1	Antes do CLP: a lógica fiada	20
2.2	1968–1969: a especificação da GM e o primeiro CLP	20
2.3	Os dez requisitos da especificação original	21
2.4	Gerações: o que mudou em cada salto	21
2.5	CLP, PAC, controlador de segurança e controlador virtual	22
2.6	O que não mudou	23
3	A estrutura da automação	25
3.1	Do chão de fábrica à gestão	25
3.2	Os cinco níveis	26
3.3	O que mudou na pirâmide	27
4	Projeto de automação industrial	28
4.1	O projeto antes do programa	28
4.2	Documentos básicos e essenciais	28
4.3	P&ID: o ponto de partida	29
4.4	Lista de I/O	29
4.5	Descritivo operacional	30
4.6	Planta baixa, I/O remoto e arquitetura	30
4.7	Fechamento: o que fica para as próximas etapas	31
II	Hardware e funcionamento	32
5	Definições, características e aplicações	33
5.1	Definição	33
5.2	Características gerais	34
5.3	Aplicações	34
5.4	Arquitetura de blocos	35
5.5	Compacto, modular e com I/O distribuído	35
5.6	O que distingue o CLP de um computador comum	36
6	Componentes, memória e tipos de dados	38

6.1	O microprocessador e a memória	38
6.2	Classificação das memórias	38
6.3	Tecnologias de memória	39
6.4	Endereçamento, barramentos e cálculo de memória	39
6.5	Tipos de dados da norma	40
6.5.1	Declaração de variáveis e o princípio da forte tipação	41
6.5.2	Incompatibilidade direta entre tipos: o que não compila	42
6.5.3	Funções padronizadas de conversão explícita de tipos	42
6.5.4	Exemplo prático: escalonamento de canal analógico	43
6.6	Números negativos: complemento de um e de dois	45
6.7	Formatos IEEE-754	45
6.8	Dados estruturados, textos e arranjos	46
6.9	Arquitetura e mapa de memória do CLP	46
7	Cartões de entrada e saída, resolução e instalação	49
7.1	O que o cartão faz com o sinal	49
7.2	Caminho do sinal em um cartão de entrada	49
7.3	Entrada digital	50
7.3.1	Lógica positiva e negativa: <i>sinking</i> e <i>sourcing</i>	52
7.4	Entrada analógica	53
7.5	Saída digital	54
7.5.1	Resumo das ligações de entrada e de saída	55
7.6	Cálculo de resolução	57
7.6.1	Definindo os bits do cartão antes de comprar	58
7.7	Instalação e troca de cartões	58
8	Porte, redundância e especificação para compra	60
8.1	O que define o porte	60
8.2	Classificação por quantidade de pontos	60
8.3	Velocidade de processamento	61
8.4	Memória	61
8.5	Formas de E/S e o efeito no porte	61
8.6	Redundância	61
8.7	Procedimento de especificação	64
8.8	Avaliação do fornecedor, do produto e do contrato	64
9	Princípio de funcionamento, ciclos e estados de operação	66
9.1	Rotina de inicialização	66
9.2	O ciclo de varredura	66
9.3	Por que o tempo de varredura importa	67
9.3.1	Watchdog	68
9.4	Modos de processamento	68
9.5	As imagens de entrada e de saída	69
9.6	Estados de operação	69
9.6.1	Auto-teste e proteção de dados	71
III	Programação e normas	72
10	Linguagens de programação	73
10.1	Por que a linguagem importa	73

10.2	Baixo nível e alto nível	73
10.3	As linguagens definidas pela norma	73
10.3.1	Exemplos comparados	75
10.4	Lendo e escrevendo Ladder	75
10.4.1	Associação em série, em paralelo e mista	76
10.5	Lista de Instruções: por que ela saiu de cena	77
10.6	Como escolher a linguagem	78
11	IEC 61131-3 em profundidade: POU's, tipagem e tarefas	79
11.1	O modelo de software	79
11.2	Função, bloco de função e programa	81
11.3	Tarefas	82
11.4	Variáveis	82
11.4.1	Escopo	82
11.4.2	Ligação com o hardware	82
11.4.3	Atributos de variável	83
11.4.4	Tipagem e alocação	83
11.5	Estruturação, modularização e reutilização	83
11.6	O que a norma não define	84
12	Família IEC 61131 e o entorno	85
12.1	Como a norma está organizada	85
12.2	As edições da parte 3	86
12.3	O entorno da norma	86
12.4	Como avaliar uma declaração de conformidade	88
13	CLP conforme IEC 61131-3 versus CLP legado	89
13.1	A pergunta que separa dois mundos	89
13.2	O caso concreto das apostilas de origem	90
13.3	Comparação, aspecto por aspecto	91
13.4	Mapa de conversão	91
13.5	As duas famílias de problemas	92
13.5.1	Problemas de linguagem — os menores	92
13.5.2	Problemas de estrutura — os maiores	92
13.6	Por que este capítulo existe	93
13.7	Como escrever código que envelhece bem	93
14	Boas práticas e padronização de código	95
14.1	Por que padronizar	95
14.2	Convenção de nomes	95
14.3	Requisitos antes da lógica	96
14.4	Segurança dentro da lógica — e fora dela	98
14.5	Eficiência do ciclo de varredura	98
14.6	Checklist de revisão de código	99
15	Documentação de programas de CLP	101
15.1	Por que documentar	101
15.2	Artefatos mínimos	102
15.3	Tabela de alocação de E/S	102
15.4	Documentação dentro do programa	103
15.5	Versionamento e backup	103

15.6 Documentação como entrega contratual	104
16 Simulação, teste e comissionamento	105
16.1 Por que testar antes	105
16.2 A escada de validação	105
16.3 Roteiro de teste	107
16.4 Limites da simulação	108
16.5 Registro de evidências	108
 IV Integração e sistemas	 110
17 Redes industriais e sistemas SCADA	111
17.1 Por que existe rede	111
17.2 Três arquiteturas de rede do CLP	111
17.3 Protocolos de campo e industriais	113
17.4 Arquitetura SCADA	115
17.5 Alarme, evento e intertravamento	116
17.6 Boas práticas de projeto de rede	116
 18 IIoT, OPC UA e MQTT	 118
18.1 O que a automação clássica deixou em aberto	118
18.2 OPC UA: camada de informação, não apenas de transporte	118
18.3 MQTT: publicação e assinatura para telemetria	119
18.4 Sparkplug: padronizar o conteúdo, não só o transporte	119
18.5 A pilha completa	120
18.6 Vantagens do IIoT na automação	121
18.7 Desafios e limites	121
18.8 Por onde começar	122
 19 Dados, telemetria e monitoramento em tempo real	 124
19.1 Do dado bruto ao indicador	124
19.2 Os indicadores usuais	126
19.3 Onde processar: CLP, borda ou nuvem	126
19.4 Detecção de desvio e manutenção preditiva simples	127
19.5 O limite do controlador	128
19.6 Qualidade do dado	128
 20 Interoperabilidade entre marcas e modelos	 130
20.1 Três palavras que não são sinônimos	130
20.2 A escada da portabilidade	131
20.3 Estratégias que funcionam	132
20.4 Onde a migração realmente quebra	133
20.5 Classificando o risco da migração	133
20.6 O que exigir na especificação e no contrato	134
 21 Convergência TI/TO	 136
21.1 Duas culturas, um mesmo sistema	136
21.2 Quem roda onde	137
21.3 Visão computacional e processamento de imagem	138
21.4 Modelos de linguagem pequenos no ambiente de controle	138
21.5 Plugins e extensões no ambiente de engenharia	139

21.6	Node-RED	140
21.7	Geração de código a partir de modelagem e simulação	140
21.8	Linux em sistemas embarcados e controladores	141
21.9	O balanço: o que ganha, o que perde, o que exige	142
22	Segurança cibernética em automação industrial	144
22.1	Safety e security: duas palavras, dois problemas	144
22.2	A base normativa: a série IEC 62443	145
22.3	A superfície de ataque do controlador	146
22.4	Casos públicos que vale conhecer	146
22.5	Contramedidas, do projeto à operação	147
22.6	Resposta a incidente	148
22.7	Segurança é processo, não projeto	148
23	Eficiência energética e sustentabilidade	150
23.1	A lógica também consome energia	150
23.2	Medir antes de agir	150
23.3	Alavancas na lógica	152
23.4	Acionamento de motores	153
23.5	Controle de demanda	153
23.6	Ciclo de vida e descarte	154
V	Operação e carreira	156
24	Manutenção preventiva e corretiva	157
24.1	Três formas de manter	157
24.2	Modos de falha típicos	158
24.3	Rotina preventiva	159
24.4	Diagnóstico por sintoma	159
24.5	Procedimento de intervenção	161
24.6	Manutenção preditiva como evolução	162
25	Formação contínua e certificações	164
25.1	Por que a formação não termina no curso	164
25.2	A trilha	165
25.3	Certificações, treinamentos e o que eles valem	166
25.4	Recursos de estudo	167
25.5	Bancada própria	168
25.6	Plano de 90 dias	169
VI	Apêndices	171
A	Práticas de laboratório	172
A.1	Como usar este apêndice	172
A.2	Prática 1 — Partida e parada com selo	173
A.3	Prática 2 — Condição de falha segura e contato NF	174
A.4	Prática 3 — Detecção de borda e pulso único	174
A.5	Prática 4 — Temporização com TON, TOF e TP	175
A.6	Prática 5 — Contagem, retenção e retomada	175
A.7	Prática 6 — Sequência por etapas com SFC	176

A.8	Prática 7 — Escala de sinal analógico	176
A.9	Prática 8 — Comparação, aritmética e alarme de faixa	177
A.10	Prática 9 — Intertravamento e partida segura	178
A.11	Prática 10 — Remota de E/S e supervisão	178
A.12	Prática 11 — Documentação e versionamento do projeto	179
B	Guia de conversão: legado para IEC 61131-3	181
B.1	Para que serve e para que não serve	181
B.2	Os bits de status e os parâmetros do legado	181
B.3	Como ler os exemplos	182
B.4	Casos de conversão	182
B.4.1	Contato instantâneo, contato invertido e bobina	182
B.4.2	Série e paralelo	183
B.4.3	Retenção de saída	183
B.4.4	Deteção de borda e pulso único	184
B.4.5	Temporização	184
B.4.6	Contagem	185
B.4.7	Comparação	185
B.4.8	Aritmética e funções	186
B.4.9	Operações bit a bit, cópia e zeragem	186
B.4.10	Deslocamento de bits	187
B.4.11	Chamada de rotina	187
B.5	As instruções do item 8.2 da fonte, com figuras e exemplos	187
B.5.1	Booleanas	188
B.5.2	Temporizadores e contadores	193
B.5.3	Comparadores	198
B.5.4	Instruções matemáticas	201
B.5.5	Operadores lógicos e de movimentação	205
B.5.6	Movimentação de bits e desvio	208
B.6	O que não se converte mecanicamente	210
B.7	Roteiro de conversão	210
C	Glossário	213
C.1	Como usar	213
D	Referências normativas e bibliografia	219
D.1	Como as referências são citadas nesta obra	219
D.2	Obras de origem desta apostila	219
D.3	Normas e documentos normativos citados	220
D.4	Bibliografia consolidada	221
D.4.1	Livros e apostilas	221
D.4.2	Artigos, relatórios e documentos técnicos	222
D.4.3	Documentação técnica e fontes de organismos	223
	Índice remissivo	225

Lista de Figuras

1.1	Pressões que levam à automação e ganhos que ela entrega.	17
2.1	Linha do tempo da evolução dos controladores industriais.	22
3.1	Níveis da automação industrial e fluxo de informação entre eles.	25
4.1	Fluxo simplificado de um projeto de automação, do requisito à entrega técnica.	28
5.1	Diagrama em blocos com os componentes básicos de um CLP: terminal de programação, fonte de alimentação, microprocessador, memória, interface de E/S e cartões de entrada e saída, ligados pelos barramentos.	35
6.1	Campos dos formatos simples (32 bits) e duplo (64 bits) da IEEE-754.	45
6.2	Áreas do mapa de memória de um controlador programável.	47
7.1	Elementos de tratamento do sinal em um cartão de entrada digital.	49
7.2	Elementos de um cartão de saída digital, da CPU ao atuador.	50
7.3	Cartão de entrada digital com alimentação em corrente contínua: os dois bornes de campo recebem 24 V CC, e o diodo em série com o resistor define o sentido da corrente pelo LED do optoacoplador.	50
7.4	Cartão de entrada digital com alimentação em corrente alternada: a ponte retificadora converte os 110 V CA ou 220 V CA em corrente contínua antes do optoacoplador.	51
7.5	Entrada digital com fonte externa e isolamento por optoacoplador.	51
7.6	Ligações de entrada e de saída digitais em tensão contínua nas configurações <i>sinking</i> e <i>sourcing</i> , com a lógica positiva ou negativa correspondente.	53
7.7	Conversão de corrente em tensão com resistor de <i>shunt</i> de 250 Ω , em laço alimentado por 24 V CC.	54
7.8	Saída digital a contato seco: o contato pertence ao módulo de saída do CLP e entra em série com a fonte externa e com a carga.	55
7.9	Esquemas de ligação de módulos de entrada (CC e CA) e de saída (CC e a relé) de um CLP.	56
8.1	Exemplo de redundância de hardware: dois CLPs em redundância ligados por um <i>switch</i> Ethernet, com a remota de I/Os compartilhada pelos dois.	62
8.2	Diagrama de fluxo do processo de redundância por software: as duas CPUs trocam informação de <i>status</i> e copiam os dados redundantes, de modo que a CPU em espera assuma o controle com o processo atualizado.	63
9.1	Ciclo de operação do CLP: leitura das entradas para a memória imagem, execução do programa e escrita das saídas, com o resultado da linha de Ladder gravado na tabela de dados.	67
9.2	Estados de operação do controlador e transições.	70

10.1	Linguagens de programação de CLP: gráficas e textuais.	74
10.2	Exemplos comparados das linguagens da IEC 61131-3: Lista de Instruções, Texto Estruturado, Diagrama de Blocos de Função, Ladder e Passos e Transições, todos para a mesma lógica.	75
10.3	Lógica em Ladder: na primeira linha, associação em série (E); na segunda, associação em paralelo (OU).	76
11.1	Modelo de software proposto pela IEC 61131-3: a configuração reúne recursos; cada recurso executa tarefas, e cada tarefa executa programas, que por sua vez usam blocos funcionais (FB).	80
12.1	A norma e as iniciativas que a complementam.	87
13.1	O que a norma padroniza e o que permanece decisão do fabricante.	90
14.1	Fluxo de elaboração do programa, dos requisitos à liberação para uso.	97
16.1	Níveis de validação, do teste de lógica ao comissionamento.	106
17.1	As três arquiteturas de rede do CLP: local, I/O remoto e rede de CLPs.	112
17.2	Arquitetura de rede de CLPs: módulos de E/S montados localmente, comunicação dos controladores com o sistema de supervisão pelo tipo mestre-escravo ou <i>polling</i>	113
17.3	Esquema básico de um sistema SCADA com uso de CLP: o processo, o equipamento de controle e a estação de supervisão com o <i>driver</i> de comunicação.	115
18.1	Pilha típica de IIoT: controlador, borda, broker e aplicações.	120
19.1	Caminho do dado até a decisão, e o retorno da decisão ao processo.	125
20.1	O que é portátil e o que cada ambiente acrescenta por conta própria.	131
21.1	Onde cada tecnologia se encaixa, do laço de controle à nuvem.	137
22.1	Zonas com requisitos distintos e os condutos que autorizam a comunicação entre elas.	145
23.1	O ciclo da eficiência: medir, agir na lógica e verificar pelo indicador.	151
24.1	Roteiro de verificação a partir do sintoma observado.	160
25.1	Trilha sugerida de estudo, da base elétrica à integração de sistemas.	165
B.1	Legado: XIC sobre <code>chave1</code> ; o contato conduz quando o dado está energizado.	188
B.2	Fluxo do exame da condição: dado binário igual a 0 deixa a saída da linha falsa; igual a 1, verdadeira.	188
B.3	Legado: XIO sobre <code>chave1</code> ; o contato conduz quando o dado está desenergizado.	189
B.4	Fluxo do exame com contato invertido: dado binário igual a 0 torna a saída verdadeira.	189
B.5	Legado: OTE aciona <code>lampada1</code> enquanto o contato <code>chave</code> permanece fechado.	189
B.6	Legado: OTL energiza <code>lampada</code> com retenção.	190
B.7	Legado: OTU desenergiza <code>lampada2</code> com retenção.	190
B.8	Legado: ONS habilita <code>ADD</code> uma única vez por subida de <code>limit_switch_1</code> , usando <code>storage_1</code> como bit de armazenamento.	191

B.9	Legado: OSR sobre <code>limit_switch_1</code> energiza <code>output_bit_1</code> uma vez por subida e soma 5 ao acumulado.	191
B.10	Fluxo do OSR: o bit de saída depende do estado do bit de armazenamento e da condição da linha.	192
B.11	Legado: OSF sobre <code>limit_switch_1</code> energiza <code>output_bit_2</code> uma vez por descida e soma 5 ao acumulado.	192
B.12	Diagrama de tempo do OSF, que é o OSR visto pela transição de descida: a instrução é executada uma vez e só volta a operar na varredura seguinte. . . .	193
B.13	Legado: TON com <code>Timer time_1</code> , <i>Preset</i> e <i>Accum</i> , acionado por <code>limit_switch_1</code> .	193
B.14	Diagrama de tempo do TON: EN acompanha a entrada, TT indica a contagem e DN sobe ao atingir a referência.	194
B.15	Legado: TOF com <code>Timer time_2</code> ; o atraso de 180 ms vale na desenergização. .	194
B.16	Diagrama de tempo do TOF: a saída cai somente depois de transcorrido o tempo de referência após a entrada desligar.	195
B.17	Legado: RTO com <code>Timer time_3</code> ; o acumulado é retido e só zera com RES. . .	195
B.18	Diagrama de tempo do RTO: o acumulado não é perdido quando a entrada desliga, ao contrário do TON.	196
B.19	Legado: CTU com referência 10; o bit DN energiza <code>light_1</code> e RES zera o acumulado.	196
B.20	Diagrama de tempo do CTU: a saída DN sobe ao alcançar o valor de referência e permanece até o reinício.	197
B.21	Diagrama de tempo do CTD: o acumulado decresce a cada pulso de entrada. .	197
B.22	Legado: RES zera <code>counter_1</code> quando <code>limit_switch_2</code> fecha.	198
B.23	Exemplo de EQU na fonte: se <code>valor_1</code> for igual a <code>valor_2</code> , <code>luz_a</code> é energizada.	198
B.24	Exemplo de NEQ na fonte: com valores diferentes, <code>luz_4</code> é energizada. . . .	198
B.25	Exemplo de LES na fonte: <code>luz_3</code> é energizada quando <code>valor_1</code> é menor que <code>valor_2</code>	199
B.26	Exemplo de GRT na fonte: <code>luz_1</code> é energizada quando <code>valor_1</code> é maior que <code>valor_2</code>	199
B.27	Exemplo de LEQ na fonte: <code>luz_2</code> é energizada quando <code>valor_1</code> é menor ou igual a <code>valor_2</code>	199
B.28	Exemplo de GEQ na fonte: <code>luz_b</code> é energizada quando <code>valor_1</code> é maior ou igual a <code>valor_2</code>	200
B.29	Legado: CMP avalia a expressão $(\text{valor}_1 * \text{valor}_2) - \text{valor}_3 < \text{valor}_4$. . .	200
B.30	Tabela de operadores aceitos na expressão de CMP e de CPT, com o tipo de dado exigido por cada um.	200
B.31	Exemplo de ADD na fonte: a soma de <code>valor_1</code> e <code>valor_2</code> é armazenada em <code>resultado</code>	201
B.32	Exemplo de SUB na fonte: <code>valor_1</code> menos <code>valor_2</code> é armazenado em <code>resultado</code> .	201
B.33	Exemplo de MUL na fonte: o produto de <code>valor_1</code> e <code>valor_2</code> é armazenado em <code>resultado</code>	202
B.34	Exemplo de DIV na fonte: <code>valor_1</code> dividido por <code>valor_2</code> , com o resultado em <code>resultado</code>	202
B.35	Exemplo de MOD na fonte: o resto da divisão do dividendo 10 pelo divisor 3 é armazenado em <code>resto</code>	203
B.36	Exemplo de SQR na fonte: a raiz quadrada de <code>valor_1</code> é armazenada em <code>resultado</code>	203
B.37	Exemplo de NEG na fonte: a negação de <code>valor_1</code> é armazenada em <code>resultado</code> .	204
B.38	Exemplo de ABS na fonte: o módulo de <code>valor_1</code> é guardado no destino. . . .	204
B.39	Legado: CPT avalia $(\text{valor}_1 * 5) / (\text{valor}_2 / 7)$ e guarda o resultado.	204
B.40	Legado: MOV copia <code>valor_1</code> para <code>valor_2</code>	205

B.41 Exemplo de AND na fonte: a operação bit a bit entre <code>valor_1</code> e <code>valor_2</code> resulta em <code>resultado</code>	205
B.42 Representação da operação AND sobre os bits: o resultado tem 1 apenas onde os dois operandos têm 1.	206
B.43 Exemplo de OR na fonte: a operação bit a bit entre <code>valor_1</code> e <code>valor_2</code> resulta em <code>resultado</code>	206
B.44 Tabela-verdade do OR bit a bit, como a fonte a apresenta.	206
B.45 Exemplo de XOR na fonte: a operação bit a bit entre <code>valor_1</code> e <code>valor_2</code> resulta em <code>resultado</code>	207
B.46 Representação da operação XOR sobre os bits, com o resultado bit a bit. . . .	207
B.47 Exemplo de NOT na fonte: a inversão bit a bit de <code>valor_1</code> resulta em <code>resultado</code>	207
B.48 Representação da operação NOT sobre os bits: todos os bits são invertidos. . .	207
B.49 Legado: CLR zera todos os bits da variável de destino.	208
B.50 Legado: BSL sobre <code>array_dint[0]</code> , com <i>Control</i> , <i>Source Bit</i> , <i>Length</i> e os bits EN e DN.	208
B.51 Efeito do BSL sobre os bits: o bit 9 é descarregado em <code>.UL</code> , os demais andam para a esquerda e <code>input_1</code> entra no bit 0.	208
B.52 Legado: BSR sobre <code>array_dint[0]</code> , com <i>Control</i> , <i>Source Bit</i> e <i>Length</i>	209
B.53 Efeito do BSR sobre os bits: o bit 9 é descarregado em <code>.UL</code> e os demais andam para a direita.	209
B.54 Legado: JSR chama a rotina indicada em <i>Routine name</i> , com parâmetros de entrada e de retorno.	209

Lista de Tabelas

1.1 Frentes de ganho da automação e como observá-las.	17
2.1 Requisitos da especificação original, reescritos e ainda válidos.	21
2.2 Gerações de controladores e o que mudou em cada uma.	22
2.3 Categorias de controlador que convivem em uma planta hoje.	23
3.1 Níveis da automação: função, equipamentos e tecnologias típicas.	26
4.1 Documentos básicos de um projeto de automação.	29
4.2 Exemplo de lista de I/O.	29
5.1 Características de um controlador programável e sua consequência prática. . .	34
5.2 Arranjos físicos de E/S e quando cada um é adequado.	36
6.1 Critérios de classificação de memórias.	39
6.2 Tecnologias de memória presentes em controladores.	39
6.3 Tipos de dados do item 4.5.1 do material de origem, com a faixa expressa em potências de dois e o valor decimal correspondente.	41
6.4 Principais funções de conversão explícita de tipos da IEC 61131-3.	43
6.5 Organização típica de uma <i>string</i> de 85 bytes.	46
6.6 As seis áreas do mapa de memória do CLP.	47

7.1	Saída de sensor a três fios e o comum exigido na entrada.	52
7.2	Tipos de saída digital e critério de escolha.	54
7.3	Efeito da resolução do cartão na resolução obtida (faixa de 4 a 20 mA).	58
8.1	Classificação por quantidade de pontos de E/S.	61
8.2	Redundância por hardware e por software.	63
9.1	Modos de processamento do programa.	68
9.2	Estado dos pontos no instante analisado e resultado do processamento.	69
10.1	Linguagens da IEC 61131-3, natureza e vocação.	74
10.2	Qual contato usar na lógica, em função da chave de campo.	76
10.3	Critérios de análise das linguagens, para escolha em projeto.	78
11.1	As três unidades de organização de programa.	81
11.2	Tipos de tarefa e consequência no comportamento do programa.	82
11.3	Atributos de variável previstos pela norma.	83
12.1	Partes da família IEC 61131 e o assunto de cada uma.	85
12.2	Edições da IEC 61131-3 e o que cada uma trouxe.	86
12.3	Perguntas para uma declaração de conformidade.	88
13.1	Implementação conforme a norma e implementação legada, lado a lado.	91
13.2	Conversão de mnemônicos de fabricante para elementos da norma.	92
13.3	Recomendações para reduzir dependência de fabricante.	93
14.1	Prefixos sugeridos por tipo de ponto.	96
14.2	O que pertence à lógica programada e o que não pertence.	98
14.3	Checklist de revisão.	99
15.1	Documentação mínima de um projeto de CLP.	102
15.2	Modelo de tabela de alocação de E/S (exemplo).	102
15.3	Rotina mínima de backup e versionamento.	103
16.1	O que cada nível detecta e o que ele não detecta.	106
16.2	Modelo de roteiro de teste.	107
16.3	O que a simulação não substitui.	108
17.1	Comparação das três arquiteturas de rede do CLP.	112
17.2	Protocolos de comunicação usuais na automação industrial e seu emprego típico.	114
17.3	Alarme, evento e intertravamento: quem faz o quê e qual é a consequência do erro.	116
18.1	O que o IIoT entrega, em termos práticos.	121
19.1	Indicadores usuais de produção e consumo, o que exigem do controle e onde costumam falhar.	126
19.2	Critérios para decidir onde processar cada tarefa.	127
20.1	Pontos de ruptura em uma migração de fabricante e o que fazer com cada um.	133
20.2	Cenários de migração e esforço típico.	134
21.1	Tecnologia da informação (TI) e tecnologia de operação (TO), lado a lado.	136
21.2	Usos de Linux na automação e o cuidado correspondente.	142

21.3	Balanco de cada tecnologia de TI trazida para a operação.	142
22.1	Segurança funcional e segurança cibernética.	144
22.2	Pontos de entrada de um sistema de automação e contramedidas usuais. . . .	146
23.1	Alavancas de eficiência que se implementam na lógica de controle.	152
24.1	Modos de falha usuais em sistemas com CLP, sinal observável e prevenção. .	158
24.2	Rotina preventiva mínima e evidência de execução.	159
24.3	Diagnóstico: sintoma, causas prováveis e verificação.	161
25.1	Etapas da trilha, o que se aprende e como provar que aprendeu.	166
25.2	Recursos úteis e o cuidado correspondente.	168
25.3	Plano de estudo de 90 dias, com entrega verificável por etapa.	169
A.1	Visão geral das práticas, linguagem sugerida e capítulos de apoio.	173
B.1	Bits de status e parâmetros do legado e o equivalente normatizado.	182
B.2	Pontos que exigem decisão na conversão, e o que verificar.	210
C.1	Termos técnicos em português, o correspondente em inglês e a observação de uso.	213
D.1	Normas citadas, assunto e edição conferida.	220

Parte I

Fundamentos e projeto

Capítulo 1

Introdução e importância da automação

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- explicar o que é automação industrial e por que ela existe;
- relacionar os objetivos da automação com os indicadores que ela melhora;
- situar o Controlador Lógico Programável (CLP) dentro do sistema de automação;
- distinguir, na prática, o que é definido por norma do que é definido por fabricante.

1.1 O que é automação industrial

Automação industrial é o uso de equipamentos e programas para operar um processo produtivo com pouca ou nenhuma intervenção humana direta. Ela não substitui apenas a força física: substitui sobretudo a *decisão repetitiva* — ligar, desligar, medir, comparar com um valor desejado e corrigir.

Essa decisão repetitiva foi, por décadas, responsabilidade de painéis de relés, temporizadores de came e sequenciadores. A lógica estava *fiada*, literalmente: mudar a sequência de uma máquina significava refazer o painel elétrico, parar a produção e refazer a documentação. Foi essa rigidez que criou a necessidade que dá origem ao CLP (Capítulo 2).

1.2 Por que automatizar

Automatizar não é um fim em si. As razões aparecem em cinco frentes, e cada uma tem uma forma de ser medida — o que importa quando é preciso justificar um investimento (Figura 1.1).

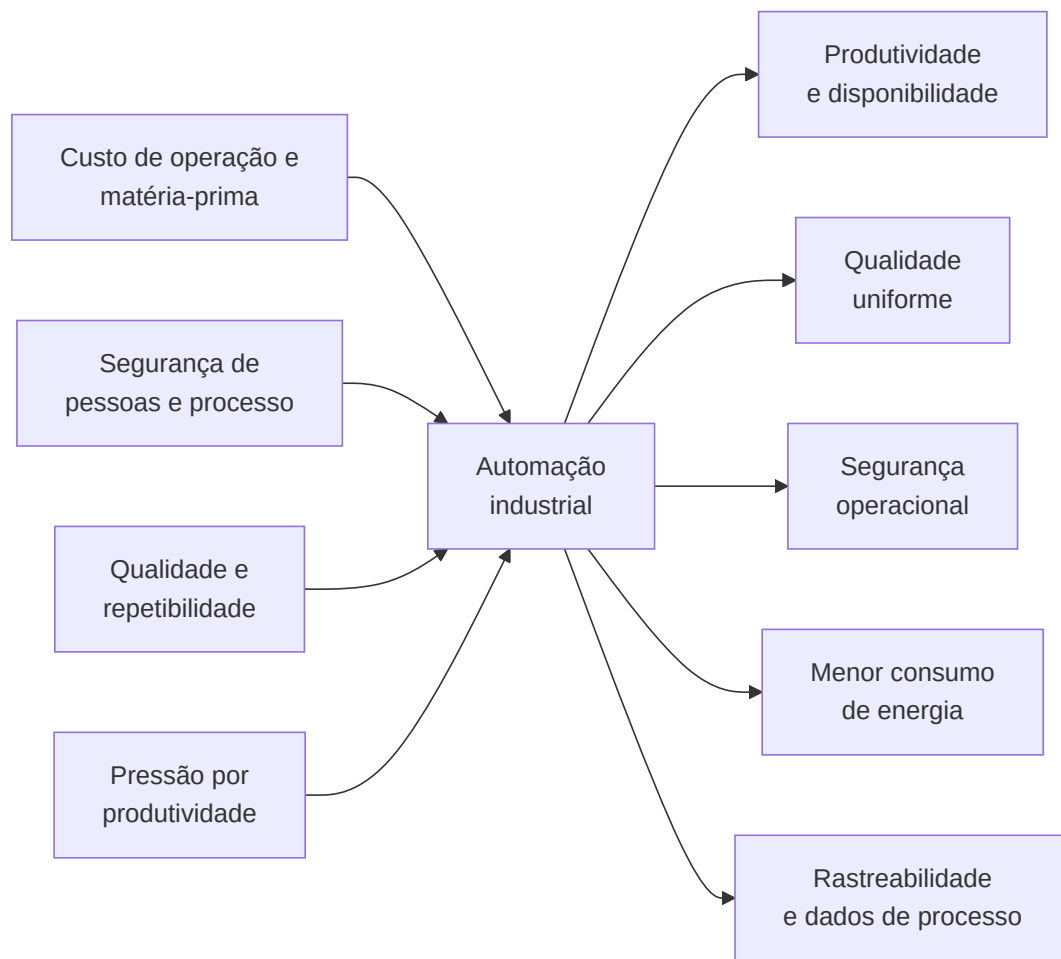


Figura 1.1: Pressões que levam à automação e ganhos que ela entrega.

Tabela 1.1: Frentes de ganho da automação e como observá-las.

Frente	O que muda na prática	Como se observa
Produtividade	Menos tempo parado, ciclo mais estável e maior volume no mesmo tempo	Disponibilidade, tempo de ciclo, peças por hora
Qualidade	Repetibilidade da sequência: a máquina executa igual a cada ciclo	Refugo, retrabalho, variação da característica
Segurança	Eliminação do homem na zona de risco e intertravamento confiável	Incidentes, intervenções em área de risco
Energia e insumo	Desligamento automático, ponto de operação otimizado, fim do funcionamento a vazio	Consumo específico (kWh por unidade), desperdício de matéria-prima
Informação	O processo passa a produzir dados de produção, falha e consumo	Existência de histórico, rastreabilidade, indicadores por turno

1.3 Onde o CLP é aplicado

Praticamente não há ramo industrial sem aplicação de controladores programáveis. Os casos mais frequentes:

- **Máquinas industriais** — usinagem, injetoras de plástico, têxteis, calçados;
- **Processos contínuos e de batelada** — siderurgia, papel e celulose, petroquímica, química, alimentos, mineração;
- **Energia** — controle de demanda e de fator de carga;
- **Supervisão e aquisição de dados** — fábricas, prédios inteligentes, utilidades;
- **Ensaio e bancadas** — teste automático de componentes, laboratórios;
- **Sistemas embarcados e móveis** — máquinas agrícolas e de construção, equipamentos de transporte.

1.4 O CLP no centro do sistema

O CLP é o equipamento que executa a lógica de controle de forma *cíclica e determinística*: ele lê as entradas, processa o programa e escreve as saídas, repetidamente, em intervalos de alguns milissegundos. É essa característica que o diferencia de um computador comum e que dá previsibilidade ao processo (Capítulo 9).

Ao redor dele ficam os instrumentos de campo, os módulos de entrada e saída, o terminal de programação e as redes de comunicação que o ligam à supervisão e à gestão (Capítulo 3).

Atenção

Nada nesta apostila autoriza intervir em painel energizado. Antes de qualquer medição, troca de módulo ou edição de programa em máquina em produção, aplique o procedimento de bloqueio e sinalização da sua instituição. CLP não é equipamento de aprendizagem em máquina viva.

1.5 Como ler esta apostila

Esta obra foi organizada em cinco partes: fundamentos e projeto, hardware e funcionamento, programação e normas, integração e sistemas, operação e carreira. Cada capítulo fecha com exercícios e, quando aplicável, uma caixa *Na prática*.

Uma convenção vale para todo o texto: **sempre que uma linguagem, instrução ou endereçamento for apresentado, ficará claro se aquilo é definido por norma ou se é proprietário de um fabricante**. Essa distinção é o assunto do Capítulo 13 e aparece marcada em cada tópico de programação.

Na prática

Escolha uma máquina da sua escola ou do seu trabalho e responda por escrito: (a) quais decisões hoje são feitas por relés, botoeiras ou operador; (b) quais dessas decisões poderiam ser automatizadas; (c) qual indicador da Tabela 1.1 melhoraria e como você mediria isso hoje, sem instalar nada novo.

Exercícios

- a) Explique, com suas palavras, por que “lógica fiada” dificulta a mudança de um processo.
- b) Cite dois ganhos da automação que não sejam de produtividade e justifique sua escolha.
- c) Um gestor afirma que automatizar sempre reduz o custo por peça. Aponte uma situação em que isso não se verifica e explique o motivo.
- d) Dê um exemplo de decisão que *não* deve ser confiada apenas ao programa do CLP, e diga quem ou o que deveria assumi-la.

Capítulo 2

Evolução dos CLPs

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- descrever o problema industrial que deu origem ao CLP;
- situar no tempo as gerações de controladores e o que mudou em cada uma;
- diferenciar CLP, PAC, controlador de segurança e controlador virtual;
- reconhecer o que permaneceu igual desde 1969 e por quê.

2.1 Antes do CLP: a lógica fiada

O controle de máquinas nasceu eletromecânico. Relés, contatos, bobinas e temporizadores de motor síncrono executavam a sequência, e a lógica estava fisicamente desenhada na fiação. Funcionava bem — mas cobrava um preço alto a cada mudança:

- mau contato e desgaste mecânico dos contatos, com falhas intermitentes;
- instalação de um número enorme de relés, com fiação ponto a ponto entre centenas de terminais;
- dificuldade de alterar a sequência, que exigia redesenhar e refazer o painel;
- manutenção periódica obrigatória e paradas recorrentes.

Mudar a lógica de uma linha de montagem significava parar a produção e trocar os quadros elétricos. Era gasto de tempo e de dinheiro em uma escala que a indústria automobilística não podia mais aceitar.

2.2 1968–1969: a especificação da GM e o primeiro CLP

Em **1968**, a divisão *Hydramatic* da General Motors — responsável pelas transmissões automáticas — publicou uma especificação para um substituto eletrônico dos painéis de relé. A especificação nasceu de um documento do engenheiro Edward R. Clark e refletia a insatisfação de muitos usuários de relés, dentro e fora da indústria automobilística. Na época, a liderança do grupo ficou associada ao engenheiro **Richard “Dick” Morley**, hoje considerado o pai do CLP.

A proposta vencedora foi da **Bedford Associates**, de Bedford, Massachusetts. O equipamento ficou pronto em **1969** e recebeu a designação **084**, porque era o octogésimo quarto projeto da empresa. A Bedford criou em seguida a **Modicon**, dedicada a fabricar e vender o produto — o nome vinha de *modular digital controller*. Em paralelo, a Allen-Bradley

desenvolvia sua própria linha, e o engenheiro Odo J. Struger ficou conhecido por cunhar a sigla *PLC* para distinguir o controlador do “PC” (*personal computer*).¹

A Modicon criou também o **Modbus**, protocolo que se tornaria padrão aberto (Capítulo 17).

2.3 Os dez requisitos da especificação original

A especificação de 1968 é um excelente roteiro de projeto porque continua atual. O novo equipamento deveria:

Tabela 2.1: Requisitos da especificação original, reescritos e ainda válidos.

#	Requisito original	Leitura atual
1	Facilidade de programar e de alterar o programa, inclusive na estação de trabalho	Programação em campo, com o processo rodando, e alteração sem refazer painel
2	Manutenção fácil, de preferência com unidades <i>plug-in</i>	Módulos plug-in e troca a quente
3	Confiabilidade superior à do painel de relés	É a razão de existir do CLP: ambiente industrial, imunidade a ruído, diagnóstico
4	Mais compacto que o painel de relés	Densidade de pontos por módulo e por rack
5	Envio direto de dados à unidade central de processamento	O embrião do que hoje é a integração com SCADA e IIoT (Capítulos 18–19)
6	Custo competitivo frente ao painel de relés	Justificativa econômica do projeto; hoje inclui custo de ciclo de vida
7	Aceitar entradas em 115 V CA	Compatibilidade com o sinal de campo da instalação: 24 V CC e 4–20 mA hoje
8	Fornecer saídas em 115 V CA acima de 2 A, capazes de acionar válvulas e partidas de motor	Acionamento direto de cargas com a saída discreta
9	Permitir ampliação do sistema com o mínimo de alteração	Arquitetura modular e expansão por módulos e remotas
10	Ser programável por pessoal de manutenção, não só por especialistas	Linguagem gráfica próxima do esquema elétrico: a razão do sucesso do Ladder

2.4 Gerações: o que mudou em cada salto

A Figura 2.1 resume essa trajetória em uma linha de tempo.

¹Fatos históricos conferidos em Laughton & Warne (2002) e Dunn (2009), reunidos em https://en.wikipedia.org/wiki/Programmable_logic_controller (consulta em 22/09/2026).

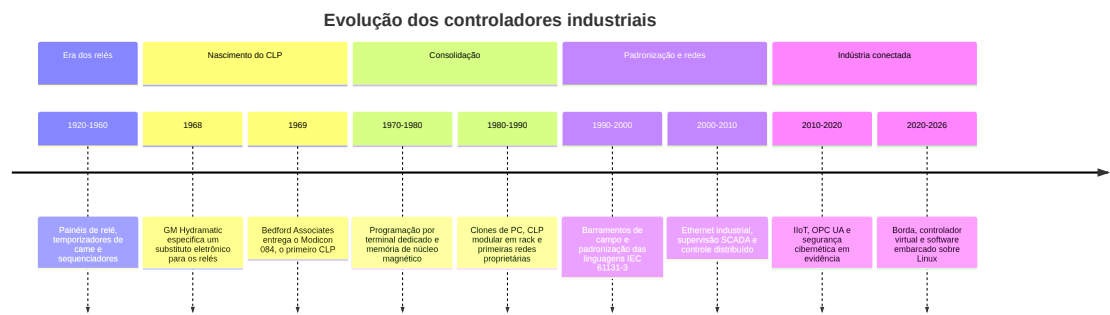


Figura 2.1: Linha do tempo da evolução dos controladores industriais.

Tabela 2.2: Gerações de controladores e o que mudou em cada uma.

Período	Programação	Hardware e memória	Limitação da época
1969–1975	Terminal dedicado, lógica booleana, sem representação gráfica; depois Ladder	Memória de núcleo magnético, componentes discretos; programa em fita cassete	Sem padronização; pouca capacidade de documentação e impressão
1975–1985	Ladder consolidado como linguagem de campo	Microprocessador substitui a lógica discreta; memória semicondutora	Redes proprietárias e fechadas; endereçamento específico de cada fabricante
1985–1995	Primeiros softwares de programação em PC; sub-rotinas	CLP modular em rack, maior densidade de pontos	Programas presos ao fabricante; troca de marca exigia reescrever tudo
1995–2005	Padronização das linguagens pela IEC 61131-3	Barramentos de campo, I/O remoto, CPUs mais rápidas	Portabilidade ainda parcial: endereçamento e blocos de sistema continuam proprietários
2005–2015	Ambiente integrado de projeto, simulação e versionamento	Ethernet industrial, controle distribuído, redundância	Integração com a rede corporativa traz o problema de segurança cibernética
2015–2026	Padrões de modelagem de informação (OPC UA) e integração com software livre	Borda (<i>edge</i>), virtualização e controladores baseados em Linux	Determinismo <i>versus</i> flexibilidade; certificação e atualização de software

2.5 CLP, PAC, controlador de segurança e controlador virtual

O termo “CLP” virou guarda-chuva. Vale separar as categorias (Tabela 2.3).

Tabela 2.3: Categorias de controlador que convivem em uma planta hoje.

Categoria	Característica central	Onde aparece
CLP clássico	Lógica discreta e sequencial com ciclo de varredura determinístico; Ladder como linguagem natural	Máquinas, linhas de montagem, utilidades
PAC (<i>programmable automation controller</i>)	Mesma base do CLP, com mais memória e processamento, multitarefa real, tipagem forte e forte integração em rede	Processos que misturam lógica, movimento e malhas PID
Controlador de segurança	Adequado a funções críticas de segurança, com nível de integridade (SIL) e conjunto de instruções específico para parada de emergência e cortinas de luz	Zonas de risco, células robotizadas
CLP virtual ou <i>soft PLC</i>	Função de controlador executada em hardware de propósito geral, em máquina virtual ou contêiner	Simulação, ensino, controle remoto, arquiteturas em nuvem
RTU (<i>remote terminal unit</i>)	Controlador rústico, de baixo consumo, cuja função principal é gerir o enlace de comunicação, com carimbo de tempo dos eventos	Sistemas a distância: saneamento, energia, dutos

Atenção

Controlador de segurança não é “um CLP mais caro”. É um equipamento certificado para uma função de segurança, com arquitetura e diagnóstico próprios. Lógica de segurança implementada em CLP comum, sem certificação, **não** cumpre função de segurança.

2.6 O que não mudou

Apesar de meio século de evolução, três decisões de 1969 continuam governando o projeto de qualquer controlador atual:

1. **o ciclo de varredura** — ler entradas, processar, escrever saídas, repetir;
2. **a linguagem gráfica próxima do esquema elétrico**, que sobreviveu como Ladder justamente por ser legível para quem vem do comando elétrico;
3. **o foco em diagnóstico** — sinalizar estado e falha, que era o requisito 3 e segue sendo o principal diferencial entre um controlador industrial e um computador comum.

Na prática

Compare um CLP antigo ainda em operação na sua região (ou a documentação dele) com um controlador atual. Monte uma tabela com três colunas: *o que era*, *o que é hoje* e *o que continua igual*. Use a Tabela 2.2 como ponto de partida, mas registre o que encontrar de concreto.

Exercícios

- a) Por que a indústria automobilística, e não a química ou a de energia, foi a primeira a exigir o controlador programável? Relacione a resposta com a frequência de mudança do processo.
- b) Escolha três dos dez requisitos da Tabela 2.1 e mostre como cada um é atendido por um CLP atual.
- c) Explique a diferença entre CLP e PAC sem usar a palavra “potência”.
- d) Um colega propõe usar um CLP comum, com lógica cuidadosamente escrita, para substituir um relé de segurança de uma prensa. Escreva a resposta técnica, em cinco linhas, explicando por que a proposta não é aceitável.
- e) Explique o que um CLP virtual permite fazer que o hardware físico não permite, e qual problema ele *não* resolve.

Capítulo 3

A estrutura da automação

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- descrever os níveis hierárquicos de um sistema de automação industrial;
- dizer o que trafega em cada nível e em que direção;
- relacionar cada nível às tecnologias e protocolos usualmente empregados;
- localizar o CLP nessa hierarquia e explicar por que ele não está sozinho.

3.1 Do chão de fábrica à gestão

Um sistema de automação não é apenas um controlador ligado a sensores e atuadores/elementos finais de controle. É uma hierarquia em que cada nível consome informação do nível inferior e devolve decisão ao nível de baixo. Essa organização, chamada de *pirâmide da automação*, existe porque os requisitos mudam a cada degrau: no campo importa o tempo de resposta; na gestão, importa o histórico e o indicador do mês.

PIRÂMIDE DA AUTOMAÇÃO INDUSTRIAL

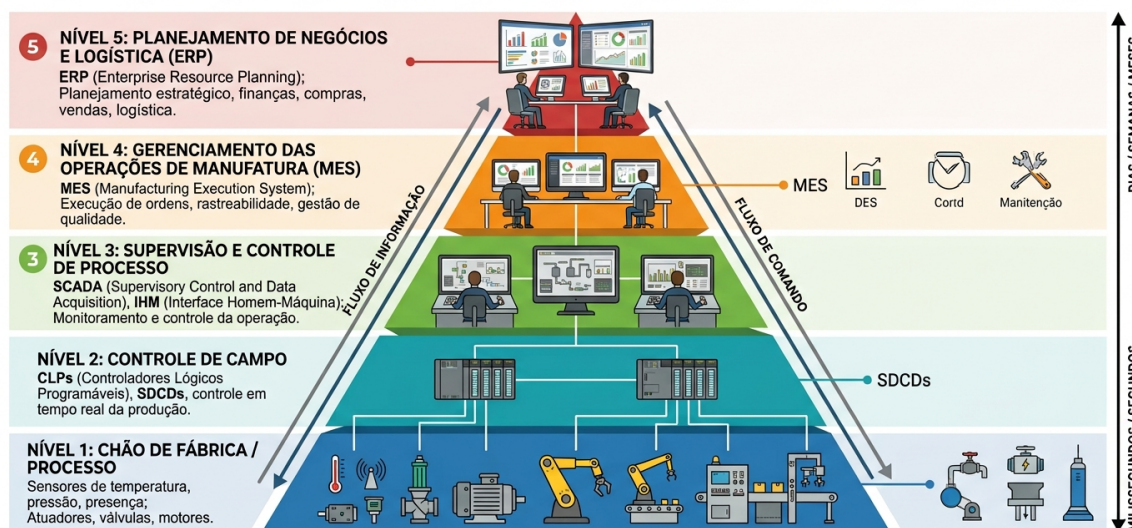


Figura 3.1: Níveis da automação industrial e fluxo de informação entre eles.

Fonte: Imagem obtida na internet e reeditada com apoio de ferramenta de IA generativa (Nano Banana).

O fluxo tem duas direções (Figura 3.1): os dados sobem — medida, estado, evento — e os comandos descem — meta, *setpoint*, ordem de produção.

3.2 Os cinco níveis

Tabela 3.1: Níveis da automação: função, equipamentos e tecnologias típicas.

Nível	Função	Equipamentos	Tecnologias usuais
5. Corporativo	Gestão financeira, administrativa e de recursos; indicadores de resultado	ERP, sistemas de <i>business intelligence</i>	Redes corporativas, bancos de dados; acesso raramente direto aos níveis fabris
4. Gerencial	Planejamento detalhado da produção, logística, indicadores e apoio à manutenção preditiva	MES, PIMS	Ethernet de alto desempenho, banco de dados histórico
3. Supervisão	Operação assistida, alarmes, telas, histórico e relatórios do processo	SCADA, IHM, <i>historian</i> , servidores de dados	Ethernet TCP/IP; OPC como camada de interoperabilidade entre sistemas
2. Controle	Executar o controle automático: lógica, intertravamento, malhas	CLP, SDCD, remotas de I/O	Profibus DP, DeviceNet, ControlNet, Modbus TCP, EtherNet/IP
1. Campo	Transformar grandeza física em sinal e executar a ação sobre o processo	Sensores, transmissores, atuadores, posicionadores	Redes de campo com baixa largura de banda: CAN, AS-i, Modbus, Profibus PA, Fieldbus Foundation H1

Nível 1 — Campo. É onde o processo encontra a instrumentação. Os dispositivos podem ser ponto a ponto, ligados diretamente ao cartão de entrada e saída, ou microprocessados e inteligentes, gerando volume grande de dados. As redes aqui são heterogêneas e, em geral, de baixa largura de banda.

Nível 2 — Controle. É onde vive o CLP. Exige inteligência local suficiente para executar lógica, intertravamento e malhas de controle, e comunicação mais sofisticada que a do campo: protocolos com velocidade maior e capacidade de resposta em tempo real.

Nível 3 — Supervisão. Concentra a operação. Aqui ficam as telas, o banco de dados do processo, o registro de eventos e alarmes. Neste nível é comum encontrar conversores de protocolo usando OPC, exatamente para garantir interoperabilidade entre sistemas de fabricantes diferentes (Capítulo 20).

Nível 4 — Gerencial. Transforma dados em informação de planejamento. Ferramentas como MES e PIMS produzem relatórios mais elaborados que os da supervisão, porque trabalham com massa de dados maior e com histórico.

Nível 5 — Corporativo. Trata da gestão de recursos da empresa. Trabalha com resultado financeiro e, tradicionalmente, tem pouco acesso direto aos dados técnicos dos níveis inferiores — é justamente essa fronteira que as arquiteturas de IIoT vieram discutir (Capítulo 18).

3.3 O que mudou na pirâmide

A pirâmide clássica pressupõe um fluxo lento de baixo para cima e uma separação clara entre rede fabril e rede administrativa. Dois movimentos alteraram esse desenho:

1. **Redes de campo substituíram a fiação ponto a ponto**, reduzindo drasticamente o cabeamento entre o processo e o sistema de controle. Em vez de levar todo sinal até o rack do CLP, os sinais vão para remotas de I/O instaladas no campo (Capítulo 17).
2. **A camada de informação se sobrepõe aos níveis**, e não os substitui: dados de um sensor de campo passam a interessar diretamente à manutenção e à gestão, sem atravessar necessariamente todos os degraus. A governança desse acesso é um problema de arquitetura e de segurança, não apenas de rede.

Atenção

Nível hierárquico não é o mesmo que prioridade de tempo. O nível de campo e o de controle respondem em milissegundos; o gerencial trabalha em horas ou dias. Confundir essas escalas é a origem de vários projetos mal dimensionados — e de acidentes, quando se tenta fazer controle rápido por caminho lento.

Na prática

Visite uma planta ou examine a arquitetura de um sistema que você conheça e preencha a Tabela 3.1 com os equipamentos *reais* de cada nível. Depois responda: (a) algum nível está ausente ou foi fundido com outro? (b) qual é o caminho que um dado de sensor percorre até virar indicador de gestão? (c) existe alguma ligação direta entre os níveis 1–2 e o nível 4 ou 5? Quem autorizou?

Exercícios

- a) Explique por que a largura de banda exigida no nível de campo é menor que a exigida no nível de supervisão.
- b) Um instrumento inteligente gera 40 parâmetros de diagnóstico, mas o projeto só usa a variável de processo. Cite duas consequências práticas e diga para qual nível esses parâmetros interessam.
- c) Descreva o caminho de um alarme desde a sua detecção no CLP até o registro no nível gerencial, nomeando cada equipamento envolvido.
- d) Por que o uso de OPC aparece tipicamente no nível de supervisão e não no nível de campo?
- e) A direção “gerência → chão de fábrica” leva que tipo de informação? Dê dois exemplos.

Capítulo 4

Projeto de automação industrial

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- situar a documentação básica dentro do ciclo de um projeto de automação;
- ler um P&ID e extrair dele as informações necessárias ao controle;
- montar uma lista de I/O a partir da instrumentação identificada;
- reconhecer o que um descritivo operacional precisa conter;
- entender como a planta baixa e a arquitetura definem I/O local ou remoto.

4.1 O projeto antes do programa

Programar o CLP é a última etapa de um processo que começa muito antes. Um projeto de automação — básico, detalhado ou executivo — se apoia em documentação consistente que define o que o sistema deve fazer. Sem ela, o programa vira registro de decisões improvisadas, e a manutenção futura fica dependente de quem escreveu o código. A Figura 4.1 mostra o encadeamento dessas etapas.

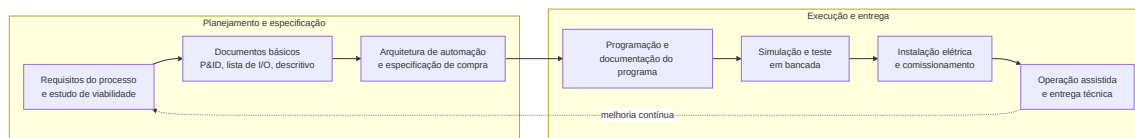


Figura 4.1: Fluxo simplificado de um projeto de automação, do requisito à entrega técnica.

Atenção

Documentação não é burocracia posterior. É insumo de projeto: a lista de I/O define o porte do controlador (Capítulo 8), e o descritivo operacional define a lógica (Capítulo 14). Documento atrasado é retrabalho garantido.

4.2 Documentos básicos e essenciais

Cinco documentos formam o núcleo do projeto. Os dois primeiros sustentam os demais:

Tabela 4.1: Documentos básicos de um projeto de automação.

Documento	O que define	Serve de base para
P&ID (fluxograma de processo e instrumentação)	Equipamentos, tubulação, instrumentação, malhas de controle, intertravamentos e segurança	Lista de I/O, descritivo operacional e definição das malhas
Lista de I/O	Quantidade e tipo de pontos: DI, DO, AI e AO, por instrumento	Dimensionamento de cartões, remotas e porte do controlador
Descritivo operacional	Como o processo funciona, o que deve acontecer em cada condição e quais são as lógicas de segurança e intertravamento	Programação e especificação de modos de operação
Planta baixa de instrumentação	Localização geográfica dos instrumentos e o caminho dos cabos	Decisão entre I/O local e I/O remoto; arquitetura de rede
Arquitetura de automação	Concepção geral do sistema e seus componentes principais	Detalhamento e especificação final de compra

4.3 P&ID: o ponto de partida

O P&ID é um diagrama de engenharia de processo que mostra o fluxo físico — equipamentos, tubulação e instrumentação — e é nele que se enxerga o sistema de controle necessário. Uma vez o P&ID definido, com a instrumentação exigida pelo funcionamento do processo e suas respectivas malhas de controle, segurança e intertravamento, inicia-se o **levantamento de pontos**.

À medida que a instrumentação é identificada, cada tag do P&ID precisa ser analisada quanto ao sinal que ocupa no controlador. É aqui que nasce a lista de I/O.

4.4 Lista de I/O

A leitura dos sinais de entrada é feita por cartões específicos, e o mesmo vale para as saídas. Os sinais mais comuns em campo são 4–20 mA, 1–5 V CC, 0/24 V CC, além de entradas para RTD (PT-100) e termopar em milivolts. Nem todos os tipos cabem no mesmo cartão — por isso os pontos são classificados e contados separadamente:

- **DI** (*discrete input*) — entrada discreta;
- **DO** (*discrete output*) — saída discreta;
- **AI** (*analog input*) — entrada analógica;
- **AO** (*analog output*) — saída analógica.

Tabela 4.2: Exemplo de lista de I/O.

Item	Tag	P&ID	Serviço	DI	DO	AI	AO
1	FIT-1003402/1	DE-001-023	Vazão de alimentação do R-2371	1		1	
2	XV-1003401/1	DE-001-023	Saída de produtos do R-2373	2	1		
3	SX-1003503	DE-001-023	Controle de vazão	2	1	1	1

A soma dos pontos define quantos cartões e quantas remotas serão necessários e, com isso, o porte do controlador mais adequado ao processo (Capítulo 8).

Na prática

Preencha a Tabela 4.2 com cinco instrumentos de um processo que você conheça. Para cada um, decida explicitamente: (a) o sinal é discreto ou analógico? (b) quantos pontos de cada tipo ele consome? (c) existe algum ponto que *não* seja de medição nem de comando, mas precise entrar na lista — por exemplo, um contato de status de partida de motor ou um alarme de falha do inversor? Esse último item é o erro mais comum em listas de I/O iniciantes.

4.5 Descritivo operacional

O descritivo operacional descreve o funcionamento do processo. É ele que permite entender tanto o processo quanto a contribuição da automação sobre ele, e é a fonte direta da lógica: condições de partida, sequência, paradas, alarmes e intertravamentos.

Um descritivo útil responde, para cada equipamento:

1. quais são as condições para partir e para parar;
2. qual é a sequência normal de operação, e o que acontece se um passo não for cumprido;
3. o que fazer em falha de energia, de comunicação, de instrumento e de equipamento;
4. quais situações exigem parada de emergência e qual o estado seguro resultante;
5. o que o operador pode comandar manualmente, e sob quais restrições.

4.6 Planta baixa, I/O remoto e arquitetura

A planta baixa com a localização dos instrumentos define o caminho dos cabos até o sistema de controle. Dependendo da distribuição e da densidade de instrumentos na área, decide-se entre I/O local — todos os sinais indo até o rack do CLP — e I/O remoto ou distribuído, com as remotas instaladas no campo.

A instalação com I/O local consome muito cabeamento, borneiras, caixas de passagem, bandejas, projeto e mão de obra. Os blocos de I/O remotos reduzem esses gastos de forma significativa, porque os sinais param de percorrer longas distâncias até o painel. A decisão é técnica e financeira, e resulta na arquitetura de rede do sistema (Capítulo 17).

O documento de **arquitetura geral de automação** consolida a concepção do sistema: define os componentes principais que orientarão o detalhamento e a especificação final de compra.

Atenção

Arquitetura definida sem a planta baixa é chute. Antes de fechar a arquitetura, responda: quantos instrumentos existem por área? Qual a distância máxima? Quanto custa o metro de cabo, considerando bandeja, passagem e mão de obra? Qual é o tempo de resposta exigido pelo processo?

4.7 Fechamento: o que fica para as próximas etapas

Ao final desta fase o projeto tem: P&ID definido, lista de I/O fechada com folga, descritivo operacional escrito, arquitetura definida e a especificação de compra encaminhada. Com isso o trabalho de programação deixa de ser criativo e passa a ser verificável — o programa implementa o que está descrito, e o descritivo permite conferir se implementou.

Exercícios

- a) Qual documento define a quantidade de cartões de entrada e saída de um projeto? Justifique.
- b) A uma lista de I/O de um motor foi anotado apenas o comando de partida (DO). Aponte o que está faltando e por que isso impediria a lógica de diagnóstico funcionar.
- c) Um P&ID indica vazão, pressão e nível em uma mesma linha, com uma válvula de controle. Descreva a malha e diga quais pontos de I/O ela provoca.
- d) Dê um exemplo de situação em que o I/O remoto é pior escolha que o I/O local no rack.
- e) Escreva, em no máximo dez linhas, o descritivo operacional de uma bomba que deve partir manualmente e parar automaticamente por nível alto, com alarme de falha elétrica.

Parte II

Hardware e funzionamento

Capítulo 5

Definições, características e aplicações

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- definir CLP a partir da definição normativa e de uma definição operacional;
- reconhecer as características que qualificam um equipamento como controlador industrial;
- identificar os blocos que compõem um CLP e o que cada um faz;
- diferenciar controlador compacto, modular e com I/O distribuído;
- distinguir o CLP de um computador de uso geral.

5.1 Definição

A definição clássica de controlador programável, dada pela *National Electrical Manufacturers Association* (NEMA), é:

“Aparelho eletrônico digital que utiliza uma memória programável para o armazenamento interno de instruções para implementações específicas, tais como lógica, sequenciamento, temporização, contagem e aritmética, para controlar, através de módulos de entradas e saídas, vários tipos de máquinas ou processos.”

Vale destacar quatro elementos dessa definição, porque cada um corresponde a uma decisão de engenharia:

1. **eletrônico digital** — não eletromecânico;
2. **memória programável** — a função é definida por programa, não por fiação;
3. **funções específicas** — lógica, sequenciamento, temporização, contagem e aritmética: este é o repertório mínimo esperado;
4. **módulos de entrada e saída** — a interface com o processo é parte do produto.

Em uma frase operacional, para o técnico de campo: *CLP é o equipamento que lê o estado das entradas, executa uma lógica gravada em memória e escreve o resultado nas saídas, repetidamente e em tempo previsível.*

5.2 Características gerais

As características abaixo são o que um comprador espera encontrar em um controlador industrial. O reconhecimento delas é útil em especificação (Capítulo 8) e em manutenção (Capítulo 24).

Tabela 5.1: Características de um controlador programável e sua consequência prática.

Característica	Consequência prática
Operação em ambiente industrial	Suporta faixa de temperatura, vibração, umidade e interferência eletromagnética que um computador comum não suporta
Programação e reprogramação rápidas, com mínima interrupção da produção	Altera-se a lógica sem refazer painel
Sinalizadores de estado e módulos <i>plug-in</i>	Troca de módulo por encaixe, sem mexer na fiação dos pontos vizinhos
Dimensão reduzida e baixo consumo	Ocupa menos espaço de painel e dissipa menos calor que o painel de relés equivalente
Compatibilidade com diferentes sinais de E/S	Aceita discreto e analógico, em corrente ou tensão, com cartões distintos
Capacidade de acionar cargas reais	A saída discreta adequada aciona cargas de alguns ampères, como bobinas de contatora e válvulas
Expansão de módulos e de memória	Cresce por módulos, e não por troca de equipamento
Comunicação em rede	Integra-se a outras CPUs, à supervisão e às redes de chão de fábrica
Monitoração do processo por computador	Permite supervisão, controle e aquisição de dados

5.3 Aplicações

O CLP automatiza processos de sequenciamento, intertravamento, controle de processo e batelada. Além da manufatura e dos processos contínuos, aparece em automação elétrica, predial e até dentro de produtos. Os casos mais frequentes são:

- máquinas industriais: usinagem, injetoras de plástico, têxteis, calçados;
- processos contínuos e de batelada: siderurgia, papel e celulose, petroquímica, química, alimentos, mineração;
- controle de energia: gestão de demanda e fator de carga;
- sinalização, intertravamento e controle PID em utilidades;
- aquisição de dados de supervisão em fábricas, edifícios inteligentes e laboratórios;
- bancadas de teste automático de componentes;
- controle embarcado em máquinas móveis, agrícolas e de construção.

5.4 Arquitetura de blocos

Um CLP é um sistema microprocessado, mais simples que um computador de uso geral, porém projetado para operar em condições severas. Seus blocos essenciais são a unidade central de processamento (UCP), as interfaces de entrada e saída e as memórias, todos apoiados por uma fonte de alimentação tratada e por um barramento interno. A Figura 5.1 reúne esses blocos e mostra como eles se ligam.

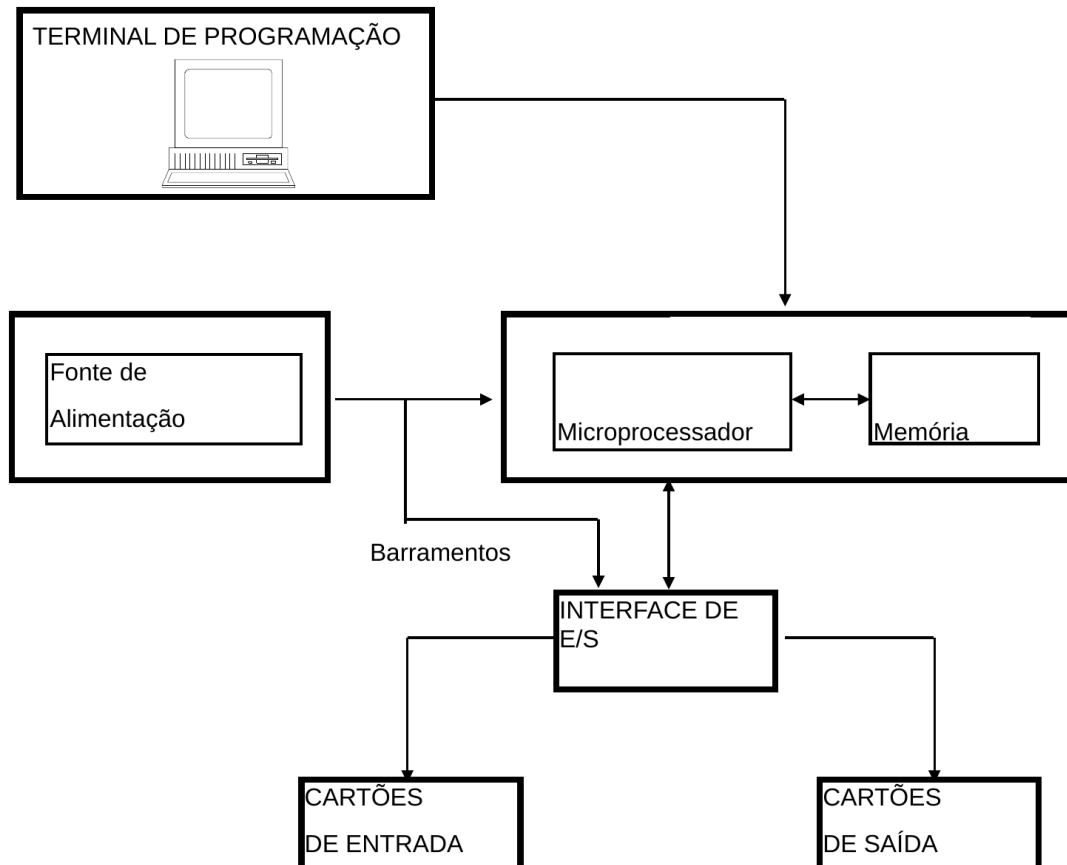


Figura 5.1: Diagrama em blocos com os componentes básicos de um CLP: terminal de programação, fonte de alimentação, microprocessador, memória, interface de E/S e cartões de entrada e saída, ligados pelos barramentos.

Fonte: VIANNA, W. S. Controlador Lógico Programável. IFF, 2023, figura 1.

O **microprocessador** é a inteligência do sistema: obtém os dados dos cartões de entrada, os processa conforme o programa do usuário e escreve o resultado nos cartões de saída. Um único processador costuma controlar dezenas ou centenas de pontos de E/S.

A **fonte de alimentação** recebe tratamento especial de filtragem e estabilização, porque disponibilidade e qualidade de energia são a diferença entre o ambiente industrial e o escritório. A alimentação dos *dispositivos de campo*, porém, é externa ao CLP — veja a caixa de atenção ao final deste capítulo.

5.5 Compacto, modular e com I/O distribuído

Quanto à estrutura física dos pontos de E/S, há três arranjos:

Tabela 5.2: Arranjos físicos de E/S e quando cada um é adequado.

Arranjo	Como é	Quando convém
Compacto	Todos os componentes em um único módulo, exceto o terminal de programação; costuma usar microcontrolador no lugar do conjunto processador, memória e interface	Aplicações pequenas, poucos pontos, custo e espaço reduzidos
Modular	Um ou mais racks, com fonte, CPU e cartões de E/S encaixáveis, escolhidos conforme a necessidade	Aplicações que crescem, exigem cartões especiais ou troca a quente
I/O distribuído	Ao menos uma CPU, fonte e módulos de E/S separados, comunicando por rede de campo (<i>fieldbus</i>)	Instrumentação espalhada por área extensa, com economia de cabeamento

Na prática os arranjos se misturam: existem controladores com E/S embutida que também aceitam cartões modulares e ainda operam com E/S distribuída por *fieldbus*.

5.6 O que distingue o CLP de um computador comum

A estrutura é parecida, o propósito não é:

- **determinismo** — o ciclo se repete em tempo previsível; no computador comum, o escalonamento do sistema operacional não garante prazo;
- **fonte e entradas tratadas** — filtragem, estabilização e imunidade a ruído são requisito de projeto, não acessório;
- **invólucro e faixa de operação** — temperatura, vibração e umidade de ambiente industrial;
- **diagnóstico** — sinalização de estado por ponto e por módulo, pensada para quem mantém a máquina.

Atenção

A alimentação dos dispositivos de campo deve ser fornecida externamente. A fonte do CLP é dimensionada para a parte interna do equipamento, não para alimentar sensores, válvulas e sinalizadores. Tentar alimentar carga de campo pela fonte interna é uma das causas mais comuns de queima de fonte e de reset indevido.

Na prática

Abra o catálogo de um controlador disponível na sua escola e localize, para ele: (a) o arranjo de E/S (compacto, modular ou distribuído); (b) a tensão de alimentação da CPU e a das saídas digitais; (c) o tempo de varredura por mil instruções; (d) quais cartões de entrada analógica existem e com quantos bits de resolução. Relacione cada dado com uma linha da Tabela 5.1.

Exercícios

- a) Reescreva a definição da NEMA em linguagem de manutenção, em no máximo três linhas.
- b) Um equipamento lê entradas, executa lógica e aciona saídas, mas não resiste a vibração e calor de painel. Ele é um CLP? Justifique pela definição.
- c) Explique por que o CLP compacto costuma empregar microcontrolador em vez do conjunto processador, memória e interface.
- d) Em que situação o I/O distribuído é mais barato que o modular, e em que situação é mais caro?
- e) Um aluno afirma que “um CLP é um computador industrial”. Aponte duas diferenças que tornam essa afirmação tecnicamente imprecisa.

Capítulo 6

Componentes, memória e tipos de dados

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- descrever a função do microprocessador e dos blocos de memória;
- classificar memórias quanto ao acesso, à volatilidade e à forma de armazenamento;
- calcular a quantidade de bits de endereçamento de uma memória;
- reconhecer os tipos de dados elementares da norma e suas faixas de valor;
- aplicar o princípio da forte tipação e utilizar funções de conversão explícita de tipos;
- representar números negativos em complemento de dois;
- interpretar os campos dos formatos simples e duplo da IEEE-754;
- localizar as seis áreas do mapa de memória de um CLP.

6.1 O microprocessador e a memória

O microprocessador é a inteligência do controlador: coleta os dados dos cartões de entrada, processa segundo o programa do usuário, armazenado em memória, e envia o resultado aos cartões de saída. A memória é o que dá sentido a esse processo — guarda as instruções e os dados necessários para executá-las.

6.2 Classificação das memórias

Uma mesma memória pode ser descrita por quatro critérios independentes (Tabela 6.1).

Tabela 6.1: Critérios de classificação de memórias.

Critério	Categorias	Exemplo e consequência
Acesso	Sequencial: chega à posição passando pelas intermediárias. Aleatório ou direto: chega diretamente à posição endereçada	Fita magnética (sequencial) e semicondutores (direto). O <i>tempo de acesso</i> é o intervalo entre a entrada do endereço e a informação disponível na saída
Volatilidade	Volátil: perde o conteúdo sem alimentação. Não volátil: mantém o conteúdo	RAM é volátil; ROM, PROM e EPROM são não voláteis
Escrita e leitura	Escrita e leitura, ou somente leitura	RAM permite ambas; ROM permite apenas leitura
Forma de armazenamento	Estática: o dado permanece indefinidamente. Dinâmica: exige reinserção periódica	Estática em flip-flop; dinâmica em capacitor

6.3 Tecnologias de memória

Tabela 6.2: Tecnologias de memória presentes em controladores.

Tecnologia	O que é	Característica de uso
ROM de máscara	Somente leitura, gravada pelo fabricante	Não volátil, não permite apagamento
PROM	Programável uma única vez, gravada pelo usuário	Não volátil, não permite apagamento
EPROM	Programável e apagável por exposição a ultravioleta	Não volátil; exige programador dedicado, o que encarece a manutenção
EEPROM	Apagável e regravável eletricamente	Não volátil; escrita e leitura bem mais lentas que a RAM
EAROM	Regravável eletricamente	Não volátil e reprogramável, porém lenta
RAM estática	Acesso aleatório, com armazenamento em flip-flop	Volátil, rápida, ocupa mais espaço e custa mais
RAM dinâmica	Acesso aleatório, com armazenamento em capacitor	Volátil, mais lenta, ocupa menos espaço e custa menos
NOVRAM	RAM mantida por acumulador ou bateria	Mantém o conteúdo por meses sem alimentação, com velocidade de RAM e custo elevado
FLASH	Substituta da ROM e da EPROM	Não volátil, alta confiabilidade, menor consumo e gravação elétrica

6.4 Endereçamento, barramentos e cálculo de memória

Uma memória eletrônica tem três barramentos: **endereços**, **dados** e **controle**. O barramento de dados é bidirecional — é o que caracteriza uma memória de leitura e escrita. A capacidade é indicada pela notação $N \times m$, em que N é o número de localidades e m o número de bits por localidade: 256×8 , $1K \times 16$, $128M \times 32$.

Nessa notação, $K = 2^{10} = 1.024$ e $M = 2^{20} = 1.048.576$. A relação entre capacidade e bits de endereçamento é:

$$N_{\text{total}} = m \times 2^{n_{\text{end}}} \iff n_{\text{end}} = \log_2 \left(\frac{N_{\text{total}}}{m} \right) \quad (6.1)$$

Exemplo 1. Um dispositivo com 393.216 bits no total e palavras de 12 bits:

$$2^x = \frac{393.216}{12} = 32.768 \implies x = \log_2 32.768 = 15 \text{ bits de endereçamento}$$

Exemplo 2. Uma memória $128\text{M} \times 32$: como 128M é 128×2^{20} , o total é $128 \times 2^{20} \times 32 = 4.294.967.296$ bits e o número de palavras é 134.217.728, o que exige 27 bits de endereçamento.

Atenção

Um erro clássico é confundir o número de *palavras* com o número de *bits*. Uma memória $128\text{M} \times 32$ tem 128 milhões de palavras e 4 bilhões de bits. A faixa de endereços — e portanto os bits de endereçamento — depende do número de palavras, não do total de bits.

6.5 Tipos de dados da norma

Os tipos de dados especificam o valor que cada variável pode assumir e como esse valor é organizado na memória. A Tabela 6.3 reproduz a tabela de tipos de dados do item 4.5.1 do material de origem, com duas correções: o tipo **REAL** é de precisão *simples* (32 bits), e a faixa do **DINT** termina em 2.147.483.647, e não em 2.147.483.648.

Tabela 6.3: Tipos de dados do item 4.5.1 do material de origem, com a faixa expressa em potências de dois e o valor decimal correspondente.

Tipo de dado	Descrição	Bits	Faixa de valor
BOOL	Booleano: falso ou verdadeiro, 0 ou 1	1	0 ou 1
SINT	<i>Short integer</i> , números inteiros curtos	8	$-(2^7)$ a $(2^7 - 1)$: -128 a 127
INT	<i>Integer</i> , números inteiros	16	$-(2^{15})$ a $(2^{15} - 1)$: -32.768 a 32.767
DINT	<i>Double integer</i> , números inteiros duplos	32	$-(2^{31})$ a $(2^{31} - 1)$: -2.147.483.648 a 2.147.483.647
USINT, BYTE	<i>Unsigned short integer</i> , inteiros curtos sem sinal	8	0 a $(2^8 - 1)$: 0 a 255
UINT, WORD	<i>Unsigned integer</i> , inteiros sem sinal	16	0 a $(2^{16} - 1)$: 0 a 65.535
UDINT, DWORD	<i>Unsigned double integer</i> , inteiros duplos sem sinal	32	0 a $(2^{32} - 1)$: 0 a 4.294.967.295
REAL, FLOAT	<i>Real</i> , números reais de precisão simples	32	conforme a IEEE 754-2019: 7 dígitos decimais
LREAL, LFLOAT	<i>Long real</i> , números reais de precisão dupla	64	conforme a IEEE 754-2019: 15 dígitos decimais
TIME	Duração (em milissegundos)	32	0 a $(2^{32} - 1)$: 0 a 4.294.967.295 ms

Fonte: Tabela do item 4.5.1 de TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021.

Note a diferença entre *com sinal* e *sem sinal*: o tipo com sinal usa o bit mais significativo para representar o sinal, e por isso tem metade da faixa positiva, ganhando a mesma faixa em valores negativos.

6.5.1 Declaração de variáveis e o princípio da forte tipação

Na norma IEC 61131-3, todo dado manipulado pelo programa deve pertencer a uma variável expressamente declarada com seu respectivo tipo de dado. O modelo adota o princípio da **forte tipação** (*strong typing*). Diferentemente de linguagens de script ou de uso geral que permitem coerção implícita (*implicit type casting*), os compiladores de CLP exigem correspondência estrita de tipos em qualquer operação ou atribuição.

Em sistemas industriais, a forte tipação não é uma restrição burocrática, mas uma exigência crítica de **segurança operacional**:

- impede que truncamentos automáticos descartem frações decimais em malhas de controle;
- evita estouros de capacidade (*overflow*) silenciosos que poderiam inverter o sinal de um comando ou zerar contadores de segurança;
- impede que grandezas físicas incompatíveis (como somar uma contagem de peças em INT com um intervalo de tempo em TIME) sejam combinadas por engano do programador.

A declaração formal de variáveis na IEC 61131-3 ocorre em blocos delimitados por VAR e END_VAR, conforme ilustrado na Listagem 6.1.

```

1 VAR
2     bBombaLigada      : BOOL;    // Estado logico do atuador (TRUE ou FALSE)
3     iLeituraBruta     : INT;      // Contagem de 16 bits do cartao analogico
4     diTotalizador     : DINT;     // Contador acumulado de ciclos de producao
5     rPressaoBar        : REAL;     // Pressao calculada do processo em bar (IEEE
6     tTempoMistura      : TIME;     // Duracao da etapa de dosagem (ex.: T#30s)
7 END_VAR

```

Listing 6.1: Declaração de variáveis com tipos distintos na IEC 61131-3.

6.5.2 Incompatibilidade direta entre tipos: o que não compila

O conceito fundamental que todo estudante e projetista de automação deve dominar é: **não é possível operar, comparar ou atribuir variáveis de tipos diferentes diretamente**. Se o código tentar misturar variáveis de tipos distintos sem conversão explícita, o ambiente de desenvolvimento acusará **erro fatal de compilação** e se recusará a gerar o código executável para a CPU do controlador.

A Listagem 6.2 exemplifica quatro erros clássicos cometidos por iniciantes que são prontamente barrados pelo compilador.

```

1 // ERRO 1: Atribuicao direta de REAL para INT
2 iLeituraBruta := rPressaoBar;
3 // Rejeitado: tipos incompativeis. Nao e possivel atribuir REAL a INT.
4
5 // ERRO 2: Operacao aritmetica direta misturando REAL e INT
6 rPressaoBar := rPressaoBar + iLeituraBruta;
7 // Rejeitado: operador '+' exige que ambos os operandos sejam do mesmo tipo.
8
9 // ERRO 3: Atribuicao direta entre inteiros de tamanhos distintos (DINT para
10 // INT)
11 iLeituraBruta := diTotalizador;
12 // Rejeitado: atribuicao invalida (risco de perda de dados: 32 bits para 16
13 // bits).
14
15 // ERRO 4: Comparacao relacional direta entre grandezas de tipos distintos
16 IF tTempoMistura > iLeituraBruta THEN
17     bBombaLigada := FALSE;
18 END_IF;
19 // Rejeitado: operador '>' nao permite comparar TIME com INT diretamente.

```

Listing 6.2: Operações inválidas rejeitadas pelo compilador por incompatibilidade de tipo.

Se essas operações fossem permitidas silenciosamente, os efeitos práticos seriam desastrosos: no Erro 1, uma pressão de 12,7 bar perderia a parte fracionária sem controle do engenheiro; no Erro 3, se o acumulador ultrapassasse 32.767, o valor sofreria estouro de complemento de dois e viraria um número negativo no cartão de saída, provocando comportamentos imprevisíveis na planta.

6.5.3 Funções padronizadas de conversão explícita de tipos

Para que variáveis de tipos diferentes possam interagir de forma legítima e segura, a norma IEC 61131-3 estabelece um conjunto de **funções de conversão explícita de tipos** (*type conversion functions*). Toda conversão deve ser declarada expressamente pelo programador.

A sintaxe padronizada pela norma segue o formato:

$$\text{TIPO_ORIGEM_TO_TIPO_DESTINO(expressão)} \quad (6.2)$$

A Tabela 6.4 lista as principais funções de conversão utilizadas em projetos industriais e seus comportamentos.

Tabela 6.4: Principais funções de conversão explícita de tipos da IEC 61131-3.

Função	Origem	Destino	Comportamento e aplicação típica
INT_TO_REAL	INT	REAL	Converte inteiro para ponto flutuante exato ($200 \rightarrow 200.0$). Usada em cálculos de engenharia
REAL_TO_INT	REAL	INT	Converte real para inteiro com arredondamento matemático ($10.7 \rightarrow 11$; $10.2 \rightarrow 10$)
TRUNC	REAL	DINT/INT	Trunca a parte decimal sem arredondamento ($10.9 \rightarrow 10$; $-4.8 \rightarrow -4$)
DINT_TO_REAL	DINT	REAL	Converte inteiro de 32 bits para real, mantendo faixa para contagens elevadas
INT_TO_DINT	INT	DINT	Expansão segura de palavra de 16 para 32 bits, sem alteração de valor numérico
DINT_TO_INT	DINT	INT	Redução de 32 para 16 bits; exige cautela contra estouro acima de 32.767
BOOL_TO_INT	BOOL	INT	Mapeia FALSE para 0 e TRUE para 1; útil para somas lógicas
TIME_TO_DINT	TIME	DINT	Extraí a duração em milissegundos como valor numérico inteiro

Com o uso das funções adequadas, as operações antes rejeitadas tornam-se perfeitamente válidas, compiláveis e transparentes quanto à intenção do projeto, como detalhado na Listagem 6.3.

```

1 // Correcao 1: Conversao explicita de REAL para INT (com arredondamento)
2 iLeituraBruta := REAL_TO_INT(rPressaoBar);
3
4 // Correcao 2: Conversao explicita de INT para REAL antes da soma aritmetica
5 rPressaoBar := rPressaoBar + INT_TO_REAL(iLeituraBruta);
6
7 // Correcao 3: Conversao explicita de DINT para INT (assegurando faixa valida
8   )
9 iLeituraBruta := DINT_TO_INT(diTotalizador);
10
11 // Correcao 4: Conversao explicita para compatibilizar tipos na comparacao
12 IF TIME_TO_DINT(tTempoMistura) > INT_TO_DINT(iLeituraBruta) THEN
13   bBombaLigada := FALSE;
14 END_IF;
```

Listing 6.3: Uso correto de variáveis com tipos distintos via conversão explícita.

6.5.4 Exemplo prático: escalonamento de canal analógico

A aplicação mais recorrente de conversão de tipos em automação é o **escalonamento de canais analógicos**. Um módulo de entrada analógica converte a corrente de um transmissor

(por exemplo, 4 a 20 mA) em uma contagem digital bruta armazenada como variável INT (faixa de 0 a 27.648 contagens no padrão industrial Siemens, ou 0 a 32.767 em cartões unipolares de 15 bits).

No programa, o algoritmo de controle precisa atuar sobre a variável física real (por exemplo, pressão de 0,0 a 10,0 bar, representada em REAL).

A relação matemática de interpolação linear é:

$$P_{\text{bar}} = \frac{N_{\text{raw}} - N_{\text{min}}}{N_{\text{max}} - N_{\text{min}}} \times (P_{\text{max}} - P_{\text{min}}) + P_{\text{min}} \quad (6.3)$$

Se um programador desavisado tentar calcular a pressão diretamente com variáveis inteiras em Texto Estruturado:

```
rPressaoBar := ((iEntradaRaw - 0) / 27648) * 10.0; (ERRADO!)
```

ocorrem dois graves problemas:

1. a subexpressão inteira $(iEntradaRaw - 0) / 27648$ executa uma divisão inteira; como o numerador é sempre menor que 27.648, o resultado é truncado para **zero** em toda a faixa útil do transmissor;
2. a mistura de termos inteiros com o literal real 10.0 é imediatamente recusada pelo compilador conforme a regra da forte tipação.

A solução normativa correta converte o valor bruto para REAL antes de efetuar qualquer cálculo aritmético, como apresentado na Listagem 6.4.

```
1  VAR
2      iEntradaRaw      : INT;      // Leitura bruta do canal analogico (0 a 27648)
3      rPressaoMedida    : REAL;    // Pressao calculada do processo em bar (0.0 a
4          10.0 bar)
5      rComandoValvula   : REAL;    // Abertura desejada da valvula de controle
6          (0.0 a 100.0 %)
7      iSaidaAnalogica   : INT;     // Valor de comando enviado ao cartao de saida
8          (0 a 27648)
9  END_VAR
10
11 // 1. Escalonamento da entrada: conversao explicita de INT para REAL
12 rPressaoMedida := (INT_TO_REAL(iEntradaRaw) / 27648.0) * 10.0;
13
14 // 2. Escalonamento inverso da saida: calculo em REAL e conversao para INT
15 iSaidaAnalogica := REAL_TO_INT((rComandoValvula / 100.0) * 27648.0);
```

Listing 6.4: Escalonamento analógico em Texto Estruturado com conversão explícita.

Atenção

Atenção às conversões estreitantes (*narrowing conversions*): converter um tipo de maior capacidade ou precisão para um menor (como DINT_TO_INT ou REAL_TO_INT) exige cuidado rigoroso do desenvolvedor. Se o valor em ponto flutuante for maior do que 32.767, a função REAL_TO_INT satura no valor máximo ou causa estouro de representação. Sempre valide os limites da grandeza antes de invocar conversões redutoras.

6.6 Números negativos: complemento de um e de dois

Para representar negativos de forma que a subtração se reduza a uma soma, usam-se representações por complemento. A representação em sinal e magnitude tem *duas* representações para o zero e exige dois circuitos distintos (um para soma e outro para subtração), o que a torna inadequada.

Complemento de um. Invertem-se todos os bits. Com 3 bits, o número 3 é 011 e o seu complemento de um é 100. O problema é que o zero continua tendo dupla representação (000 e 111) — a redundância não é eliminada.

Complemento de dois. Invertem-se todos os bits e soma-se uma unidade. Com 3 bits, o número 2 é 010; invertendo, 101; somando 1, chega-se a 110, que é -2 .

Com o complemento de dois, uma palavra de n bits representa:

$$-2^{n-1} \text{ a } +2^{n-1} - 1 \quad (6.4)$$

Por isso o INT, de 16 bits, vai de -32.768 (-2^{15}) a 32.767 ($2^{15} - 1$).

6.7 Formatos IEEE-754

A norma IEEE-754, publicada em 1985, unificou a representação de números reais em ponto flutuante. O formato tem três campos: o bit de **sinal**, um conjunto de bits de **expoente** — cujo tamanho determina a faixa representável — e a **mantissa**, que determina a precisão.

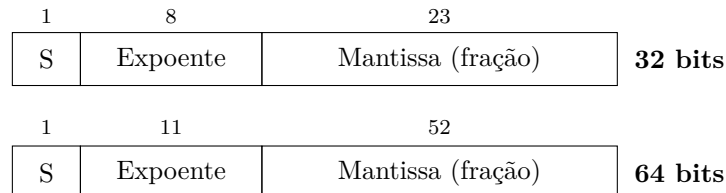


Figura 6.1: Campos dos formatos simples (32 bits) e duplo (64 bits) da IEEE-754.

Além do zero, o formato prevê valores especiais: infinito e *NaN* (*not a number*), usado para resultados indefinidos como $0/0$. Dois eventos precisam ser previstos no programa:

- **overflow** — o expoente é grande demais para caber no campo;
- **underflow** — o valor é tão pequeno (ou o expoente negativo tão grande) que não é representável.

Na prática, é por isso que a comparação de valores **REAL** exige tolerância: nunca se testa “igual a zero”, mas “menor que um valor mínimo”.

Atenção

Cálculos com **REAL** podem acumular erro e nunca fechar exatamente no valor esperado. Em lógicas que dependem de comparação de reais — nível, vazão, posição — use sempre uma faixa de tolerância definida no projeto, e registre essa escolha na documentação do programa.

6.8 Dados estruturados, textos e arranjos

Além dos tipos elementares existem dados que agrupam outras informações:

Tipos estruturados. Várias variáveis de tipos possivelmente diferentes agrupadas em um único nome. O exemplo mais presente na prática é o bloco de temporização: uma variável TON disponibiliza internamente

- `Timer_01.PRE` — valor de preset, que é do tipo DINT;
- `Timer_01.ACC` — tempo acumulado, DINT;
- `Timer_01.EN` — condição de funcionamento, BOOL;
- `Timer_01.DN` — fim da contagem, BOOL.

Entender essa estrutura é o que permite, por exemplo, exibir o tempo acumulado na supervisão.

Tipos customizados. Blocos de instrução definidos pelo usuário, com as variáveis escolhidas por ele — por exemplo, um bloco VALVULA_ON_OFF com os parâmetros LIGA, DESLIGA e STATUS, todos BOOL.

Texto. O tipo *string* é uma sequência de caracteres. Em muitos controladores ocupa 85 bytes, com a estrutura da Tabela 6.5.

Tabela 6.5: Organização típica de uma *string* de 85 bytes.

Bytes	Conteúdo
0–1	<i>offset</i> e comprimento máximo (0 a 80 bytes)
2–3	Comprimento atual
4–83	Caracteres que formam a palavra ou a frase
84	Terminador

Arranjos. Um *array* armazena variáveis de mesmo tipo, em vetor ou matriz. Por exemplo, `ARR_INT[0..99]` OF INT cria 100 posições de inteiros; o acesso a cada posição é feito pelo índice: `ARR_INT[0]` é a primeira, `ARR_INT[1]` a segunda, e assim por diante.

6.9 Arquitetura e mapa de memória do CLP

As memórias de um controlador não têm todas a mesma função. A memória de *programa* guarda um conteúdo fixo, e a memória de *dados* guarda o que varia durante a execução. Na prática, o mapa de memória se divide em seis áreas: A Figura 6.2 organiza essas áreas no conjunto.

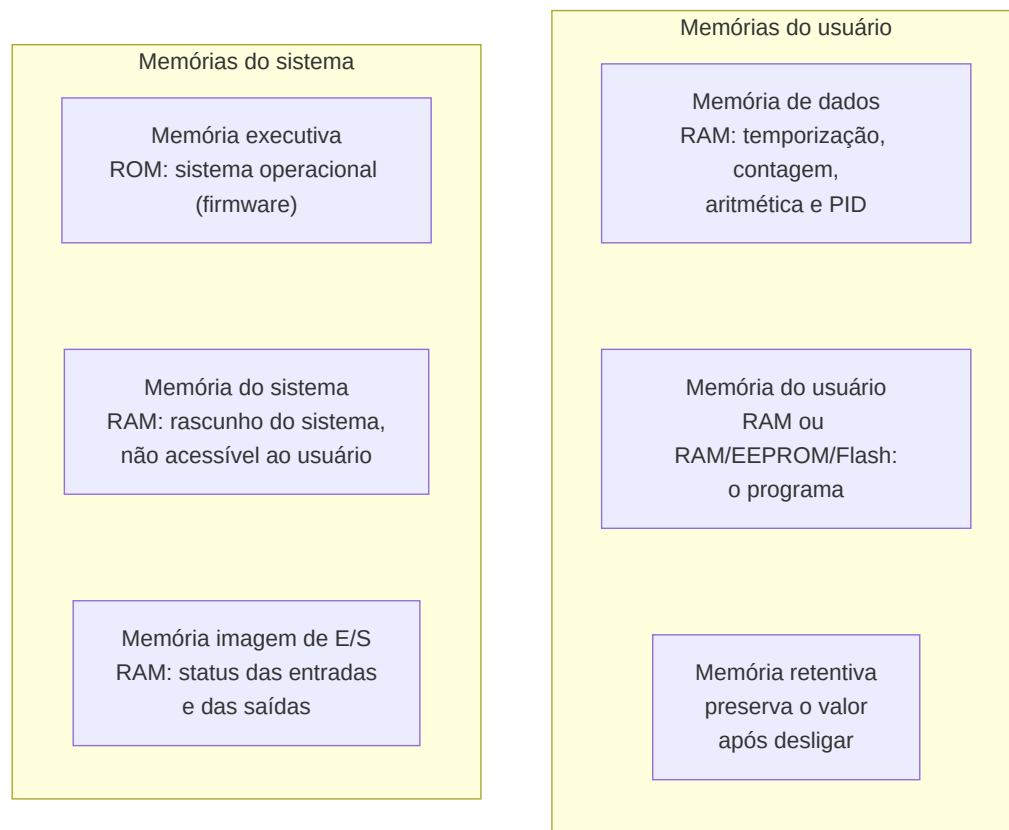


Figura 6.2: Áreas do mapa de memória de um controlador programável.

Tabela 6.6: As seis áreas do mapa de memória do CLP.

Área	Tecnologia	O que guarda e quem acessa
Executiva	ROM ou EPROM	Sistema operacional (<i>firmware</i>) que gerencia o hardware; somente leitura para o usuário
Do sistema	RAM	Resultados e operações intermediárias do próprio sistema; é um rascunho, não acessível ao usuário
Imagem de E/S	RAM	Status das entradas (gravado após a leitura) e status das saídas (gravado após o processamento)
De dados	RAM	Valores de temporização, contagem, aritmética, PID e manipulação de dados
Do usuário	RAM ou RAM/E-EPROM/-Flash	O programa do usuário, lido pela CPU instrução por instrução
Retentiva	RAM com bateria ou área dedicada	Preserva o valor após o desligamento; a variável precisa ser declarada como retentiva

A **memória retentiva** merece atenção: em alguns controladores existe uma área exclusiva, e a marca de retenção é feita na declaração da variável; em outros, não há divisão e toda

a RAM é mantida por bateria. O sintoma de bateria fraca é o comportamento que “muda sozinho” após uma falta de energia — assunto do Capítulo 24.

Na prática

Em um controlador com programador disponível, crie quatro variáveis: um **BOOL**, um **INT** no limite superior (32.767), um **DINT** e uma **STRING**. Depois: (a) some 1 ao **INT** de valor máximo e observe o que acontece; (b) inverta um número negativo para positivo e escreva o resultado; (c) veja no *software* de programação em que área de memória cada variável foi alocada. O resultado da letra (a) é uma das causas mais frequentes de defeito intermitente em programas reais.

Exercícios

- Classifique uma memória *flash* pelos quatro critérios da Tabela 6.1.
- Quantos bits de endereçamento exige uma memória de 65.536 bits com palavras de 8 bits?
- Explique, sem usar fórmula, por que o **UINT** vai de 0 a 65.535 e o **INT** de -32.768 a 32.767.
- Represente -5 em complemento de dois com 8 bits, mostrando os dois passos.
- No formato simples da IEEE-754, quantos dígitos significativos se pode esperar de um **REAL**? E no duplo? Relacione com a largura da mantissa (Figura 6.1).
- Por que uma comparação de igualdade entre duas variáveis **REAL** é uma má prática?
- Explique o princípio da forte tipação adotado pela norma IEC 61131-3. Por que a linha `rVazao := rVazao + iAjuste;` (sendo `rVazao` do tipo **REAL** e `iAjuste` do tipo **INT**) causa erro de compilação, e como ela deve ser escrita para ser executada pelo CLP?
- Um programa usa `Timer_01.ACC` para calcular a taxa de produção. Em qual área do mapa de memória (Tabela 6.6) esse valor está armazenado?
- Um técnico relata que a contagem de peças do turno anterior se perdeu depois de uma falta de energia. Em que área de memória a variável provavelmente estava alocada, e como corrigir o programa?

Capítulo 7

Cartões de entrada e saída, resolução e instalação

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- explicar o que um cartão de entrada e saída faz com o sinal;
- reconhecer os tipos de entrada e de saída digital e escolher o adequado ao campo;
- ligar entradas analógicas em corrente e em tensão, dimensionando o *shunt*;
- calcular a resolução de um canal e definir os bits necessários do cartão;
- diferenciar troca a frio de troca a quente.

7.1 O que o cartão faz com o sinal

Os cartões de entrada e saída são os circuitos que tratam eletricamente os sinais para que possam ser processados pela CPU. Na entrada, convertem um sinal elétrico de campo em dado binário; na saída, convertem o dado binário da CPU em sinal elétrico capaz de acionar um atuador.

Há uma razão elétrica para que esse tratamento exista: em ambiente industrial, os sinais de E/S carregam ruído elétrico, e ruído que alcança os circuitos da CPU provoca operação falsa. A estrutura de E/S isola e filtra o sinal para entregar à CPU uma informação confiável.

7.2 Caminho do sinal em um cartão de entrada

Os elementos de um cartão de entrada digital, na ordem em que o sinal os atravessa: A Figura 7.1 mostra essa sequência de tratamento.

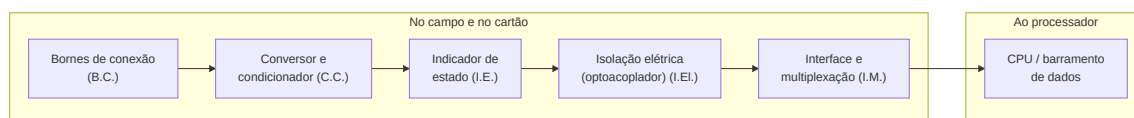


Figura 7.1: Elementos de tratamento do sinal em um cartão de entrada digital.

B.C. — **bornes de conexão:** interligam o sensor ao cartão, em geral com sistema *plug-in*.

C.C. — **conversor e condicionador:** converte em contínua o sinal alternado e rebaixa a tensão a valores compatíveis com o restante do circuito.

I.E. — **indicador de estado:** dá indicação visual do estado de cada ponto.

I.El. — **isolação elétrica:** separa eletricamente o campo do processador.

I.M. — **interface e multiplexação:** informa ao processador o estado de cada variável.

O cartão de saída percorre o caminho inverso: A Figura 7.2 percorre o caminho inverso, da CPU ao atuador.

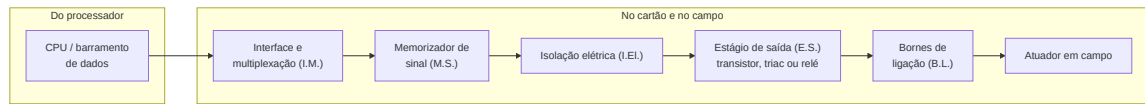


Figura 7.2: Elementos de um cartão de saída digital, da CPU ao atuador.

O **M.S. (memorizador de sinal)** guarda o estado multiplexado, a **I.El.** isola eletricamente o processador do campo e o **E.S. (estágio de saída)** transforma sinais lógicos de baixa potência em sinais capazes de operar o atuador.

7.3 Entrada digital

As entradas digitais aparecem nas formas abaixo, conforme a especificação:

- contato seco;
- 24 V CC;
- TTL;
- 110 V CA e 220 V CA.

O que separa os dois casos de alimentação é o que o circuito precisa fazer com o sinal antes de isolar: em corrente contínua ele já chega na forma que o optoacoplador usa; em corrente alternada, precisa primeiro ser retificado. As figuras 7.3 e 7.4 mostram os dois circuitos lado a lado.

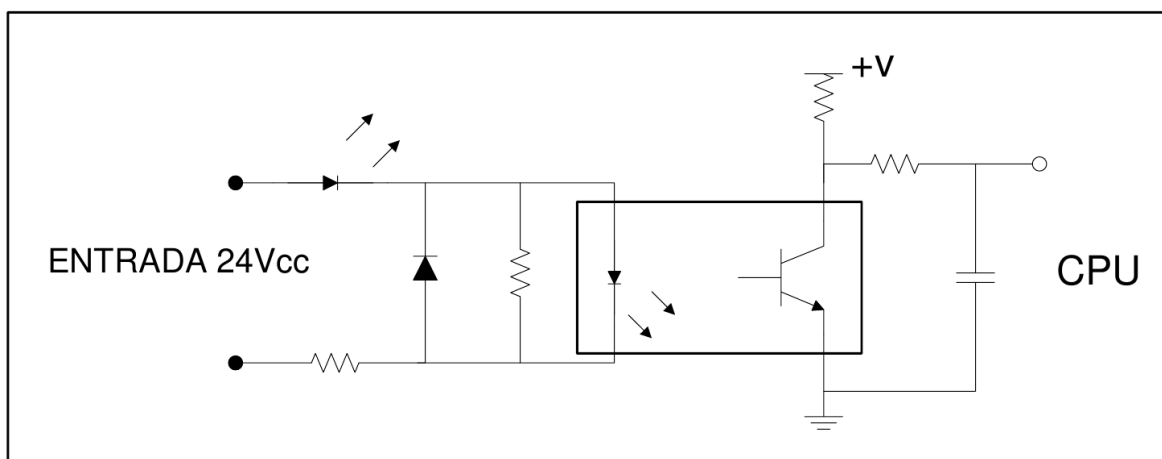


Figura 7.3: Cartão de entrada digital com alimentação em corrente contínua: os dois bornes de campo recebem 24 V CC, e o diodo em série com o resistor define o sentido da corrente pelo LED do optoacoplador.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, figura 8.

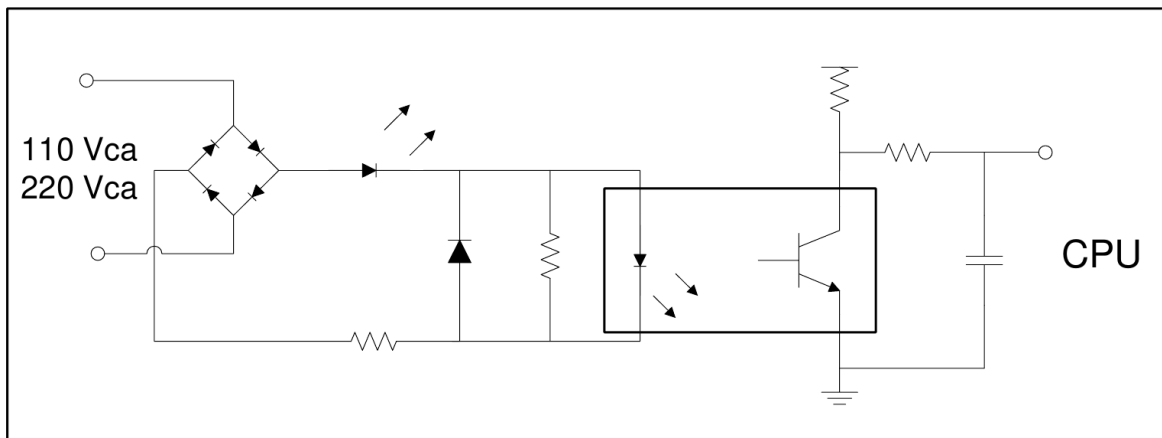


Figura 7.4: Cartão de entrada digital com alimentação em corrente alternada: a ponte retificadora converte os 110 V CA ou 220 V CA em corrente contínua antes do optoacoplador.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, figura 9.

A entrada digital com **fonte externa** é a mais utilizada: as chaves que acionam a entrada ficam no campo e a fonte de alimentação é externa ao CLP. O circuito interno é isolado por *optoacoplador* — um LED do lado de campo e um fototransistor do lado interno, sem ligação elétrica entre os dois. A Figura 7.5 detalha esse circuito.

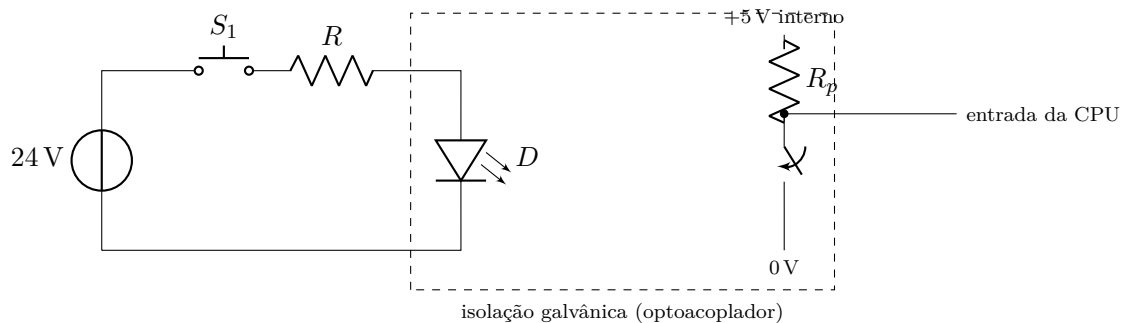


Figura 7.5: Entrada digital com fonte externa e isolamento por optoacoplador.

Com a botoeira aberta, não circula corrente pelo LED e o fototransistor permanece aberto: o resistor de *pull-up* leva nível alto à entrada da CPU. Com a botoeira fechada, o LED acende, o fototransistor satura e leva a entrada a nível baixo. É essa inversão — contato fechado, nível baixo — que o programa precisa levar em conta.

Sensores a três fios: NPN e PNP. Sensores capacitivos, indutivos ou ópticos com saída a transistor e alimentação de 8 a 30 V CC ligam-se diretamente à entrada digital, desde que o cartão seja de 24 V CC e o comum seja escolhido conforme o tipo de sensor:

Tabela 7.1: Saída de sensor a três fios e o comum exigido na entrada.

Tipo	Como aciona a saída	Implicação na ligação
NPN (<i>sinking</i>)	Chaveia o negativo: a saída puxa o sinal para o comum	Entrada com comum positivo
PNP (<i>sourcing</i>)	Chaveia o positivo: a saída entrega tensão ao sinal	Entrada com comum negativo

7.3.1 Lógica positiva e negativa: *sinking* e *sourcing*

Nem toda entrada digital aceita qualquer sensor. O que decide a compatibilidade é o potencial em que está o **terminal comum (COM)** do cartão, e desse detalhe resultam dois arranjos:

- **entrada *sinking* (lógica positiva)** — o comum é ligado ao polo negativo (0 V) e o módulo “drena” a corrente para o negativo. A chave de campo leva o polo positivo (24 V) ao borne de entrada. É o arranjo que aceita sensores **PNP**, que entregam tensão positiva na saída quando acionados;
- **entrada *sourcing* (lógica negativa)** — o comum é ligado ao polo positivo (24 V) e o módulo fornece a corrente. A chave de campo leva o polo negativo ao borne de entrada. É o arranjo que aceita sensores **NPN**, que chaveiam o negativo.

A mesma nomenclatura vale para a saída, e é aí que a confusão costuma aparecer: inverter o comum do módulo inverte também o ponto em que a carga é ligada.

- **saída *sinking* (lógica negativa)** — a saída chaveia o polo negativo. O terminal $+V$ recebe o positivo; um lado da carga vai ao positivo e o outro ao borne de saída;
- **saída *sourcing* (lógica positiva)** — a saída chaveia o polo positivo. O comum recebe o positivo; um lado da carga vai direto ao borne de saída e o outro ao negativo.

Vale registrar o duplo sentido do termo: o sensor NPN também é chamado de *sinking*, porque drena a corrente, e o PNP de *sourcing*. Aqui a palavra qualifica o **módulo**, e por isso a associação com a Tabela 7.1 é cruzada — o sensor *sinking* (NPN) pede a entrada *sourcing*. A Figura 7.6 reúne as quatro combinações de entrada e de saída em tensão contínua.

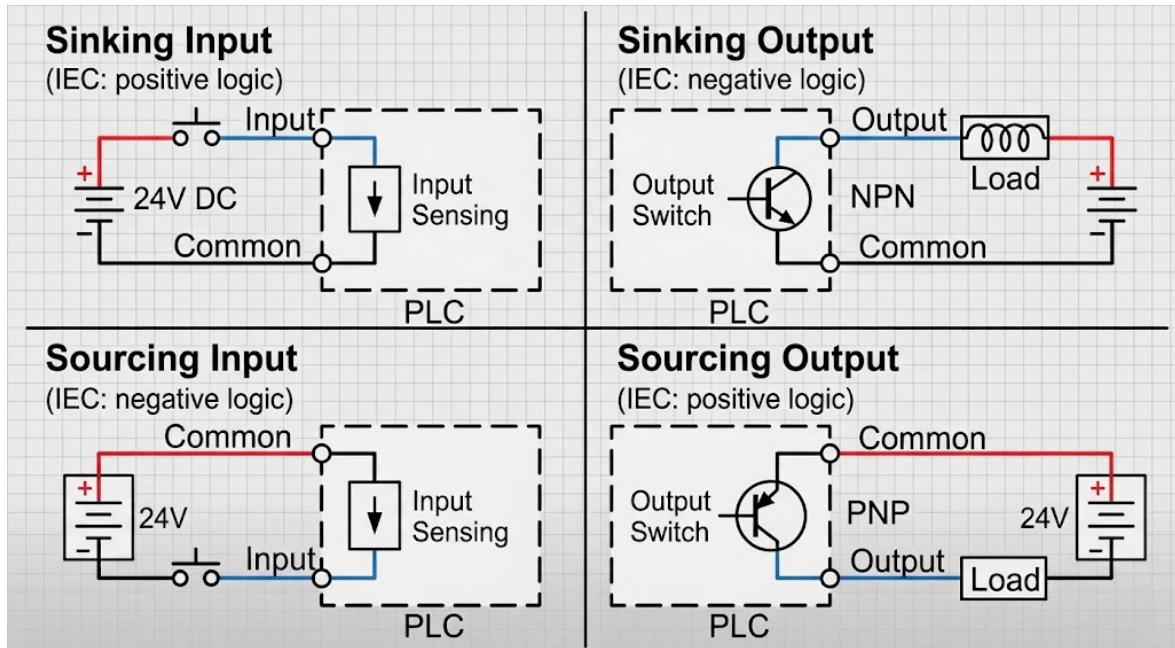


Figura 7.6: Ligações de entrada e de saída digitais em tensão contínua nas configurações *sinking* e *sourcing*, com a lógica positiva ou negativa correspondente.

Fonte: Imagem obtida na internet e reeditada com apoio de ferramenta de IA generativa (Nano Banana).

Sensor a dois fios em CA. As entradas de CLP têm alta impedância e por isso não são acionadas diretamente por um triac, como ocorre com sensores a dois fios em corrente alternada. Se o sensor é desse tipo, é necessário acrescentar uma derivação resistivo-capacitiva em paralelo com a entrada, para manter a corrente de manutenção do tiristor. Os valores constam no manual do controlador — e não devem ser estimados em campo.

Atenção

A entrada digital do tipo contato seco fica limitada a dispositivos que abrem ou fecham um contato. Antes de ligar um sensor à entrada do CLP, confirme no manual: tensão, tipo de comum e corrente máxima. Ligar um sensor PNP em entrada de comum positivo não queima o CLP por acaso — e o defeito aparece como “entrada que nunca atua”.

7.4 Entrada analógica

As entradas analógicas trabalham em corrente ou em tensão:

- **corrente:** 4 a 20 mA, 0 a 10 mA, 0 a 50 mA;
- **tensão:** 1 a 5 V CC, 0 a 10 V CC, −10 a 10 V CC.

A entrada em corrente é implementada diretamente no transmissor, o que é uma vantagem: a corrente não se altera ao longo do cabo, enquanto a tensão cai com a distância. Na entrada em tensão é preciso um resistor de *shunt* para converter corrente em tensão. O valor depende da faixa do transmissor e da entrada; na indústria é comum 250 Ω para sinais de 4 a 20 mA, o que produz 1 a 5 V (pela lei de Ohm, $V = R \cdot I$).

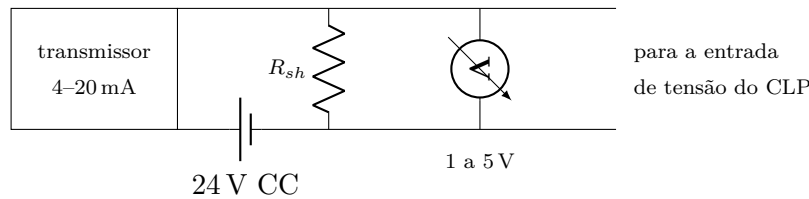


Figura 7.7: Conversão de corrente em tensão com resistor de *shunt* de $250\ \Omega$, em laço alimentado por 24 V CC.

No circuito da Figura 7.7, a fonte de 24 V CC fecha o laço: no transmissor a dois fios, **o mesmo par de condutores leva a alimentação e devolve o sinal**. A tensão da fonte precisa cobrir a queda no próprio transmissor, a queda no *shunt* — 5 V quando a corrente é de 20 mA — e a queda no cabo; o pior caso é justamente o de corrente máxima, porque é nele que a queda no *shunt* é maior.

Para medir temperatura, a entrada analógica precisa aceitar RTD (PT-100) ou termopar em milivolts — são cartões específicos, com compensação de junta fria no caso do termopar. Um cartão de 4 a 20 mA não mede termopar.

7.5 Saída digital

Quanto ao elemento de comutação, a saída digital pode ser de quatro tipos:

Tabela 7.2: Tipos de saída digital e critério de escolha.

Tipo	Natureza da carga	Comutação e vida útil	Limite prático
Transistor	Somente tensão contínua	Muito rápida; vida longa	Tensão e corrente inferiores às do contato seco
Triac	Somente tensão alternada	Rápida; vida útil maior que o contato seco	Não serve para contínuo
Contato (relé)	Tensão contínua ou alternada	Lenta; vida útil inferior aos componentes de estado sólido	Desgaste mecânico do contato
TTL	Cargas de sinal, em placa	Rápida	Baixa corrente; uso restrito

A Figura 7.8 mostra essa ligação.

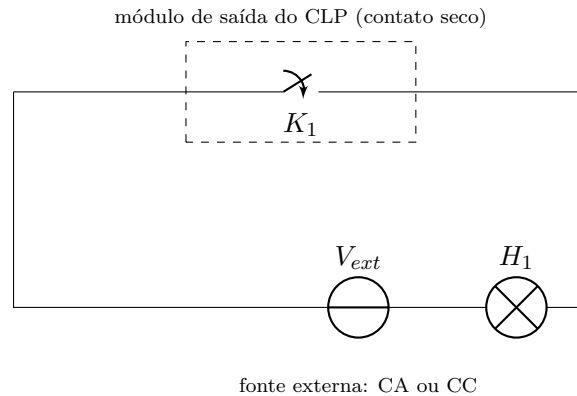


Figura 7.8: Saída digital a contato seco: o contato pertence ao módulo de saída do CLP e entra em série com a fonte externa e com a carga.

Note o que é interno e o que é externo. O contato e a bobina que o aciona fazem parte do módulo de saída: a bobina é energizada pela lógica do próprio controlador, e o que se leva a campo são os dois terminais do contato. O circuito de campo, por isso, é um laço simples — fonte externa, contato e carga em série — e a fonte pode ser contínua ou alternada, que é a vantagem do contato seco sobre a saída a transistor e a saída a triac (Tabela 7.2). O contato ainda isola eletricamente o circuito de campo da eletrônica do módulo.

Independentes ou com ponto comum. As saídas digitais podem ser *independentes*, quando cada ponto aciona carga de fonte distinta sem risco de interligá-las, ao custo de mais cabos; ou *com ponto comum*, que simplifica o painel, mas exige fonte comum. Para acionar cargas incompatíveis entre si em um cartão com ponto comum, usa-se relé intermediário *externo*: a bobina desse relé é acionada pela saída do CLP e o contato dele comanda a carga.

7.5.1 Resumo das ligações de entrada e de saída

A Figura 7.9 reúne, em um mesmo quadro, os esquemas de ligação elétrica para módulos de entrada e de saída digitais de um CLP, divididos em corrente contínua (CC), corrente alternada (CA) e saída a relé. São três arranjos de entrada — CC *sinking*, CC *sourcing* e CA sem polaridade — e três de saída — CC *sinking*, CC *sourcing* e contato de relé. A figura serve de consulta rápida: cada arranjo tem um potencial de referência diferente no terminal comum, e é esse potencial que decide o que pode ser ligado ao ponto.

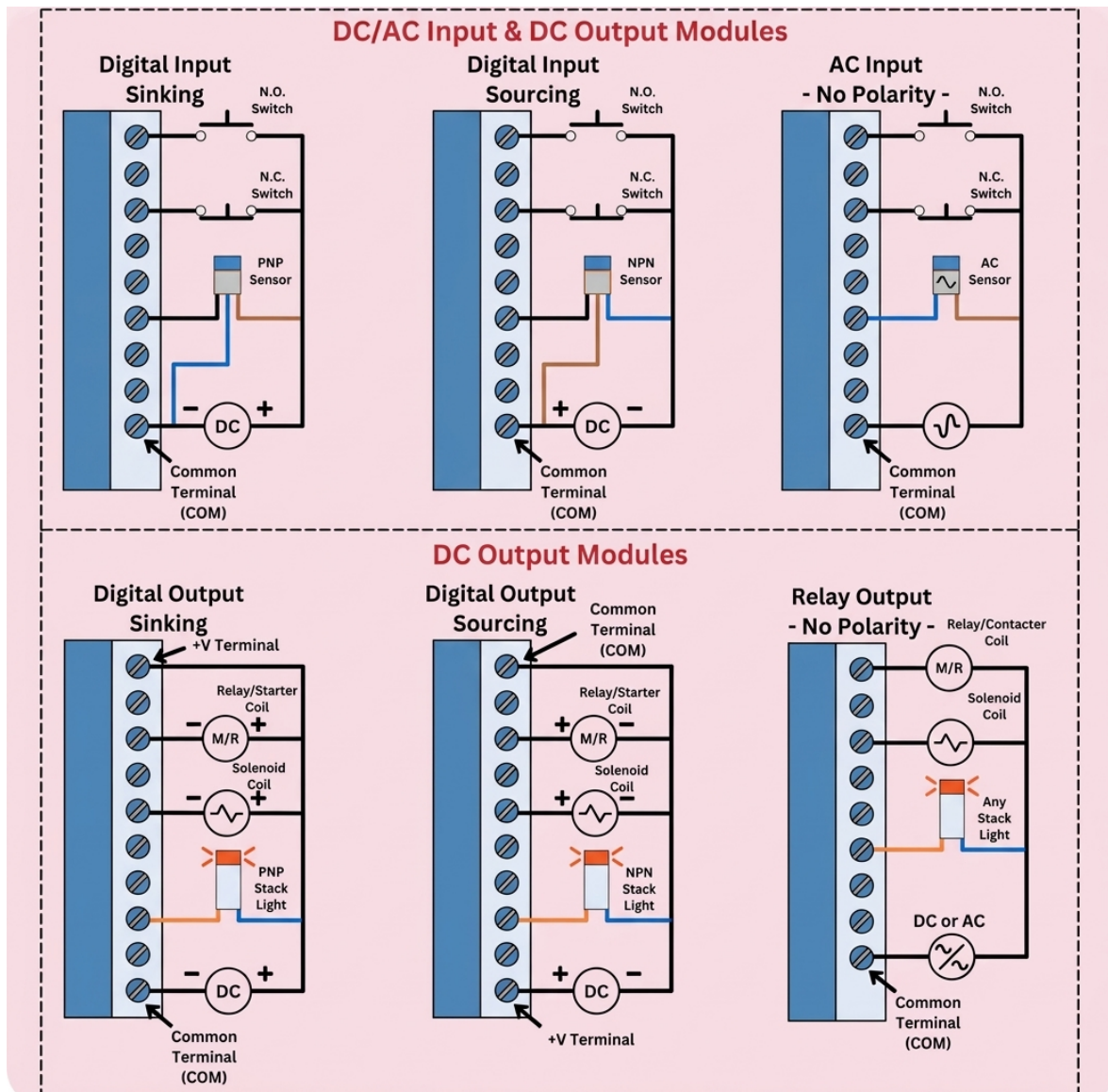


Figura 7.9: Esquemas de ligação de módulos de entrada (CC e CA) e de saída (CC e a relé) de um CLP.

Fonte: Imagem obtida na internet e reeditada com apoio de ferramenta de IA generativa (Nano Banana).

Entrada digital *sinking*. **Conceito:** o terminal comum (COM) do módulo é ligado ao polo negativo (CC -); o módulo atua drenando a corrente para o negativo. **Ligações:** as chaves NA e NF chaveiam o polo positivo (CC +) para os terminais de entrada, e sensores do tipo PNP são os usados nessa configuração, porque fornecem sinal positivo (CC +) na saída quando acionados.

Entrada digital *sourcing*. **Conceito:** o terminal comum (COM) é ligado ao polo positivo (CC +); o módulo fornece a corrente aos dispositivos externos. **Ligações:** as chaves NA e NF chaveiam o polo negativo (CC -) para os terminais de entrada, e sensores do tipo NPN são os usados nessa configuração, porque chaveiam o sinal negativo (CC -) na saída quando acionados.

Entrada CA — sem polaridade. Conceito: usada para sinais em corrente alternada (CA), em que não há polaridade positiva ou negativa fixa. **Ligações:** o terminal comum (COM) é ligado a um dos lados da fonte CA, em geral o neutro ou uma fase; chaves e sensores CA chaveiam o outro lado da fonte (a fase) para os terminais de entrada.

Saída digital *sinking*. Conceito: as saídas do CLP chaveiam o polo negativo (CC –); o terminal interno de saída conecta a carga ao negativo quando ativado. **Ligações:** o terminal +V, no topo do conector, é ligado ao polo positivo (CC +) para alimentar o barramento interno, e o terminal COM, na base, é conectado ao polo negativo (CC –). Um dos lados das cargas — bobina de relé ou de contator, solenoide, torre de sinalização — é ligado ao positivo (CC +) e o outro lado vai diretamente ao terminal de saída do CLP.

Saída digital *sourcing*. Conceito: as saídas do CLP chaveiam o polo positivo (CC +); quando a saída ativa, ela fornece tensão positiva à carga. **Ligações:** o terminal COM, no topo, é conectado ao positivo (CC +), e o terminal +V, na base, é conectado ao negativo (CC –). Um dos lados das cargas é ligado diretamente ao terminal de saída do CLP, onde recebe o positivo, e o outro lado é conectado ao negativo (CC –).

Saída a relé — sem polaridade. Conceito: as saídas usam contatos secos de relés eletromecânicos, o que permite acionar cargas em CC ou em CA com isolamento elétrica entre o campo e a eletrônica do módulo. **Ligações:** o terminal COM é ligado a uma das linhas da alimentação (CC ou CA) e cada carga — bobina de relé, solenoide, sinalizador — é conectada entre o terminal de saída correspondente e o outro lado da fonte de alimentação.

Atenção

O mesmo ponto físico do cartão muda de comportamento conforme o potencial do comum. Por isso a consulta à Figura 7.9 não substitui o manual do módulo: a designação *sinking/sourcing* não é uniforme entre fabricantes, e alguns adotam o nome pelo lado do dispositivo de campo, não pelo lado do módulo. Antes de energizar, confirme no manual o potencial do COM e o tipo de sensor que o fabricante declara compatível.

7.6 Cálculo de resolução

A resolução é o menor degrau que o cartão consegue distinguir, e depende do número de bits do conversor A/D (ou D/A) e da faixa configurada.

Um cartão de 12 bits tem $2^{12} = 4.096$ possibilidades de nível lógico, o que corresponde a 0 a 4.095 na contagem. Com sinal de 4 a 20 mA, faixa de engenharia de 0 a 4.000 PSI e indicação em porcentagem:

$$\frac{20 - 4}{4.095 - 0} = \frac{16}{4.095} = 0,00391 \text{ mA por nível}$$

$$\frac{4.000 - 0}{4.095 - 0} = 0,977 \text{ unidade de engenharia por nível}$$

$$\frac{100 - 0}{4.095 - 0} = 0,0244 \% \text{ por nível}$$

Ou seja: cada incremento de 1 no nível lógico corresponde a 0,00391 mA de sinal, o que na faixa do instrumento vale 0,977 PSI.

7.6.1 Definindo os bits do cartão antes de comprar

A resolução necessária decorre da aplicação, e não do catálogo. Em um processo com medição de vazão de 0 a 500 m³/h, sinal de 4 a 20 mA, cartão também de 4 a 20 mA, deseja-se leitura melhor que 0,02 m³/h. Então:

$$2^n - 1 = \frac{500 - 0}{0,02} = 25.000 \implies 2^n = 25.001$$

$$n = \frac{\log 25.001}{\log 2} \cong 14,61 \implies n = 15 \text{ bits}$$

A conclusão é objetiva: um cartão de 15 bits atende; um de 12 bits não. Como o preço do cartão cresce com a resolução, esse cálculo precisa ser feito *antes* da compra — e é uma das informações que alimentam a especificação do Capítulo 8.

Tabela 7.3: Efeito da resolução do cartão na resolução obtida (faixa de 4 a 20 mA).

Bits de resolução	Níveis (0 a $2^n - 1$)	mA por nível	% por nível
12 bits	4.095	0,0039	0,024
12 bits (0 a 20 mA)	4.095	0,0049	0,031

Atenção

Note a segunda linha da Tabela 7.3: a *mesma* resolução de 12 bits apresenta resolução pior em mA quando a faixa é 0 a 20 mA, porque o mesmo número de níveis cobre uma faixa maior. Resolução em bits não se compara entre cartões de faixas diferentes.

7.7 Instalação e troca de cartões

Um cartão pode precisar de manutenção ou troca por queima ou mau funcionamento. Há dois procedimentos:

- **troca a frio** — o barramento onde está o cartão é desenergizado, evitando dano à eletrônica do cartão e do barramento;
- **troca a quente** (*hot swap*) — a troca é feita com o sistema energizado, sem interromper o funcionamento dos cartões vizinhos.

A troca a quente não é um recurso universal: precisa estar prevista na especificação dos componentes. Se o cartão não tem essa característica, trocá-lo energizado danifica o cartão e o rack.

Atenção

Troca a quente é um recurso de projeto, não uma habilidade do mantenedor. Antes de retirar qualquer cartão com o sistema energizado, confirme no manual do equipamento: (a) que o módulo suporta *hot swap*; (b) qual o procedimento de remoção; (c) o efeito da retirada sobre os pontos vizinhos. Na dúvida, desenergize.

Na prática

Baixe o manual de um cartão de entrada digital de 24 V CC e de um cartão de saída a relé do mesmo fabricante e monte uma tabela comparativa com: (a) tensão de operação e faixa de corrente por ponto; (b) tipo de isolamento e tensão de isolamento; (c) tempo de resposta (*on* e *off*); (d) tipo de comum e quantidade de pontos. Depois responda: por que o tempo de resposta do cartão de relé é maior que o do cartão de entrada?

Exercícios

- a) Explique a função do optoacoplador e o que ocorreria se ele fosse removido do cartão.
- b) Um sensor indutivo PNP deve ser ligado a um cartão de entrada. Que comum deve ser usado, e por quê?
- c) Calcule a resolução em mA por nível de um cartão de 16 bits na faixa de 4 a 20 mA.
- d) Uma balança envia 4 a 20 mA para uma faixa de 0 a 6.000 kg. O processo exige leitura de 0,5 kg. Quantos bits o cartão precisa ter? Mostre o cálculo.
- e) Por que a entrada analógica em corrente é preferida em relação à entrada em tensão, quando o instrumento fica longe do painel?
- f) Uma saída digital precisa acionar uma bobina de válvula em 220 V CA e outra em 12 V CC. Que tipo de saída usar para cada uma, e por quê?
- g) Explique a diferença entre troca a frio e troca a quente, e o requisito de projeto que a segunda impõe.

Capítulo 8

Porte, redundância e especificação para compra

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- identificar os quatro critérios que definem o porte de um controlador;
- classificar um controlador pela quantidade de pontos de E/S;
- diferenciar redundância por hardware de redundância por software;
- conduzir um procedimento de especificação, com folga e critério de desempenho;
- avaliar fornecedor, produto e contrato antes de fechar a compra.

8.1 O que define o porte

Os controladores são classificados em micro, pequeno, médio e grande porte. O porte não é definido por uma única característica, mas por quatro, diretamente proporcionais entre si:

1. **velocidade de processamento;**
2. **memória;**
3. **quantidade de pontos de E/S;**
4. **capacidade de redundância.**

Cada fabricante define seus modelos dentro dessas faixas, e é o tipo de processo que determina qual porte serve. Controladores de micro e pequeno porte atendem automação predial, residencial e pequenos processos industriais, com menor custo e menos recursos. Os de médio e grande porte existem para processos complexos, com muitos pontos de E/S e requisitos de disponibilidade.

8.2 Classificação por quantidade de pontos

Uma classificação corrente de mercado (de origem americana) usa a quantidade de pontos de E/S:

Tabela 8.1: Classificação por quantidade de pontos de E/S.

Porte	Pontos de E/S
Micro	até 64
Pequeno	até 256
Médio	até 1.024
Grande	até 4.096
Muito grande	acima de 4.096

Atenção

Essa tabela é uma convenção de mercado, não uma norma. Como cada fabricante agrupa seus modelos de forma própria, a quantidade de pontos serve para *orientar* a conversa com o fornecedor — nunca para escolher o equipamento. Quem define a especificação é a lista de I/O mais os requisitos de desempenho e de disponibilidade.

8.3 Velocidade de processamento

É a capacidade de processar certa quantidade de instruções em determinado tempo — por exemplo, 1 K de instruções a cada 1 ms. O valor importa porque define o tempo de varredura (Capítulo 9): processo com exigência de resposta rápida precisa de CPU veloz tanto quanto de programa enxuto.

8.4 Memória

A quantidade de memória está relacionada ao tamanho do programa e, portanto, à quantidade de instruções e de variáveis internas e externas. Dimensionar memória pelo programa atual é erro comum; o critério deve ser o programa previsto com as ampliações já mapeadas no plano da planta.

8.5 Formas de E/S e o efeito no porte

As três formas de E/S — embutida, modular e distribuída — foram apresentadas na Seção anterior (Tabela 5.2). Para a especificação, o que importa é o efeito de cada uma:

- **embutida:** limita o crescimento; ampliar significa trocar de controlador;
- **modular:** cresce por cartões, mas exige rack, fonte e espaço de painel previstos;
- **distribuída:** troca cabo por rede — economia grande quando a instrumentação está espalhada, custo adicional quando os pontos estão todos no mesmo painel.

Hoje é comum o mesmo controlador reunir as três possibilidades: E/S embutida, expansão por cartões modulares e operação com E/S distribuída em *fieldbus*.

8.6 Redundância

Em qualquer processo de automação industrial deseja-se máxima disponibilidade, porque uma falha do sistema de controle pode interromper o processo ou gerar dano. As falhas

vêm de software e de hardware: erro de programação, falha de rede, erro de fluxo de dados, cabeamento e degradação física. Redundância é o uso de um segundo recurso, físico ou lógico, apto a assumir o controle quando o primário falha — é o meio mais simples de obter alta disponibilidade. A Figura 8.1 mostra o arranjo mais comum: dois controladores completos ligados por rede, cada um com sua própria fonte.

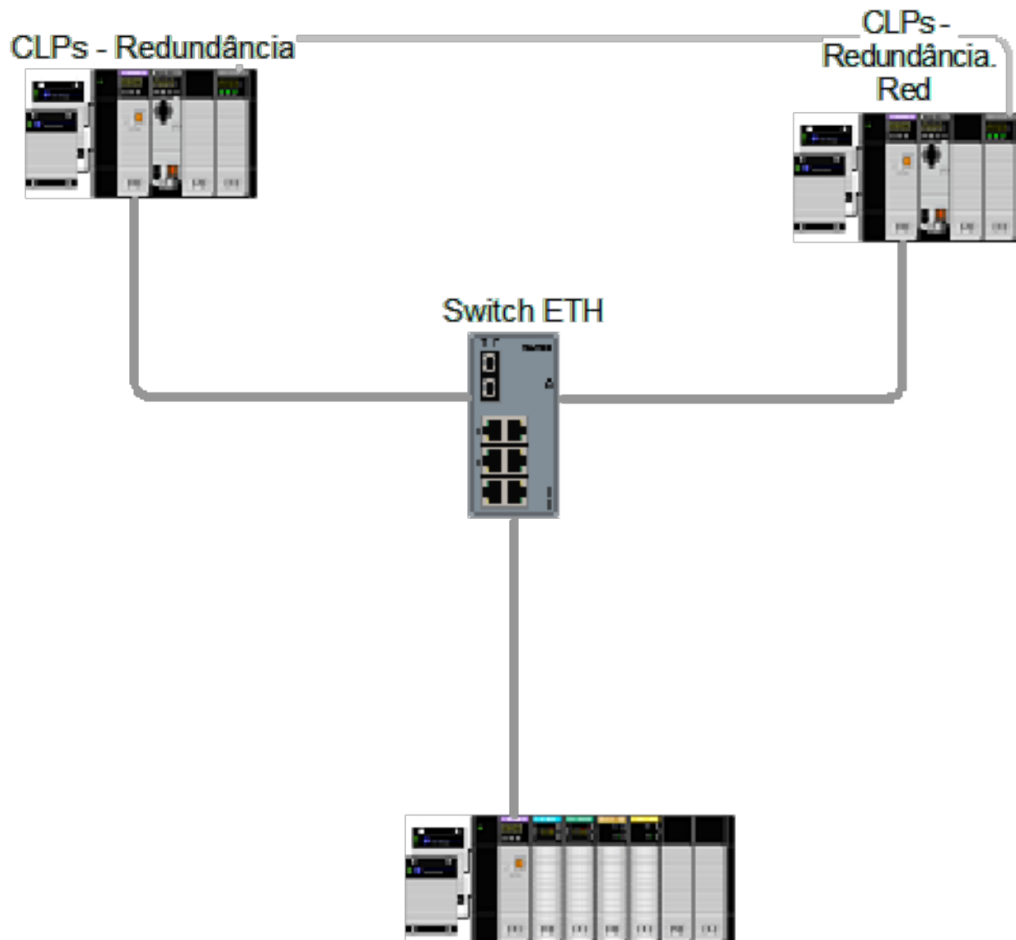


Figura 8.1: Exemplo de redundância de hardware: dois CLPs em redundância ligados por um *switch* Ethernet, com a remota de I/Os compartilhada pelos dois.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, figura 25.

Tabela 8.2: Redundância por hardware e por software.

Aspecto	Redundância por hardware	Redundância por software
Como é feita	Duplicação de componentes físicos, com cartão específico que monitora e sincroniza os mapas de memória	Dois controladores distintos, em geral idênticos, interligados por rede de comunicação
Onde vive a lógica de comutação	No cartão de redundância, sem código adicional do usuário	No programa do usuário, com instruções específicas configuradas no desenvolvimento
Robustez	Maior: o programa está nas duas CPUs e o estado é mantido atualizado	Menor: depende do acerto da lógica escrita pelo usuário
Custo e complexidade	Maior custo de hardware	Menor custo de hardware, maior custo de projeto e de testes

Na arquitetura redundante, em condições normais a CPU principal lê as entradas, executa o programa e escreve as saídas. A cada ciclo, ela transfere continuamente o estado atual do processo para a CPU em espera, de modo que esta acompanhe tudo sem atuar nas saídas. Se a principal falha, a reserva assume com o estado já atualizado, e o processo não reinicia do zero. É o que a Figura 8.2 detalha: as duas CPUs executam o mesmo programa de usuário, mas só a principal escreve saída.

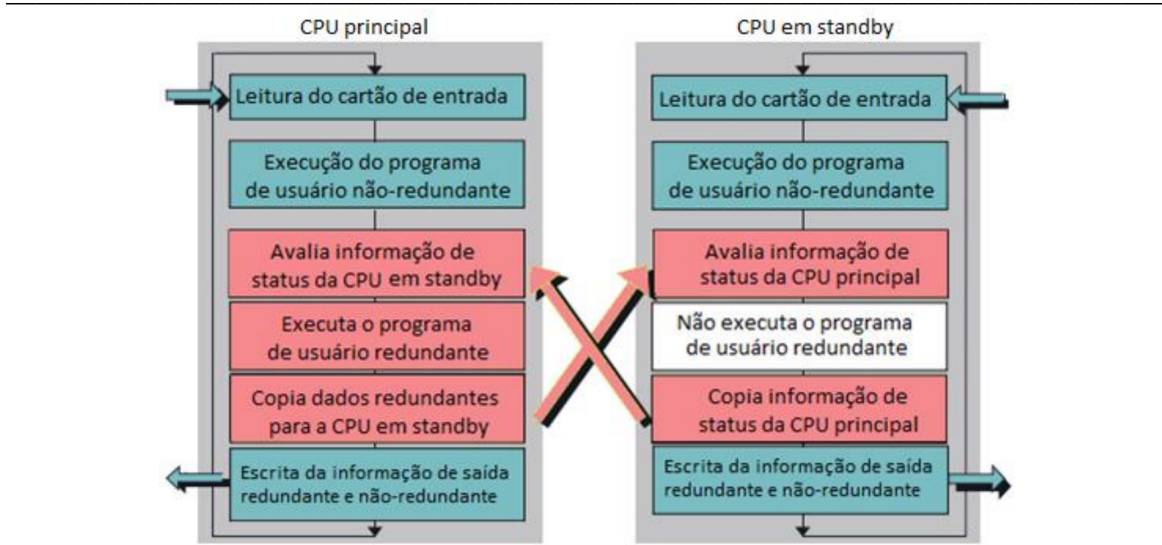


Figura 8.2: Diagrama de fluxo do processo de redundância por software: as duas CPUs trocam informação de *status* e copiam os dados redundantes, de modo que a CPU em espera assuma o controle com o processo atualizado.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, figura 26.

Atenção

Redundância não substitui o projeto de segurança nem corrige programa mal escrito: uma lógica incorreta é executada de forma redundante. Antes de especificar redundância, verifique se o ganho de disponibilidade justifica o custo — e se o orçamento já cobre

backup, condicionamento de energia e manutenção preventiva, que atacam outras causas de parada.

8.7 Procedimento de especificação

1. **Definir a configuração mínima** que atenda a todas as necessidades de operação e de processo. Pergunta inicial: uma CPU simples basta, ou é necessária configuração redundante (*hot-standby*) e troca de cartões com o sistema energizado (*hot-swap*)?
2. **Acrescentar folga:** mesmo com a contagem de E/S fechada, prever no mínimo 10 % de reserva sobre a configuração prevista.
3. **Analisar alternativas:** se o porte é grande, avaliar arquitetura distribuída — controladores menores em rede podem atender melhor e custar menos.
4. **Definir os índices mínimos de desempenho:** considerar o *scan* total (processamento mais atualização de E/S) e os retardos entre sinal lógico e sinal físico, ou seja, o intervalo entre o CLP perceber uma entrada e a saída virar ação em campo.
5. **Dimensionar a memória** para o programa, os dados, os operandos auxiliares e os valores de contagem e temporização.
6. **Verificar as funções especiais** exigidas pelo processo: módulos inteligentes (PID), comunicação em rede, rotinas periódicas, rotinas com frequência de execução diferente do ciclo normal, procedimentos de emergência em alarme.

8.8 Avaliação do fornecedor, do produto e do contrato

Fornecedor. Verificar se tem instalações semelhantes à estudada, se mantém equipe capacitada para suporte pré e pós-venda e se tem condições de manter a continuidade do produto no mercado ao longo da vida da planta.

Produto. Evitar equipamento com menos de um ano de mercado e sem instalação industrial em operação comprovada por esse período; exigir modularidade, ou seja, capacidade de ampliação; suportar rede de comunicação mesmo que não se use agora, prevenindo integração futura com outras áreas (Capítulo 17); e empregar a tecnologia mais avançada disponível no momento da compra.

Contrato. Acordar visitas de inspeção e procedimentos de teste elétrico e funcional; exigir suporte ao equipamento por pelo menos a vida útil da instalação; se a data de entrega é crítica no cronograma, defini-la com cláusula de multa por atraso; e ter cuidado com alterações de arquitetura depois do pedido, porque adendos podem custar mais caro que o equipamento original.

Preço. Depois de dois ou mais fornecedores atenderem a todos os aspectos anteriores, o preço é o critério de desempate — não o critério inicial.

Na prática

Monte a especificação de um controlador para uma máquina com a lista de I/O da Tabela 4.2 ampliada para 40 pontos digitais e 8 analógicos. Defina: (a) o porte justificado pela Tabela 8.1; (b) a quantidade de cartões necessários, com folga de 10 %; (c) o tempo de varredura máximo aceitável para um ciclo de máquina de 200 ms; (d) se há justificativa técnica para redundância — e o custo aproximado disso.

Exercícios

- a) Cite os quatro critérios de porte e explique qual deles costuma definir o custo final.
- b) Um processo tem 300 pontos de E/S. Em que classificação de mercado ele cai? Isso basta para definir o controlador? Justifique.
- c) Explique por que a redundância por software é considerada menos robusta que a por hardware.
- d) O que a CPU em espera faz durante a operação normal, e por que isso é necessário?
- e) Por que a folga mínima de 10 % sobre a contagem de pontos existe? Dê um exemplo real de ampliação que a justifique.
- f) Um fornecedor oferece um modelo lançado há três meses, com preço 20 % menor. Quais perguntas técnicas e contratuais você faz antes de aceitar?
- g) Explique a diferença entre decidir pelo preço no início e decidir pelo preço no final do processo de especificação.

Capítulo 9

Princípio de funcionamento, ciclos e estados de operação

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- descrever a rotina de inicialização executada antes do primeiro ciclo;
- descrever as três etapas do ciclo de varredura;
- explicar o que é tempo de varredura e quais fatores o afetam;
- justificar o *watchdog* e reconhecer a falha que ele sinaliza;
- diferenciar processamento cíclico, por interrupção, por tempo e por evento;
- interpretar o estado dos pontos nas imagens de entrada e de saída;
- distinguir os estados de operação do controlador.

9.1 Rotina de inicialização

Antes de executar o programa do usuário, o controlador cumpre uma rotina gravada em seu sistema operacional. Ao ser energizado no estado de execução, ele:

1. limpa a memória imagem;
2. testa a memória RAM;
3. verifica a possibilidade de executar o programa gravado.

Só depois disso a CPU passa a varrer as instruções em laço, indefinidamente — é o ciclo de varredura descrito na seção seguinte, repetido enquanto o controlador estiver em execução.

9.2 O ciclo de varredura

O controlador funciona como um microcomputador dedicado, executando continuamente ciclos de varredura. A Figura 9.1 mostra o ciclo completo sobre a memória imagem: a entrada é lida do campo para a imagem das entradas, o programa é resolvido sobre essa imagem e o resultado é escrito na imagem das saídas, de onde segue para os cartões de saída. O ciclo se divide em três etapas:

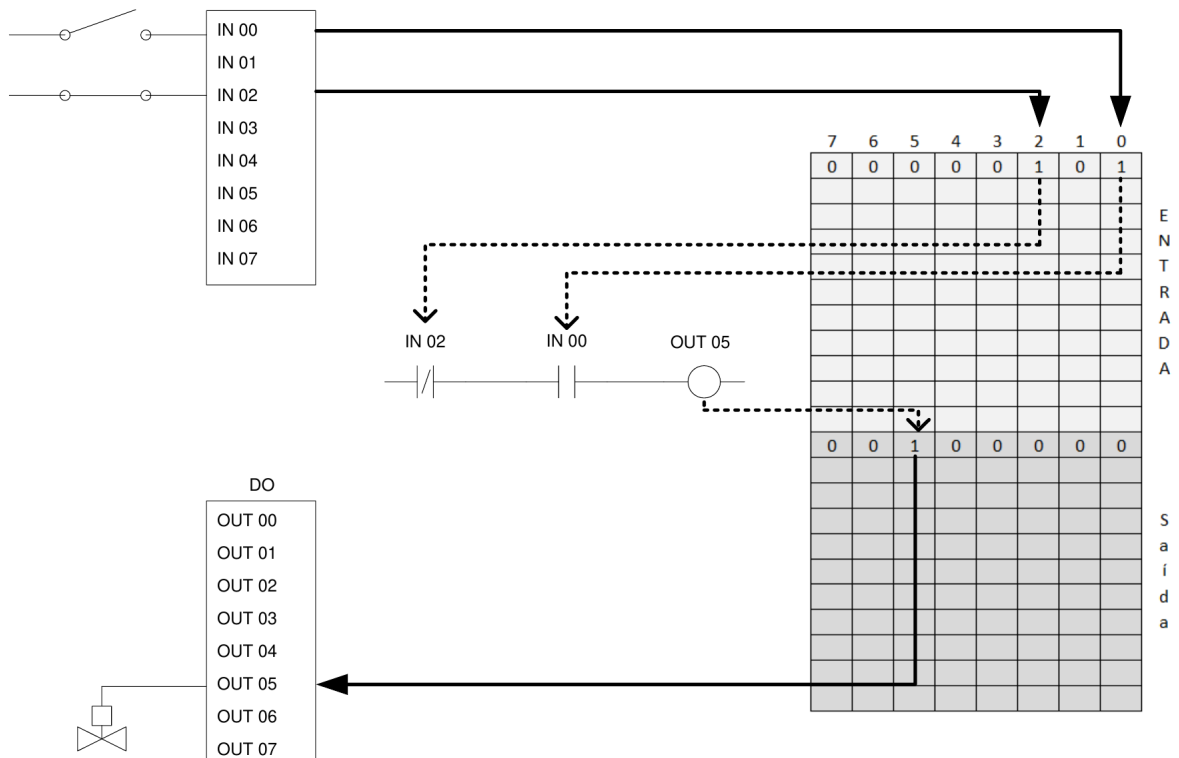


Figura 9.1: Ciclo de operação do CLP: leitura das entradas para a memória imagem, execução do programa e escrita das saídas, com o resultado da linha de Ladder gravado na tabela de dados.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, figura 28.

1. **Entrada** — os dados dos sensores e transdutores são lidos e armazenados na *memória imagem das entradas*;
2. **Processamento** — o programa do usuário é executado sobre esse conteúdo;
3. **Saída** — os resultados são gravados na *memória imagem das saídas* e depois transferidos aos cartões de saída, que acionam os atuadores.

O ciclo completo é chamado de *scan*, varredura ou *loop*. O tempo para percorrê-lo é o **tempo de varredura** (ou tempo de *scan*), e depende de dois fatores: o tamanho do programa do usuário e a quantidade de pontos de entrada e saída. Valores típicos são da ordem de poucos milissegundos; com a eletrônica embarcada atual, controladores rápidos chegam a dezenas de microssegundos para programas pequenos.

9.3 Por que o tempo de varredura importa

Se o tempo de varredura é maior que a duração de um pulso de entrada, o pulso pode ser perdido: a entrada vai de 0 para 1 e volta a 0 enquanto o controlador ainda está processando, e nenhum ciclo chega a ler esse estado. Se a informação de entrada se perde, a saída correspondente não é gerada.

Exemplo numérico. Um sensor de fim de curso gera um pulso de 5 ms na passagem da peça. Com tempo de varredura de 20 ms, existe chance real de o pulso cair entre duas leituras e simplesmente não ser visto. Duas soluções possíveis: aumentar a velocidade de processamento (controlador mais rápido ou programa mais enxuto) ou usar cartão de *contagem rápida* ou entrada com interrupção, que captura o evento fora do ciclo — o assunto da Seção seguinte.

Por isso o tempo de varredura é critério de escolha do controlador (Capítulo 8) e critério de revisão do programa (Capítulo 14).

9.3.1 Watchdog

Para evitar comportamento indefinido quando o processamento se estende além do previsto, existe um temporizador de supervisão, o *watchdog time*. Ele supervisiona o tempo de processamento; ultrapassado o valor máximo — tipicamente algo em torno de 150 ms, mas definido por fabricante — o funcionamento da CPU é interrompido imediatamente, o controlador assume estado de erro e sinaliza a falha de forma visual e, às vezes, sonora.

Atenção

Um controle que reinicia no meio da produção não é um defeito misterioso: é frequentemente watchdog disparado. As causas mais comuns são laço de programa mal construído, temporização muito longa e programa grande demais para o tempo configurado. A primeira providência é ler o diagnóstico do controlador, não reiniciar o equipamento até o problema voltar.

9.4 Modos de processamento

Além do modo cíclico, existem três modos que atendem situações em que o ciclo normal não é adequado:

Tabela 9.1: Modos de processamento do programa.

Modo	Como funciona	Quando é necessário
Cíclico	As instruções são lidas sequencialmente do início ao fim e o ciclo recomeça	Modo predominante em praticamente todas as CPUs
Por interrupção	Ao reconhecer uma ocorrência prioritária, a CPU interrompe o ciclo e executa uma rotina de interrupção; ao terminar, retoma o ponto onde parou	Eventos que não podem esperar o ciclo completo, como emergência ou captura de pulso
Comandado por tempo	Trechos do programa são executados em intervalos regulares de tempo, dentro do ciclo normal	Controle com amostragem periódica fixa, na ordem de milissegundos
Por evento	O programa reage a eventos específicos: retorno de energia, falha de bateria em controlador legado e estouro do tempo de supervisão do ciclo	Procedimentos de inicialização, diagnósticos e resposta a falha

A distinção tem consequência prática: lógica de segurança ou de captura de evento rápido não deve ser resolvida no modo cíclico comum.

9.5 As imagens de entrada e de saída

Durante o ciclo, a comunicação com o campo acontece pela memória imagem. Um exemplo mostra por que isso é importante para ler o programa de outra pessoa.

Considere um trecho de Ladder com um contato NA da entrada IN00 em série com um contato NF da entrada IN02, acionando a saída OUT05. No instante analisado, IN00 está acionada (nível lógico 1) e IN02 está desativada (nível lógico 0).

Tabela 9.2: Estado dos pontos no instante analisado e resultado do processamento.

Ponto	Estado de campo	Bit na imagem	Efeito no programa
IN00	contato fechado	1	contato NA conduz
IN01	contato aberto	0	—
IN02	contato aberto	0	contato NF conduz (lógica de negação)
OUT05	—	1	saída acionada

Em lógica combinacional, o que o programa implementa é:

$$OUT05 = IN00 \cdot \overline{IN02}$$

A saída é acionada quando o nível das *imagens* obedece a essa relação. Note o ponto que costuma confundir: o bit da imagem depende de o ponto estar energizado, e não de o contato de campo ser NA ou NF. Um contato NF de campo energizado resulta em bit 1; a inversão que se vê no programa vem do tipo de contato usado na lógica, não do campo.

Na prática

Usando um simulador de CLP ou a função de simulação do *software* de programação: (a) monte o trecho do exemplo e force IN00 e IN02 para reproduzir a Tabela 9.2; (b) monte uma tabela-verdade completa com as quatro combinações de IN00 e IN02 e confira com a expressão; (c) troque o contato NF por um NA e explique, por escrito, em que situação a saída passa a se comportar de forma perigosa.

9.6 Estados de operação

A UCP pode assumir um estado de erro e dois estados de programação: A Figura 9.2 reúne os estados e as transições entre eles.

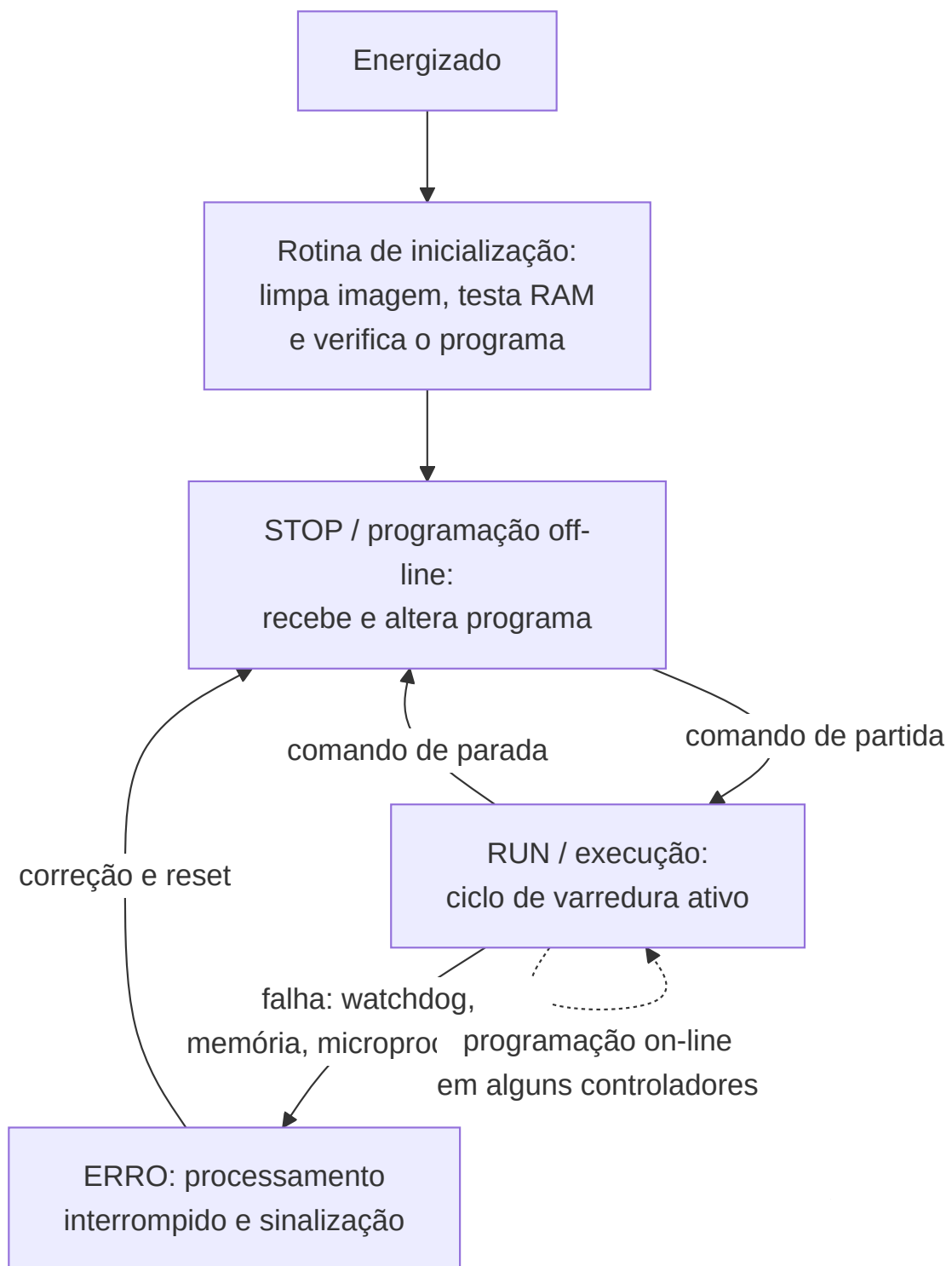


Figura 9.2: Estados de operação do controlador e transições.

Programação (STOP). O controlador não executa o programa e não assume nenhuma lógica de controle. Fica pronto para ser configurado, receber programa novo ou ser modificado. A programação feita nesse estado é chamada *off-line*.

Execução (RUN). O controlador executa o programa do usuário. Em alguns controladores é possível alterar o programa com ele em execução — programação *on-line*. Quando a lógica modificada é enviada, o novo programa entra em operação sem que o estado mude e sem interromper o ciclo.

Erro. Estado que aponta falha de operação ou de execução do programa.

9.6.1 Auto-teste e proteção de dados

A cada inicialização de ciclo o controlador executa um auto-teste do circuito interno; se houver falha — memória ou microprocessador, por exemplo — um sinal de alarme é emitido. Em caso de falta de energia, as memórias são alimentadas automaticamente pelo sistema por baterias de reserva, para evitar perda de dados.

Atenção

Programação *on-line* é recurso poderoso e perigoso. Alterar lógica com a máquina produzindo exige: autorização formal, procedimento de bloqueio quando o trecho for de segurança, e registro da alteração feita. Sem isso, o próximo mantenedor não saberá qual programa está efetivamente na memória — exatamente o problema que o Capítulo 15 trata.

Exercícios

- Descreva as três etapas do ciclo de varredura indicando, em cada uma, onde o dado está armazenado.
- Um programa tem tempo de varredura de 30 ms e precisa detectar um pulso de 8 ms. Que soluções existem, e qual delas não exige trocar o controlador?
- Explique a função do *watchdog* e o que o operador deve fazer quando ele dispara.
- Por que o *watchdog* é classificado como processamento por evento?
- Calcule o valor de OUT05 para as quatro combinações de IN00 e IN02 e monte a tabela-verdade.
- Um técnico afirma: “o contato NF de campo apareceu como bit 1, então o programa está errado”. Corrija a afirmação.
- Explique a diferença entre programação *off-line* e *on-line*, e uma consequência de controle de qualidade de cada uma.
- Por que o auto-teste de RAM acontece antes do ciclo, e não durante?

Parte III

Programação e normas

Capítulo 10

Linguagens de programação

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- diferenciar linguagem de baixo nível de linguagem de alto nível no contexto do CLP;
- reconhecer as linguagens definidas pela norma e a vocação de cada uma;
- ler e escrever lógica de contatos: associação em série, em paralelo e mista;
- usar corretamente o contato normalmente aberto e o normalmente fechado;
- explicar por que a Lista de Instruções não deve ser a primeira linguagem ensinada hoje.

10.1 Por que a linguagem importa

A linguagem é a ferramenta com que se gera o programa que coordena e sequencia as operações do microprocessador. Ela determina três coisas ao mesmo tempo: quanto tempo se leva para escrever, quem consegue ler o código depois e o quanto esse código sobrevive a uma troca de equipamento.

10.2 Baixo nível e alto nível

- **baixo nível** — linguagem de máquina (código binário ou hexadecimal) e *assembly*, com mnemônicos como MVI, OUT, MOV. Cada processador tem seu conjunto próprio de registradores e instruções, o que torna o programa intransportável;
- **alto nível** — linguagem próxima da comunicação humana. Exige compilador ou interpretador para chegar à linguagem de máquina, mas reduz muito o tempo de elaboração e dispensa conhecer a arquitetura do processador. O preço é o tempo de processamento maior.

A norma IEC 61131-3 padroniza linguagens de *alto nível* para automação. Controladores que não atendem à norma costumam oferecer apenas Ladder, no dialeto do próprio fabricante.

10.3 As linguagens definidas pela norma

A Figura 10.1 organiza essa classificação.

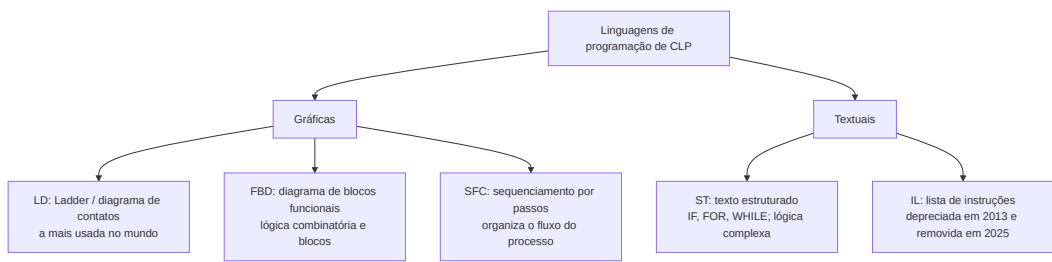


Figura 10.1: Linguagens de programação de CLP: gráficas e textuais.

Tabela 10.1: Linguagens da IEC 61131-3, natureza e vocação.

Sigla	Nome	Natureza	Para que serve melhor
LD	<i>Ladder diagram</i> , diagrama de contatos	Gráfica	Lógica discreta e intertravamento; a mais próxima do comando elétrico e a mais usada internacionalmente
FBD	<i>Function block diagram</i> , diagrama de blocos	Gráfica	Lógica combinatória e composição de blocos; permite construir blocos complexos a partir de blocos simples
SFC	<i>Sequential function chart</i> , passos e transições	Gráfica	Processos sequenciais: decompõe o problema em estados e condições de transição, mantendo a visão global da solução
ST	<i>Structured text</i> , texto estruturado	Textual	Lógica complexa: laços, condicionais, cálculo e manipulação de dados
IL	<i>Instruction list</i> , lista de instruções	Textual	Linguagem depreciada: ver a Seção 10.5

Texto estruturado (ST). Tem base na informática: usa IF-THEN, ELSE, FOR, WHILE e por isso é recomendado para lógicas complexas. É a linguagem mais próxima do que o aluno encontra em outras linguagens de programação.

Diagrama de blocos (FBD). Representa a lógica por símbolos de porta e blocos, como na lógica digital. É a linguagem natural quando o problema é combinar funções: comparar, calcular, temporizar e encadear resultados.

Passos (SFC). Também chamada antigamente de GRAFCET, parte do pressuposto de que o processo tem sequencialidade. Consiste em *passos* ligados a blocos de ação e *transições*: cada passo representa um estado particular do sistema, e a transição é a condição que, quando verdadeira, desativa o passo anterior e ativa o seguinte.

Ladder (LD). A linguagem de relé: contatos e bobinas, com a bobina na extremidade direita, alimentados por duas barras verticais. Cada linha horizontal é uma sentença lógica em que os contatos são as entradas e a bobina é a saída. É a mais simples de assimilar para quem já conhece circuito de comando elétrico — e é essa familiaridade que explica seu sucesso.

10.3.1 Exemplos comparados

A Figura 10.2 coloca as linguagens lado a lado, resolvendo a mesma lógica elementar — $C = A \cdot \overline{B}$ — em cada uma delas. Comparar os quadros é a forma mais rápida de perceber a diferença de vocação: o texto estruturado e a lista de instruções descrevem a operação passo a passo, enquanto o Ladder, o diagrama de blocos e os passos e transições mostram a *estrutura* da solução.

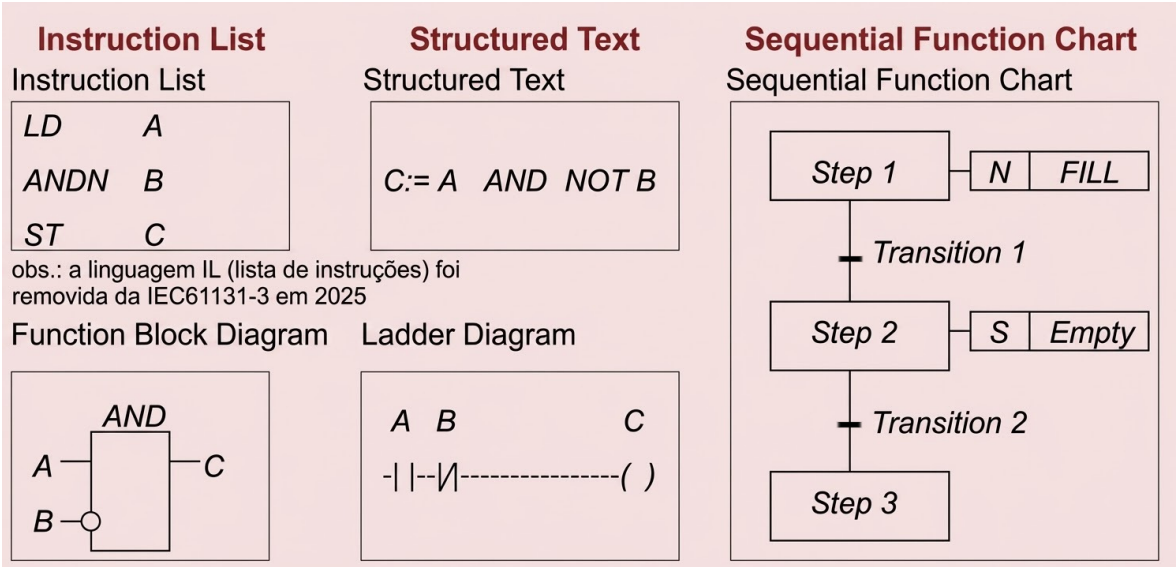


Figura 10.2: Exemplos comparados das linguagens da IEC 61131-3: Lista de Instruções, Texto Estruturado, Diagrama de Blocos de Função, Ladder e Passos e Transições, todos para a mesma lógica.

Fonte: Imagem obtida na internet e reeditada com apoio de ferramenta de IA generativa (Nano Banana).

Repare que apenas três quadros descrevem a lógica de forma gráfica, e que o exemplo em Lista de Instruções aparece aqui como registro do que a norma já removeu — a Seção 10.5 volta ao ponto.

10.4 Lendo e escrevendo Ladder

Cada operando — contato ou bobina — é identificado por um endereço de memória. O endereço aparece como nome simbólico escolhido pelo fabricante, o que é uma das razões da baixa portabilidade entre marcas (Capítulo 13).

- **contato** — conduz conforme o estado do endereço que o identifica. Os contatos endereçados como entrada se fecham se forem NA e se abrem se forem NF, conforme o par de terminais do módulo de entrada esteja acionado;
- **bobina** — aciona o seu endereço: enquanto uma bobina de saída estiver atuada, o par de terminais correspondente no módulo de saída permanece em condução;
- **bit auxiliar** — bits da memória de dados manipuláveis pelo programa, equivalentes aos relés auxiliares do comando elétrico. Não fazem parte da memória imagem de E/S.

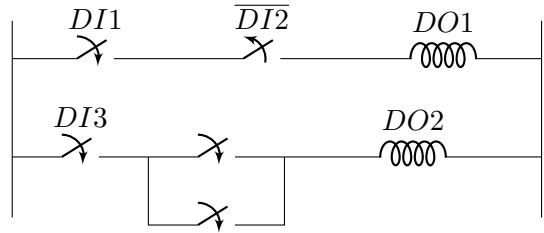


Figura 10.3: Lógica em Ladder: na primeira linha, associação em série (E); na segunda, associação em paralelo (OU).

Tabela 10.2: Qual contato usar na lógica, em função da chave de campo.

Objetivo com a chave externa	Se a chave externa for	O contato no Ladder deve ser
Ligar	NA	NA
Ligar	NF	NF
Desligar	NA	NF
Desligar	NF	NA

Pode-se usar chave externa de qualquer tipo, desde que o contato no programa seja o conveniente. Mesmo assim, **não se deve usar chave NF para ligar nem NA para desligar**: a prática correta é buscar a condição de *falha segura*, ou seja, que a perda de sinal leve o processo ao estado mais seguro.

O contato NF é a negação: campo fechado resulta em ausência de condução; campo aberto resulta em condução. Essa inversão é a origem da maior parte dos erros de leitura de programa por quem está começando.

10.4.1 Associação em série, em paralelo e mista

- **série (E lógico)** — a saída é acionada quando *todos* os contatos conduzem;
- **paralelo (OU lógico)** — a saída é acionada se *pelo menos um* ramo conduz;
- **mista** — combinações das duas, criando condições compostas.

Os três casos, escritos em Ladder e em texto estruturado:

```
1 IF DI1 AND NOT DI2 AND DI3 THEN
2     DO1 := TRUE;
3 ELSE
4     DO1 := FALSE;
5 END_IF;
```

Listing 10.1: Lógica E: série de DI1, $\overline{DI2}$ e DI3.

```
1 IF DI1 OR NOT DI2 OR DI3 THEN
2     DO1 := TRUE;
3 ELSE
4     DO1 := FALSE;
5 END_IF;
```

Listing 10.2: Lógica OU: ramos paralelos de DI1, $\overline{DI2}$ e DI3.

```
1 IF (DI1 OR NOT DI2) AND DI3 THEN
2     DO1 := TRUE;
3 ELSE
4     DO1 := FALSE;
5 END_IF;
```

Listing 10.3: Lógica mista: DI3 em série com o paralelo de DI1 e $\overline{DI2}$.

A familiaridade dos exemplos não é coincidência: Ladder e texto estruturado descrevem a mesma álgebra booleana. A diferença é de forma — e é por isso que a norma admite as duas no mesmo projeto.

Na prática

Usando o simulador do ambiente de programação disponível: (a) reproduza a Figura 10.3 e monte a tabela-verdade das três entradas; (b) escreva a mesma lógica em ST e compare com as listagens 10.1 a 10.3; (c) inverta um dos contatos e explique, por escrito, em que condição de campo a saída passa a comportar-se de forma insegura.

10.5 Lista de Instruções: por que ela saiu de cena

A Lista de Instruções (IL) é textual e sequencial, semelhante ao *assembly*: uma lista de mnemônicos executados em ordem. O material didático antigo costumava apresentá-la como linguagem principal, com o argumento de que é “diretamente entendida pelo controlador”. Esse argumento não se sustenta hoje, por três razões:

1. o programa em IL tem leitura difícil — não se enxerga a lógica, apenas a sequência de operações;
2. a norma **depreciou a Lista de Instruções na sua terceira edição (2013)** e a **removeu na quarta edição (2025)**, cuja publicação é de maio de 2025. Ensinar IL como linguagem de partida é ensinar a escrever código que o próprio normativo abandonou;
3. o que ela oferecia — acesso ao nível da máquina — hoje é resolvido pela compilação a partir de ST, sem que o programador precise descer a esse nível.

Consequência prática para esta apostila: IL aparece aqui como registro histórico e para leitura de código legado, nunca como linguagem recomendada para programa novo. Quando o objetivo for portabilidade, escreva em ST ou LD.

Atenção

A citação de edições é obrigatória quando se fala de conformidade. “Conforme IEC 61131-3” sem dizer a edição é afirmação vazia: um controlador que implementa a segunda edição (2003) pode ser conforme e, ao mesmo tempo, oferecer uma linguagem que a quarta edição já removeu.

10.6 Como escolher a linguagem

Tabela 10.3: Critérios de análise das linguagens, para escolha em projeto.

Critério	O que observar
Forma de programação	<i>Linear</i> : programa em um único bloco. <i>Estruturada</i> : permite organizar, criar bibliotecas de rotinas utilitárias e dividir o programa por critério funcional, operacional ou geográfico
Forma de representação	Diagrama de contatos, diagrama de blocos ou lista de instruções; a representação escolhida define quem consegue revisar o código
Documentação	A documentação é recurso do ambiente de programação mais do que da linguagem, mas quanto mais rico em comentários e mais estruturado o código, melhor. Com muitos profissionais envolvidos entre concepção e manutenção, isso deixa de ser detalhe
Conjunto de instruções	Conjunto de funções disponível: temporização, contagem, aritmética, PID, manipulação de dados e blocos de comunicação
Portabilidade	Se a linguagem e os blocos usados são da norma ou do fabricante (Capítulo 13)

Programação estruturada é o que permite divisão por área, biblioteca de rotinas e manutenção por outra pessoa que não o autor. Em curso técnico, esse é o critério que mais pesa no médio prazo: o programa será mantido por quem não o escreveu.

Exercícios

- Explique a diferença entre linguagem de baixo e de alto nível e dê um exemplo de cada no contexto do CLP.
- Associe cada situação à linguagem mais adequada e justifique: (a) intertravamento de uma prensa; (b) cálculo de conversão de unidade com laço; (c) sequência de partida de uma máquina de envase; (d) encadeamento de blocos de cálculo.
- Escreva a equação booleana e o trecho em ST para uma saída acionada por DI1 em série com o paralelo de DI2 e DI3.
- Preencha a Tabela 10.2 para uma botoeira de emergência que deve desligar o processo, e explique a escolha pelo critério de falha segura.
- Um programa usa uma chave NF de campo para *ligar* um motor. Explique o risco e como corrigir.
- Explique, com suas palavras, por que o contato NF é chamado de negação e por que isso induz erro de leitura.
- Cite as duas edições da norma citadas neste capítulo e o que cada uma fez com a Lista de Instruções.
- Se um fabricante afirma que seu controlador “é conforme à IEC 61131-3”, que pergunta você faz antes de aceitar a afirmação?

Capítulo 11

IEC 61131-3 em profundidade: POUs, tipagem e tarefas

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- descrever o modelo de software da norma, da configuração ao POU;
- diferenciar função, bloco de função e programa;
- configurar tarefas contínua, periódica e por evento, e explicar o papel do *watchdog*;
- classificar variáveis quanto ao escopo e à ligação com o hardware;
- usar os atributos de variável, em especial a retenção;
- dizer o que a norma *não* padroniza — e por que isso é a origem dos problemas de portabilidade.

11.1 O modelo de software

A norma não padroniza apenas a forma das linguagens: padroniza também como o software se organiza. Essa organização é hierárquica e é o que permite que um programa grande seja dividido em partes gerenciáveis. A Figura 11.1 apresenta o modelo completo, do topo da hierarquia até os blocos funcionais.

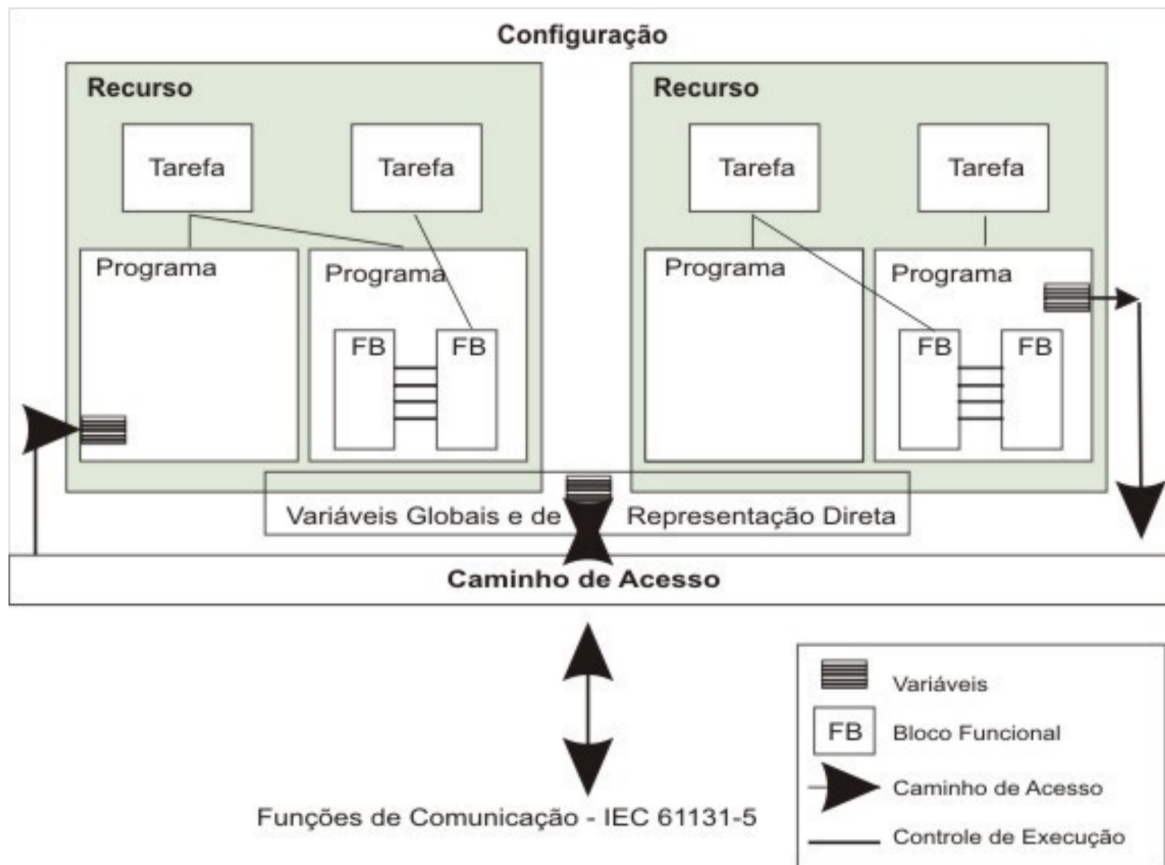


Figura 11.1: Modelo de software proposto pela IEC 61131-3: a configuração reúne recursos; cada recurso executa tarefas, e cada tarefa executa programas, que por sua vez usam blocos funcionais (FB).

Fonte: VIANNA, W. S. Controlador Lógico Programável. IFF, 2023, figura 35.

Configuração. Corresponde ao software necessário a um controlador ou a um conjunto de controladores para cumprir suas funções. Define os elementos que interagem entre si, com suas configurações individuais, e a configuração total resultante.

Recursos. Qualquer elemento com capacidade de processamento dentro da configuração, capaz de executar programas. Pode ser físico — a CPU do processador, uma interface de operação, um *gateway* de comunicação — ou virtual, quando há compartilhamento de memória de processamento por softwares distintos.

Tarefas. Controlam a execução dos programas, de forma periódica ou por disparo de evento. É a tarefa que define *quando* cada parte do programa roda (Seção 11.3).

POUs. A *unidade de organização de programa* é a forma definida pela norma para implementar o software: associação de variáveis e instruções, em qualquer das linguagens. O software aplicativo do controlador consiste na criação e associação de POUs entre si — POUs podem ser formadas a partir de outras POUs, o que permite estruturar o programa e reutilizar código.

11.2 Função, bloco de função e programa

Tabela 11.1: As três unidades de organização de programa.

POU	Memória interna	in- Saídas	Exemplos e uso
Função (FUN)	Não tem: gera o resultado a cada execução, a partir das entradas	Um único valor	ADD, SQRT, SIN, COS, GT, MIN, MAX, AND, OR; também funções definidas pelo usuário. Uso geral e sem estado
Bloco de função (FB)	Tem: os dados persistem entre execuções — é isso que o diferencia da função	Um ou mais valores	PID, temporizadores, contadores e blocos criados para funções específicas, como controle de motores. Podem compor blocos mais complexos
Programa (PRG)	Tem estado próprio	—	Construído a partir de funções e blocos de função, em qualquer linguagem da norma; pode acessar diretamente entradas e saídas e comunicar-se com outros programas. Suas partes podem ser controladas por tarefas diferentes

A distinção função *versus* bloco de função é a que mais causa erro em prova e em projeto: a função é um cálculo; o bloco de função é um *estado que evolui*. Um temporizador precisa lembrar quanto tempo já acumulou — por isso é bloco de função, e não função.

Atenção

Se você declarar o mesmo bloco de função em dois pontos do programa esperando compartilhar o estado, o resultado não será o esperado: cada *instância* tem seu próprio estado. Compartilhar estado se faz por uma única instância declarada e usada em todo o programa. Este é um dos defeitos mais difíceis de localizar em lógica de temporização.

11.3 Tarefas

Tabela 11.2: Tipos de tarefa e consequência no comportamento do programa.

Tarefa	Como se comporta
Contínua	Executa em segundo plano, com menor prioridade: todo tempo de CPU não alocado a outras operações (comunicação, tarefas periódicas) é usado por ela. Ao terminar um ciclo de varredura, recomeça imediatamente. Um projeto não exige tarefa contínua; se houver, só pode existir uma
Periódica	Executa em intervalos regulares de tempo. Quando o período se renova, interrompe qualquer tarefa de prioridade mais baixa e, ao terminar, devolve o controle ao ponto em que a tarefa anterior foi interrompida
Por evento	Executa apenas quando ocorre o disparo configurado. Ao ocorrer, interrompe todas as tarefas de prioridade mais baixa, executa uma vez e devolve o controle ao ponto onde a anterior parou

Cada tarefa tem um controle de tempo de execução configurável, o *watchdog* (WDT). Ao estourar o tempo de ciclo da CPU, o processamento para em condição de falha e o evento é sinalizado ao operador de forma visual e, às vezes, sonora.

Separar o programa em tarefas é o principal instrumento de engenharia para conciliar exigências diferentes no mesmo controlador: uma malha de controle que precisa rodar a cada 10 ms não deve esperar a rotina de comunicação, e um laço de diagnóstico lento não deve consumir o tempo do controle.

11.4 Variáveis

11.4.1 Escopo

- **globais** — declaradas acima de todos os programas; podem ser usadas pelo programa principal e por todas as rotinas e sub-rotinas;
- **locais** — declaradas no início de uma rotina e visíveis apenas por ela.

A norma admite que uma variável global seja acessada por outra configuração ou mesmo por outro controlador, desde que devidamente declarada em cada um.

11.4.2 Ligação com o hardware

- **internas** — sem vínculo direto com o hardware de E/S. Servem para resultados intermediários de cálculo, comparações e variáveis auxiliares. Não fazem parte da memória imagem de E/S;
- **externas** — com vínculo direto com o hardware: são as variáveis de pontos de entrada e saída físicos, que recebem ou enviam valores para dispositivos do campo.

Confundir as duas é a origem de uma pergunta frequente em manutenção: “por que este bit não aparece no diagnóstico de E/S?”. Se ele não é externo, não tem ponto físico correspondente.

11.4.3 Atributos de variável

Tabela 11.3: Atributos de variável previstos pela norma.

Atributo	Efeito
RETAIN	Preserva o valor após o desligamento: é a variável retentiva do Capítulo 6. Use para contagens de turno, estados de processo que devem sobreviver à falta de energia
CONSTANT	Valor fixo, não pode ser alterado em tempo de execução. Use para limites de projeto e parâmetros de calibração
AT	Liga a variável a um endereço físico direto de E/S, dispensando o mapeamento por imagem. Recurso poderoso e traiçoeiro: o valor <i>não</i> passa pela memória imagem

E ainda os tipos de variável: global, direta (local), de mapeamento de E/S (entrada, saída, entrada e saída), externa e temporária.

Atenção

Variável com AT não passa pela memória imagem: ela lê ou escreve o ponto físico no momento do acesso, e não no início ou no fim do ciclo. Isso quebra o determinismo do programa e, em lógica de segurança, é inaceitável. Use AT apenas com justificativa registrada.

11.4.4 Tipagem e alocação

Os tipos elementares e derivados foram apresentados no Capítulo 6. Além deles, a norma permite criar novos tipos: dados estruturados que reúnem variáveis de tipos diferentes, tipos derivados de um tipo já existente, tipos enumerados, subfaixas de valor com limites e arranjos.

Quanto à memória, os controladores atuais fazem *alocação dinâmica* das variáveis: elas são criadas com o tipo definido na declaração e não precisam ter tamanho fixo nem ocupar posições sequenciais. Isso muda a pergunta de projeto: em vez de calcular endereços, declara-se a variável com o tipo adequado e o ambiente resolve a alocação.

11.5 Estruturação, modularização e reutilização

O modelo de software é uma combinação de conceitos que dá estrutura aos projetos. Três princípios o sustentam:

1. **modularização** — dividir o problema em partes com responsabilidade definida;
2. **estruturação** — organizar essas partes em hierarquia, com interfaces claras;
3. **reutilização** — escrever uma vez e usar em vários pontos, por instanciação de blocos de função.

Trocando em miúdos: o programa deixa de ser uma sequência longa de linhas e passa a ser um conjunto de blocos nomeados, cada um testável isoladamente. É o que torna possível dividir o trabalho entre programadores e, depois, entregar a manutenção a outra equipe.

11.6 O que a norma não define

Este ponto é decisivo para o capítulo seguinte. A norma *não* define:

- o endereçamento físico das entradas e saídas (%IO.0, I:1/0, DI1...);
- o ambiente de programação, sua interface e seu formato de arquivo;
- os protocolos de rede usados para programar e para comunicar;
- a implementação dos blocos de sistema e de comunicação específicos do fabricante.

Portanto, “conforme IEC 61131-3” descreve um conjunto de garantias sobre *software aplicativo* — e nada além disso. Todo o resto continua sendo decisão do fabricante, e é exatamente aí que a portabilidade se perde.

Na prática

Em um controlador com ambiente de programação disponível: (a) crie uma função que converta uma leitura de 4 a 20 mA em unidade de engenharia; (b) crie um bloco de função que acumule o tempo em que uma entrada permanece acionada, com **RETAIN**; (c) instancie o bloco em dois pontos do programa e verifique se os estados são independentes; (d) crie duas tarefas — uma periódica de 20 ms para o controle e uma lenta para diagnóstico — e observe a diferença no comportamento do programa.

Exercícios

- Desenhe a hierarquia do modelo de software e explique o papel de cada nível.
- Por que o temporizador é bloco de função e não função? Responda em termos de memória.
- Explique a diferença entre tarefa contínua, periódica e por evento, dando um uso real para cada uma.
- O que o *watchdog* de uma tarefa protege, e o que o operador deve fazer quando ele dispara?
- Classifique como interna ou externa: (a) contador de peças do turno; (b) estado do pressostato de linha; (c) resultado intermediário de um cálculo de vazão; (d) comando da válvula de saída.
- Um técnico precisa que a contagem de ciclos não se perca na falta de energia. Que atributo de variável usar, e onde mais o mesmo atributo é necessário no sistema?
- Explique por que **AT** quebra o determinismo do ciclo de varredura.
- Liste três itens que a norma *não* padroniza e explique como cada um deles pode impedir a troca de fabricante.

Capítulo 12

Família IEC 61131 e o entorno

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- identificar o assunto de cada parte da família IEC 61131;
- situar as quatro edições da parte 3 e o que mudou em cada uma;
- reconhecer as lacunas de cobertura do material antigo;
- explicar o papel da PLCopen, da IEC 61499 e do OPC UA;
- avaliar uma declaração de conformidade com a norma.

12.1 Como a norma está organizada

A IEC 61131 é uma norma de controladores programáveis dividida em partes, preparadas e mantidas por um grupo de trabalho específico (WG 7, sistemas de controle programáveis, do SC 65B do TC 65 da IEC). Ela se chamava IEC 1131 antes da mudança de numeração da IEC.

Tabela 12.1: Partes da família IEC 61131 e o assunto de cada uma.

Parte	Assunto
1	Informações gerais: definições de termos usados nas demais partes e as principais propriedades funcionais e características dos controladores
2	Requisitos de equipamento e testes: condições normais de serviço, requisitos construtivos, ensaios de tipo e compatibilidade eletromagnética
3	Linguagens de programação: arquitetura básica de software e linguagens do programa de controle
4	Orientações para o usuário: especificação, instalação, comissionamento e certificação
5	Comunicações: módulo de comunicação, blocos de função e mecanismos de conexão entre controladores e outros dispositivos
6	Segurança funcional
7	Programação de controle <i>fuzzy</i> (FCL)
8	Orientações para aplicação e implementação das linguagens de programação
9	Interface de comunicação digital de ponto único para sensores e atuadores pequenos (SDCI, comercializada como IO-Link)
10	Formato de intercâmbio XML para exportação e importação de projetos IEC 61131-3

Atenção

Três partes acima merecem atenção de quem aprendeu pela apostila antiga. A **parte 6** (segurança funcional) e as partes **9** e **10** não existiam quando o material de referência foi escrito. Um texto que apresenta “a família IEC 61131” cobrindo apenas as partes 1 a 5, 7 e 8 está desatualizado, e a ausência da parte 6 é a mais grave: é ela que trata de controlador em função de segurança.

12.2 As edições da parte 3

A parte 3 é a que interessa diretamente à programação, e ela evoluiu:

Tabela 12.2: Edições da IEC 61131-3 e o que cada uma trouxe.

Edição	Marco
1ª, 1993	Primeira publicação da padronização das linguagens de programação
2ª, 2003	Edição usada como referência pelo material didático das apostilas de origem: define as cinco linguagens (LD, FBD, IL, ST e SFC)
3ª, 2013	Descreve como implementar programação orientada a objetos no software aplicativo; deprecia a Lista de Instruções (IL)
4ª, maio de 2025	Edição vigente: define três linguagens gráficas (LD, FBD, SFC) e uma textual (ST); a Lista de Instruções foi removida

Duas consequências práticas para quem escreve material didático:

1. dizer apenas “conforme IEC 61131-3” é insuficiente — a edição precisa ser declarada;
2. a IL deve aparecer como linguagem histórica e para leitura de código legado, nunca como recomendação de escrita (Capítulo 10).

12.3 O entorno da norma

A norma não vive sozinha. Três iniciativas completam o que ela não cobre: A Figura 12.1 situa essas iniciativas em relação à norma.

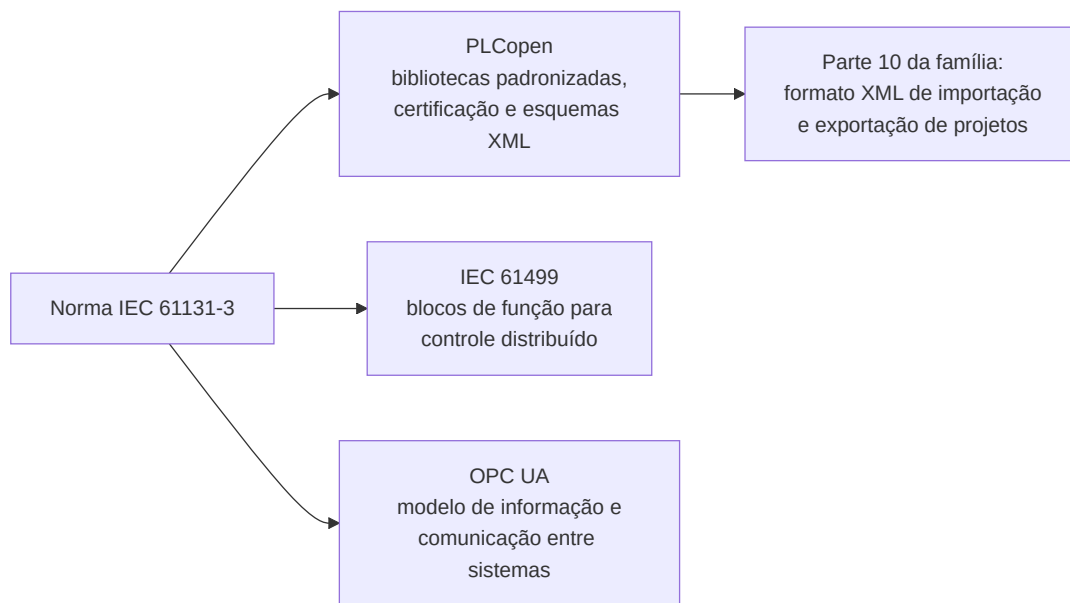


Figura 12.1: A norma e as iniciativas que a complementam.

PLCopen. Organização sem fins lucrativos que apoia e estende a norma. É dela a biblioteca padronizada de blocos de função, o programa de certificação de ambientes de programação e os esquemas XML de intercâmbio de projetos — este último incorporado à família como parte 10 da IEC 61131. Para o usuário, a PLCopen responde a uma pergunta prática: “este programa roda no controlador de outro fabricante se ambos forem conformes?” — e a resposta depende de quais bibliotecas foram usadas.

IEC 61499. Norma de blocos de função para *controle distribuído*. Enquanto a IEC 61131-3 organiza o programa dentro de um controlador, a IEC 61499 organiza a aplicação espalhada por vários dispositivos. É assunto de arquiteturas distribuídas, não substitui a 61131-3 no dia a dia da máquina.

OPC UA. Não é norma de programação, e sim camada de *informação e comunicação*: modelo de dados padronizado, independente de fabricante, com sessão segura. É o que permite que um sistema supervisório ou um ERP leia dados de controladores de marcas diferentes sem escrever um *driver* para cada um. Detalhes no Capítulo 18.

12.4 Como avaliar uma declaração de conformidade

Tabela 12.3: Perguntas para uma declaração de conformidade.

Pergunta	Por que ela importa
Qual edição da norma?	Há diferença relevante entre 2ª, 3ª e 4ª edições, inclusive sobre linguagens disponíveis
Quais linguagens são suportadas?	“Conforme” não significa “todas as linguagens”: o ambiente pode implementar apenas um subconjunto
Há certificação de terceira parte?	Programa de certificação existe e é verificável; declaração do próprio fabricante não é o mesmo
Os blocos usados são padronizados?	Blocos proprietários são o principal fator de dependência na migração
Como é o endereçamento de E/S?	Endereçamento é justamente o que a norma não padroniza (Capítulo 13)

Na prática

Escolha um controlador e busque a declaração de conformidade à IEC 61131-3 disponível do fabricante. Responda, com o documento em mãos: (a) qual edição é citada? (b) quais linguagens são efetivamente suportadas? (c) existe certificação de terceira parte? (d) quais blocos de função padronizados estão disponíveis? Registre as respostas e as que ficaram sem resposta — este último item é o mais informativo.

Exercícios

- Qual parte da família trata de segurança funcional, e por que a ausência dela em um material de CLP é uma lacuna grave?
- Em que edição a Lista de Instruções foi depreciada e em qual foi removida?
- Explique a diferença de escopo entre IEC 61131-3 e IEC 61499.
- O que a PLCopen oferece que a norma não oferece diretamente?
- Um fornecedor apresenta um controlador “conforme IEC 61131-3” que implementa apenas Ladder. A afirmação é falsa? Justifique.
- Por que o formato de intercâmbio XML importa para quem precisa migrar de fabricante?
- Se uma planta tem controladores de três marcas diferentes, o OPC UA resolve o problema de portabilidade do *programa*? Explique o que ele resolve e o que não resolve.

Capítulo 13

CLP conforme IEC 61131-3 versus CLP legado

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- dizer, diante de uma instrução qualquer, se ela é normatizada ou proprietária;
- identificar um programa escrito em dialeto de fabricante e reescrevê-lo em elementos da norma;
- reconhecer as duas famílias de problemas que impedem a portabilidade;
- avaliar o custo real de uma migração de fabricante.

13.1 A pergunta que separa dois mundos

Todo programa de CLP é escrito em algo. A pergunta decisiva, que este capítulo ensina a responder, é: *o que estou usando pertence à norma ou ao fabricante?*

A distinção não é acadêmica. Ela determina se o programa sobrevive à troca de um equipamento, se um técnico formado em outra marca consegue mantê-lo e se o material didático que você estudou continua válido depois da próxima revisão. A Figura 13.1 separa as duas esferas.

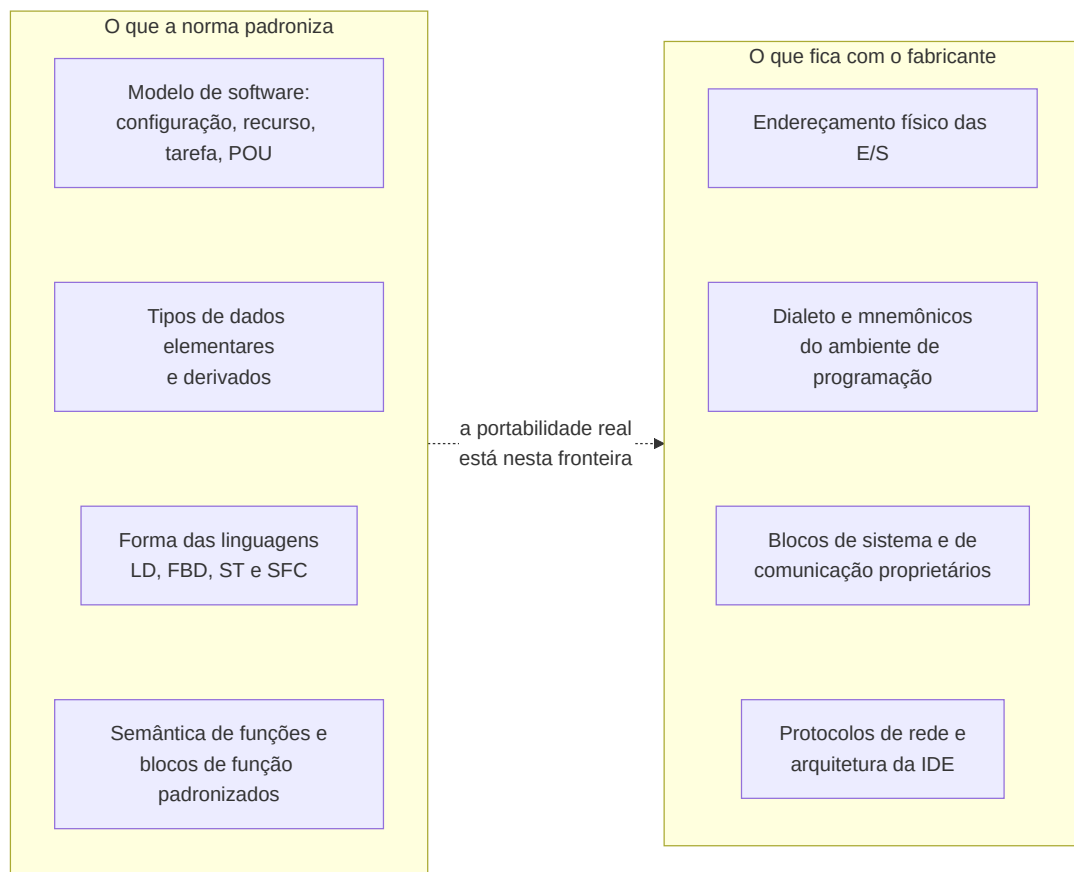


Figura 13.1: O que a norma padroniza e o que permanece decisão do fabricante.

13.2 O caso concreto das apostilas de origem

O material didático que serviu de base a esta apostila foi escrito a partir de 2017 e, na parte de programação, ensina um conjunto de instruções com nomes como XIC, XIO, OTL e JSR. Sua bibliografia declara a origem: o *manual de referência geral do conjunto de instruções* de uma família específica de controladores.

Ou seja: o capítulo de “prática – programação” do material original ensina um **dialeto de fabricante**, não a norma — e o faz logo depois de um capítulo que apresenta a norma e afirma que ela existe “para que os CLP’s de diferentes fabricantes possam ser utilizados por programas distintos”. Há uma contradição interna no próprio texto.

Esta apostila corrige o problema de duas formas: reescreve o conteúdo em elementos normatizados (este capítulo) e mantém o legado documentado no Apêndice B, para quem precisa ler código existente.

13.3 Comparação, aspecto por aspecto

Tabela 13.1: Implementação conforme a norma e implementação legada, lado a lado.

Aspecto	Conforme IEC 61131-3	Legado proprietário
Linguagens	LD, FBD, ST e SFC definidas pela norma (IL até a 3ª edição)	Conjunto de mnemônicos do ambiente do fabricante
Contato normalmente aberto	Contato NA; em IL, LD; em ST, uso direto da variável booleana	XIC — <i>examine if closed</i>
Contato normalmente fechado	Contato NF; em IL, LDN; em ST, NOT da variável	XIO — <i>examine if opened</i>
Bobina simples	Bobina de saída; em ST, atribuição	OTE — <i>output energize</i>
Retenção de saída	Bobina de retenção (SET) e de desretenção (RESET), definidas na norma	OTL e OTU — <i>output latch e unlatch</i>
Detecção de borda	Blocos de detecção de borda de subida e de descida	ONS, OSR e OSF
Temporização	Blocos padronizados TON e TOF	TON e TOF com parâmetros próprios (PRE, ACC, DN) e base de tempo fixa
Aritmética	Funções padronizadas: ADD, SUB, MUL, DIV, MOD, SQRT, ABS, NEG	Mesmas operações com nomes do fabricante, como SQR e CPT
Deslocamento de bits	SHL e SHR	BSL e BSR — <i>bit shift left/right</i>
Chamada de rotina	Chamada de POU: função, bloco de função ou programa	JSR — <i>jump to subroutine</i>
Tipagem	Tipagem forte, com tipos elementares e derivados padronizados (Capítulo 6)	Tipagem própria do ambiente; endereçamento por área de memória
Endereçamento de E/S	Não padronizado pela norma	Próprio de cada família, com sintaxe e convenção particulares
Blocos de sistema e comunicação	Não padronizados pela norma	Específicos do fabricante; principal obstáculo na migração

13.4 Mapa de conversão

A Tabela 13.2 é o instrumento de trabalho para ler código antigo. A coluna da direita indica o elemento normatizado equivalente.

Tabela 13.2: Conversão de mnemônicos de fabricante para elementos da norma.

Mnemônico legado	Significado	Equivalente normatizado
XIC	examina se fechado	contato NA (em IL: LD)
XIO	examina se aberto	contato NF (em IL: LDN; em ST: NOT)
OTE	energiza saída	bobina de saída / atribuição
OTL	travamento de saída	bobina de retenção SET
OTU	destravamento de saída	bobina de desretenção RESET
ONS, OSR	pulso único na subida	bloco de detecção de borda de subida
OSF	pulso único na descida	bloco de detecção de borda de descida
TON	temporizador na energização	bloco TON padronizado
TOF	temporizador na desenergização	bloco TOF padronizado
MOV	move valor	bloco MOVE padronizado
CLR	zera o valor	MOVE com valor zero
CPT	bloco de cálculo	expressão em texto estruturado
SQR	raiz quadrada	função SQRT
ABS, NEG	valor absoluto e negação	funções ABS e NEG
BSL, BSR	deslocamento de bits	SHL e SHR
AND, OR, XOR, NOT	operações em palavra	mesmos operadores padronizados
JSR	salto para sub-rotina	chamada de POU

O Apêndice B traz o mesmo mapa com exemplo de código lado a lado, para uso em laboratório.

13.5 As duas famílias de problemas

Quem avalia portabilidade costuma olhar apenas a linguagem, e é aí que erra a avaliação. Os obstáculos reais se dividem em dois grupos:

13.5.1 Problemas de linguagem — os menores

Mnemônicos diferentes para a mesma função. São inconvenientes, mas a conversão é mecânica: a Tabela 13.2 resolve, e a maior parte das diferenças desaparece com uma reescrita cuidadosa.

13.5.2 Problemas de estrutura — os maiores

- **endereço físico:** cada família usa uma sintaxe e uma organização de memória próprias. Mesmo escrevendo em Ladder idêntico, o programa não roda sem refazer a ligação entre variável e ponto físico;
- **blocos de sistema e comunicação:** leitura de módulo, controle de rede, diagnóstico de rack e blocos de protocolo proprietário não têm equivalente direto;
- **temporização e contagem:** parâmetros, base de tempo e comportamento de acumulação variam, e uma diferença de base de tempo muda o comportamento do processo;
- **ambiente de programação:** formato de arquivo, versionamento e bibliotecas são do fabricante, o que inviabiliza a reutilização direta do projeto.

Só o primeiro grupo é resolvido por conformidade com a norma. Os demais exigem projeto de migração, teste e comissionamento — e é isso que precisa estar no orçamento.

Atenção

Nunca prometa portabilidade a partir de conformidade normativa. O correto é dizer: “o software aplicativo é escrito em linguagem padronizada, o que reduz o esforço de migração; endereçamento, blocos de sistema e protocolos permanecem específicos e exigem reescrita e teste”. Afirmar o contrário é criar expectativa que se transforma em prejuízo na próxima troca de fornecedor.

13.6 Por que este capítulo existe

Quatro razões práticas:

1. **manutenção:** quem aprendeu só um dialeto fica incapaz de manter a máquina do lado;
2. **formação:** um curso técnico que ensina dialeto forma profissional para um fornecedor, não para o mercado;
3. **custo:** o custo de migração é definido por decisões tomadas anos antes, na escrita do programa;
4. **documentação:** registrar em que padrão o código foi escrito é parte da entrega técnica (Capítulo 15).

13.7 Como escrever código que envelhece bem

Tabela 13.3: Recomendações para reduzir dependência de fabricante.

Prática	Efeito
Usar as linguagens e os blocos padronizados sempre que existirem	Elimina a maior parte dos problemas de linguagem
Isolar o que é específico do fabricante em poucos blocos, bem nomeados e comentados	Concentra o problema de migração em pontos conhecidos, em vez de espalhá-lo pelo programa
Separar a lógica de processo da lógica de comunicação e de diagnóstico	Permite reaproveitar a lógica de processo na migração
Declarar tipos explicitamente, sem depender de conversão implícita	Evita comportamento diferente entre ambientes
Nomear variáveis com o significado do processo, não com o endereço físico	O nome sobrevive à troca de endereçamento
Registrar em que edição da norma o programa foi escrito	Torna a conformidade verificável anos depois

Uma palavra em defesa do legado. Nada disso significa que instalação com controlador legado deva ser descartada. Ela funciona, tem lógica documentada (ou deveria) e substituí-la pode não se pagar. O que se pede é consciência: saber que aquilo é dialeto, manter a documentação em dia e planejar a migração antes que ela seja forçada por falta de peça de reposição.

Na prática

Pegue um trecho de programa escrito em dialeto de fabricante — pode ser o do Apêndice B ou um da sua escola. Então: (a) classifique cada instrução como normatizada ou proprietária, usando a Tabela 13.2; (b) reescreva o trecho em LD e em ST conforme a norma; (c) liste o que *não* foi possível converter e explique por quê; (d) escreva um parágrafo de avaliação sobre o esforço de migrar aquele trecho para outro fabricante.

Exercícios

- a) Classifique como normatizado ou proprietário: LD, XIC, TON, OTL, SHL, BSR, MOVE, JSR, ABS.
- b) Por que o endereçamento de E/S não é padronizado pela norma, e qual é a consequência prática disso em uma migração?
- c) Reescreva em texto estruturado: XIC chave1 em série com XI0 chave2, acionando OTL saida1.
- d) Explique a diferença entre bobina simples, bobina de retenção e bobina de desretenção.
- e) Um gestor afirma: “contratamos controlador conforme IEC 61131-3, então podemos trocar de fabricante sem custo”. Escreva a resposta técnica, com três argumentos.
- f) Dê um exemplo de bloco que *nunca* será portátil entre fabricantes, e explique por quê.
- g) Escolha duas das recomendações da Tabela 13.3 e explique como aplicá-las em um programa que você conhece.
- h) Um controlador legado em operação apresenta falha de peça de reposição. Descreva o procedimento que você recomenda, considerando o que este capítulo apresenta.

Capítulo 14

Boas práticas e padronização de código

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- aplicar uma convenção de nomes de variáveis e tags;
- estruturar o programa por área e por responsabilidade;
- situar os requisitos de segurança antes da lógica de funcionamento;
- reduzir o tempo de varredura sem sacrificar legibilidade;
- revisar um programa com um checklist objetivo.

14.1 Por que padronizar

Um programa de CLP é lido muito mais vezes do que é escrito. Ele é lido na partida, no turno, na manutenção corretiva e, anos depois, por quem não participou do projeto. Padronizar é reduzir o tempo que cada uma dessas pessoas gasta para entender o que está escrito — e é o que separa um programa que funciona de um programa que *continua* funcionando.

14.2 Convenção de nomes

Regras mínimas, que valem mais do que qualquer padrão sofisticado:

1. **nome pelo significado do processo**, não pelo endereço: Bomba_Agua_Liga e não D012;
2. **prefixo por tipo de ponto**: evita confundir entrada, saída, bit interno e variável analógica;
3. **sem acento e sem caractere especial**: o nome vai circular entre sistemas, planilhas e documentação;
4. **verbo para comando, substantivo para estado**: Liga, Desliga, Em_Operacao, Falha;
5. **um nome por conceito**: se o mesmo sinal aparece com dois nomes no programa, alguém vai errar;
6. **prefixo de área**: em planta com várias áreas, o nome começa pela área (ENV_ para envase, PRE_ para prensa).

Tabela 14.1: Prefixos sugeridos por tipo de ponto.

Prefixo	Tipo	Exemplo
DI_	entrada digital	DI_Porta_Fechada
AI_	entrada analógica	AI_Nivel_Tanque
DO_	saída digital	DO_Bomba_Liga
AO_	saída analógica	AO_Valvula_Posicao
B_	bit interno / auxiliar	B_Ciclo_Iniciado
T_	temporizador	T_Partida_Bomba
C_	contador	C_Pecas_Turno
V_	variável de cálculo	V_Vazao_Filtrada

14.3 Requisitos antes da lógica

O desenvolvimento de qualquer sistema — independentemente da técnica — exige primeiro conhecer a tecnologia, as possibilidades de ligação e as características dos elementos. E exige reconhecer que equipamentos podem causar danos ao processo por defeito de peças, erro de programação ou erro de instalação, podendo colocar vidas em risco.

Por isso, todo projeto deve ser conduzido de modo a atender, nesta ordem de prioridade:

1. **segurança de pessoas;**
2. **segurança da instalação;**
3. **garantia de funcionamento;**
4. **facilidade de manutenção;**
5. **facilidade de operação;**
6. **custo.**

Custo é o último critério, e essa ordem não é retórica: um projeto mais barato que falha no item 1 ou 2 não é um projeto viável. A Figura 14.1 mostra esse fluxo.

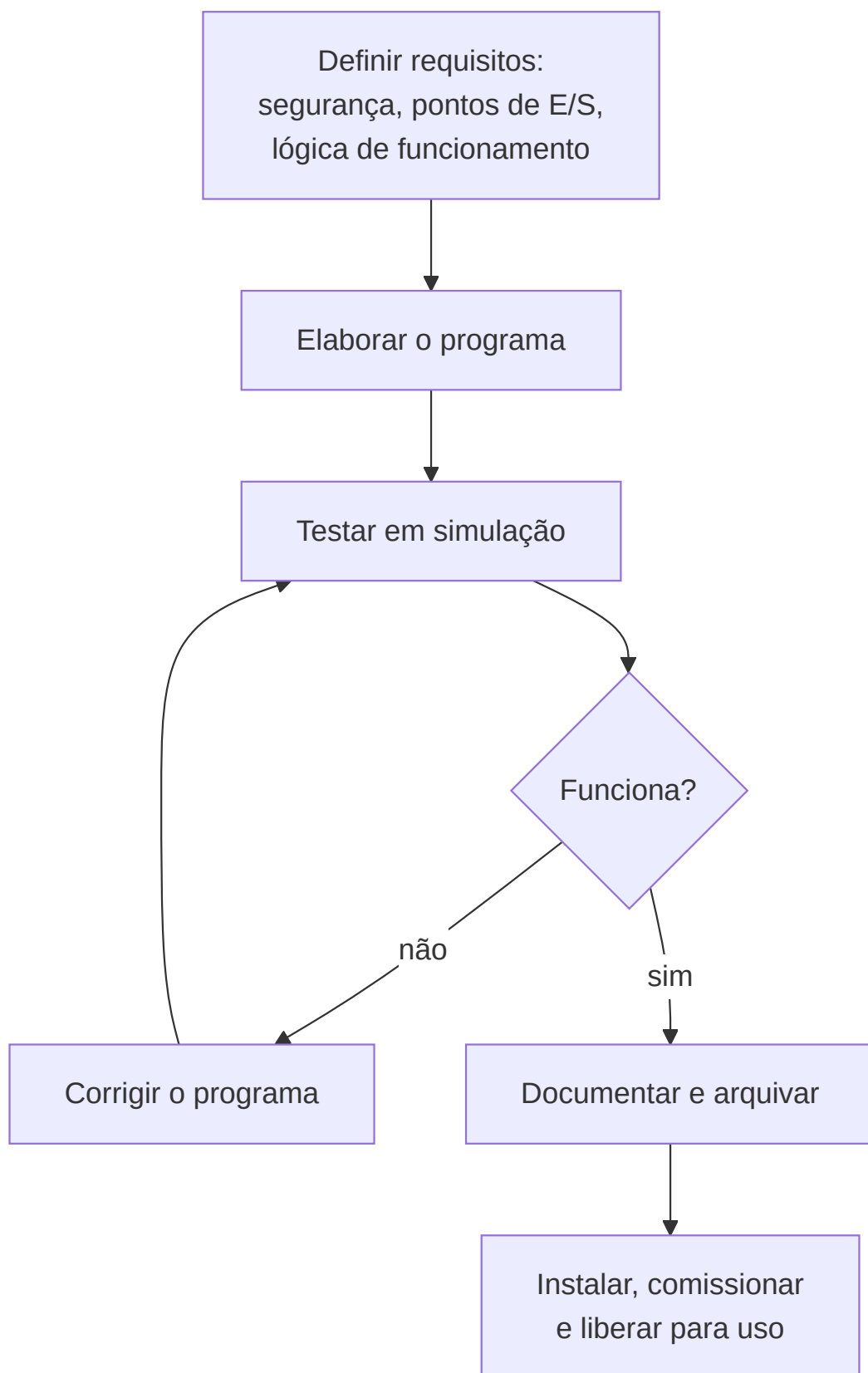


Figura 14.1: Fluxo de elaboração do programa, dos requisitos à liberação para uso.

Depois do ciclo básico esquematizado, entram as *condições marginais* de funcionamento e segurança: ciclo único ou contínuo; partida e parada; manual e automático; parada de emergência e desbloqueio.

14.4 Segurança dentro da lógica — e fora dela

A lógica programada deve implementar intertravamento, condição de falha segura e tratamento de falha. Mas há um limite, e ele precisa estar claro para todo aluno:

Tabela 14.2: O que pertence à lógica programada e o que não pertence.

Pode ser resolvido no programa	Não pode depender apenas do programa
Intertravamento de processo: só abre a válvula se a pressão estiver abaixo do limite	Parada de emergência de pessoa: exige circuito de segurança dedicado ou controlador de segurança certificado
Sequência de partida e de parada com verificação de condições	Deteção de presença em zona de risco: exige dispositivo de segurança (cortina de luz, relé de segurança)
Diagnóstico, alarme e registro de falha	Proteção contra falha do próprio controlador (o programa não pode detectar sua própria ausência de execução)
Retomada segura após falta de energia	Bloqueio de energização: exige meios externos ao software

Atenção

Programa não é dispositivo de segurança. Uma lógica de emergência escrita em CLP comum, por mais cuidadosa que seja, não cumpre função de segurança: se o controlador travar, a lógica de emergência também trava. Use controlador de segurança ou circuito dedicado — e trate isso na especificação (Capítulo 8), não na programação.

14.5 Eficiência do ciclo de varredura

O tempo de varredura depende do tamanho do programa e da quantidade de pontos (Capítulo 9). Para reduzir sem perder clareza:

- **use tarefas:** o que precisa ser rápido roda em tarefa periódica curta; diagnóstico e relatório ficam em tarefa lenta;
- **evite laços longos** dentro do ciclo: processamento pesado deve ir para tarefa própria, com período maior;
- **não duplique lógica:** cada regra existe uma vez e é chamada de onde for preciso;
- **cuidado com a comunicação** dentro do ciclo: troca de dados pela rede é justamente o que o *watchdog* vigia;
- **meça, não estime:** comissione lendo o tempo de ciclo real, não o calculado.

14.6 Checklist de revisão de código

Use esta lista antes de liberar qualquer programa — inclusive o do seu trabalho de conclusão.

Tabela 14.3: Checklist de revisão.

#	Verificação
1	Todos os nomes seguem a convenção e descrevem o processo
2	Cada POU tem uma responsabilidade única e um cabeçalho de documentação (Capítulo 15)
3	Não há lógica duplicada em pontos diferentes do programa
4	Temporizadores e contadores têm instância única e base de tempo declarada
5	Variáveis que devem sobreviver à falta de energia estão declaradas como retentivas
6	Não há uso de escrita direta de E/S que contorne a memória imagem sem justificativa registrada
7	Condições de partida e de parada estão implementadas conforme o descritivo operacional
8	Intertravamentos e limites estão presentes e testados
9	O programa trata a primeira varredura e a retomada após falta de energia
10	Falha de sensor e de comunicação tem comportamento definido (e seguro)
11	O tempo de ciclo medido atende ao requisito do processo
12	Não há força (<i>force</i>) ativa em saída
13	O programa foi testado conforme o roteiro do Capítulo 16
14	A revisão final foi feita por alguém que não escreveu o código

O item 14 é o mais ignorado e o mais barato. Quem escreveu não enxerga o próprio pressuposto errado; outra pessoa enxerga em minutos.

Na prática

Pegue um programa de exercício seu e aplique o checklist da Tabela 14.3. Depois: (a) reescreva os nomes de todas as variáveis segundo a Tabela 14.1; (b) identifique qualquer lógica duplicada e transforme em bloco de função reutilizável; (c) registre quantos itens falharam e o que isso significa para quem herdar o programa.

Exercícios

- Explique, com um exemplo, por que nomear variável pelo endereço físico cria problema na manutenção.
- Ordene, do mais para o menos prioritário, os seis requisitos de projeto e justifique as duas primeiras posições.
- Dê um exemplo de intertravamento que pode ficar no programa e um que *não* pode. Explique a diferença.
- Um programa tem tempo de varredura alto. Cite três providências de projeto que reduzem o tempo sem cortar funcionalidade.
- Explique por que “força” ativa em saída é um risco e como evitá-lo.
- Escolha três itens do checklist e descreva o defeito concreto que cada um previne.

- g) Por que a revisão por outra pessoa encontra erros que o autor não encontra?
- h) Cite uma prática de padronização que reduz custo de migração de fabricante (Capítulo 13) e uma que reduz custo de manutenção.

Capítulo 15

Documentação de programas de CLP

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- listar os artefatos mínimos que compõem a documentação de um projeto;
- montar uma tabela de alocação de E/S;
- documentar um bloco de programa com cabeçalho padronizado;
- definir uma rotina de versionamento e backup compatível com a realidade da planta;
- reconhecer a documentação como requisito contratual de entrega.

15.1 Por que documentar

Um programa sem documentação só funciona enquanto seu autor está disponível. Na prática industrial, isso significa: parada mais longa na manutenção corretiva, dificuldade de formar novo operador, incapacidade de provar o que foi alterado e risco em auditoria de segurança.

A própria apostila de origem já registrava, no capítulo de programação em Ladder, que “uma prática indispensável é a elaboração das tabelas de alocação ou mapa de memória dos dispositivos de entrada/saída”. Esta apostila retoma e amplia essa prática, porque ela é o item que costuma faltar justamente nas instalações mais antigas — as que mais precisam de manutenção.

15.2 Artefatos mínimos

Tabela 15.1: Documentação mínima de um projeto de CLP.

Artefato	Conteúdo	Quem usa
Arquitetura de automação	Diagrama dos controladores, redes, remotas e estações	Manutenção, elétrica, novos projetos
Tabela de alocação de E/S	TAG, descrição, localização e endereço de cada ponto	Manutenção, operação, programação
Lista de tags e variáveis	Nome, tipo, escopo, retenção e descrição de cada variável	Programação e manutenção
Descritivo operacional	Como o processo funciona e o que acontece em cada condição	Operação e programação
Diagrama de sequência ou de fluxo	Passos do processo e condições de transição	Operação, manutenção e testes
Listagem comentada	Programa com comentários de bloco e de linha	Manutenção
Manual de operação e de alarmes	Modos, comandos, alarmes e o que fazer em cada um	Operação
Rotina de backup e versionamento	Frequência, local, nomenclatura e responsável	Manutenção e gestão

15.3 Tabela de alocação de E/S

É o documento mais consultado na manutenção, porque liga três mundos: o instrumento em campo, o ponto físico no cartão e o nome que aparece no programa.

Tabela 15.2: Modelo de tabela de alocação de E/S (exemplo).

TAG	Descrição	Localização	Endereço
PSL-100	Chave de pressão baixa	Topo do tanque pressurizado	DI1
TT-400	Transmissor de temperatura	Saída do misturador	AI1
FS-200	Chave de fluxo	Saída de óleo do aquecedor	DI2
TV-400	Válvula de temperatura	Ao lado do tanque	DO1

Note a estrutura: **TAG** (identificação de instrumentação), **descrição** (o que é), **localização** (onde está fisicamente) e **endereço** (onde o programa o encontra). Sem a localização, o endereço não serve para quem está em campo com o painel aberto; sem o endereço, o TAG não serve para quem está programando.

Atenção

Mantenha a tabela de alocação atualizada *junto* com o programa. Em instalação em que a tabela é feita uma vez, no comissionamento, e nunca revisada, o técnico do plantão descobre o endereço testando ponto por ponto com a máquina em operação — exatamente o cenário que as boas práticas do Capítulo 14 querem evitar.

15.4 Documentação dentro do programa

Documentar não é só produzir documento externo: é deixar o próprio programa legível. Um cabeçalho padronizado por POU resolve a maior parte do problema.

```
1 (* -----
2   POU:          FB_Control_Bomba
3   Tipo:         Bloco de função
4   Objetivo:     Comandar bomba com protecao de nivel e registro de horas
5   Entradas:     Liga (BOOL), Desliga (BOOL), Nivel (REAL, m), NivelMin (REAL,
6                 m)
7   Saidas:       Bomba (BOOL), Horas (DINT, h), Falha (BOOL)
8   Regras:       Liga somente se Nivel >= NivelMin. Nivel < NivelMin
9                 desliga a bomba. Falha bloqueia liga ate reset.
10  Retencao:     Horas (RETAIN)
11  Conforme:     IEC 61131-3, 4a ed. (2025) - blocos padronizados
12  Autor/Data:   <nome> / <data>          Revisao: 02 (2026-09-22)
13  ----- *)
```

Listing 15.1: Modelo de cabeçalho de POU.

Três regras práticas para comentário:

- 1. **comente o porquê, não o quê**: “liga a bomba” o código já diz; “protege contra cavitação” diz o que ninguém deduz;
- 2. **numere as revisões e registre o que mudou** — a pergunta “o que mudou depois do comissionamento?” é a mais frequente na manutenção;
- 3. **descreva cada tag na lista de variáveis**, não apenas no programa.

15.5 Versionamento e backup

Tabela 15.3: Rotina mínima de backup e versionamento.

Item	Recomendação
O que versionar	O arquivo-fonte do projeto (que abre no ambiente de programação), não apenas o PDF ou o arquivo compilado. Sem a fonte, não há manutenção possível
Frequência	Sempre após alteração testada, antes e depois de qualquer intervenção em campo e em periodicidade fixa (por exemplo, mensal)
Nomenclatura	Nome da máquina, número de revisão, data e responsável: ENV01_rev07_2026-09-22_WSV
Onde guardar	Fora do controlador, em pelo menos dois locais, um deles fora da planta. Backup só no computador da sala de controle não é backup
Quem responde	Um responsável nomeado. Backup sem responsável não acontece
Restauração	Testada pelo menos uma vez por ano. Backup nunca testado é suposição, não garantia

Alteração em campo. Programação *on-line* (Capítulo 9) é o ponto em que a documentação se perde com mais facilidade: altera-se a lógica com a máquina rodando e ninguém registra.

A regra prática: **toda alteração em campo gera uma revisão nova do arquivo, com data e autor**, mesmo que seja de uma linha.

15.6 Documentação como entrega contratual

A documentação discutida aqui é a mesma que a Seção de especificação e compra exige do fornecedor (Capítulo 8). Ela deve estar prevista no contrato, com lista de artefatos, formato e prazo — porque no fim da obra a pressa é grande e o que não está no contrato não é entregue.

Para a instituição de ensino, vale a mesma lógica: trabalho de conclusão, projeto integrador e programa didático entregues sem documentação ensinam ao aluno o hábito errado.

Na prática

Monte a documentação de um trabalho prático seu, com quatro entregas: (a) arquitetura de automação em diagrama; (b) tabela de alocação de E/S no modelo da Tabela 15.2; (c) cabeçalho de POU para cada bloco do programa, no modelo da listagem 15.1; (d) rotina de backup preenchida na Tabela 15.3, com nome do responsável. Depois troque o material com um colega e peça que ele explique o que o seu programa faz sem abrir o código. O que ele não conseguir explicar é o que falta documentar.

Exercícios

- a) Explique por que a tabela de alocação de E/S é o documento mais consultado na manutenção.
- b) Uma instalação tem backup semanal do programa, guardado no computador da sala de controle. Aponte dois problemas dessa rotina e como corrigi-los.
- c) Por que versionar apenas o PDF do programa não é suficiente?
- d) Escreva o cabeçalho de POU para um bloco que controla uma esteira com sensor de presença e contador de peças.
- e) Dê um exemplo de comentário que explica “o quê” e outro que explica “porquê”. Qual dos dois tem mais valor e por quê?
- f) Uma alteração de uma linha foi feita em campo, com a máquina rodando, sem registro. Descreva o risco que isso cria para o próximo mantenedor.
- g) Liste o que deve constar em contrato de fornecimento de sistema de automação sobre documentação.
- h) Por que restaurar um backup de teste, uma vez por ano, é diferente de apenas fazer backup?

Capítulo 16

Simulação, teste e comissionamento

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- explicar por que testar antes de energizar é obrigatório;
- percorrer os seis níveis de validação, do teste de lógica ao *dry-run*;
- montar um roteiro de teste com casos, estímulos, resultados esperados e evidências;
- reconhecer os limites da simulação — o que ela não detecta.

16.1 Por que testar antes

Um controlador mal programado pode causar perda de produção e criar condições perigosas para equipamentos e pessoas. Simular permite encontrar e corrigir defeitos cedo no projeto, quando a correção custa minutos em vez de horas de parada. Além disso, melhora a segurança associada ao equipamento e reduz o tempo de instalação e comissionamento, porque muitos cenários já foram testados antes de o sistema ser energizado.

16.2 A escada de validação

A validação é uma escada: cada degrau é mais caro e mais realista que o anterior, e cada degrau elimina uma classe de defeito. Subir todos os degraus antes de energizar a máquina é o que evita surpresa no comissionamento. A Figura 16.1 organiza esses níveis.

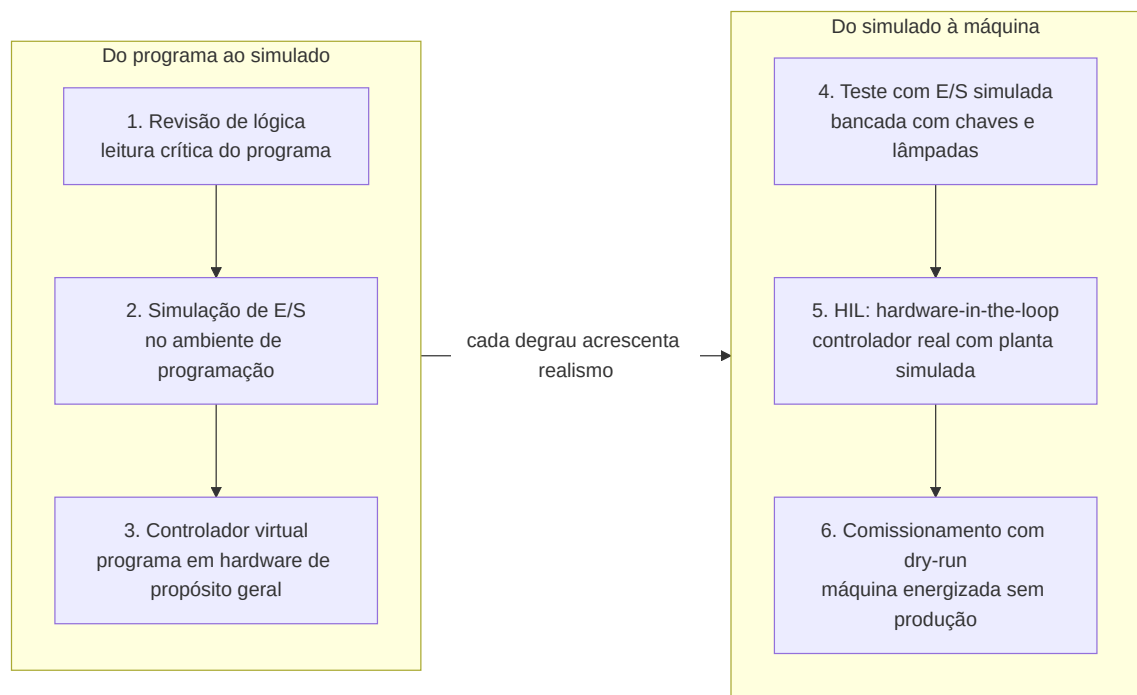


Figura 16.1: Níveis de validação, do teste de lógica ao comissionamento.

Tabela 16.1: O que cada nível detecta e o que ele não detecta.

Nível	Detecta	Não detecta
1. Revisão de lógica	Erro de lógica, endereço trocado, condição faltante, laço mal construído	Nada de comportamento em tempo real
2. Simulação de E/S	Sequência, temporização relativa, intertravamento, tabela-verdade	Comportamento de sinal elétrico e de campo
3. Controlador virtual	Portabilidade do código, desempenho relativo, integração com o ambiente	Diferenças de implementação do hardware real
4. Bancada com E/S	Fiação de campo, tensões, sensores e atuadores reais, ruído	Comportamento da máquina sob carga
5. HIL	Interação controlador–planta simulada, casos difíceis e de falhas	Desgaste e efeitos mecânicos reais
6. <i>Dry-run</i> em campo	Tudo o que envolve a instalação completa, com o processo energeticamente seguro	Desempenho em produção plena

Simulação de E/S. É o nível que todo aluno tem disponível: o ambiente de programação permite forçar o estado das entradas e observar as saídas sem nenhum hardware conectado. Com ela se percorre a árvore de decisões do programa — inclusive as combinações que a máquina real demora horas para alcançar.

Controlador virtual (*soft PLC*). A função de controlador é executada em hardware de propósito geral, em máquina virtual ou contêiner. Permite ensaiar o programa em outra

plataforma e é útil em ensino e em testes de integração.

HIL (*hardware-in-the-loop*). O controlador real roda conectado a um modelo computacional da planta. É o nível que permite ensaiar falha de sensor, sobrecurso de válvula e situação de emergência sem risco e sem parar a produção.

Dry-run em campo. Máquina energizada, sem produção: verifica-se fim de curso, polaridade, tempo de resposta e cada intertravamento, ponto por ponto, com a equipe de campo presente.

16.3 Roteiro de teste

Teste sem roteiro é passeio. O roteiro abaixo é um modelo para adaptação; a coluna de evidência é a que costuma ser esquecida e é a que sustenta a entrega técnica.

Tabela 16.2: Modelo de roteiro de teste.

#	Caso	Estímulo	Resultado esperado	Evidência
T01	Partida em condições normais	Todas as condições de partida satisfeitas; comando de partida	Máquina parte na sequência prevista	Registro no relatório de comissionamento
T02	Bloqueio de partida por condição	Uma condição de partida não satisfeita; comando de partida	Máquina não parte e alarme indicado	Lista de alarmes ativos
T03	Intertravamento em operação	Simular perda da condição de intertravamento durante a operação	Parada segura na sequência prevista	Registro de eventos com horário
T04	Parada de emergência	Atuar o dispositivo de emergência	Parada conforme projeto; retomada só após desbloqueio e reset	Teste presencial com assinatura
T05	Retomada após falta de energia	Desenergizar e reenergizar o painel	Estado seguro; variáveis retentivas preservadas	Leitura das variáveis retentivas
T06	Falha de sensor	Desconectar sensor em bancada	Programa assume o comportamento definido no descritivo (e seguro)	Registro de falha e ação tomada
T07	Falha de comunicação	Interromper o enlace com a rede ou com a remota de E/S	Comportamento seguro declarado; alarme de comunicação	Registro de eventos
T08	Tempo de ciclo	Operação em carga máxima	Tempo de varredura medido dentro do requisito	Medição registrada no relatório

Os casos T04 a T07 são os que a simulação isolada **não** cobre adequadamente, porque dependem de comportamento de hardware, de energia ou de rede. Eles precisam ser testados com equipamento real — e sempre pelo procedimento de bloqueio da instalação.

16.4 Limites da simulação

Tabela 16.3: O que a simulação não substitui.

Limite	Por quê
Sinal elétrico real	Tensão, corrente, ruído, tempo de comutação e falha de isolamento só aparecem com módulo e carga reais
Dinâmica mecânica	Inércia, folga, atrito e sobrecurso não existem no modelo
Atraso de rede	Latência e perda de pacote dependem da instalação
Falha do próprio controlador	Nenhuma simulação executa o programa com o controlador travado
Erro de campo	Sensor instalado no lugar errado, inversão de polaridade e fiação trocada só aparecem na bancada e no <i>dry-run</i>

Atenção

Nunca energize máquina com o programa recém-carregado sem ter percorrido o roteiro de teste. E nunca faça *dry-run* com pessoa em zona de risco: a máquina está energizada, com lógica nova e sem histórico — a condição de maior risco de todo o projeto.

16.5 Registro de evidências

O resultado do teste precisa sair do ambiente e virar documento:

- roteiro preenchido, com data, responsável e assinatura nos casos de segurança;
- registro de eventos com data e hora dos alarmes e paradas testadas;
- lista de pendências encontradas e o que foi feito com cada uma;
- versão do programa testada (número de revisão) — teste sem versão identificada não vale como evidência.

Esse conjunto é o que liga este capítulo ao anterior: a documentação da Capítulo 15 é o que permite provar, meses depois, que o sistema foi testado.

Na prática

Monte o roteiro de teste para um dos exercícios de laboratório desta apostila, preenchendo as colunas da Tabela 16.2. Depois: (a) execute os casos possíveis no simulador de E/S e registre o resultado obtido; (b) identifique quais casos só podem ser executados com hardware real; (c) escreva, em cinco linhas, o risco de liberar o programa sem executar esses casos.

Exercícios

- Cite três benefícios de testar em simulação antes da instalação.
- Ordene os seis níveis de validação do menos para o mais realista e explique o que

cada um acrescenta.

- c) Por que a simulação de E/S não substitui o teste com módulo de entrada real?
- d) O que é HIL e que tipo de falha ele permite ensaiar sem risco?
- e) Escreva dois casos de teste adicionais para uma máquina de envase, no formato da Tabela 16.2.
- f) Por que a versão do programa é parte obrigatória da evidência de teste?
- g) Um fornecedor entrega o sistema afirmando que “já testou tudo no simulador”. Que questionamento você faz, com base neste capítulo?
- h) Explique por que o *dry-run* é a etapa de maior risco do projeto.

Parte IV

Integração e sistemas

Capítulo 17

Redes industriais e sistemas SCADA

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- reconhecer as três arquiteturas de rede de um CLP e escolher entre elas;
- identificar os protocolos de campo e industriais mais usados e o emprego típico de cada um;
- descrever a arquitetura de um sistema SCADA, componente por componente;
- distinguir alarme, evento e intertravamento e dizer quem executa cada um;
- justificar por que a rede de automação não pode ser projetada como a rede de escritório.

17.1 Por que existe rede

Um CLP isolado só automatiza uma máquina. A partir do momento em que há duas máquinas, um painel de operação ou uma sala de controle, surge a necessidade de *transportar informação*. O transporte pode ser feito a cabo dedicado, ponto a ponto, mas o custo cresce depressa: cada sinal exige par de condutores, borneira, caixa de passagem, bandeja e mão de obra. Em uma planta com centenas de pontos espalhados, o cabo passa a custar mais que o controlador.

A rede resolve esse problema trocando **fios** por **mensagens**: em vez de levar cada sinal até o painel, leva-se um cabo de comunicação até uma unidade que concentra os sinais no próprio campo. É o mesmo raciocínio dos níveis apresentados no Capítulo 3, visto agora pelo lado do meio físico.

17.2 Três arquiteturas de rede do CLP

A classificação prática considera onde estão os módulos de entrada e saída e como se dá a comunicação entre o controlador e a estação de programação ou supervisão.

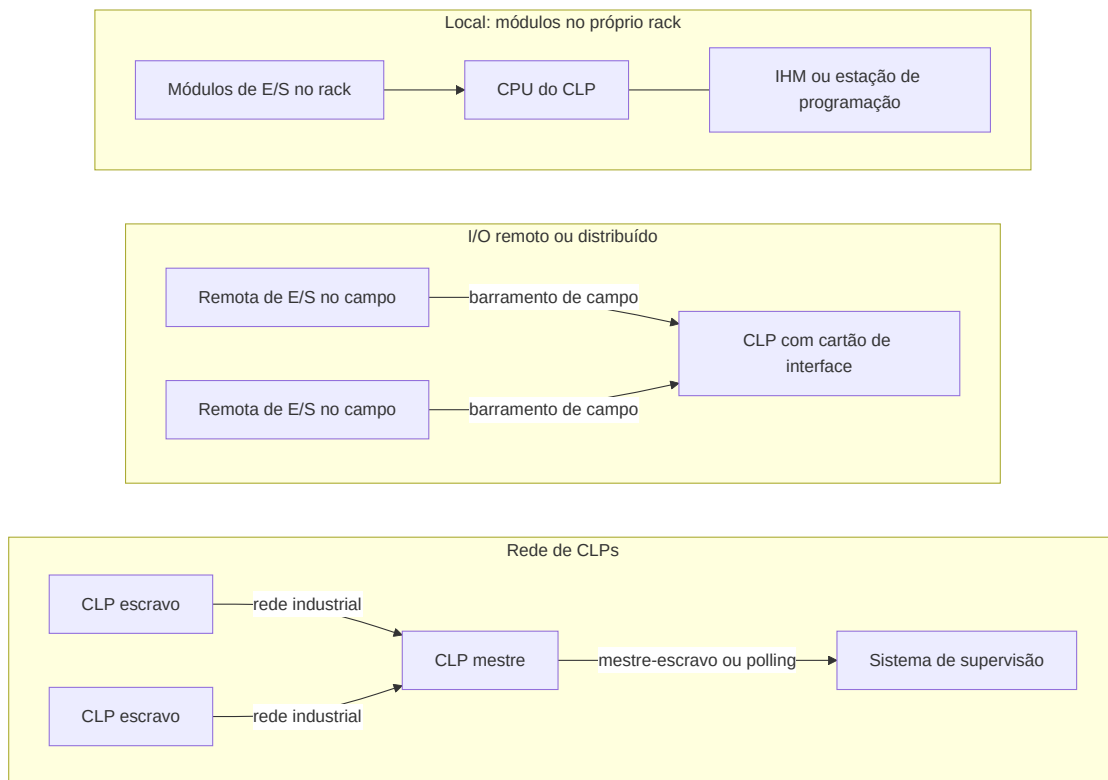


Figura 17.1: As três arquiteturas de rede do CLP: local, I/O remoto e rede de CLPs.

Tabela 17.1: Comparação das três arquiteturas de rede do CLP.

Arquitetura	Onde ficam as E/S	Quando compensa	Cuidado
Local	No rack do próprio controlador	Bancada, máquina pequena, pontos concentrados e próximos	Todo sinal vindo do campo chega ao painel: o custo do cabo cresce rápido com a distância
I/O remoto ou distribuído	Em remotas instalações no campo	Campo distante, pontos espalhados por uma área extensa	Exige cartão de interface, endereçamento da remota e diagnóstico do barramento; a queda do barramento derruba o conjunto de pontos ligado a ele
Rede de CLPs	Módulos locais em cada controlador	Linhas com várias máquinas que precisam coordenar entre si	É preciso definir quem comanda quem; dois controladores escrevendo na mesma saída é falha de projeto, não de programa

Na arquitetura de rede de CLPs, a comunicação com a supervisão costuma ser *mestre-escravo* ou por consulta periódica (*polling*): a estação de supervisão pergunta, o controlador responde. É um modelo simples e previsível, que funciona bem para supervisão e mal para troca de dados rápida entre controladores. A Figura 17.2 mostra essa arquitetura: o supervisório

atende a rede por uma interface, e cada controlador mantém seus módulos de E/S locais, ligados aos instrumentos do trecho de processo que ele comanda.

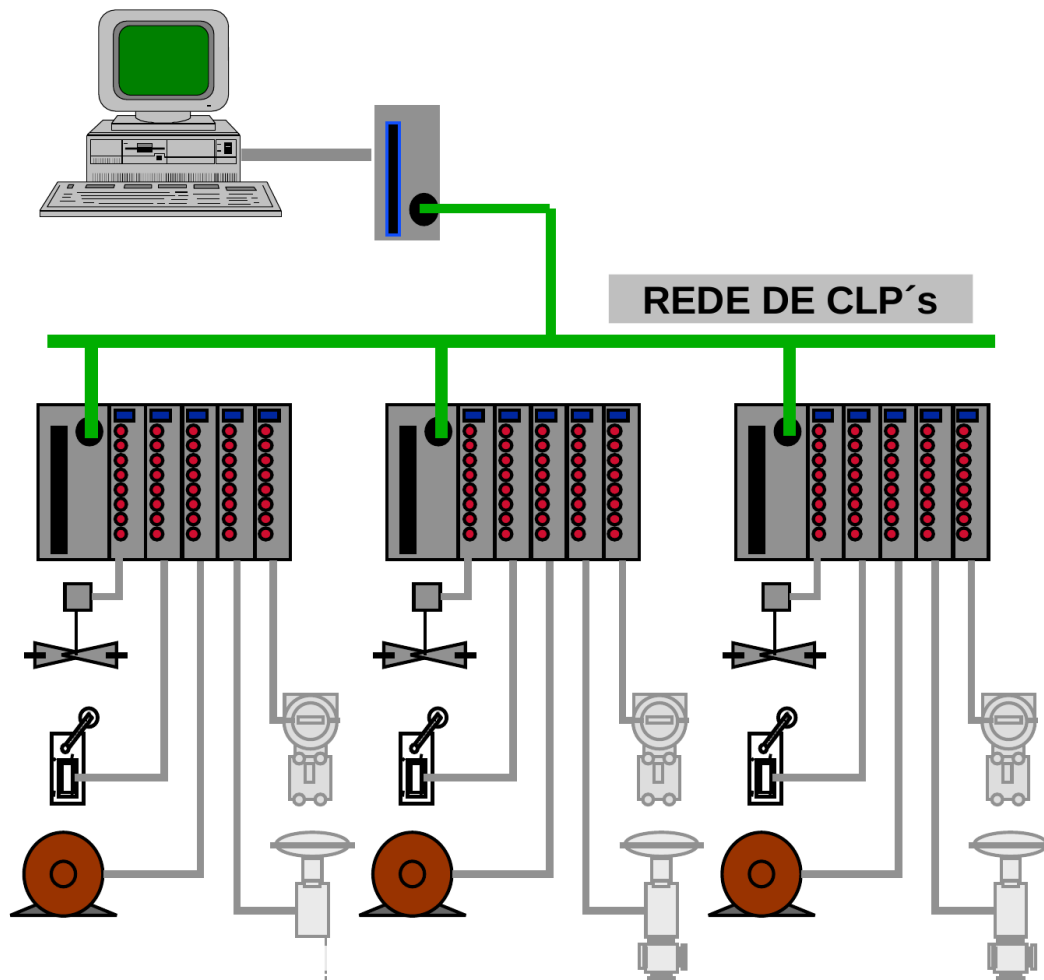


Figura 17.2: Arquitetura de rede de CLPs: módulos de E/S montados localmente, comunicação dos controladores com o sistema de supervisão pelo tipo mestre-escravo ou *polling*.

Fonte: VIANNA, W. S. Controlador Lógico Programável. IFF, 2023, figura 68.

Atenção

Na arquitetura de I/O remoto, a maior parte dos sinais deixou de passar pelo painel — mas passou a depender de *um* cabo de comunicação. Antes de adotá-la, responda: o que acontece com o processo se esse barramento cair? A resposta precisa estar escrita no projeto e no programa, não descoberta no dia da parada.

17.3 Protocolos de campo e industriais

“Protocolo” aqui significa o conjunto de regras que permite a dois equipamentos de fabricantes diferentes trocar dados. A Tabela 17.2 traz os mais encontrados em instalações industriais, com o emprego típico de cada um. Ela é um guia de conversa com o fornecedor, não uma especificação: os limites de taxa, alcance e número de nós de cada protocolo variam com o meio físico e com a implementação, e devem ser confirmados na documentação do produto.

Tabela 17.2: Protocolos de comunicação usuais na automação industrial e seu emprego típico.

Protocolo	Meio e modelo usuais	Emprego típico e o que observar
AS-i	Barramento de dois fios que alimenta e comunica	Ligação de sensores e atuadores discretos próximos ao processo. Transporta pouco dado por dispositivo; serve para sinais simples, não para variáveis de controle.
IO-Link	Ponto a ponto, sobre o cabo do próprio sensor	Sensores inteligentes com parâmetros e diagnóstico. Referenciado como parte 9 da família IEC 61131 (SDCI). Não substitui o barramento: é o último trecho até o sensor.
Modbus RTU	Serial, tipicamente RS-485, modelo mestre-escravo	Instrumentos, medidores, inversores de frequência e remotas simples. Protocolo aberto, simples e largamente implementado. Não padroniza o significado dos registros: a tabela de cada equipamento é que diz o que cada endereço contém.
Modbus TCP	Ethernet, modelo cliente-servidor	Integração com supervisão e troca de dados entre equipamentos de marcas diferentes. Leva o Modbus para a rede Ethernet; o determinismo passa a depender do projeto da rede, não do protocolo.
HART	Comunicação digital sobreposta ao sinal analógico	Transmissor analógico que também informa diagnóstico e parâmetros. Mantém o sinal de 4–20 mA que o sistema já lê e acrescenta informação digital.
Profibus DP	Serial, mestre-escravo	Controle de campo com I/O remoto. Rede consolidada, muito presente em instalações mais antigas.
PROFINET	Ethernet industrial	Controle em tempo real e integração de I/O remoto. Exige projeto de rede dedicado: topologia, switches e priorização de tráfego.
EtherNet/IP	Ethernet industrial sobre o modelo CIP	Controle e integração de equipamentos de uma mesma família de produtos. Comum em instalações que já usam controladores dessa família.
CANopen	Barramento CAN, baseado em difusão de mensagens	Máquinas, eixos e módulos compactos embarcados. Difundido em equipamentos móveis, em que o alcance físico é curto.

Duas conclusões práticas saem da Tabela 17.2:

1. **protocolo aberto não é o mesmo que dado padronizado.** Saber ler um registro Modbus não diz o que ele significa. É por isso que o Capítulo 18 discute uma camada de informação, e não apenas de transporte.
2. **o protocolo não resolve a arquitetura.** Escolher Profinet ou EtherNet/IP não decide quantas zonas de rede haverá, quem fica em qual e o que acontece quando um trecho cai.

17.4 Arquitetura SCADA

O sistema SCADA (*Supervisory Control and Data Acquisition*) é a camada que transforma o conjunto de controladores da planta em uma operação visível e comandável a partir da sala de controle. Ele reúne cinco funções, sempre as mesmas, qualquer que seja o produto. Antes de detalhá-las, a Figura 17.3 fixa o esquema básico: de um lado o processo, do outro a estação de supervisão, e entre os dois o equipamento de controle e a via de comunicação que os interliga.

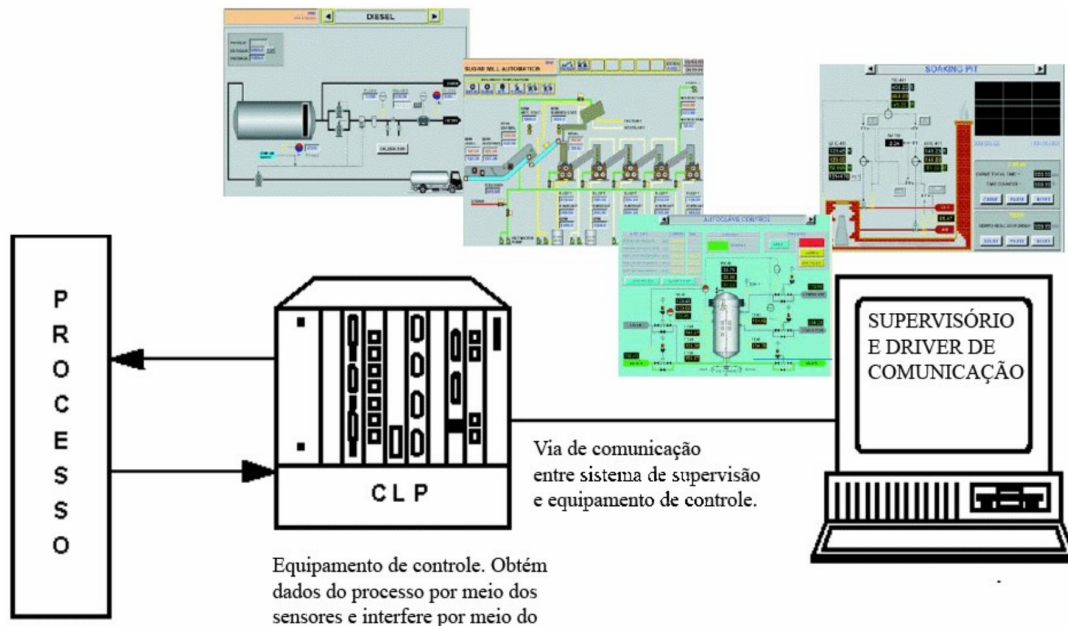


Figura 17.3: Esquema básico de um sistema SCADA com uso de CLP: o processo, o equipamento de controle e a estação de supervisão com o *driver* de comunicação.

Fonte: VIANNA, W. S. Controlador Lógico Programável. IFF, 2023, figura 65.

1. **servidor de dados e *driver* de comunicação** — é quem fala com os controladores. O *driver* codifica o protocolo de cada equipamento; sem ele, nada entra no sistema. É também o ponto em que uma mudança de fabricante custa caro (Capítulo 20);
2. **interface homem-máquina (IHM)** — telas de operação, comandos, sinóticos e tendências. É o único componente que o operador efetivamente usa;
3. **histórico (*historian*)** — armazenamento de longo prazo das variáveis, com carimbo de tempo. Sustenta relatórios, comparação entre turnos e análise de causa;
4. **gerenciador de alarmes e eventos** — recebe, registra, ordena e apresenta o que fugiu do esperado, com reconhecimento pelo operador;
5. **relatórios e integração gerencial** — consolida indicadores e entrega dados ao MES ou ao PIMS, no nível gerencial (Capítulo 3).

O ganho de reunir tudo isso é operacional: o operador deixa de caminhar por centenas de instrumentos distribuídos em uma sala extensa e passa a acompanhar o processo em uma tela, com registro do que aconteceu. O benefício real não é a tela — é o **registro**. Uma planta sem histórico é uma planta que só sabe explicar o incidente por memória de quem estava lá.

17.5 Alarme, evento e intertravamento

Este é o ponto em que a maior parte dos projetos erra, porque as três palavras soam parecidas. Não são, e trocá-las é risco de acidente.

Tabela 17.3: Alarme, evento e intertravamento: quem faz o quê e qual é a consequência do erro.

Função	O que é	Quem executa	Consequência do erro
Intertravamento	Condição que impede a máquina de assumir um estado inseguro, ou que a leva a um estado seguro	A lógica de controle — no CLP, e quando há risco de ferimento, em controlador ou dispositivo dedicado de segurança	Tratar intertravamento como aviso coloca pessoas em risco: o sistema alerta e continua operando
Alarme	Aviso de que um valor ou estado saiu da faixa esperada e exige atenção do operador	O sistema de supervisão, com registro e reconhecimento	Alarme demais faz o operador ignorar todos: é a fadiga de alarmes
Evento	Registro de algo que aconteceu, sem exigir ação imediata: partida, parada, troca de receita	O sistema de supervisão	Sem registro não existe análise de causa depois do incidente

A regra que evita o erro: **a proteção mora no controle, não na supervisão**. Se a única coisa que impede o transbordamento de um tanque é um alarme que o operador precisa ver e reconhecer, não há proteção — há esperança. Intertravamento que depende da rede de supervisão cai junto com a rede de supervisão.

Atenção

Uma rede de supervisão não é determinística: a tela pode demorar, a estação pode travar, o operador pode estar em outro ponto da planta. Nada que precise responder *sempre* em tempo previsível pode depender dela. Essa fronteira é o que separa supervisão de controle e é o mesmo raciocínio que reaparece no Capítulo 18, quando se discute levar dados para a nuvem.

17.6 Boas práticas de projeto de rede

- **separe por função, não por conveniência:** rede de controle, rede de supervisão e rede administrativa são domínios diferentes, com requisitos diferentes (Capítulo 22);
- **documente o endereçamento:** faixa de endereços por área, nome de cada nó, mapa de portas e *switch* usado. Rede não documentada é meia hora perdida em cada manutenção;
- **padronize os nomes** de variáveis publicadas na supervisão, como no Capítulo 14. Um mesmo dado não deve chegar com dois nomes diferentes;

- **defina o comportamento na falha:** o que o CLP faz se perder a comunicação; o que a supervisão mostra quando o dado está velho. Dado sem carimbo de tempo não é dado — é suposição;
- **redundância onde dói:** redundância de rede e de servidor deve ser decidida pelo custo da parada, não por simpatia pela tecnologia.

Na prática

Escolha uma arquitetura de rede que você conheça e monte o desenho dela no formato da Figura 17.1. Depois responda: (a) onde estão as E/S: no rack, em remotas ou em outros CLPs? (b) qual protocolo é usado em cada trecho e quem é mestre em cada um? (c) se o trecho entre o controlador e a supervisão cair por dez minutos, o que a planta faz? (d) quem é avisado, e por qual alarme?

Exercícios

- Explique, em três linhas, por que a rede reduz o custo de instalação em uma área grande.
- Uma máquina tem 40 pontos digitais concentrados no painel e fica a 8 m do quadro de supervisão. Qual arquitetura da Tabela 17.1 você escolhe? Justifique.
- Cite duas situações em que a arquitetura de I/O remoto é claramente mais vantajosa e uma em que ela é um risco.
- Escolha dois protocolos da Tabela 17.2 e diga em que situação real cada um seria a escolha natural.
- Por que saber ler um registro Modbus não garante entender o que ele significa?
- Liste os cinco componentes de um sistema SCADA e diga qual deles é responsável por sustentar um relatório de turno.
- Classifique cada caso como intertravamento, alarme ou evento: (i) porta de proteção aberta com a máquina em movimento; (ii) temperatura da água 5 °C acima da faixa normal; (iii) registro automático da troca de receita de produção; (iv) ausência de confirmação de fim de curso na partida.
- Um colega propõe desligar a bomba de resfriamento “porque o supervisório avisa”. Explique por que a proposta é inaceitável, citando a Tabela 17.3.
- Por que o *driver* de comunicação é o componente do SCADA que mais encarece uma troca de fabricante?
- Escreva três itens que obrigatoriamente devem constar da documentação de rede de uma planta.

Capítulo 18

IIoT, OPC UA e MQTT

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- explicar o que a arquitetura cliente-servidor clássica não resolve;
- descrever o que OPC UA acrescenta a um protocolo de transporte;
- explicar o modelo *publish/subscribe* do MQTT e o papel dos níveis de serviço;
- dizer o que o Sparkplug padroniza e por que isso importa;
- apontar os limites do IIoT: latência, determinismo, custo de rede e borda versus nuvem.

18.1 O que a automação clássica deixou em aberto

O Capítulo 3 mostrou a hierarquia em que cada nível fala com o de cima por um caminho próprio. Esse arranjo funcionou por décadas e tem duas limitações que ficaram caras com o tempo:

1. **o problema $N \times M$:** cada combinação de aplicação e equipamento exige um *driver* próprio. Três sistemas de supervisão e cinco famílias de controladores significam até quinze integrações para manter, atualizar e testar a cada versão de *firmware*;
2. **dado sem significado:** um protocolo de transporte diz *onde* está o valor, não *o que* ele é. Sem essa camada, cada consumidor precisa de uma tabela mantida à mão dizendo que o registro 40001 é a temperatura e que a unidade é grau Celsius.

O conjunto de tecnologias que ataca esses dois problemas ficou conhecido como **IIoT** (*Industrial Internet of Things*): usar padrões de comunicação abertos para levar dado industrial a qualquer consumidor autorizado, dentro ou fora da fábrica, sem que cada consumidor precise conhecer o equipamento que o originou.

18.2 OPC UA: camada de informação, não apenas de transporte

OPC UA (*Unified Architecture*) foi lançado em 2008 pela OPC Foundation como uma arquitetura orientada a serviços, independente de plataforma, que reuniu as especificações anteriores do OPC “clássico” em um único conjunto. Quatro características o distinguem dos protocolos de transporte apresentados no capítulo anterior (OPC Foundation, *Unified Architecture*, especificação disponível publicamente):

- **modelagem de informação:** o dado é exposto como um espaço de endereços hierárquico, com objetos, variáveis e tipos definidos. Um cliente pode *descobrir* a estrutura, em vez de depender de uma tabela externa;

- **assinatura de dados:** o cliente se inscreve em um item e recebe notificação quando o valor muda (relatório por exceção), em vez de perguntar em laço;
- **eventos e métodos:** além de valores, o servidor notifica eventos e pode executar métodos, ou seja, há canal para invocar ações e não só para ler e escrever;
- **sessão segura:** autenticação por certificado, cifragem das mensagens, assinatura e sequenciamento, com registro de auditoria.

O detalhe que interessa a quem compra equipamento é o primeiro item da lista acima. Sobre a base do OPC UA existe um programa de perfis setoriais (*companion specifications*), mantido por organizações de cada segmento industrial. Um perfil publicado define, para um tipo de equipamento, *quais* variáveis devem existir e com que nome — que é exatamente o que faltava no Modbus, como observado no Capítulo 12.

O mesmo padrão também prevê uma variante de publicação e assinatura (PubSub), pensada para comunicação de muitos para muitos, em que o produtor publica sem saber quem consome. Guarde essa distinção: é a mesma ideia que o MQTT explora, por outro caminho.

18.3 MQTT: publicação e assinatura para telemetria

MQTT é um padrão aberto de mensagens mantido pela OASIS (versões 3.1.1 e 5.0), projetado para dispositivos com pouca memória e rede de banda limitada ou instável. O desenho é *publish/subscribe*:

- um **broker** central recebe as mensagens publicadas;
- quem publica envia para um **tópico**, sem saber quem recebe;
- quem consome assina o tópico que lhe interessa, sem saber quem publica.

Esse desacoplamento é a razão do sucesso do MQTT em telemetria. Acrescentar um consumidor novo — um painel de indicadores, uma ferramenta de análise, um sistema de manutenção — não exige tocar no publicador. Em uma arquitetura ponto a ponto, cada consumidor novo é uma integração nova.

Dois recursos merecem atenção:

- **níveis de serviço** (*QoS* 0, 1 e 2, na documentação da OASIS): entregar no máximo uma vez, ao menos uma vez e exatamente uma vez. Escolher o nível é uma decisão de engenharia: contagem de peças tolera repetição; consumo acumulado de energia, não;
- **sessão persistente:** o cliente pode reconectar e receber o que foi publicado durante a ausência. É o que viabiliza o padrão *store and forward* em enlaces que caem, comuns em rede celular e em campo remoto.

18.4 Sparkplug: padronizar o conteúdo, não só o transporte

MQTT padroniza *como* a mensagem viaja, não *o que* ela diz. Sem essa camada, cada integração inventa seu formato de tópico e seu formato de carga útil, e volta-se ao problema $N \times M$ — agora em cima de um protocolo aberto.

O **Eclipse Sparkplug** (versão 3.0, mantida pela Eclipse Foundation) resolve isso definindo, para o mundo industrial, a estrutura dos tópicos e o formato da carga útil, além de dois mecanismos que valem a pena conhecer:

- **certificados de nascimento e de morte:** ao conectar-se, o dispositivo publica seu estado e a lista do que vai informar; ao cair, é possível detectar a ausência. O consumidor sabe *se* o dado é válido, não apenas se ele chegou;
- **consciência de estado** (*state awareness*): a informação de qualidade acompanha o dado, o que permite distinguir “valor parado” de “equipamento desligado”.

18.5 A pilha completa

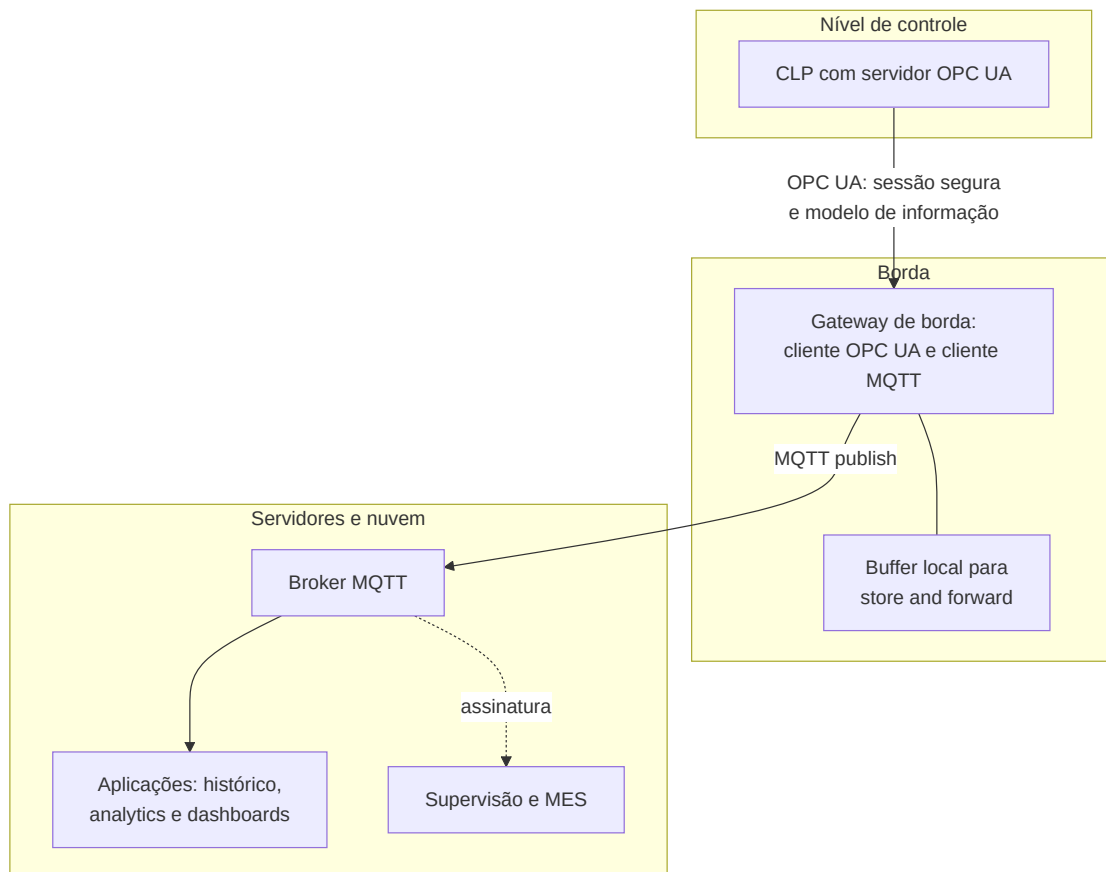


Figura 18.1: Pilha típica de IIoT: controlador, borda, broker e aplicações.

A Figura 18.1 mostra a montagem mais comum:

1. o **controlador** expõe dados por OPC UA. Em instalações antigas, o caminho é um *driver* do protocolo proprietário dentro do gateway;
2. o **gateway de borda** é o tradutor: fala OPC UA (ou Modbus) de um lado e MQTT do outro, mantém um *buffer* local e aplica alguma regra de filtro — mudança de valor, limite de taxa, agregação;
3. o **broker** distribui;
4. as **aplicações** consomem: histórico, painéis, análise, manutenção e, quando fizer sentido, o próprio MES.

Note o que *não* está na figura: nenhum caminho de comando descendo pelo broker até o laço de controle. Essa ausência é deliberada e é o conteúdo da próxima seção.

18.6 Vantagens do IIoT na automação

Tabela 18.1: O que o IIoT entrega, em termos práticos.

Vantagem	Efeito prático
Acesso a mais variáveis	Parâmetros de diagnóstico que já existiam no equipamento e não eram lidos passam a ser visíveis — sem alterar a lógica de controle
Integração desacoplada	Um consumidor novo entra por assinatura, sem alterar o publicador nem o programa do controlador
Dado com contexto	Modelo de informação e carga útil padronizada reduzem a dependência de tabelas de registradores mantidas à mão
Efeito de escala	Muitas máquinas podem ser acompanhadas por um número pequeno de especialistas, em vez de exigir presença física em cada ponto
Histórico para a melhoria	Tendência, consumo e contagem de ciclos alimentam manutenção e eficiência energética, temas dos Capítulos 19 e 23

18.7 Desafios e limites

Se o IIoT fosse apenas vantagem, toda planta já o teria adotado por completo. Não é o caso. Os obstáculos reais são de quatro naturezas:

1. **latência e determinismo.** A cadeia inclui gateway, broker, fila, rede compartilhada e aplicação. Cada elo acrescenta atraso e variabilidade. Um caminho que passa por broker e por rede corporativa não é determinístico — e, por isso, não serve para controle. O critério prático: se a função precisa de prazo garantido, ela fica no controlador;
2. **banda e custo de rede.** Enviar toda variável a cada segundo, de cada máquina, custa tráfego, licença e nuvem. O gateway existe também para isso: filtrar, agregar e transmitir por exceção. Sem essa disciplina, o projeto de IIoT entrega uma conta mensal e não um indicador;
3. **segurança.** Cada caminho novo de leitura é superfície nova de ataque. Um broker acessível a partir da rede corporativa, com credencial padrão e sem TLS, transforma a melhoria em porta de entrada (Capítulo [22](#));
4. **governança do dado.** Quem é o dono da variável? Quem aprova mudar o seu significado? Duas áreas publicando a mesma grandeza com nomes e unidades diferentes produzem dois indicadores e nenhuma decisão.

Atenção

O IIoT amplia a *visão* sobre o processo; não amplia a *autoridade* sobre a máquina. Nada que dependa de prazo garantido — intertravamento, parada de emergência, proteção de máquina — pode depender de broker, de gateway ou de nuvem. A fronteira correta é simples de enunciar: dado sobe, comando de segurança não desce por esse caminho.

O Capítulo 21 retoma essa fronteira ao tratar de tecnologias de TI no chão de fábrica.

18.8 Por onde começar

A ordem que costuma dar certo em planta em operação:

1. **inventariar** o que já existe: quais controladores, quais protocolos, quais dados já são coletados e quem usa cada um;
2. começar por um **indicador que a planta quer** — consumo por linha, disponibilidade de um equipamento crítico — e trabalhar de trás para frente, para saber quais variáveis faltam;
3. **piloto em um equipamento**, com o gateway isolado da rede de controle;
4. só então padronizar, documentando tópicos, nomes e unidades.

Começar pela tecnologia — “vamos instalar a plataforma” — produz projeto bonito e nenhum indicador. Começar pelo indicador produz uma lista de variáveis, que é o que a engenharia precisa.

Na prática

Escolha um equipamento da sua planta ou da sua bancada e responda, por escrito: (a) quais variáveis de processo ele já disponibiliza em rede hoje? (b) quais delas interessariam a um indicador de disponibilidade? (c) qual seria o caminho do dado, no formato da Figura 18.1, até esse indicador? (d) qual dessas variáveis *não* pode ser levada para fora da rede de controle, e por quê?

Exercícios

- a) Explique o problema $N \times M$ com um exemplo numérico de três sistemas e quatro famílias de equipamentos.
- b) Qual das quatro características do OPC UA resolve diretamente o problema do significado do dado? Justifique.
- c) Compare OPC UA e MQTT quanto a: modelo de comunicação, papel do intermediário e adequação a controle de tempo real.
- d) Descreva o caminho de um dado de temperatura desde o CLP até um painel de indicadores na nuvem, nomeando cada elemento da Figura 18.1.
- e) Por que a escolha do nível de serviço (QoS) muda conforme a variável? Dê um exemplo de variável que tolera repetição e um que não tolera.
- f) O que os certificados de nascimento e de morte informam ao consumidor que um simples valor recebido não informa?
- g) Cite dois riscos concretos de ligar o broker MQTT diretamente na rede corporativa.
- h) Um fornecedor propõe fazer o intertravamento de uma prensa “pelo servidor OPC UA, para centralizar a lógica”. Escreva a recusa técnica em cinco linhas.

- i) Escolha uma das vantagens da Tabela 18.1 e diga qual desafio da seção seguinte pode anulá-la se o projeto for mal feito.
- j) Descreva, em quatro passos, como você conduziria um piloto de IIoT em uma máquina isolada.

Capítulo 19

Dados, telemetria e monitoramento em tempo real

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- distinguir dado, informação e indicador, e construir o terceiro a partir dos dois primeiros;
- calcular e interpretar os indicadores usuais de produção e de consumo;
- decidir onde processar cada tarefa: no CLP, na borda ou na nuvem;
- reconhecer os sinais simples de degradação usados em manutenção preditiva;
- dizer, com clareza, o que o controlador pode e o que ele não pode fazer.

19.1 Do dado bruto ao indicador

Os três termos são usados como sinônimos no dia a dia e não são:

- **dado**: o valor lido, com carimbo de tempo. “Contador de peças = 4.812, às 14h32”;
- **informação**: o dado com contexto. “4.812 peças produzidas no turno A, contra 5.100 no mesmo turno de ontem”;
- **indicador**: a informação transformada em medida de desempenho, comparável entre períodos e máquinas. “Eficiência global do equipamento: 71 %”.

Toda a cadeia de monitoramento existe para chegar ao terceiro. Uma planta pode ter milhares de tags e nenhum indicador — e, nesse caso, tem telemetria sem gestão.

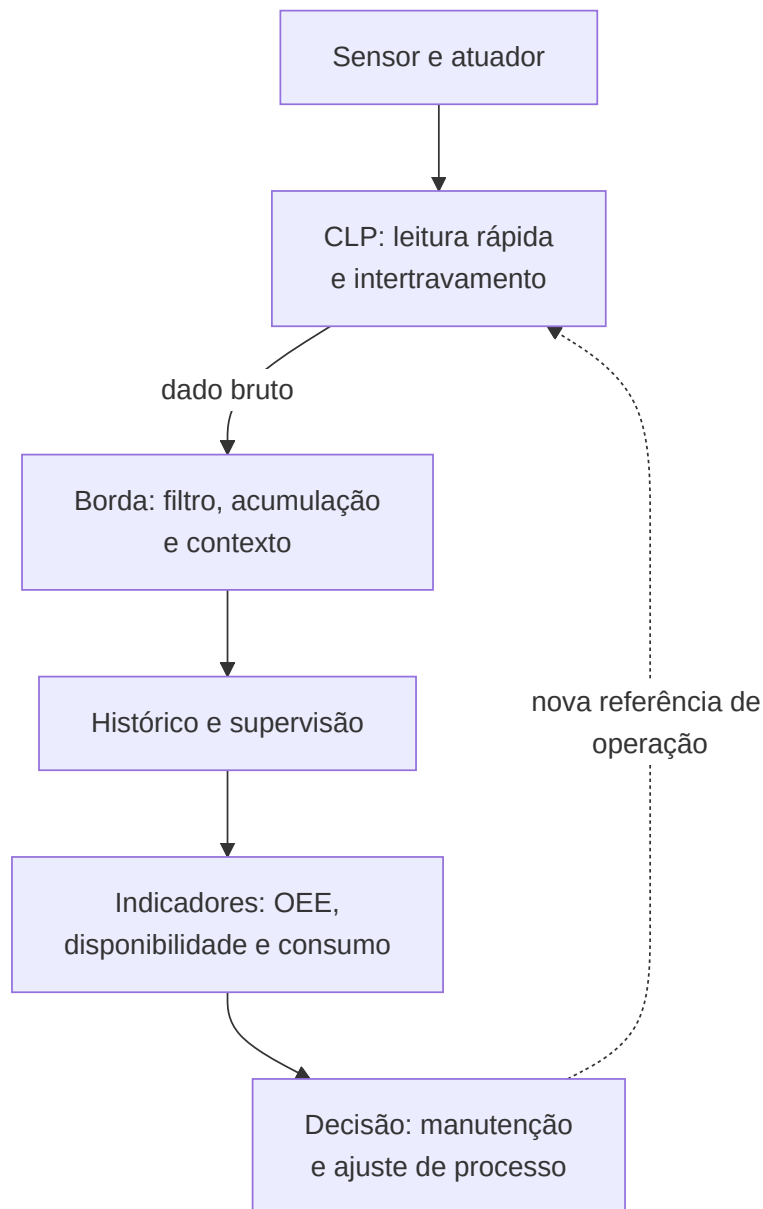


Figura 19.1: Caminho do dado até a decisão, e o retorno da decisão ao processo.

Atenção

A etapa mais esquecida da Figura 19.1 é a última. Indicador que não volta como decisão — mudar um *setpoint*, antecipar uma manutenção, reprogramar um lote — é relatório de gaveta. Antes de instrumentar qualquer coisa, escreva qual decisão depende daquele número.

19.2 Os indicadores usuais

Tabela 19.1: Indicadores usuais de produção e consumo, o que exigem do controle e onde costumam falhar.

Indicador	Definição conceitual	O que o CLP fornece	Onde costuma falhar
OEE (eficiência global do equipamento)	Disponibilidade $\times$ desempenho $\times$ qualidade	Estado da máquina (parada, operando, parada, contagem de peças boas e refugadas, tempo disponível	Classificação de parada mal definida: toda parada vira “outros”, e o indicador deixa de apontar causa
Disponibilidade	Tempo em operação dividido pelo tempo programado	Sinal de máquina em ciclo, tempo em falha	Não distinguir parada planejada de não planejada
Tempo de ciclo	Tempo entre duas peças conforme, ou por etapa do ciclo	Marcações de início e fim, com resolução adequada	Medir no supervisão um fenômeno que acontece em milissegundos: a média esconde a variação
Contagem de produção	Peças ou lotes produzidos por turno, produto e linha	Contador de ciclo, com tratamento de primeira varredura e de retomada	Retenção de contador perdida na queda de energia, ou contagem dupla na partida
Consumo energético	Energia por tonelada, por peça ou por turno	Leitura de medidor por rede ou por pulso; estado de motores e de iluminação	Comparar períodos com produtos diferentes e concluir que a eficiência mudou (Capítulo 23)

Uma observação sobre o OEE: ele é útil exatamente porque separa disponibilidade, desempenho e qualidade. Quando combinado em um único número sem detalhamento, perde a capacidade de apontar *onde* intervir. O indicador agregado é para a reunião de gestão; o detalhamento é para a engenharia.

19.3 Onde processar: CLP, borda ou nuvem

Esta é a decisão de arquitetura que define o custo e a utilidade do sistema.

Tabela 19.2: Critérios para decidir onde processar cada tarefa.

Critério	No CLP	Na borda	Na nuvem ou na sala de controle
Prazo	Prazo garantido, em milissegundos	Responde em centenas de milissegundos a segundos	Prazo não garantido; depende de banda e de fila
Volume	Baixo: o que cabe na memória de trabalho	Intermediário: filtra e agrega antes de enviar	Alto: meses ou anos de histórico de muitas máquinas
Criticidade	Só o que não pode parar a máquina	O que melhora a operação sem risco imediato	O que orienta decisão de dias ou meses
Manutenção	Exige parada e equipe de automação	Atualizável em janela curta	Atualizável continuamente, por equipe de TI

Lendo a Tabela 19.2 de cima para baixo chega-se à regra prática: **leitura rápida e intertravamento no controlador; transformação e contexto na borda; acúmulo e análise pesada fora do chão de fábrica.**

Essa divisão também é uma proteção. Cada função que sai do controlador deixa de depender do determinismo dele — e, em troca, ganha capacidade de processamento. Mover a decisão certa para o lugar errado é a origem tanto de máquinas inseguras quanto de indicadores inúteis.

19.4 Detecção de desvio e manutenção preditiva simples

Nem toda manutenção preditiva exige aprendizado de máquina. Quatro recursos simples, implementáveis em lógica de controlador ou em ferramenta de histórico, cobrem boa parte dos casos:

1. **limite absoluto:** o valor passou de um patamar. É o alarme clássico;
2. **taxa de mudança:** a variável está subindo mais rápido que o normal. Um aumento lento e persistente de corrente em um motor indica desgaste ou obstrução, mesmo dentro da faixa “normal”;
3. **contagem de ciclos:** o atuador número n vai superar a vida declarada. Simples, verificável e diretamente acionável;
4. **desvio do padrão:** o tempo de uma etapa do ciclo cresceu 8% em relação à referência do mesmo produto, sem que nenhum limite tenha sido atingido.

Os dois últimos são os mais valiosos, porque antecipam a falha sem depender de um limite fixo — que existe justamente para não ser atingido em operação normal.

Atenção

Detecção de desvio sem contexto gera alarme falso, e alarme falso ensina o operador a ignorar alarmes. Antes de publicar um indicador de degradação, defina três coisas: a referência, a banda de tolerância e o que fazer quando o desvio aparecer. Um indicador

que ninguém sabe como usar é ruído com carimbo de tempo.

19.5 O limite do controlador

Vale ser explícito, porque é o erro mais comum em projetos de monitoramento:

- o CLP faz **aquisição rápida**, lógica de **intertravamento**, contagem e temporização com precisão conhecida — e faz isso sem depender de rede;
- o CLP **não** é o lugar de treinar modelo, cruzar meses de histórico, gerar relatório gráfico ou calcular indicador que depende de dados de várias máquinas. Havia memória contada para isso no Capítulo 6, e a memória de trabalho do controlador precisa sobrar para o programa de controle;
- o critério de fronteira é o **prazo**: o que precisa de resposta garantida fica no controlador; o resto sai.

19.6 Qualidade do dado

Indicador calculado sobre dado ruim é pior que nenhum indicador, porque produz confiança injustificada. Quatro cuidados resolvem a maior parte dos problemas:

1. **carimbo de tempo no ponto de origem**, não na chegada. Em rede não determinística, o instante da chegada não é o instante do evento;
2. **qualidade acompanhando o valor**: distinguir “zero” de “sem leitura”. São estados diferentes e exigem decisões diferentes;
3. **resolução e período de amostragem coerentes** com o fenômeno (Capítulo 7). Amostrar devagar um evento rápido produz média enganosa;
4. **unidade e escala documentadas** na lista de tags (Capítulo 15). Duas áreas usando escalas diferentes para a mesma grandeza geram dois indicadores contraditórios — e a discussão que ninguém quer ter.

Na prática

Escolha uma máquina da sua planta ou bancada e construa um indicador de disponibilidade: (a) liste as variáveis necessárias e diga se cada uma já existe ou precisa ser criada; (b) defina as categorias de parada e quem vai classificá-las; (c) monte o cálculo na Tabela 19.1; (d) decida, pela Tabela 19.2, onde cada cálculo roda; (e) escreva qual decisão será tomada com o resultado.

Exercícios

- a) Dê um exemplo de dado, um de informação e um de indicador para uma linha de envase.
- b) Por que o OEE detalhado é mais útil para a engenharia que o OEE agregado?
- c) Uma máquina produziu 4.500 peças em 480 minutos programados, dos quais 40 foram de parada por falha e 20 de troca de produto planejada. Calcule a

disponibilidade e diga que decisão a diferença entre as duas paradas permite tomar.

- d) Explique por que medir tempo de ciclo no supervisório pode esconder o problema que se quer investigar.
- e) Aplique a Tabela 19.2 e decida onde rodar: (i) intertravamento de porta; (ii) média de consumo por turno; (iii) comparação de consumo entre meses; (iv) contagem de peças para o indicador de produção.
- f) Escreva a regra de fronteira entre CLP e borda em uma frase, e aplique-a a um caso seu.
- g) Descreva dois indicadores de degradação que se pode extrair apenas de contagem de ciclos e de tempo de etapa, sem sensor novo.
- h) Por que o carimbo de tempo deve ser gerado na origem? Dê um exemplo em que a diferença importa.
- i) Qual é a diferença prática entre “valor zero” e “sem leitura” em um indicador de consumo?
- j) Um colega propõe calcular o OEE de dez máquinas dentro de um dos CLPs. Aponte dois problemas técnicos com base no Capítulo 6.

Capítulo 20

Interoperabilidade entre marcas e modelos

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- distinguir compatibilidade declarada, interoperabilidade e portabilidade;
- aplicar as estratégias que realmente reduzem dependência de fabricante;
- localizar, no projeto, os pontos em que a migração quebra;
- classificar o risco de uma migração antes de orçá-la;
- escrever exigências contratuais que preservem a liberdade futura da planta.

20.1 Três palavras que não são sinônimos

No material de fornecedor, as três aparecem trocadas:

- **compatibilidade declarada:** o fabricante afirma que o produto atende a uma norma. É uma declaração, não uma verificação independente — e cabe perguntar a qual edição da norma ela se refere (Capítulo 12);
- **interoperabilidade:** equipamentos diferentes conseguem trocar dados em operação. Depende de protocolo e de perfil de informação, não de linguagem de programação;
- **portabilidade:** o mesmo *programa* roda em outro ambiente com esforço previsível. É a propriedade mais difícil das três, porque depende de tudo o que a norma *não* padroniza.

Interoperabilidade resolve o problema de *conversar*. Portabilidade resolve o problema de *migrar*. São problemas distintos e frequentemente confundidos em conversa de especificação.

20.2 A escada da portabilidade

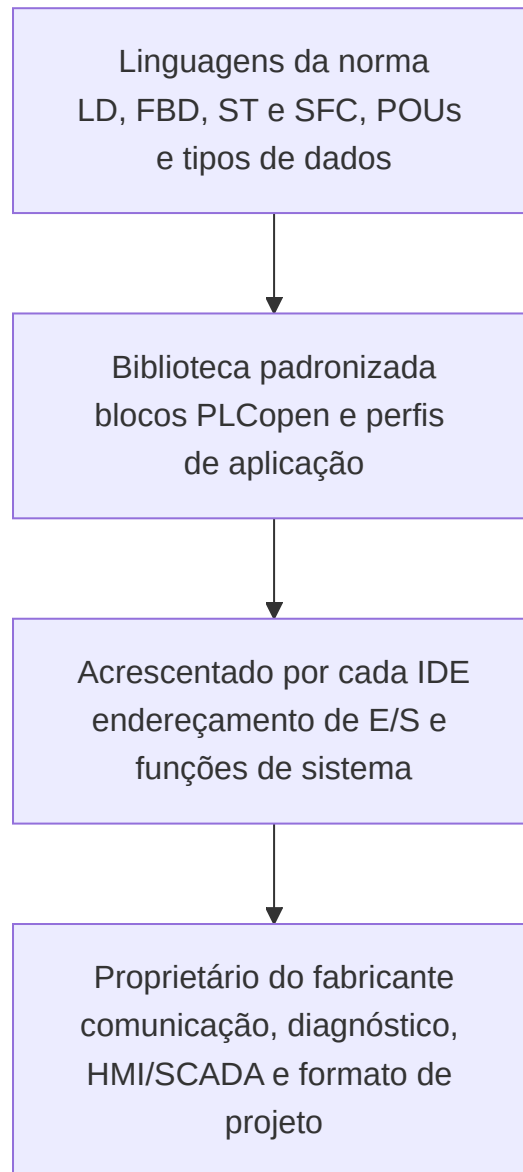


Figura 20.1: O que é portátil e o que cada ambiente acrescenta por conta própria.

A Figura 20.1 mostra que a portabilidade não é uma propriedade binária: ela diminui à medida que se desce na pilha. No topo, linguagens, POUs e tipos de dados são padronizados e viajam bem. Nos degraus seguintes, cada ambiente acrescenta o seu — e é aí que o custo aparece.

O Capítulo 13 já separou os problemas de linguagem dos problemas de estrutura e mostrou que os primeiros são mecânicos e os segundos não. Este capítulo trata os segundos.

20.3 Estratégias que funcionam

1. **escrever conforme a norma, sempre que houver equivalente padronizado.** É a decisão de maior efeito e de menor custo, tomada no dia da escrita — não no dia da migração (Tabela 13.3);
2. **usar biblioteca padronizada**, e não blocos proprietários, para as funções que existem em versão normatizada. Bibliotecas setoriais de blocos de função existem justamente para isso (Capítulo 12);
3. **isolar o que é específico:** endereçamento, diagnóstico de rack, comunicação e qualquer função de sistema devem ficar concentrados em poucos blocos, com nome e comentário explícitos;
4. **preferir texto estruturado** para lógica complexa. Ladder é excelente para lógica combinacional e para quem lê em campo, mas algoritmos longos ficam mais legíveis e mais facilmente transportáveis em ST;
5. **usar protocolo aberto como ponte** quando o código não é portátil. Nem sempre vale reescrever o programa: muitas vezes a integração se resolve mantendo os dois controladores e ligando-os por OPC UA (Capítulo 18) ou Modbus TCP (Capítulo 17);
6. **usar o formato de intercâmbio** previsto na família da norma para exportar e importar projetos entre ambientes, quando ambos o implementarem. Ele transporta a estrutura, não as bibliotecas proprietárias.

Atenção

As estratégias 1 a 4 custam quase nada se aplicadas desde o início e custam muito quando aplicadas depois. Reescrita de programa em operação consome parada de produção, teste e revalidação, comissionamento e risco de introduzir defeito em código que já funcionava. Quem decide sobre arquitetura de software conversa com quem decide sobre orçamento de manutenção — e raramente são a mesma pessoa.

20.4 Onde a migração realmente quebra

Tabela 20.1: Pontos de ruptura em uma migração de fabricante e o que fazer com cada um.

Ponto de ruptura	Por que quebra	O que reduz o impacto
Endereçamento físico de E/S	Cada família organiza e nomeia os pontos de um jeito. A norma não padroniza endereçamento	Nomear variáveis pelo processo, nunca pelo endereço; manter tabela de alocação (Capítulo 15)
Blocos de sistema	Leitura de estado do módulo, diagnóstico de rack e tratamento de falha são próprios de cada ambiente	Concentrar em um bloco de adaptação, documentado; testar esse bloco primeiro na migração
Temporização e contagem	Parâmetros, base de tempo e comportamento de acumulação variam entre ambientes	Declarar explicitamente base de tempo e comportamento esperado; testar com cronômetro, não por leitura de código
Comunicação entre controladores	Blocos de troca de dados proprietários não têm equivalente direto	Padronizar em protocolo aberto desde o projeto, ou em camada de gateway
HMI e SCADA	Telas, animações e <i>scripts</i> ficam presos ao produto e à versão	Separar o endereçamento da tela do endereço físico, com camada de tags intermediária
Formato de projeto e bibliotecas	Arquivo, versionamento e bibliotecas são do fabricante	Exigir entrega do projeto-fonte e exportação em formato aberto; manter o fonte em versionamento próprio
Peças e ciclo de vida	Migração forçada por falta de reposição é a mais cara de todas	Acompanhar avisos de descontinuação; planejar substituição antes de a parada ser obrigatória

20.5 Classificando o risco da migração

Nem toda migração é igual. A Tabela 20.2 dá uma escala de trabalho que ajuda a dimensionar esforço antes de orçar.

Tabela 20.2: Cenários de migração e esforço típico.

Cenário	Risco	Por quê
Troca de modelo dentro da mesma família, com o mesmo ambiente de programação	Baixo	Endereçamento, blocos e formato de projeto permanecem; o esforço é de conferência e teste de compatibilidade de <i>firmware</i>
Troca de família com o mesmo fabricante e o mesmo ambiente	Médio	A estrutura de projeto e a biblioteca mudam; a linguagem, em geral, não
Troca de fabricante, mantendo o projeto funcional e a arquitetura de rede	Alto	Reescrever aplicativo, refazer endereçamento, revalidar comunicação e reescrever a interface com a supervisão
Troca de fabricante com substituição de HMI, supervisão e rede de campo	Muito alto	É um projeto novo, com comissionamento novo; deve ser orçado como tal, e não como “atualização de controlador”

Não existe, nesta tabela, promessa de compatibilidade entre marcas. Existe uma escala de esforço verificável no campo: quanto mais camadas mudam ao mesmo tempo, mais caro fica — e é justamente por isso que quem migra ganha ao migrar por partes.

20.6 O que exigir na especificação e no contrato

Toda liberdade futura da planta é decidida no momento da compra. Do que foi discutido até aqui, quatro exigências resolvem a maior parte do problema (ver também o Capítulo 8):

1. **entrega do projeto-fonte**, não apenas do arquivo executável carregado no controlador, com a versão do ambiente de programação identificada;
2. **declaração de conformidade com edição e ano** da norma, e a informação de quais linguagens e bibliotecas foram efetivamente usadas;
3. **lista de blocos proprietários** utilizados no aplicativo, com o que cada um faz — essa lista é o orçamento da migração futura;
4. **exportação em formato de intercâmbio** sempre que o ambiente oferecer, e a tabela de tags em formato aberto (planilha ou texto), nunca apenas dentro do projeto.

Na prática

Escolha um sistema real com pelo menos um controlador, uma IHM e uma rede de campo. Produza um relatório de portabilidade com: (a) a lista de blocos proprietários usados no programa, com a função de cada um; (b) a classificação do cenário de migração na Tabela 20.2; (c) os três pontos de ruptura da Tabela 20.1 que exigiriam mais trabalho no seu caso, justificando; (d) as exigências contratuais que faltaram na compra original.

Exercícios

- a) Explique a diferença entre interoperabilidade e portabilidade com um exemplo de cada.

- b) Por que a portabilidade diminui à medida que se desce na Figura 20.1?
- c) Um projeto usa apenas LD e blocos TON padronizados, mas lê o estado dos módulos por bloco proprietário. Classifique o risco de migração e justifique.
- d) Cite dois casos em que manter dois controladores ligados por protocolo aberto é mais vantajoso que reescrever o programa.
- e) Por que nomear variáveis pelo processo, e não pelo endereço, sobrevive à migração?
- f) Explique o risco de temporização e diga por que a conferência precisa ser feita com cronômetro.
- g) Classifique na Tabela 20.2: (i) troca de um controlador por outro do mesmo modelo, por fim de vida; (ii) substituição completa de um painel, com controlador, IHM e rede nova; (iii) troca de fabricante mantendo a IHM existente.
- h) Justifique, com base na Tabela 20.1, por que a migração forçada por falta de peça é a mais cara.
- i) Escreva três cláusulas de especificação para a compra de um controlador, cada uma ligada a um ponto de ruptura.
- j) Um fornecedor afirma: “nosso controlador é conforme IEC 61131-3, então seu programa roda em qualquer marca”. Escreva a resposta técnica em cinco linhas.

Capítulo 21

Convergência TI/TO

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- explicar por que as práticas de TI e de tecnologia de operação (TO) divergem;
- dizer o que roda dentro do controlador, o que roda na borda e o que roda fora da fábrica;
- avaliar o uso de visão computacional, de modelos de linguagem pequenos, de plugins, de Node-RED, de código gerado a partir de modelos e de Linux embarcado;
- reconhecer, em cada tecnologia, o ganho, a perda e a exigência que ela traz.

21.1 Duas culturas, um mesmo sistema

O chão de fábrica e o data center passaram a compartilhar rede, dados e até hardware. As práticas, porém, continuam diferentes — e é essa diferença que explica a maior parte dos atritos de projeto.

Tabela 21.1: Tecnologia da informação (TI) e tecnologia de operação (TO), lado a lado.

Aspecto	TI	TO
Prioridade	Confidencialidade e integridade do dado	Disponibilidade e segurança física do processo
Tempo	Melhor esforço; a resposta pode variar	Prazo garantido, em milissegundos, para as funções de controle
Ciclo de vida	Atualização frequente, reinício aceitável	Equipamento em operação por muitos anos; reinício é parada de produção
Mudança	Implantação contínua, versão nova a cada semana	Mudança planejada, com janela, teste e aprovação
Quem opera	Equipe de infraestrutura, à distância	Equipe de automação e manutenção, no local

Convergência TI/TO não é escolher uma das colunas da Tabela 21.1. É decidir, item por item, onde cada prática vale — e reconhecer que “atualizar tudo automaticamente” e “prazo garantido de 2 ms” são requisitos incompatíveis no mesmo componente.

21.2 Quem roda onde

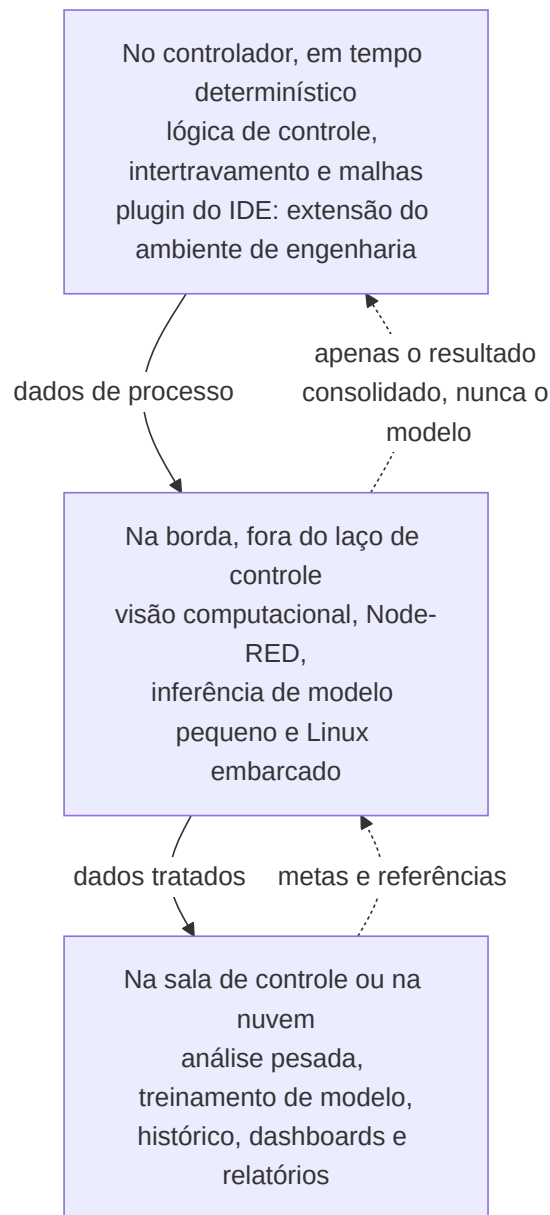


Figura 21.1: Onde cada tecnologia se encaixa, do laço de controle à nuvem.

A Figura 21.1 organiza o capítulo inteiro. A regra que ela representa:

- no **controlador** fica o que é determinístico e o que protege a máquina. Só entra ali o que tem prazo previsível e comportamento verificável;
- na **borda** — um computador industrial, um gateway ou um módulo de processamento no painel — ficam as funções que precisam de mais poder de cálculo e admitem latência maior: imagem, integração de protocolos, inferência leve;
- **fora da fábrica** ficam o acúmulo de longo prazo e a análise pesada, que não participam de nenhum laço de controle.

O retorno de informação da borda para o controlador existe, mas passa por um filtro: chega o *resultado* (uma referência, um valor de correção, um indicador), nunca o modelo nem o julgamento automático de uma função não determinística.

21.3 Visão computacional e processamento de imagem

Inspeção por imagem deixou de ser exclusividade de grandes linhas. Um conjunto moderno tem quatro partes: a **câmera** industrial (padrões de interface como GigE Vision e USB3 Vision existem justamente para desacoplar câmera de software), a **iluminação**, a **óptica** e o **processamento**.

Os usos típicos são de verificação, não de medição de precisão:

- presença e ausência de componente, tampa, rótulo ou parafuso;
- posição e orientação de peça antes do manuseio;
- leitura de código de barras e de código bidimensional, com conferência contra a ordem de produção;
- verificação de cor e de aspecto superficial, com limite de aceitação definido.

Para o processamento, a biblioteca aberta OpenCV é a base mais difundida; softwares de inspeção comerciais montam sobre ela ou sobre bibliotecas próprias.

O ponto de integração com o CLP é mais simples do que parece: o resultado da inspeção é um **sinaleiro discreto** — passa, não passa, peça presente — entregue por uma saída do sistema de visão para uma entrada do controlador, ou por variável publicada em OPC UA (Capítulo 18). O CLP decide o que fazer com o resultado; não processa a imagem.

Atenção

Em visão computacional, o resultado depende mais de *iluminação e fixação* do que do algoritmo. Uma câmera excelente com iluminação instável produz reprovação intermitente, e o operador aprende a desligar a inspeção. Antes de escolher biblioteca, resolva o gabarito mecânico e a luz — e defina o que o sistema deve fazer quando não consegue decidir.

21.4 Modelos de linguagem pequenos no ambiente de controle

Modelos de linguagem de pequeno porte (*small language models*, SLM) são modelos compactos, que podem ser executados localmente, sem depender de serviço externo. É a forma mais realista de levar capacidade de linguagem para perto do processo, porque mantém o dado dentro da planta.

O que é viável hoje. Funções de apoio, com o resultado passando por confirmação humana:

- **ajuda ao operador:** responder, em linguagem natural, sobre procedimento, alarme ou significado de uma variável, a partir de documentação e do histórico da própria planta;
- **diagnóstico assistido:** receber o conjunto de alarmes e estados no momento da falha e sugerir causas prováveis, com base em casos anteriores registrados;
- **geração de relatório:** transformar dados de turno em texto de ocorrência, para revisão e assinatura;

- **apoio à engenharia:** explicar um trecho de código, sugerir nome de variável, resumir diferenças entre duas revisões de programa.

O que é irrealista. Inferência de modelo dentro do ciclo de varredura. Um modelo, mesmo pequeno, responde em frações de segundo a alguns segundos e ocupa memória na casa de centenas de megabytes a alguns gigabytes — uma a três ordens de grandeza acima do que o ciclo de controle admite (Capítulo 9). Além do tempo, faltam as propriedades que o controle exige: comportamento determinístico, resultado reproduzível e verificável, e rastreabilidade da decisão.

Atenção

Nenhuma função de segurança, intertravamento ou proteção de máquina pode depender de modelo de linguagem. Um modelo pode errar com fluência: produz respostas plausíveis e incorretas, sem sinalizar dúvida. Colocar isso dentro do laço de controle significa aceitar que a máquina tome decisões imprevisíveis — e que ninguém consiga explicar depois por que ela tomou. Se a IA vai ajudar, que ajude *o operador* e *o engenheiro*, com o resultado registrado, conferido e assinado por gente.

Duas questões de governança precisam ser respondidas antes de instalar qualquer assistente:

1. **quem valida a resposta** e como o erro é detectado, antes que vire ação;
2. **o que sai da planta.** Enviar dados de processo, receitas e parâmetros para um serviço externo é decisão de segurança da informação, não de engenharia de automação (Capítulo 22).

21.5 Plugins e extensões no ambiente de engenharia

Ambientes de programação aceitam extensões: bibliotecas de blocos, verificadores de padronização, geradores de documentação e integrações com ferramentas de versionamento. O efeito prático é grande — por exemplo, um verificador que aplica automaticamente o checklist do Capítulo 14 encontra problemas antes da revisão humana.

O que se ganha e o que se perde:

- **ganha:** produtividade, padronização e geração de documentação a partir do próprio projeto;
- **perde:** portabilidade. O plugin é do ambiente, não da norma; um projeto que depende de extensão proprietária não migra nem para outra versão, em alguns casos;
- **exige:** política de versão. Qual versão do plugin foi usada para gerar aquele programa? Sem essa informação, o projeto deixa de ser reconstruível. É o mesmo problema de reprodutibilidade que existe em qualquer *build* de software, com a diferença de que aqui o artefato controla uma máquina.

Há ainda a dimensão de segurança: um plugin executa código dentro do ambiente que fala diretamente com o controlador. Ele deve ser tratado como software de terceiro em rede industrial — origem conhecida, versão controlada e instalado com aprovação.

21.6 Node-RED

Node-RED é uma ferramenta de programação visual de fluxos, construída sobre Node.js, muito usada em borda industrial. Cada nó faz uma coisa — ler um protocolo, transformar um valor, publicar em um tópico, enviar um aviso — e o programador liga os nós.

Os usos em que ela se destaca:

- **prototipagem rápida** de integração, para provar que o dado chega e tem o significado esperado;
- **gateway de protocolo**, traduzindo entre Modbus, OPC UA e MQTT (Capítulos 17 e 18);
- **orquestração de avisos**: consolidar condições e notificar a manutenção;
- **painéis simples** de acompanhamento.

E os limites, que precisam estar claros antes de a ferramenta virar infraestrutura crítica:

1. **não é ambiente determinístico**. Roda sobre um sistema de propósito geral, com *garbage collector* e rede compartilhada; o tempo de resposta varia;
2. **não substitui lógica de segurança**. Nenhum intertravamento pode morar em um fluxo Node-RED, ainda que funcione perfeitamente em teste;
3. **exige disciplina de manutenção**. Fluxo visual sem versionamento, sem teste e sem comentário vira “caixa preta” que ninguém se atreve a alterar — o mesmo problema que a documentação do Capítulo 15 procura evitar;
4. **herda os riscos do ambiente**. Portas abertas, credenciais padrão e módulos de terceiros desatualizados são o caminho mais comum para um incidente.

A forma segura é tratar Node-RED como parte da camada de *informação*: ele lê, transforma e publica. Ele não comanda proteção.

21.7 Geração de código a partir de modelagem e simulação

Ambientes de modelagem e simulação — MATLAB/Simulink é o exemplo mais difundido — permitem descrever o comportamento do sistema, simular antes de embarcar e gerar código para o controlador. O uso típico no curso técnico e na indústria é:

1. modelar a planta e o controlador;
2. testar o algoritmo em simulação, com cenários difíceis, sem risco;
3. gerar código (texto estruturado ou linguagem de mais baixo nível, conforme o ambiente);
4. embarcar, testar em bancada e depois em campo.

O ganho é real: algoritmos de controle que seriam penosos de escrever à mão — filtros, controle preditivo, escalonamento de ganhos — passam a ser testáveis antes de tocar a máquina.

Os limites também são reais:

- **código gerado precisa ser revisto.** Ele é correto em relação ao modelo, e o modelo pode estar errado em relação à planta;
- **rastreabilidade:** é preciso guardar o modelo, a versão do gerador e o código resultante. Alterar o código à mão e perder o vínculo com o modelo é caminho sem volta;
- **nem todo ambiente aceita** o código gerado, ou o aceita com restrições de desempenho e de uso de memória;
- **uso em função de segurança** exige processo próprio de verificação, com a norma de segurança funcional aplicável (Capítulo 22) — a geração automática não substitui a validação.

21.8 Linux em sistemas embarcados e controladores

Distribuições Linux viraram base de computadores industriais, gateways de borda, IHMs avançadas e de controladores que executam *soft-PLC* — o controlador implementado em software sobre hardware de propósito geral. As razões são práticas: ecossistema amplo, isolamento por contêineres, virtualização, atualização gerenciada e integração natural com ferramentas de TI.

O que se encontra no equipamento, porém, raramente é a distribuição de um computador pessoal. O usual é uma **imagem sob medida**, gerada pelo fabricante com apenas os componentes necessários. A ferramenta mais difundida para isso é o *Yocto Project*, com a camada de metadados OpenEmbedded: em vez de instalar um sistema completo e retirar o que sobra, descrevem-se *receitas* e o resultado é uma imagem mínima, reproduzível e versionada. É o caminho adotado por vários fabricantes de controladores — os controladores PLCnext da Phoenix Contact, por exemplo, executam uma distribuição Linux gerada com o Yocto, conforme a documentação do fabricante.

A diferença não é só de vocabulário. Uma imagem sob medida chega a campo sem gerenciador de pacotes aberto ao usuário, sem serviços que ninguém pediu e com uma lista de componentes que se pode auditar: é menos superfície exposta (Capítulo 22) e menos coisa para quebrar numa atualização. Em troca, acrescentar um *driver* ou um programa novo deixa de ser um comando e passa a depender da cadeia de compilação cruzada do fabricante — negocia-se escopo, versão e prazo, e não se resolve na madrugada anterior à partida. Vale saber sobre que base roda o seu controlador antes de prometer uma integração que exija pacote de terceiro.

O ponto que interessa à automação é o **determinismo**. Linux comum prioriza aproveitamento e não prazo: a resposta pode variar conforme a carga. Núcleos com preempção de tempo real reduzem bastante essa variabilidade e são adequados a muitas tarefas de controle. Ainda assim, o pior caso de um sistema de propósito geral com *soft-PLC* não é o mesmo de um controlador dedicado com sistema operacional de tempo real, e a diferença está justamente no que a norma de segurança funcional exige demonstrar.

Tabela 21.2: Usos de Linux na automação e o cuidado correspondente.

Uso	É adequado?	Cuidado
Gateway de borda, integração e histórico local	Sim, é o uso mais comum	Manter atualizado e sem serviços desnecessários expostos
IHM e supervisão em computador industrial	Sim	Isolar de outras funções; não acumular servidor de arquivos e supervisão na mesma máquina
<i>Soft-PLC</i> em controle de máquina	Depende do requisito de prazo	Medir o pior caso e verificar contra o requisito do processo, não contra a média
Função de segurança	Só com hardware e sistema dedicados e certificados	Não usar computador de propósito geral como elemento de segurança

21.9 O balanço: o que ganha, o que perde, o que exige

Tabela 21.3: Balanço de cada tecnologia de TI trazida para a operação.

Tecnologia	O que ganha	O que perde	O que exige
Visão computacional	Inspeção automática de características difíceis de medir por sensor	Determinismo do resultado, se a iluminação ou a peça variar	Gabarito mecânico, iluminação estável e critério de decisão escrito
SLM na instalação	Apoio ao operador e ao engenheiro; dado não sai da planta	Confiabilidade da resposta; nenhuma garantia de prazo	Ficar fora do laço de controle, com resultado conferido e registrado
Plugin no IDE	Produtividade, padronização e documentação automática	Portabilidade do projeto	Política de versão e aprovação de software de terceiro
Node-RED	Integração rápida entre protocolos e prototipagem	Determinismo e rastreabilidade	Versionar os fluxos e nunca usá-lo para proteção
Código gerado por modelo	Algoritmo complexo testado em simulação antes de embarcar	Vínculo com o modelo, se o código for editado à mão	Rastreabilidade entre modelo, gerador e código embarcado
Linux embarcado	Flexibilidade, atualização e integração com TI	Pior caso de tempo, frente a um controlador dedicado	Medir o pior caso e mantê-lo atualizado

Nenhuma linha da Tabela 21.3 é gratuita: cada tecnologia se paga em um contexto e cobra em outro, e a decisão depende do que a máquina precisa garantir — prazo, segurança ou simplesmente sair do papel.

Na prática

Escolha uma das tecnologias da Tabela 21.3 e prepare uma proposta de uso em uma máquina real, respondendo: (a) qual problema concreto ela resolve, medido em indicador? (b) em qual camada da Figura 21.1 ela vai rodar, e por quê? (c) qual é o pior caso de tempo aceitável, e como você o mediria? (d) o que acontece com a máquina se essa tecnologia falhar ou parar de responder? (e) quem mantém isso depois de você sair?

Exercícios

- a) Explique, em três linhas, por que atualização automática e prazo garantido são requisitos incompatíveis no mesmo componente.
- b) Localize na Figura 21.1 onde roda um verificador de padrões de código e onde roda um modelo de linguagem que resume alarmes.
- c) Cite quatro usos típicos de visão computacional em linha de montagem e diga qual deles é o mais fácil de implantar.
- d) Por que, em visão computacional, iluminação e fixação importam mais que o algoritmo?
- e) Como o resultado de uma inspeção por imagem chega ao CLP? Descreva dois caminhos.
- f) Liste três usos viáveis de SLM em uma planta e explique, para cada um, onde entra a confirmação humana.
- g) Explique por que a inferência de modelo não cabe no ciclo de varredura, citando tempo e memória.
- h) Um integrador propõe usar um assistente de IA para decidir o bloqueio de uma prensa. Escreva a recusa técnica, com dois argumentos.
- i) Cite dois ganhos e duas perdas de adotar plugin de terceiro no ambiente de engenharia.
- j) Por que Node-RED não pode executar intertravamento, mesmo que o teste tenha funcionado?
- k) Descreva o problema de rastreabilidade criado quando alguém edita à mão um código gerado automaticamente.
- l) Aplique a Tabela 21.2 a um computador industrial que já executa supervisão e passa a executar também *soft-PLC* de uma máquina. Justifique sua decisão.
- m) Escolha uma linha da Tabela 21.3 e escreva a exigência técnica que o “o que exige” representa, em forma de cláusula de especificação.

Capítulo 22

Segurança cibernética em automação industrial

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- distinguir segurança funcional (*safety*) de segurança cibernética (*security*);
- aplicar os conceitos de zona, conduto e nível de segurança da série IEC 62443;
- reconhecer a superfície de ataque de um controlador industrial;
- escolher contramedidas proporcionais ao risco de cada zona;
- descrever um plano de resposta a incidente sem imprevisto.

22.1 Safety e security: duas palavras, dois problemas

O português usa “segurança” para as duas coisas, e essa coincidência esconde uma diferença que decide projetos.

Tabela 22.1: Segurança funcional e segurança cibernética.

Aspecto	Segurança funcional (<i>safety</i>)	Segurança cibernética (<i>security</i>)
Protege contra	Falha do sistema que cause dano a pessoas, ao ambiente ou ao patrimônio	Ação humana intencional que explore uma vulnerabilidade
Pergunta central	O sistema falha de forma segura?	Alguém de fora consegue fazer o sistema fazer o que não deveria?
Base normativa usual	Séries IEC 61508 (genérica) e IEC 61511 (indústria de processo), com níveis de integridade de segurança (SIL)	Série IEC 62443, para sistemas de automação e controle; ISO/IEC 27001 para o sistema de gestão corporativo
Métrica típica	Probabilidade de falha perigosa; nível SIL	Nível de segurança (SL) por zona, e requisitos atendidos
Documento típico	Análise de risco e matriz de causa e efeito dos intertravamentos	Análise de risco por zona, arquitetura de rede e plano de resposta

As duas se cruzam em um ponto perigoso, e a Tabela 22.1 resume a distinção antes de detalhá-la: **um incidente cibernético pode anular a proteção funcional**. Se um invasor altera a lógica que desliga a máquina, o sistema perde a característica de segurança que havia

sido projetada e validada. É por isso que proteção de máquina não se resolve só com projeto de *safety*: ela depende também de quem consegue escrever no controlador.

22.2 A base normativa: a série IEC 62443

A série **IEC 62443** trata da segurança cibernética de sistemas de automação e controle industriais. Ela nasceu do comitê ISA99, em 2002, e passou a ser desenvolvida em conjunto com o grupo de trabalho IEC TC65 WG10; hoje é publicada pela ISA como ANSI/ISA 62443 nos Estados Unidos e pela IEC como IEC 62443 internacionalmente, com conteúdo técnico equivalente quando as duas organizações aceitam o texto. A IEC a reconhece como norma horizontal: normas setoriais de segurança em tecnologia de operação devem usá-la como base.

Os conceitos que interessam ao profissional de automação:

- **papéis:** o proprietário do ativo, o fornecedor do produto e o prestador de serviço têm responsabilidades diferentes. “Segurança cibernética é responsabilidade do fabricante” é uma frase falsa em qualquer um dos três casos;
- **zonas e condutos:** a instalação é dividida em zonas — agrupamentos de ativos com requisitos de segurança comuns — e os condutos são os caminhos de comunicação autorizados entre zonas. É o conceito que substitui a ideia de “rede plana”, em que tudo fala com tudo;
- **níveis de segurança (SL 0 a 4):** indicam a resistência exigida contra classes distintas de atacante. Na definição da série, o nível 0 dispensa proteção; o nível 1 protege contra uso indevido não intencional; o nível 2, contra uso indevido intencional por meios simples e poucos recursos; o nível 3, contra meios sofisticados e recursos moderados; e o nível 4, contra meios sofisticados com recursos extensos. A avaliação é feita requisito por requisito, e não como “nota” do produto;
- **níveis de maturidade (1 a 4):** avaliam o processo de quem *desenvolve* o produto ou o serviço, do trabalho informal até a melhoria contínua medida.

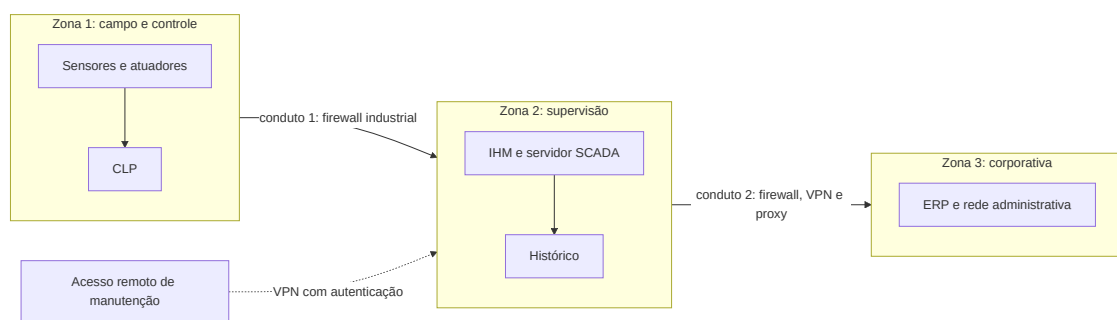


Figura 22.1: Zonas com requisitos distintos e os condutos que autorizam a comunicação entre elas.

A série é revisada continuamente e publicada em partes. Para citar uma parte é preciso citar a edição e o ano, e conferir a versão vigente na fonte — por exemplo, entre as partes publicadas estão IEC 62443-3-3 (requisitos de segurança de sistema, 2013), IEC 62443-4-1 (ciclo de vida de desenvolvimento seguro, 2018), IEC 62443-4-2 (requisitos técnicos de componentes, 2019), IEC 62443-3-2 (avaliação de risco para projeto de sistema, 2020) e IEC

62443-2-1 (programa de segurança do sistema de automação, 2024). Como a série evolui, o ano citado aqui corresponde à publicação consultada, não a uma promessa de permanência.

Para o lado corporativo e de gestão, dois documentos complementam o quadro:

- **ISO/IEC 27001:2022** (terceira edição, com emenda de 2024): requisitos para um sistema de gestão de segurança da informação. Cobre a empresa inteira, inclusive a parte administrativa; não substitui a IEC 62443 no que é específico de controle industrial;
- **NIST Cybersecurity Framework, versão 2.0**: referência de programa, organizada em funções de governança, identificação, proteção, detecção, resposta e recuperação. Serve como roteiro de maturidade, não como especificação técnica.

22.3 A superfície de ataque do controlador

Um controlador industrial não é um computador comum, mas compartilha com ele uma característica: possui vários pontos de entrada. A Tabela 22.2 lista os que aparecem com mais frequência e a contramedida correspondente.

Tabela 22.2: Pontos de entrada de um sistema de automação e contramedidas usuais.

Ponto de entrada	Como o problema aparece	Contramedida
Ambiente de engenharia	Estação de programação com acesso à rede de controle e à internet	Estação dedicada, sem navegação, em zona própria; acesso apenas pelo conduto autorizado
Serviços de rede externos	Servidor web do controlador, FTP, telnet ou diagnóstico abertos	Desabilitar o que não é usado; inventariar portas e serviços por equipamento
Credenciais padrão	Senha de fábrica mantida em painel, IHM ou gateway	Trocar antes do comissionamento; conta por pessoa, não compartilhada
Mídia removível	Pen drive para levar o programa até a máquina	Política de mídia controlada e dedicada; verificação antes do uso
<i>Firmware</i> e atualização	Versão antiga, sem correção, ou atualização sem controle	Inventário de versões e janela de atualização planejada
Rede plana	Rede de controle no mesmo domínio da rede administrativa	Segmentação em zonas, com conduto intermediado por <i>firewall</i> industrial
Acesso remoto	Acesso direto para suporte do fornecedor, sem registro	Acesso por VPN com autenticação, sessão registrada e autorizada por chamado

22.4 Casos públicos que vale conhecer

Estudar incidentes reais é mais eficiente que decorar taxonomia. Três exemplos são públicos, documentados e suficientes para fixar o problema — e nenhum deles precisa ser detalhado em termos de exploração:

- **Stuxnet (2010)**. Programa malicioso que atingiu sistemas de controle industrial e demonstrou publicamente que uma instalação física pode ser alvo de manipulação de lógica. É o caso que tirou a segurança cibernética industrial da pauta teórica;
- **vulnerabilidade de credencial em controladores Logix (2021, CVE-2021-22681)**. O ambiente de engenharia e as versões de *firmware* correspondentes usavam uma chave para verificar se quem se comunica com o controlador é o software de programação legítimo. Um atacante não autenticado poderia burlar essa verificação e autenticar-se no controlador. A severidade é crítica (base 9,8 na escala CVSS 3.1 do NVD) e o caso consta do catálogo de vulnerabilidades exploradas conhecidas da agência norte-americana CISA;
- **injeção de código pelo ambiente de engenharia (2022, CVE-2022-1159)**. Um atacante que obtivesse acesso de administrador na estação de trabalho com o software de programação poderia inserir código no controlador de forma indetectável ao usuário. Severidade alta (base 7,2 no NVD e 7,7 na avaliação do ICS-CERT). O caso mostra que o ponto fraco não é o controlador, é a **estação de engenharia**.

Atenção

O padrão dos três casos é o mesmo: o alvo preferido não é a lógica de controle, é o *caminho* até ela — a estação de engenharia, a credencial, o serviço aberto. Investir em proteção do controlador e deixar a estação de programação navegando na internet no mesmo segmento de rede é proteger a porta e deixar a janela aberta.

22.5 Contramedidas, do projeto à operação

A ordem abaixo segue o princípio de que arquitetura vale mais que produto: nenhum equipamento resolve o que a segmentação resolve.

1. **segmentar em zonas e definir os condutos**, como na Figura 22.1. É a decisão de maior efeito e a mais barata quando tomada no projeto;
2. **intermediar todo tráfego entre zonas** por *firewall* industrial, com regras explícitas por endereço, porta e direção. Sem regra escrita, não há conduto;
3. **controlar o acesso remoto**: VPN com autenticação individual, sessão registrada e autorização por chamado. Acesso de fornecedor é acesso privilegiado e deve ser tratado como tal;
4. **eliminar credenciais padrão** e individualizar contas. Nenhum sistema de automação deve ter usuário “admin” compartilhado por quatro pessoas;
5. **desabilitar serviços não utilizados**: o servidor web do controlador que ninguém usa é superfície de ataque sem contrapartida;
6. **controlar mídia removível**, com mídia da empresa, verificada e destinada a esse uso;
7. **fazer *hardening* de IHM e dos computadores industriais**: sistema atualizado, serviços mínimos, sem navegação e sem *e-mail*;
8. **registrar e fazer backup**: registro de eventos do controlador, da rede e dos acessos; backup do projeto, guardado fora do equipamento e testado. Backup que nunca foi restaurado não é backup, é expectativa;

9. **gerenciar atualização com janela planejada.** Em automação, a correção de segurança compete com a parada de produção: a decisão é de risco, tomada com inventário de versões em mãos;
10. **treinar quem opera.** A maior parte dos incidentes começa em uma ação humana — conexão indevida, mídia desconhecida, senha anotada.

22.6 Resposta a incidente

Ter plano de resposta não impede o incidente; impede o imprevisto durante ele. O plano precisa caber em uma página e responder a cinco perguntas:

1. **quem decide** desligar ou isolar um trecho da planta, e quem é o substituto;
2. **qual é o modo seguro** para a planta operar sem o sistema comprometido — operação manual, com supervisão local, ou parada programada;
3. **de onde se restaura:** qual é a última versão confiável do programa e da configuração, e onde ela está guardada;
4. **como se registra a evidência:** anotar horário, telas, registro de eventos e o que foi alterado, antes de mexer — inclusive para permitir análise posterior;
5. **como se volta à produção:** retorno gradual, com conferência de lógica, intertravamento e teste dos casos críticos, como no Capítulo 16.

Duas armadilhas comuns nessa hora: restaurar um backup que já contém o código alterado pelo atacante — daí a importância de guardar versões fora da rede — e reiniciar o equipamento “para ver se resolve”, o que apaga evidência sem resolver a causa.

22.7 Segurança é processo, não projeto

Um sistema de automação é seguro por decisões mantidas ao longo do tempo: inventário atualizado, zonas respeitadas, acessos revisados, versões acompanhadas e equipe treinada. A parte técnica é a menor parte do trabalho; a maior é disciplina.

No contexto do curso técnico, isso se traduz em três hábitos que valem desde a bancada:

- **proteger desde o primeiro teste:** trocar senha padrão, não deixar o laboratório em rede aberta e não usar pen drive de origem desconhecida;
- **documentar o que foi conectado:** qualquer equipamento novo em rede, com endereço, porta e finalidade (Capítulo 15);
- **tratar o acesso como privilégio:** quem programa altera o comportamento físico de uma máquina. Esse poder não deve estar disponível em qualquer computador.

Na prática

Desenhe as zonas e os condutos de uma planta que você conheça, no formato da Figura 22.1, e depois: (a) atribua o nível de segurança que você julgar adequado a cada zona, justificando; (b) liste os serviços que estão abertos hoje e diga quais poderiam ser desabilitados; (c) escreva as três primeiras regras de *firewall* entre as zonas; (d) redija o plano de resposta a incidente em uma página, respondendo às cinco perguntas da

seção anterior.

Exercícios

- a) Explique a diferença entre *safety* e *security* usando um exemplo de cada.
- b) Por que um incidente cibernético pode anular a proteção funcional de uma máquina?
- c) Descreva o conceito de zona e o de conduto, e diga por que ele substitui a ideia de “rede plana”.
- d) Explique os níveis de segurança 1 a 4 da série IEC 62443, com suas próprias palavras.
- e) Qual é a diferença entre nível de segurança e nível de maturidade? A quem cada um se aplica?
- f) Cite três papéis definidos pela série IEC 62443 e diga uma responsabilidade de cada um.
- g) Um gerente afirma: “segurança cibernética da planta é responsabilidade do fabricante do controlador”. Responda com base nos papéis definidos pela norma.
- h) Escolha quatro linhas da Tabela 22.2 e descreva a contramedida correspondente em uma frase cada.
- i) O que os casos CVE-2021-22681 e CVE-2022-1159 têm em comum quanto ao ponto de entrada?
- j) Por que a estação de engenharia costuma ser o alvo preferido? Cite duas medidas práticas de proteção.
- k) Explique por que “backup que nunca foi restaurado não é backup”.
- l) Escreva as cinco perguntas que o plano de resposta a incidente precisa responder.
- m) Por que restaurar um backup antigo pode não resolver um incidente, e como evitar esse problema?
- n) Escolha uma medida da seção de contramedidas que dependa de disciplina, e não de produto, e explique como você garantiria que ela seja mantida na rotina da planta.

Capítulo 23

Eficiência energética e sustentabilidade

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- identificar, na lógica de controle, as decisões que alteram o consumo de energia;
- definir indicadores de consumo comparáveis entre períodos, linhas e produtos;
- escolher entre partida direta, partida suave e acionamento por variador de frequência;
- aplicar controle de demanda sem comprometer a segurança do processo;
- considerar o ciclo de vida do equipamento, da escolha ao descarte.

23.1 A lógica também consome energia

Eficiência energética em automação costuma ser tratada como assunto de equipamento — trocar motor, instalar variador, colocar painel solar. Boa parte do resultado, porém, está em *quando* e *como* a máquina opera, e isso é decidido na lógica.

Três exemplos do dia a dia em que o programa decide o consumo:

- **operação a vazio:** a esteira continua ligada depois de a última peça passar. É o consumo mais fácil de eliminar e o mais frequentemente esquecido;
- **excesso de tempo de espera:** um temporizador longo demais mantém aquecedor, bomba ou exaustor ligados sem necessidade;
- **pico de partida desnecessário:** todos os motores partem juntos, elevando a demanda instantânea — e a fatura — sem ganho de produção.

Atenção

Eficiência nunca compete com segurança. Desligar automaticamente um sistema de exaustão, uma pressurização, uma refrigeração de painel ou qualquer função de proteção para economizar energia é falha grave de projeto. Antes de automatizar qualquer desligamento, confirme por escrito quais funções *não* podem ser interrompidas, ouvindo quem opera e quem mantém — e registre essa lista na documentação (Capítulo 15).

23.2 Medir antes de agir

Sem medição, toda ação de eficiência é opinião. O mínimo necessário:

1. **medidor de energia** no ponto que se quer avaliar: quadro de uma linha, de um grupo de máquinas ou de um equipamento que consome muito;
2. **leitura por rede** quando o medidor oferece comunicação, ou por **pulso** em entrada digital, com o cuidado de escolher a resolução adequada à frequência do pulso (Capítulo 7);
3. **energia por unidade produzida**, que é o indicador comparável. Quilowatt-hora por tonelada, por peça ou por metro produzido permite comparar turnos e produtos diferentes, o que o consumo absoluto não permite;
4. **contexto**: produto, velocidade, turno e temperatura ambiente. Comparar consumo de dois períodos com produtos diferentes e concluir que a eficiência mudou é erro clássico (Capítulo 19).

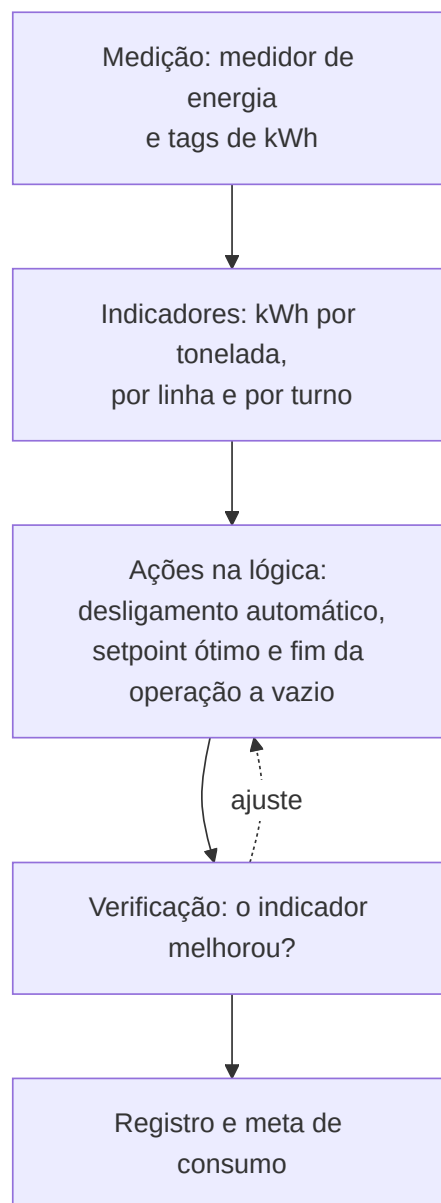


Figura 23.1: O ciclo da eficiência: medir, agir na lógica e verificar pelo indicador.

A Figura 23.1 tem um retorno explícito: a seta pontilhada significa que a ação precisa ser reavaliada pelo indicador. Alavanca instalada sem verificação é despesa, não resultado.

Com notação de grandezas: a energia E em quilowatt-hora é o produto da potência média $\bar{P}$ em quilowatt pelo intervalo de tempo Δt em horas,

$$E = \bar{P} \cdot \Delta t,$$

e o indicador de eficiência é a razão entre a energia e a produção Q no mesmo período:

$$e = \frac{E}{Q} \quad [\text{kWh por unidade produzida}].$$

É essa razão — e não E sozinho — que deve entrar em meta. Produzir mais consumindo proporcionalmente mais não é eficiência.

23.3 Alavancas na lógica

Tabela 23.1: Alavancas de eficiência que se implementam na lógica de controle.

Alavanca	O que fazer no programa	Indicador afetado
Desligamento automático por inatividade	Temporizar a ausência de peça ou de demanda e desligar o que pode ser desligado: esteira, exaustão, iluminação de área, aquecimento de espera	Consumo da linha no turno
Fim da operação a vázio	Detectar “sem carga” e reduzir rotação ou parar, em vez de manter o equipamento girando	Energia por peça produzida
Setpoint ótimo	Revisar referências que ficaram com folga grande (temperatura, cesso pressão, vazão) e ajustar ao necessário, com confirmação de que a qualidade se mantém	Consumo específico do pro-
Ajuste por demanda	Reduzir rotação de bomba ou ventilador quando a demanda cai, em vez de manter rotação constante com estrangulamento	Energia do sistema de bombeamento ou ventilação
Redução de tempo de espera	Encurtar temporizações de aquecimento, estabilização e transição, dentro do que o processo exige	Energia por lote
Sequenciamento de partida	Escalonar a partida de motores para reduzir a demanda instantânea	Demanda contratada e multa por ultrapassagem
Recuperação de calor e reúso	Usar energia que já está no processo: reaproveitar calor de compressor, de exaustão ou de resfriamento	Consumo total da planta

Atenção

Ajuste de *setpoint* é a alavanca com melhor retorno e maior risco. Reduzir a temperatura de um banho, a pressão de uma linha de ar ou a vazão de resfriamento pode afetar qualidade, tempo de ciclo ou segurança de processo. Toda alteração de referência precisa de teste documentado e aprovação de quem responde pelo produto e pela instalação — nunca de ajuste anônimo em tempo de execução.

23.4 Acionamento de motores

A escolha do modo de partida tem efeito sobre o consumo, sobre a rede elétrica e sobre a vida útil do equipamento:

- **partida direta:** simples e barata. A corrente de partida é várias vezes a nominal, o que produz queda de tensão, esforço mecânico e pico de demanda. Adequada a motores pequenos e a partidas pouco frequentes;
- **partida estrela-triângulo:** reduz a corrente de partida, com degrau de conjugado e limitação de carga no momento da transição. Serve a cargas que partem descarregadas;
- **partida suave (*soft-starter*):** controla a tensão aplicada na partida, sem controlar a rotação em regime. É solução de compromisso para reduzir esforço mecânico e pico de corrente;
- **variador de frequência:** controla a rotação em regime. É o único dos quatro que permite eficiência energética em carga variável, porque atua no ponto de operação, e não apenas na partida.

Em bombas centrífugas e ventiladores, o ponto de operação define o consumo. Reduzir a vazão fechando uma válvula ou um *damper* aumenta a perda de carga e mantém o motor consumindo quase o mesmo; reduzir a rotação, por outro lado, reduz o consumo de forma muito mais acentuada — em sistemas centrífugos a potência varia de forma aproximadamente cúbica com a rotação, o que faz da regulação por rotação a alavanca mais forte disponível em bombeamento e ventilação.

Isso não significa que variador é sempre a resposta: em carga constante, o inversor acrescenta perdas próprias e complexidade sem ganho. A pergunta correta é “a carga varia?”.

23.5 Controle de demanda

A fatura de energia industrial depende tanto da *energia* consumida quanto da *demand*a — a potência média no intervalo de medição — e multa a ultrapassagem do valor contratado. Dois mecanismos reagem a isso:

1. **limitação por prioridade:** quando a demanda medida se aproxima do limite, o sistema desliga cargas de menor prioridade por tempo determinado. É preciso que a lista de prioridades seja explícita e que as funções críticas nunca estejam nela;
2. **gerenciamento por horário:** deslocar operações com folga — carregamento de baterias, limpeza, teste de equipamento — para fora do horário de ponta.

A limitação de demanda é uma intervenção *na operação*, e por isso precisa de três cuidados: lista de cargas bloqueadas revisada e aprovada, alarme claro para o operador quando a limitação atuar, e registro de cada atuação — sem isso, a produção perde uma máquina e a causa fica desconhecida.

23.6 Ciclo de vida e descarte

Eficiência e sustentabilidade também se decidem antes da instalação:

- **vida útil e disponibilidade de reposição:** equipamento com fim de vida anunciado força substituição no pior momento, e a substituição emergencial consome mais recurso que a planejada (Capítulo 24);
- **atualização sem troca total:** parte dos casos de obsolescência se resolve com atualização de *firmware*, troca de módulo ou de HMI, preservando o que ainda funciona;
- **sobressalentes críticos** em estoque, com regra de descarte por validade — especialmente baterias de retenção e filtros;
- **descarte correto:** eletroeletrônicos, baterias e componentes não seguem o lixo comum; seguem logística reversa, conforme a política de resíduos adotada pela instituição ou pela empresa;
- **eficiência do próprio sistema de automação:** fontes sobredimensionadas, painéis mal ventilados e iluminação de painel permanente são consumos pequenos que se somam em uma planta inteira.

Na prática

Escolha um equipamento de consumo significativo na sua planta ou bancada e aplique o ciclo da Figura 23.1: (a) instale ou identifique a medição e calcule a energia por unidade produzida no período; (b) escolha duas alavancas da Tabela 23.1 que se apliquem ao equipamento; (c) escreva a alteração de lógica necessária para cada uma, em texto estruturado; (d) defina o que *não* pode ser desligado e registre a justificativa; (e) proponha o prazo de reavaliação do indicador e o critério de sucesso.

Exercícios

- Dê três exemplos de consumo causado por decisão de programação, e não por equipamento.
- Por que o indicador “kWh por tonelada” é preferível ao consumo absoluto em kWh?
- Escreva a expressão da energia a partir de potência média e intervalo de tempo, e explique cada grandeza com a unidade.
- Uma linha consumiu 4.800 kWh para produzir 60 t em um turno e 5.200 kWh para produzir 65 t no turno seguinte. Calcule o indicador nos dois turnos e diga o que melhorou.
- Explique por que a limitação de demanda exige lista de prioridades aprovada e registro de atuação.
- Compare partida direta, estrela-triângulo, partida suave e acionamento por variador quanto a corrente de partida e controle de regime.
- Por que, em bombeamento, reduzir a rotação economiza mais que fechar uma válvula? Relacione com o ponto de operação.

- h) Um colega propõe instalar variador em um motor que trabalha sempre em carga nominal. Faça a análise técnica da proposta.
- i) Cite dois perigos de reduzir *setpoint* de temperatura em um processo, e diga quem deve aprovar a alteração.
- j) Selecione duas alavancas da Tabela 23.1 e escreva o trecho de lógica necessário para cada uma.
- k) Liste três funções que *nunca* podem entrar em um esquema de desligamento automático, e justifique cada uma.
- l) Descreva duas medidas de sustentabilidade ligadas ao ciclo de vida do controlador, e diga em que momento do projeto cada uma deve ser decidida.
- m) Um painel elétrico tem as fontes sobredimensionadas e a ventilação desligada nos fins de semana. Explique, com base neste capítulo, se a decisão é correta.

Parte V

Operação e carreira

Capítulo 24

Manutenção preventiva e corretiva

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- reconhecer os modos de falha típicos de um sistema com CLP;
- montar uma rotina preventiva com período e evidência para cada item;
- diagnosticar por sintoma, seguindo um roteiro de verificação;
- executar uma intervenção em campo com bloqueio e registro;
- identificar quando a manutenção preditiva é viável e quando é apenas dado sem uso.

24.1 Três formas de manter

- **corretiva:** age depois da falha. É inevitável em algum grau, mas cara: a parada não é planejada, o sobressalente pode não existir e a produção está perdida;
- **preventiva:** age em intervalo programado, substituindo ou verificando antes de a falha acontecer. Custa mesmo quando o componente estaria bom — mas a parada é planejada;
- **preditiva:** age quando a condição medida indica degradação. Reduz intervenção desnecessária, mas exige medição, histórico e critério — sem os três, vira preventiva com nome novo.

Em automação, o ganho maior não está em escolher uma das três, e sim em *deslocar o esforço para o lado barato*: registro de falha bem feito (que permite o diagnóstico rápido), sobressalente disponível e intervenção com procedimento escrito.

24.2 Modos de falha típicos

Tabela 24.1: Modos de falha usuais em sistemas com CLP, sinal observável e prevenção.

Componente	Modo de falha	Sinal observável	Prevenção
Bateria de retenção	Descarga: as variáveis retentivas e o relógio de tempo real são perdidos quando a energia cai	Perda de contador e de horário após parada; aviso de bateria fraca no diagnóstico do controlador	Substituição em intervalo definido, com registro da data, e conferência do aviso de diagnóstico (Capítulo 6)
Cartão de entrada ou saída	Canal queimado por surto, curto ou sobrecarga	Um ponto específico não lê ou não atua, enquanto os demais do mesmo cartão funcionam	Proteção de circuito, separação de cargas indutivas, conferência dos sinais de campo e registro de falha por ponto
Fonte de alimentação	Envelhecimento de capacitores, sobretensão, sobrecarga	Reinícios aleatórios, falha intermitente de módulos, reinício em partida de motor	Medição de tensão sob carga, dimensionamento correto, reserva de capacidade e ventilação do painel
Térmica e poeira	Acúmulo de pó, filtro saturado, ventilação obstruída	Falha que aparece no verão ou depois de horas de operação e desaparece depois de esfriar	Limpeza e troca de filtro em rotina; medição de temperatura no interior do painel
Programa e memória	Corrupção de programa, bateria fraca, edição indevida em campo	Comportamento diferente do documentado, sem falha de hardware	Backup versionado, restrição de acesso e controle de quem altera o quê (Capítulos 15 e 22)
Rede de comunicação	Cabo, conector, terminador, endereço duplicado, interferência	Comunicação cai e volta; erro em um nó específico	Conferência de cabeamento e de aterramento, documentação do endereçamento e teste de continuidade
Campo	Sensor descalibrado, atuador travado, fim de curso desregulado	Leitura plausível e errada — o pior tipo de falha	Conferência periódica com instrumento padrão e teste de atuação (Capítulo 16)

24.3 Rotina preventiva

A rotina precisa ser executável por quem está no turno e verificável por quem responde pela planta. Cada item tem período, responsável e evidência.

Tabela 24.2: Rotina preventiva mínima e evidência de execução.

Item	Período	Evidência
Backup do projeto do controlador	A cada alteração e em intervalo fixo	Arquivo versionado, com identificação de versão do programa e data; cópia guardada fora do equipamento
Leitura do diagnóstico do controlador	Mensal	Registro dos avisos ativos, incluindo bateria, temperatura e estatística de erros de comunicação
Inspeção visual e limpeza do painel	Semestral ou conforme ambiente	Lista assinada: pó, ventilação, filtros, conexões, sinais de aquecimento
Medição de temperatura interna	Semestral, nos meses quentes	Valor registrado, comparado ao histórico
Teste de bateria de retenção	Conforme o fabricante e o histórico	Data de substituição anotada no painel e no sistema de manutenção
Conferência de pontos de E/S	Anual, ou após intervenção	Lista de pontos testados, com resultado (Capítulo 7)
Teste dos intertravamentos e da parada de emergência	Anual e após qualquer alteração de lógica	Roteiro de teste assinado, com data e responsável (Capítulo 16)
Restauração de backup em bancada	Anual	Registro do teste de restauração: sem isso, o backup é apenas uma esperança
Revisão da documentação e da lista de tags	Anual	Documentação atualizada e publicada na versão vigente

Atenção

Teste de intertravamento e de parada de emergência só pode ser feito com a máquina em condição segura, com a área liberada e com autorização de quem responde pela produção. Testar proteção com pessoa em zona de risco é a receita do acidente que a proteção existia para evitar (Capítulo 16).

24.4 Diagnóstico por sintoma

O roteiro da Figura 24.1 parte do sintoma e desce pela cadeia, sempre do mais provável e mais barato de verificar para o menos provável. A ordem importa: conferir o programa antes de conferir a fonte é o erro mais comum e o mais caro em tempo.

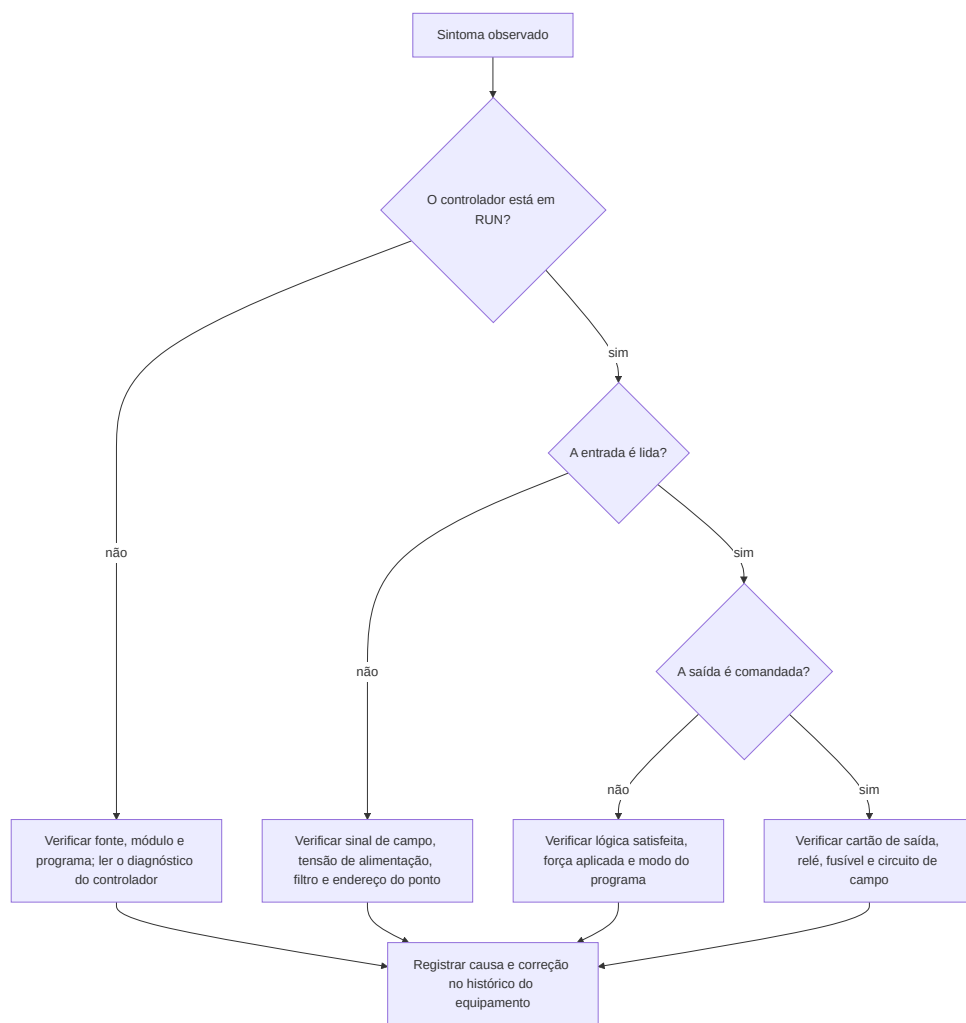


Figura 24.1: Roteiro de verificação a partir do sintoma observado.

Tabela 24.3: Diagnóstico: sintoma, causas prováveis e verificação.

Sintoma	Causas prováveis	Como verificar
Saída não atua	Ponto forçado no ambiente de programação; canal do cartão queimado; lógica não satisfeita; circuito de campo aberto; fusível ou relé	Conferir a força ativa (<i>force</i>); observar o LED do canal; verificar a condição que deveria acionar a saída; medir tensão no borne com o ponto comandado
Entrada não lê	Sinal de campo ausente; tensão de alimentação incorreta; filtro de entrada longo demais para o evento; endereço errado na lista de tags; borneira solta	Medir o sinal no borne; conferir o tipo de entrada (<i>sourcing</i> ou <i>sinking</i>); comparar o tempo do evento com o filtro configurado; conferir a alocação do ponto
CLP em <i>fault</i>	Falha de fonte; módulo com erro; programa incompatível com o <i>firmware</i> ; erro de configuração de rack	Ler o diagnóstico do controlador; conferir tensões; testar módulo por módulo; verificar a versão do ambiente de programação
Comportamento diferente do documentado	Programa alterado em campo; variável retentiva perdida; temporizador com base de tempo diferente da prevista	Comparar o programa carregado com o backup versionado; conferir bateria e retenção; medir o tempo real de operação com cronômetro
Comunicação cai e volta	Cabo, conector, terminador, endereço duplicado, aterramento, interferência de acionamento	Conferir endereçamento e terminadores; revisar aterramento e roteamento dos cabos junto aos cabos de potência
Leitura plausível, mas errada	Sensor descalibrado ou com sinal de campo degradado; escala ou unidade trocadas	Comparar com instrumento padrão; conferir escala e unidade na lista de tags (Capítulo 15)

24.5 Procedimento de intervenção

Este procedimento vale para qualquer intervenção em máquina em produção. Ele não é uma sugestão de boa prática: é a sequência mínima que separa a manutenção do acidente.

1. **comunicar** quem opera e quem responde pela área antes de começar;
2. **bloquear e etiquetar a energia**: desligar, bloquear o dispositivo de seccionamento e identificar o responsável. Energia residual — pressão, mola, gravidade, capacitor, inércia — precisa ser dissipada ou contida. *Nenhum trabalho de ligação, medição ou troca de componente é feito em equipamento energizado*;
3. **conferir a ausência de tensão** com instrumento adequado, no ponto onde se vai trabalhar, antes de tocar no circuito;
4. **fazer backup antes de alterar** qualquer coisa no programa, e anotar a versão;
5. **se a alteração for no programa**, usar o modo de operação mais seguro possível e manter a lógica de proteção ativa. Forçar ponto para “testar” com a máquina em produção é proibido: a força desabilita a proteção;

6. **testar após a intervenção** pelos casos críticos do roteiro (Capítulo 16), antes de liberar a produção;
7. **registrar**: o que foi feito, por quê, quando, por quem, qual a versão do programa resultante e qual sobressalente foi consumido. Registro é o que transforma a próxima ocorrência em diagnóstico rápido, em vez de investigação do zero;
8. **liberar** com a área segura e as proteções restabelecidas, comunicando quem bloqueou.

Atenção

Dois hábitos causam acidente com controlador: forçar saída para testar com a máquina em operação e editar o programa *on-line* sem conferir o efeito imediato na máquina. A edição em modo de execução altera o comportamento no ciclo seguinte — o que significa que o efeito é instantâneo. Se a edição for inevitável, faça-a com o equipamento em condição segura e com o intertravamento intacto.

24.6 Manutenção preditiva como evolução

Predição útil não exige sensor exótico. Três fontes de dado simples, já presentes ou de fácil obtenção, cobrem boa parte dos casos:

- **contagem de ciclos e de manobras** por atuador, comparada à vida declarada pelo fabricante. É o dado mais confiável e o mais barato de manter (Capítulo 19);
- **temperatura**, no painel e no motor, com tendência comparada ao histórico. Degradação aparece como desvio lento, não como limite atingido;
- **corrente e tempo de etapa**: corrente de motor que sobe lentamente indica desgaste ou obstrução; tempo de etapa que cresce indica perda de desempenho (Capítulo 23).

O critério de viabilidade é econômico e operacional: só compensa predição quando o custo da falha não planejada é maior que o custo de medir, guardar e analisar o dado — e quando existe alguém encarregado de olhar o resultado. Indicador de degradação publicado e não acompanhado é a pior forma de preventiva: parece controle e não é.

Na prática

Escolha um equipamento automatizado da sua planta ou bancada e monte o plano de manutenção dele: (a) liste os cinco modos de falha mais prováveis, no formato da Tabela 24.1; (b) preencha a Tabela 24.2 com período e responsável para cada item aplicável; (c) redija o procedimento de intervenção em uma página, na ordem dos oito passos; (d) descreva dois sintomas e o roteiro de verificação de cada um, usando o formato da Tabela 24.3.

Exercícios

- a) Explique a diferença entre manutenção preventiva e preditiva, e diga o que a preditiva exige além da medição.
- b) Por que a perda de variáveis retentivas após uma queda de energia é um modo de falha previsível?

- c) Uma saída não atua. Descreva o roteiro de verificação na ordem correta, com base na Tabela 24.3.
- d) Uma entrada não lê um pulso curto, mas lê um sinal contínuo. Qual é a causa mais provável e como confirmar?
- e) Por que a falha mais perigosa é a leitura plausível e errada? Dê um exemplo.
- f) Liste, em ordem, os oito passos do procedimento de intervenção e explique a função de cada um.
- g) Por que não se deve forçar ponto de saída para testar uma máquina em produção?
- h) Por que a edição *on-line* tem efeito imediato, e que cuidado isso exige?
- i) Um painel apresenta falhas que só acontecem nos meses mais quentes. Que modo de falha suspeitar e como confirmar?
- j) Por que é obrigatório testar a restauração de um backup, e não apenas produzi-lo?
- k) Escolha três itens da Tabela 24.2 e diga qual evidência prova que foram executados.
- l) Cite duas fontes de dado já disponíveis que permitem manutenção preditiva sem sensor novo, e diga o que cada uma antecipa.
- m) Em que condições a manutenção preditiva não compensa? Dê duas condições.
- n) Um fornecedor propõe um pacote de predição que “avisa tudo automaticamente”. Que perguntas você faz antes de assinar? Relacione com os três requisitos da predição viável.

Capítulo 25

Formação contínua e certificações

Objetivos do capítulo

Ao final deste capítulo você deve ser capaz de:

- situar sua formação atual em uma trilha de estudo coerente;
- distinguir certificação de produto, de competência e de treinamento pontual;
- escolher recursos de estudo por critério, e não por propaganda;
- montar uma bancada de prática com custo controlado e segurança;
- executar um plano de 90 dias com entrega verificável ao final de cada etapa.

25.1 Por que a formação não termina no curso

O controlador mudou três vezes em vinte anos: passou de ilha de lógica a nó de rede, de rede a sistema com camada de informação, e agora convive com visão computacional, borda e modelos de linguagem (Capítulo 21). O profissional que parou em “programa em Ladder” fica preso a uma geração de equipamento.

Duas consequências práticas para quem está se formando:

1. **a base não envelhece:** eletricidade, comandos, lógica, instrumentação e a norma IEC 61131-3 — com suas edições — sustentam qualquer marca. É onde vale investir primeiro;
2. **o entorno muda:** protocolos, ferramentas e plataformas de dados se renovam. Aqui o esforço é de atualização contínua, em dose pequena e frequente, e não de recomeço.

25.2 A trilha

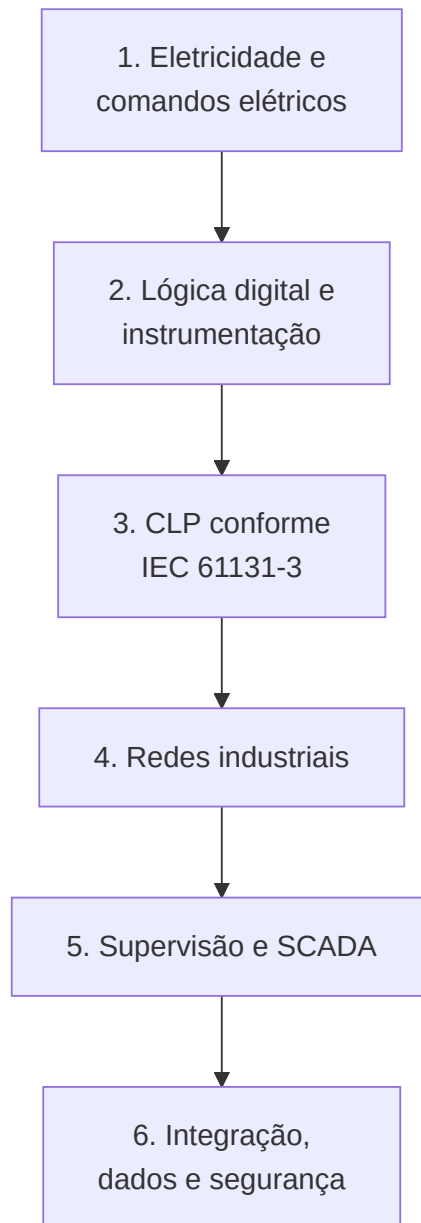


Figura 25.1: Trilha sugerida de estudo, da base elétrica à integração de sistemas.

A Figura 25.1 apresenta a sequência; a Tabela 25.1 detalha cada etapa e a entrega que comprova o domínio.

Tabela 25.1: Etapas da trilha, o que se aprende e como provar que aprendeu.

Etapa	Conteúdo	Evidência de domínio
1. Eletricidade e comandos elétricos	Leitura de diagrama, contator, relé, selo, botoeira, proteção de circuito	Montar um comando de partida e parada com selo em bancada, e explicar cada contato
2. Lógica digital e instrumentação	Tabela-verdade, álgebra booleana, sinal 4–20 mA, escalas, precisão	Converter um sinal de 4–20 mA em grandeza de engenharia e justificar a resolução obtida (Capítulo 7)
3. CLP conforme IEC 61131-3	Modelo de software, tipos de dados, LD, FBD, ST e SFC, tarefas	Escrever o mesmo controle em LD e em ST, com tipos declarados, e documentar a diferença
4. Redes industriais	Modbus RTU e TCP, protocolos industriais, arquiteturas de rede	Ligar dois equipamentos por Modbus e construir a tabela de registros usados (Capítulo 17)
5. Supervisão e SCADA	Telas, tags, histórico, alarmes e eventos	Montar uma tela com histórico e um alarme classificado corretamente (Capítulo 17)
6. Integração, dados e segurança	OPC UA, MQTT, indicadores, zonas e condutos	Publicar uma variável por protocolo aberto, calcular um indicador e desenhar as zonas da bancada (Capítulos 18 e 22)

O erro mais comum na trilha é começar pela etapa 5, porque a tela é a parte visível. Sem a etapa 3, a tela mostra dado que ninguém sabe se está certo, como a Tabela 25.1 deixa explícito ao exigir a entrega de cada etapa antes de seguir.

25.3 Certificações, treinamentos e o que eles valem

Vale separar três coisas que costumam aparecer misturadas em conversa:

- **treinamento de fabricante:** curso sobre uma família de produtos. Ensina profundamente o equipamento e é o que o mercado de manutenção procura. Programas de treinamento existem em fabricantes de peso no país, como Siemens (SITRAIN), Rockwell Automation, Schneider Electric, WEG, Novus, Mitsubishi Electric, Omron e Beckhoff. O conteúdo, a duração e a validade mudam com o tempo — confirme sempre na fonte, no site oficial do fabricante;
- **certificação de competência profissional:** avalia o profissional, não o produto. Existem programas internacionais com esse perfil, como os da ISA (*International Society of Automation*) voltados a automação e a sistemas de controle. Exigem experiência comprovada e são mantidos por recertificação periódica;
- **certificação de produto ou de ferramenta:** atesta que um produto ou ambiente de programação atende a requisitos de uma norma ou de uma biblioteca padronizada. É o caso das certificações ligadas à PLCopen para ambientes de programação (Capítulo 12). *Ela não certifica pessoas* — não confunda com certificação profissional.

Atenção

Preço, carga horária, pré-requisitos e prazo de validade de certificações mudam com frequência e variam por país. Nenhuma informação deste capítulo deve ser usada como referência de custo ou de validade: consulte sempre a entidade emissora antes de se inscrever ou de anunciar um título no currículo.

Três critérios para escolher onde investir tempo e dinheiro:

1. **a instituição tem laboratório?** Curso de automação sem bancada é curso de leitura;
2. **o conteúdo ensina norma ou só uma marca?** Um bom curso diz, em cada instrução, se ela é normatizada ou proprietária — é a distinção que o Capítulo 13 ensina;
3. **existe avaliação prática?** Certificado que se obtém apenas respondendo a um questionário prova menos.

25.4 Recursos de estudo

A Tabela 25.2 reúne o que costuma ser útil e o cuidado correspondente a cada recurso.

Tabela 25.2: Recursos úteis e o cuidado correspondente.

Recurso	Para que serve	Cuidado
Ambientes de programação em versão de avaliação ou de uso educacional	Praticar a linguagem e a estrutura de POU's sem comprar licença	Nem todo recurso fica disponível na versão gratuita; verifique os limites antes de investir tempo em material didático de terceiros
Ambientes de desenvolvimento conforme a norma, com versão de uso livre	Aprender o modelo de software da norma, independente de fabricante	Verifique em qual edição da norma o ambiente se declara conforme
Simuladores de processo	Testar lógica sem hardware, inclusive casos de falha	Simulação não substitui bancada para aprender elétrica, aterramento e ruído
Plataformas abertas de controlador (por exemplo, projetos abertos de CLP executando em Linux)	Entender o que há dentro de um controlador: varredura, memória, E/S	Não são substitutos de controlador industrial certificado em aplicação real
Documentação técnica do fabricante	A fonte primária: manual de instruções, de comunicação e de diagnóstico	É a referência a citar; fórum é apoio, não fonte
Normas	A definição correta de conceito, linguagem e tipo de dado	São documentos pagos, obtidos junto aos organismos de normatização. No Brasil, a ABNT publica adoções de normas IEC com a designação NBR IEC — confira no catálogo quais partes existem e em que edição. Nunca use cópia não autorizada
Comunidades técnicas e fóruns	Resolver problema concreto e ver como outros resolvem	Verifique a resposta na documentação; a resposta com mais votos não é necessariamente a correta

Nunca reproduza trechos de norma, figura de catálogo ou manual de fabricante em trabalho publicado sem autorização. Cite a referência: obra, fabricante, título, versão e data.

25.5 Bancada própria

Uma bancada simples cobre quase todo o conteúdo deste material e cabe em uma mesa:

- um controlador compacto, mesmo de porte pequeno, com entrada digital, saída a relé e uma entrada analógica;
- fonte de alimentação, disjuntor e proteção na entrada — e a bancada *não* deve ser ligada à rede elétrica sem supervisão de quem conhece o risco;
- botoeiras, chaves seletoras, sinaleiros, fim de curso e um relé como carga;

- um sensor qualquer para a entrada analógica: potenciômetro para simular, transmissor para aprender escala;
- um computador com o ambiente de programação e a planilha de documentação;
- itens de segurança: protetor de bancada, destino claro para os cabos e **regra escrita de que não se monta nem se mede circuito energizado**.

Duas adições baratas aumentam muito o valor didático: um módulo de entrada/saída remota acessível por Modbus, para praticar rede, e uma tela de supervisão simples, para praticar tag, alarme e histórico.

25.6 Plano de 90 dias

O plano da Tabela 25.3 é uma sugestão de sequência para quem já fez o básico de eletricidade. O que importa não é o prazo, é a regra: **toda etapa termina com algo que outra pessoa pode ver** — e não com horas de vídeo assistido.

Tabela 25.3: Plano de estudo de 90 dias, com entrega verificável por etapa.

Período	Foco	Prática	Entrega
Semanas 1–2	Tipos de dados, endereçamento e leitura de diagrama de comando	Montar um comando de partida e parada com selo em bancada	Diagrama desenhado, lista dos pontos usados e vídeo curto do funcionamento
Semanas 3–4	Ladder e texto estruturado conforme a norma	Escrever o mesmo controle nas duas linguagens, com tipos declarados	Os dois programas versionados, com comentário explicando a diferença
Semanas 5–6	Temporização, contagem, borda e retenção	Implementar uma sequência com etapas, intertravamento e retomada após falta de energia	Lista de casos de teste preenchida, com resultado observado e servado
Semanas 7–8	Rede industrial e supervisão	Ligar remota por Modbus, criar tags e monitorar tela com histórico e alarme	Tabela de registros, tela funcionando e alarme classificado
Semanas 9–10	Documentação e boas práticas	Documentar o projeto das semanas anteriores como entrega de engenharia	Descritivo, lista de E/S, tabela de tags e rotina de backup preenchida
Semanas 11–12	Integração e dados	Publicar uma variável por OPC UA ou MQTT e calcular um indicador	Diagrama da arquitetura, indicador calculado e desenhado das zonas da bancada

Ao final do plano, o conjunto das entregas forma um **portfólio**. Em processo seletivo, ele vale mais que uma lista de cursos: mostra o que a pessoa sabe fazer, como documenta e como testa.

Na prática

Monte o seu plano de 90 dias: (a) escreva as seis etapas adaptadas à sua realidade, com o tempo que você tem por semana; (b) defina, para cada etapa, a entrega verificável; (c) liste o que falta comprar ou conseguir emprestado na bancada, com prioridade; (d) agende a revisão da etapa 1 para uma data específica — com alguém que possa assistir à demonstração.

Exercícios

- a) Explique por que a base da trilha (etapas 1 a 3) não envelhece, e o que muda nas etapas 4 a 6.
- b) Por que começar pela supervisão é um erro comum, e o que ele produz?
- c) Diferencie treinamento de fabricante, certificação de competência e certificação de produto.
- d) Por que a certificação de um ambiente de programação *não* certifica o profissional?
- e) Cite três critérios para escolher um curso de automação e explique cada um.
- f) Escolha uma das etapas da Tabela 25.1 e descreva uma entrega sua, real, que sirva como evidência de domínio dela.
- g) Por que simulação não substitui bancada no aprendizado de elétrica?
- h) Que cuidado se deve ter ao usar resposta de fórum como referência técnica?
- i) Liste cinco itens mínimos de uma bancada de estudo e um item de segurança obrigatório.
- j) Por que a bancada deve ter regra escrita sobre trabalho em circuito energizado?
- k) Cite duas adições baratas à bancada e diga o que cada uma ensina.
- l) Explique a regra “toda etapa termina com algo que outra pessoa pode ver” e por que ela reduz o risco de estudo sem prática.
- m) Monte a entrega da semana 5–6 do plano de 90 dias para um processo simples, no formato da Tabela 16.2.
- n) Por que um portfólio de entregas pode valer mais que uma lista de cursos em um processo seletivo?

Parte VI

Apêndices

Capítulo A

Práticas de laboratório

Objetivos do capítulo

Ao final das práticas deste apêndice você deve ser capaz de:

- escrever e testar lógica em **linguagens padronizadas pela IEC 61131-3**, sem depender de dialeto de fabricante;
- aplicar detecção de borda, temporização, contagem e retenção com comportamento previsível;
- converter um sinal analógico em grandeza de engenharia e justificar a resolução obtida;
- documentar cada prática como entrega técnica, com lista de E/S, tags e casos de teste.

A.1 Como usar este apêndice

O material de origem trazia uma bateria de práticas escrita inteiramente em mnemônicos de um fabricante. Aqui as mesmas práticas foram reescritas em **elementos normatizados**: contatos, bobinas, blocos de função padronizados (TON, TOF, TP, CTU, CTD), operadores e expressões em texto estruturado. O Apêndice B traz o mapa de conversão para quem precisar ler o código antigo.

Cada prática tem a mesma estrutura: **objetivo, material, procedimento, critério de aceite e o que entregar**. O critério de aceite é escrito de forma verificável — “a saída liga”, não “entender o funcionamento”.

Atenção

Regras de segurança do laboratório. Nenhum procedimento deste apêndice autoriza trabalho em circuito energizado.

1. só monte, desmonte ou meça com a bancada **desenergizada e bloqueada**, conferindo a ausência de tensão antes de tocar no circuito (Capítulo 24);
2. ligação de motor, rede elétrica ou qualquer tensão acima da alimentação da bancada exige supervisão do professor;
3. força de ponto (*force*) só é permitida em bancada, com o equipamento sem carga mecânica e com autorização;
4. teste de intertravamento com máquina energizada exige área liberada e autorização de quem responde pela produção;
5. ao terminar, desenergize, organize o cabeamento e registre o que foi alterado no projeto.

A Tabela A.1 dá a visão geral das onze práticas e indica, para cada uma, a linguagem sugerida e os capítulos de apoio.

Tabela A.1: Visão geral das práticas, linguagem sugerida e capítulos de apoio.

Nº	Tema	Linguagem	Apoio teórico
1	Partida e parada com selo	LD	Capítulos 10 e 14
2	Condição de falha segura e contato NF	LD	Capítulo 10
3	Detecção de borda e pulso único	LD e ST	Capítulos 10 e 11
4	Temporização com TON, TOF e TP	FBD	Capítulo 11
5	Contagem, retenção e retomada	FBD e ST	Capítulos 6 e 11
6	Sequência por etapas com SFC	SFC	Capítulo 10
7	Escala de sinal analógico	ST	Capítulo 7
8	Comparação, aritmética e alarme de faixa	ST	Capítulos 6 e 10
9	Intertravamento e partida segura	LD	Capítulos 14 e 16
10	Remota de E/S e supervisão	Integração	Capítulos 17 e 19
11	Documentação e versionamento do projeto	—	Capítulo 15

A.2 Prática 1 — Partida e parada com selo

Objetivo. Implementar o comando clássico de partida e parada com retenção, em Ladder conforme a norma, e explicar o papel do contato de selo.

Material. Controlador com módulo de entradas e saídas digitais; botoeira NA (partida) e botoeira NF (parada); uma saída com sinalizador.

Procedimento.

1. declare as variáveis com nome do processo (`bt_partida`, `bt_parada`, `motor`), no padrão do Capítulo 14;
2. escreva a lógica: o contato de partida em paralelo com o contato do próprio motor, em série com o contato de parada, acionando a bobina do motor;
3. carregue o programa, ligue a bancada e teste as quatro combinações de partida e parada;
4. acrescente uma segunda saída que só ligue com o motor em operação e confirme o comportamento;
5. desligue a bancada no meio da operação e religue: o motor deve permanecer parado.

Critério de aceite. Um toque na partida liga o motor e ele permanece ligado; um toque na parada desliga; soltar as botoeiras não altera o estado; após queda e retorno de energia o motor não religa sozinho.

O que entregar. Diagrama Ladder comentado, lista de variáveis com o ponto físico de cada uma e registro do teste das quatro combinações.

A.3 Prática 2 — Condição de falha segura e contato NF

Objetivo. Reconhecer que o tipo de contato no programa decorre da condição de falha segura, e não do tipo de chave disponível em campo.

Material. A bancada da Prática 1 acrescida de uma chave fim de curso NA e de uma chave fim de curso NF.

Procedimento.

1. monte uma saída que só permaneça ligada com a proteção fechada, usando a chave NA;
2. refaça com a chave NF, mantendo o mesmo comportamento final;
3. preencha a tabela de contato em função da chave (Capítulo 10) para os dois casos;
4. desconecte o fio de uma das chaves com a bancada desenergizada, religue e observe o comportamento;
5. explique por escrito qual das duas montagens é a correta para uma proteção de máquina.

Critério de aceite. A montagem escolhida leva a máquina ao estado seguro quando o sinal da proteção é perdido; a justificativa cita a condição de falha segura.

O que entregar. Tabela preenchida, os dois programas e o parágrafo de justificativa.

A.4 Prática 3 — Detecção de borda e pulso único

Objetivo. Diferenciar nível de borda, e usar detecção de borda para contar um evento único por acionamento.

Material. A bancada da Prática 1 e um contador na saída.

Procedimento.

1. escreva uma lógica que incremente um contador *enquanto* a entrada estiver acionada e observe o resultado;
2. reescreva usando um bloco de detecção de borda de subida, de modo que cada acionamento conte **uma única vez**;
3. repita com detecção de borda de descida;
4. escreva a versão em texto estruturado e compare com a versão em Ladder;
5. anote, com o cronômetro do ambiente de programação, o tempo em que a entrada permaneceu acionada.

Critério de aceite. A primeira versão conta muitas vezes por acionamento; a segunda conta exatamente uma vez por transição; as versões em Ladder e em texto estruturado têm o mesmo comportamento.

O que entregar. Os três programas, a medição de tempo e uma frase explicando por que a detecção de borda é indispensável em contagem de peças.

A.5 Prática 4 — Temporização com TON, TOF e TP

Objetivo. Aplicar os três blocos de temporização padronizados e distinguir os seus comportamentos.

Material. A bancada da Prática 1; duas saídas com sinalizador.

Procedimento.

1. com TON, acione uma saída 3 s após a entrada ser ativada; desative a entrada e observe o tempo acumulado;
2. com TOF, mantenha a saída ligada por 3 s *depois* de a entrada ser desativada;
3. com TP, gere um pulso de 3 s a partir da borda de subida da entrada, mesmo que a entrada continue ativa;
4. para cada bloco, desative a entrada antes de o tempo completar e registre o que acontece com a saída e com o tempo acumulado;
5. declare o tempo de referência (PT) como constante nomeada no início do programa, e não como número solto no bloco.

Critério de aceite. Os três comportamentos são distintos e descritos corretamente no relatório; o tempo cronometrado confere com o programado dentro da tolerância do ciclo de varredura.

O que entregar. Diagrama de blocos dos três casos, registro dos tempos medidos e a explicação da diferença entre TON, TOF e TP.

Atenção

Confira a **base de tempo** declarada no ambiente antes de comparar resultados. Um temporizador escrito com base de 10 ms lido como se fosse de 1 ms produz um erro de dez vezes — e é o tipo de defeito que só aparece na máquina, quando o tempo do processo não é mais o esperado.

A.6 Prática 5 — Contagem, retenção e retomada

Objetivo. Usar contagem com valor de referência e variável retentiva, e verificar o comportamento do sistema após falta de energia.

Material. A bancada da Prática 3, com sinalizador de fim de lote.

Procedimento.

1. conte 10 acionamentos de uma entrada e acione uma saída ao atingir o valor de referência;
2. use um contador decrescente partindo de 10 e acione a mesma saída ao chegar a zero;
3. acrescente um botão de reinício que zere o acumulado;
4. declare o acumulado como variável **retentiva** e repita o ensaio desenergizando a bancada no meio da contagem;
5. com o acumulado *não* retentivo, repita o mesmo ensaio e compare.

Critério de aceite. Com retenção, a contagem continua de onde parou; sem retenção, o acumulado volta ao valor inicial; em ambos os casos o reinício zera no momento correto.

O que entregar. Os dois programas, o registro dos ensaios e a explicação do efeito da retenção, ligando com o mapa de memória do Capítulo 6.

A.7 Prática 6 — Sequência por etapas com SFC

Objetivo. Descrever uma sequência de máquina com passos, ações e transições, na linguagem de sequenciamento da norma.

Material. A bancada das práticas anteriores; duas saídas com sinalizador.

Procedimento.

1. descreva em texto a sequência: passo inicial $\rightarrow$ avanço do atuador A $\rightarrow$ avanço do atuador B $\rightarrow$ recuo de B $\rightarrow$ recuo de A $\rightarrow$ retorno ao passo inicial;
2. defina a **condição de transição** de cada passo, como expressão booleana sobre as entradas;
3. implemente em SFC, com um passo ativo por vez;
4. acrescente um passo de **falha** que seja ativado se uma transição não ocorrer dentro de 5 s, e que desligue todas as saídas;
5. force a falha por tempo e confirme que a sequência para em estado seguro.

Critério de aceite. A sequência avança apenas quando a transição é verdadeira; nunca há dois passos ativos ao mesmo tempo; a falha por tempo desliga as saídas e é sinalizada.

O que entregar. Diagrama SFC, tabela de transições com a condição de cada uma e registro do teste de falha por tempo.

A.8 Prática 7 — Escala de sinal analógico

Objetivo. Converter a leitura bruta de uma entrada analógica em grandeza de engenharia e avaliar a resolução obtida.

Material. Controlador com entrada analógica; potenciômetro ou transmissor de 4–20 mA; instrumento de referência.

Procedimento.

1. identifique a resolução da entrada, em bits, e a faixa elétrica aceita (Capítulo 7);
2. defina duas variáveis de referência, `bruto_min` e `bruto_max`, correspondentes ao fundo de escala;
3. escreva a expressão de escala em texto estruturado, usando tipo real para o resultado;
4. aplique três valores de entrada e compare a leitura com o instrumento de referência;
5. calcule a resolução obtida na grandeza de engenharia e registre o número de casas decimais que faz sentido exibir;
6. provoque sinal abaixo do mínimo e acima do máximo e defina o que o programa deve fazer nesses casos.

Critério de aceite. A grandeza calculada confere com a referência dentro da resolução declarada; o programa trata sinal fora de faixa; a documentação registra a resolução obtida e não apenas a do cartão.

O que entregar. Cálculo da resolução, a expressão de escala comentada e a tabela de comparação com o instrumento.

A.9 Prática 8 — Comparação, aritmética e alarme de faixa

Objetivo. Construir uma lógica de alarme com histerese e temporização, a partir da comparação de grandezas.

Material. A bancada da Prática 7 e um sinalizador de alarme.

Procedimento.

1. defina os limites alto e baixo de operação e os limites de alarme;
2. acione o alarme quando a grandeza ultrapassar o limite alto, e só o desligue quando ela cair abaixo de um valor *menor* — a histerese;
3. faça o mesmo para o limite baixo, na direção oposta;
4. acrescente uma temporização de **confirmação**: o alarme só é registrado se a condição persistir por 5 s;
5. registre a contagem de acionamentos do alarme em variável retentiva;
6. explique, em três linhas, por que histerese e confirmação por tempo reduzem alarme falso.

Critério de aceite. O alarme não oscila perto do limite; a oscilação transitória não gera registro; o contador de acionamentos confere com o número de eventos provocados.

O que entregar. O programa comentado, a tabela de limites e histerese e o registro dos eventos provocados.

A.10 Prática 9 — Intertravamento e partida segura

Objetivo. Implementar intertravamento a partir de uma tabela de causa e efeito e provar o seu funcionamento antes de energizar a máquina.

Material. Bancada com pelo menos duas entradas de proteção e duas saídas de atuação; roteiro de teste em branco (Capítulo 16).

Procedimento.

1. monte a tabela de causa e efeito do processo: para cada proteção, o que deve acontecer;
2. escreva a lógica de intertravamento em Ladder, com a proteção **em série** no comando de cada atuador que ela deve bloquear;
3. escreva também os alarmes correspondentes, deixando claro que o alarme *informa* e o intertravamento *age* (Capítulo 17);
4. preencha o roteiro de teste: caso, estímulo, resultado esperado e evidência;
5. execute o roteiro em bancada, com as saídas sem carga mecânica, e registre o resultado de cada caso;
6. provoque a partida com a proteção violada e confirme que o atuador não parte.

Critério de aceite. Nenhum atuador parte com a proteção violada; toda atuação de intertravamento tem alarme correspondente; o roteiro está preenchido e assinado.

O que entregar. Tabela de causa e efeito, programa, roteiro preenchido e a lista do que ficou impossível testar em bancada.

A.11 Prática 10 — Remota de E/S e supervisão

Objetivo. Ampliar a bancada com uma remota de E/S em rede e levar uma variável até uma tela de supervisão, com registro histórico.

Material. Controlador, remota de E/S acessível por protocolo aberto, computador com ambiente de supervisão.

Procedimento.

1. conecte a remota e documente endereço, protocolo e meio físico;
2. mapeie duas entradas e duas saídas da remota em variáveis com nome de processo;
3. monte uma tabela de registros ou de nós usados, com o significado de cada posição;
4. no programa, faça uma entrada da remota acionar uma saída local, e uma entrada local acionar uma saída da remota;

5. publique a variável na supervisão, crie uma tela simples e habilite histórico para ela;
6. com a bancada em operação, desconecte o cabo de rede da remota e registre o que acontece: o programa para? A saída da remota assume que estado?

Critério de aceite. A troca de sinais funciona nos dois sentidos; a tabela de registros está completa; o histórico registra a variável; o comportamento na perda de comunicação está descrito e é seguro.

O que entregar. Desenho da arquitetura de rede, tabela de registros, tela, e o parágrafo sobre o comportamento na falha de rede, ligando com o Capítulo 22.

A.12 Prática 11 — Documentação e versionamento do projeto

Objetivo. Fechar o conjunto de práticas com a entrega documental, que é o que permite a outra pessoa manter o sistema (Capítulo 15).

Material. Todos os arquivos produzidos nas práticas anteriores.

Procedimento.

1. monte o descritivo de arquitetura: o que o sistema faz, quais são as entradas, as saídas e os atuadores;
2. preencha a **lista de alocação de E/S**, ligando cada variável ao ponto físico;
3. monte a **tabela de tags** com nome, tipo, unidade, faixa e descrição;
4. versione o projeto-fonte e registre a versão no cabeçalho do programa;
5. execute o backup e faça o teste de **restauração** em bancada, registrando o resultado;
6. escreva o procedimento de operação em uma página: como partir, como parar, o que fazer em alarme.

Critério de aceite. Um colega que não participou da montagem consegue ligar a bancada e executar a sequência apenas com a documentação entregue; a restauração do backup foi testada e registrada.

O que entregar. O conjunto documental completo e a versão identificada do projeto.

Na prática

Escolha três práticas deste apêndice, monte-as como um único sistema — por exemplo: comando de partida com selo, temporização de partida e alarme de faixa — e integre-as em um programa com POUs separadas, uma por função. Depois:

- (a) documente o conjunto com o mesmo checklist da Prática 11;
- (b) monte um roteiro de teste que cubra os casos críticos (partida, parada, falha de proteção e retomada após queda de energia);
- (c) execute o roteiro em bancada e registre a evidência de cada caso;
- (d) escreva duas linhas sobre o que mudou no seu entendimento sobre “programar CLP” depois de separar as funções em POUs, em vez de escrever tudo em uma única varredura longa.

Exercícios

- a) Explique por que o contato de selo é necessário em um comando de partida e parada.
- b) Descreva a diferença entre o que se mede em uma **entrada** com contato NA e a lógica que se escreve para ligar um atuador a partir dela.
- c) Por que a escolha entre contato NA e NF em uma proteção de máquina não pode ser feita pelo tipo de chave disponível em campo?
- d) Liste duas situações em que contar *nível* produz resultado errado e a detecção de borda corrige.
- e) Compare o comportamento de TON, TOF e TP quando a entrada é desativada antes de o tempo completar.
- f) O que muda no resultado de uma contagem quando o acumulado é declarado retentivo? Relacione com o mapa de memória do Capítulo 6.
- g) Por que uma sequência em SFC precisa de um passo de falha com temporização?
- h) Na Prática 7, por que a resolução na grandeza de engenharia difere da resolução do cartão?
- i) Explique o papel da histerese e o da confirmação por tempo em uma lógica de alarme.
- j) Qual é a diferença prática entre alarme e intertravamento na Prática 9?
- k) Na Prática 10, o que deve acontecer com as saídas da remota quando a comunicação cai, e por que essa decisão não pode ser deixada para o momento da falha?
- l) Um colega entregou o programa sem lista de E/S, dizendo que “o código está comentado”. Aponte duas consequências concretas dessa decisão.
- m) Escolha uma das práticas e escreva o critério de aceite dela em uma frase verificável, como se fosse uma cláusula de contrato.

Capítulo B

Guia de conversão: legado para IEC 61131-3

Objetivos do capítulo

Ao final deste apêndice você deve ser capaz de:

- ler um trecho de programa escrito em dialeto de fabricante e dizer o que ele faz;
- reescrevê-lo em elementos padronizados pela IEC 61131-3, em Ladder e em texto estruturado;
- reconhecer os pontos em que a conversão **não** é mecânica;
- seguir um roteiro de conversão e registrar o que ficou pendente.

B.1 Para que serve e para que não serve

Este apêndice existe por um motivo prático: existe muito programa em operação escrito em dialeto de fabricante, e quem entra na manutenção precisa entendê-lo. O guia faz três coisas:

1. mostra a correspondência entre os mnemônicos do legado e os elementos que a IEC 61131-3 padroniza;
2. apresenta o **mesmo trecho nas duas formas**, lado a lado, para leitura comparada;
3. lista o que **não** se converte automaticamente, para que ninguém prometa migração mecânica.

Ele não serve para justificar escrever código novo em dialeto, e não substitui o estudo da norma: a correspondência apresentada é de *função*, e não garante equivalência em todos os casos de borda. A comparação completa está no Capítulo 13; aqui o foco é o código.

B.2 Os bits de status e os parâmetros do legado

Antes dos exemplos, é preciso nomear o que aparece no código antigo. As instruções legadas expõem *bits de status* e *parâmetros* próprios, que a norma substitui por entradas e saídas declaradas dos blocos padronizados — a Tabela B.1 faz essa correspondência.

Tabela B.1: Bits de status e parâmetros do legado e o equivalente normalizado.

Legado	O que é	Equivalente normalizado
.EN	instrução habilitada	entrada de habilitação do bloco (IN ou EN)
.DN	instrução concluída	saída de conclusão (Q, ENO)
.TT	temporizador em contagem	tempo decorrido ET maior que zero, ou estado do bloco
.ACC	tempo acumulado, em milissegundos	ET: tempo decorrido, tipado como duração
.PRE	valor de referência do tempo	PT: tempo de referência, tipado como duração
.POS e .DN do contador	acumulado e estado de conclusão	CV e Q
.UL	bit que “saiu” do registrador de deslocamento	não existe no bloco normalizado: precisa ser lido antes de deslocar
storage bit	endereço de armazenamento escolhido pelo programador, usado na detecção de borda	não existe : o estado fica dentro da instância do bloco de borda
RES	instrução que zera temporizador, contador ou palavra	entrada de reinício do bloco (RESET) ou atribuição de valor inicial

B.3 Como ler os exemplos

Cada caso traz três blocos:

- **legado** — na forma de mnemônicos, como o ambiente antigo exibe;
- **Ladder IEC** — os mesmos elementos gráficos, com os contatos, as bobinas e os blocos que a norma define;
- **texto estruturado** — a mesma lógica em linguagem textual, que costuma ser o destino mais útil da conversão.

Em Ladder, a convenção usada nos desenhos é:] [para contato normalmente aberto,]/[para contato normalmente fechado, () para bobina simples, (S) para bobina de retenção e (R) para bobina de desretenção.

B.4 Casos de conversão

B.4.1 Contato instantâneo, contato invertido e bobina

```
; leitura da chave e acionamento da lampada
XIC chave1      ; contato NA: conduz se a entrada estiver energizada
XIO chave2      ; contato NF: conduz se a entrada estiver desenergizada
OTE lampada1    ; bobina simples
```

Listing B.1: Legado: mnemônicos de contato e bobina.

```

      chave1          chave2          lampada1
|----] [-----+-----]/[------( )-----|
|  lampada1      |
|----] [-----+

```

Listing B.2: Ladder conforme a norma: mesmos elementos, sem mnemônico proprietário.

```

1 (* Leitura: a bobina vale enquanto a condicao for verdadeira *)
2 lampada1 := (chave1 OR lampada1) AND NOT chave2;

```

Listing B.3: O mesmo trecho em texto estruturado.

Observação. Aqui a diferença é de nomenclatura, não de lógica. O contato NA, o contato NF e as bobinas são elementos da norma; o que o legado acrescenta são nomes de instrução para eles. O que muda de fato é a *origem* do operando: no legado é um endereço de área de memória; na norma, um nome declarado pelo programador (Capítulo 13).

B.4.2 Série e paralelo

```

; serie = E logico: todos os contatos devem conduzir
XIC chave1    XIC chave2    XIO chave3        OTE lampada1
; paralelo = OU logico: basta um contato conduzir
XIC chave1
XIC chave2                                OTE lampada2

```

Listing B.4: Legado: associação em série (E) e em paralelo (OU).

```

1 lampada1 := chave1 AND chave2 AND NOT chave3;
2 lampada2 := chave1 OR chave2;

```

Listing B.5: Os dois casos em texto estruturado.

Observação. Conversão direta. Vale registrar que a associação em Ladder e a expressão booleana em texto estruturado são a mesma coisa escrita de duas formas — e que a expressão é mais fácil de revisar quando a condição cresce (Capítulo 14).

B.4.3 Retenção de saída

```

XIC  bt_liga      OTL lampada1      ; OTL energiza com retencao
XIC  bt_desliga   OTU lampada1      ; OTU desenergiza com retencao

```

Listing B.6: Legado: travamento e destravamento de saída.

```

      bt_liga          lampada1
|----] [------(S)-----|
      bt_desliga       lampada1
|----] [------(R)-----|

```

Listing B.7: Ladder conforme a norma: bobinas de retenção e de desretenção.

```

1 IF bt_liga THEN
2     lampada1 := TRUE;
3 END_IF;
4 IF bt_desliga THEN
5     lampada1 := FALSE;
6 END_IF;

```

Listing B.8: Retenção em texto estruturado, com ordem explícita.

Observação — e este é o caso mais delicado do apêndice. Se as duas condições forem verdadeiras na *mesma* varredura, o resultado depende da **ordem** em que as instruções aparecem no programa. A norma define blocos de retenção com precedência declarada (SR e RS), justamente para eliminar essa ambiguidade. Ao converter, portanto, não basta trocar OTL/OTU por S/R: é preciso garantir que as condições sejam mutuamente exclusivas ou documentar, de forma explícita, qual delas deve vencer.

B.4.4 Detecção de borda e pulso único

```
XIC limit_switch_1  ONS storage_1      ; um pulso por subida
XIC storage_1       ADD contador 1      ; incrementa uma unica vez
XIC limit_switch_1  OSR output_bit_1    ; pulso na subida
XIC limit_switch_1  OSF output_bit_2    ; pulso na descida
```

Listing B.9: Legado: pulso único na subida e na descida.

```
1 (* R_TRIG: borda de subida; F_TRIG: borda de descida *)
2 trig_subida(CLK := limit_switch_1);
3 IF trig_subida.Q THEN
4     contador := contador + 1;
5 END_IF;
6
7 trig_descida(CLK := limit_switch_1);
8 pulso_descida := trig_descida.Q;
```

Listing B.10: Blocos de detecção de borda em texto estruturado.

Observação. No legado, o programador escolhe um endereço de armazenamento — e reaproveitar esse endereço em duas linhas diferentes é uma fonte clássica de defeito difícil de achar. Na norma, o estado da borda vive dentro da instância do bloco, e cada uso tem a sua instância. Esta é uma conversão que **melhora** o programa original.

B.4.5 Temporização

```
XIC limit_switch_1  TON timer_1, 180    ; base em ms
XIC timer_1.dn      OTE light_2         ; apos 180 ms
XIC timer_1.tt      OTE light_3         ; enquanto cronometra
XIC timer_1.acc      MOV N7:0           ; acumulado para N7:0
```

Listing B.11: Legado: temporizador na energização e leitura dos bits de status.

```
1 (* PT e ET sao duracoes tipadas; nada de base implicita *)
2 temporizador(IN := limit_switch_1, PT := T#180ms);
3 light_2 := temporizador.Q;
4 light_3 := NOT temporizador.Q;
5 tempo_acumulado := temporizador.ET;
```

Listing B.12: Bloco de temporização padronizado, em texto estruturado.

Observação. Três armadilhas:

1. **base de tempo** — no legado, o valor 180 vale o que a base daquele temporizador disser; na norma, T#180ms diz o tempo por extenso e não admite ambiguidade. Converter o número sem conferir a base é o erro mais comum;

2. **retenção** — temporizadores não retentivos perdem o acumulado quando a entrada desliga; o padrão define blocos distintos para esse comportamento, e a escolha precisa ser consciente;
3. **comparar tempo** — ao comparar o tempo decorrido, use a mesma unidade nos dois lados da expressão.

B.4.6 Contagem

```
XIC limit_switch_1    CTU conter_1, 10    ; valor de referencia 10
XIC limit_switch_2    RES conter_1        ; zera o acumulado
XIC conter_1.dn        OTE light_1        ; referencia atingida
```

Listing B.13: Legado: contador crescente, reinício e leitura do bit de conclusão.

```
1 contador(CU := limit_switch_1, RESET := limit_switch_2, PV := 10);
2 light_1 := contador.Q;
3 valor_atual := contador.CV;
```

Listing B.14: Contador padronizado, em texto estruturado.

Observação. No legado, o acumulado continua incrementando *depois* de atingir o valor de referência, e o bit de conclusão permanece energizado até o reinício — comportamento que vários programas antigos usam de propósito. Ao converter, confirme no ambiente de destino o que acontece com o valor acumulado após a referência ser atingida e depois do reinício; não presuma.

B.4.7 Comparação

```
EQU valor_1 valor_2      OTE luz_a      ; igual
NEQ valor_1 valor_2      OTE luz_4      ; diferente
LES valor_1 valor_2      OTE luz_3      ; menor que
GRT valor_1 valor_2      OTE luz_1      ; maior que
LEQ valor_1 valor_2      OTE luz_2      ; menor ou igual
GEQ valor_1 valor_2      OTE luz_b      ; maior ou igual
CMP (valor_1*valor_2)-valor_3<valor_4    OTE luz_c      ; expressao
```

Listing B.15: Legado: instruções de comparação.

```
1 luz_a := (valor_1 = valor_2);
2 luz_4 := (valor_1 <> valor_2);
3 luz_3 := (valor_1 < valor_2);
4 luz_1 := (valor_1 > valor_2);
5 luz_2 := (valor_1 <= valor_2);
6 luz_b := (valor_1 >= valor_2);
7 luz_c := ((valor_1 * valor_2) - valor_3) < valor_4;
```

Listing B.16: As mesmas comparações como expressões.

Observação. A instrução de expressão do legado desaparece na conversão: o mesmo resultado se obtém escrevendo a expressão diretamente, com parênteses. É uma das conversões em que o código de destino fica mais curto que o original.

B.4.8 Aritmética e funções

```

XIC condicao ADD valor_1 valor_2 resultado
XIC condicao SUB valor_1 valor_2 resultado
XIC condicao MUL valor_1 valor_2 resultado
XIC condicao DIV valor_1 valor_2 resultado
XIC condicao MOD 10 3 resto
XIC condicao SQR valor_1 raiz
XIC condicao NEG valor_1 valor_negado
XIC condicao ABS valor_1 valor_absoluto
XIC condicao CPT (valor_1+valor_2)/2 media

```

Listing B.17: Legado: instruções aritméticas com destino explícito.

```

1 (* Um comando por instrucao legada vira uma atribuicao *)
2 resultado := valor_1 + valor_2;
3 resto := 10 MOD 3;
4 raiz := Sqrt(valor_1);
5 valor_negado := NEG(valor_1);
6 valor_absoluto := ABS(valor_1);
7 media := (valor_1 + valor_2) / 2;

```

Listing B.18: As mesmas operações como funções e expressões padronizadas.

Observação. Aqui aparece uma diferença que o legado esconde: **tipagem**. Em texto estruturado, o tipo de cada variável é declarado, e a divisão entre inteiros trunca o resultado. Se o projeto precisa de parte fracionária, a variável de destino tem de ser de tipo real e a conversão precisa ser explícita (Capítulo 6).

B.4.9 Operações bit a bit, cópia e zeragem

```

XIC c AND valor_1 valor_2 resultado
XIC c OR valor_1 valor_2 resultado
XIC c XOR valor_1 valor_2 resultado
XIC c NOT valor_1 resultado
XIC c MOV valor_1 N7:0
XIC c CLR N7:0

```

Listing B.19: Legado: lógica em palavra, cópia e zeragem.

```

1 (* Operadores bit a bit sobre bit strings *)
2 resultado := valor_1 AND valor_2;
3 resultado := valor_1 OR valor_2;
4 resultado := valor_1 XOR valor_2;
5 resultado := NOT valor_1;
6
7 (* Cópia e zeragem viram atribuicao *)
8 copia := valor_1;
9 palavra := 0; (* equivale a CLR *)

```

Listing B.20: As mesmas operações, com os operadores da norma.

Observação. Atenção ao duplo sentido dos operadores: **AND** e **OR** aplicados a variáveis booleanas são operadores lógicos; aplicados a palavras, são operações bit a bit. O símbolo é o mesmo; o tipo dos operandos é que decide. Programas legados frequentemente dependem disso sem documentar.

B.4.10 Deslocamento de bits

```
XIC c    BSL array_dint[0] input_1 10      ; desloca 10 bits a esquerda
; array_dint[0].ul recebe o bit que saiu
XIC c    BSR array_dint[0] input_1 10      ; desloca 10 bits a direita
```

Listing B.21: Legado: deslocamento de registrador com comprimento e bit de saída.

```
1 (* Leia o bit que vai sair ANTES de deslocar *)
2 bit_saida := extrair_bit(array_dint, comprimento);
3
4 (* Deslocamento sobre bit string *)
5 deslocado := SHL(palavra, 1);
6 deslocado := SHR(palavra, 1);
```

Listing B.22: Deslocamento com os operadores da norma.

Observação. Os operadores padronizados deslocam *uma palavra*, com número de posições declarado; não existe parâmetro de comprimento nem bit de saída automático. Quando o projeto precisa do comportamento de registrador de deslocamento com comprimento variável — o caso do “bit que sai” —, ele tem de ser montado: extrair o bit antes de deslocar e descartar o que passa do comprimento. Esta é uma conversão que **não** é um para um.

B.4.11 Chamada de rotina

```
XIC condicao    JSR rotina_2      ; executa a rotina e retorna
```

Listing B.23: Legado: salto para sub-rotina.

```
1 (* Chamada de bloco de funcao, com interface declarada *)
2 controlador_pid(SP := referencia, PV := medida, KP := 2.0, KI := 0.5);
3
4 (* Chamada de funcao: devolve valor, sem estado interno *)
5 media := MEDIA(valor_1, valor_2);
6
7 (* Alternativa em Ladder: o nome da POU entra na propria linha *)
8 (*   condicao ----[ ROTINA_B ]---- *)
```

Listing B.24: Chamadas de POU conforme a norma.

Observação. A sub-rotina do legado não tem interface declarada: ela recebe e devolve dados por variáveis compartilhadas, o que torna impossível saber, olhando a chamada, de que ela depende. A POU normatizada declara as suas entradas e saídas — e é isso que permite reutilizá-la em outro programa (Seção 11.3 e Capítulo 13).

B.5 As instruções do item 8.2 da fonte, com figuras e exemplos

O item 8.2 do material de origem (Rev. 05D) é a parte prática da apostila antiga: apresenta, uma a uma, as instruções e os blocos de função do ambiente **Allen-Bradley/RSLogix**. Isso é dialeto de fabricante — não é a IEC 61131-3. As figuras desta seção foram extraídas daquele item e são reproduzidas com o crédito da fonte, para consulta de quem precisa ler um programa que já existe.

Cada ficha traz o que a instrução faz, o elemento que a norma usa em seu lugar e a figura original com o exemplo da fonte. As conversões com código lado a lado estão nas seções anteriores deste apêndice; aqui o foco é o reconhecimento.

Atenção

Este material serve para **ler** código existente, não para escrever código novo. Escrever programa em dialeto de fabricante amarra a planta ao fornecedor (Capítulo 13).

B.5.1 Booleanas

XIC — *Examine If Closed*

Examina o dado binário e conduz quando ele está energizado: é o contato normalmente aberto. Se o dado estiver energizado, o restante da linha é habilitado. Equivale ao contato] [da norma e, em texto estruturado, à condição de um IF (Figura B.1). A Figura B.2 reproduz o exemplo da fonte.

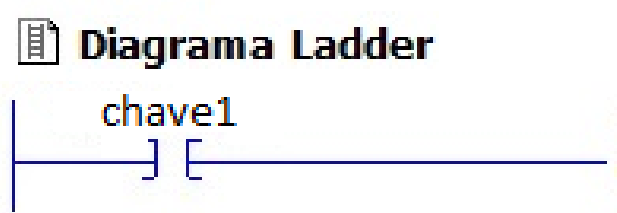


Figura B.1: Legado: XIC sobre chave1; o contato conduz quando o dado está energizado.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

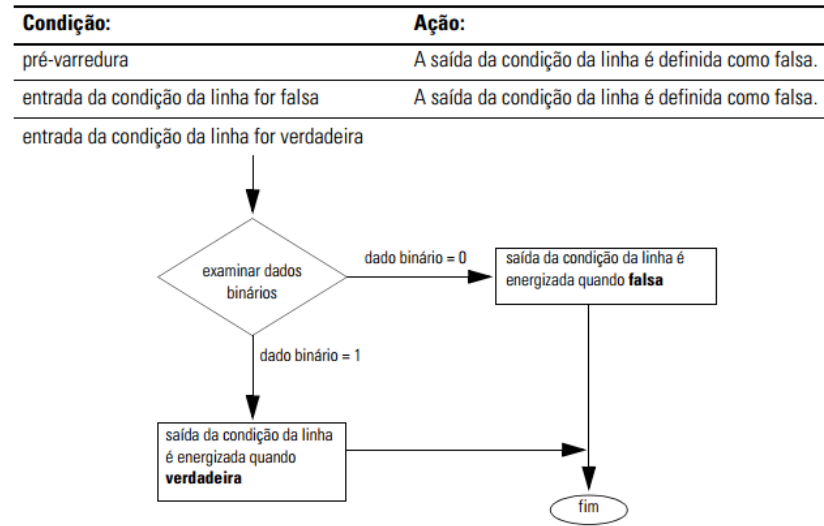


Figura B.2: Fluxo do exame da condição: dado binário igual a 0 deixa a saída da linha falsa; igual a 1, verdadeira.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

XIO — *Examine If Opened*

Examina o dado e conduz quando ele está *desenergizado*: é o contato normalmente fechado. Na norma, é o contato]/[e, em texto estruturado, a negação da condição. Os desenhos da

fonte estão nas Figuras B.3 e B.4.

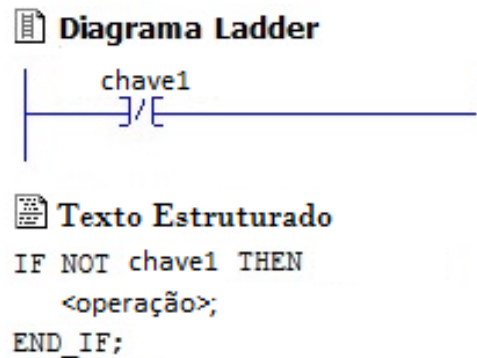


Figura B.3: Legado: XIO sobre `chave1`; o contato conduz quando o dado está desenergizado.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

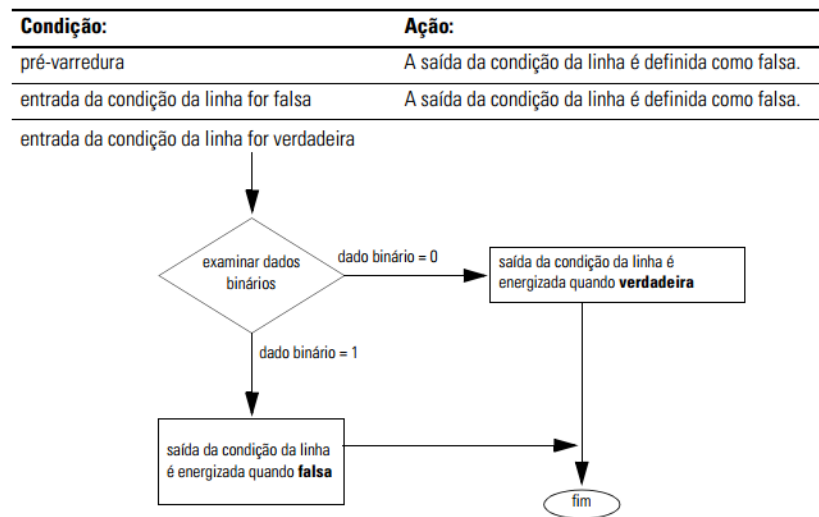


Figura B.4: Fluxo do exame com contato invertido: dado binário igual a 0 torna a saída verdadeira.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

OTE — Output Energize

Bobina simples: energiza ou desenergiza o dado binário conforme o resultado da linha. A saída só permanece energizada enquanto a condição da linha for verdadeira — é a bobina () da norma. O caso da fonte aparece na Figura B.5.



Figura B.5: Legado: OTE aciona `lâmpada1` enquanto o contato `chave` permanece fechado.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

OTL — *Output Latch*

Bobina de retenção: energiza o dado e o mantém em 1 mesmo depois de a linha ficar falsa, até que outra instrução o desenergize. Corresponde à bobina (S) da norma; a conversão discutida em *Retenção de saída* mostra por que ela exige cuidado com a ordem das instruções. A fonte ilustra esse caso na Figura B.6.

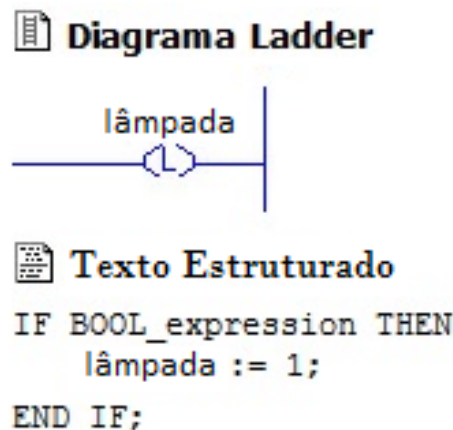


Figura B.6: Legado: OTL energiza lâmpada com retenção.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

OTU — *Output Unlatch*

Bobina de desretenção: devolve o dado a 0 quando habilitada e não altera nada quando desabilitada. É a bobina (R) da norma. A Figura B.7 reproduz o exemplo da fonte.

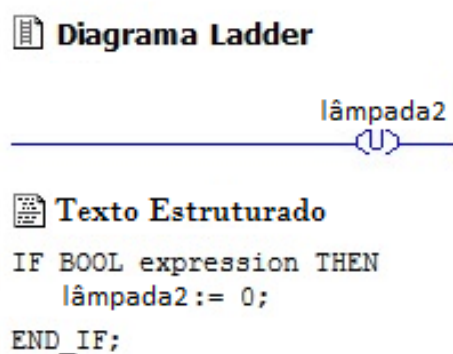


Figura B.7: Legado: OTU desenergiza lâmpada2 com retenção.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

ONS — *One Shot*

Deixa passar um único pulso: quando o bit de armazenamento está desenergizado, habilita o resto da linha e marca o bit; enquanto a entrada permanecer energizada, o restante da linha

não é habilitado de novo. Na norma, quem faz isso é um bloco de detecção de borda (**R_TRIG**) com instância própria — e é aí que a conversão melhora o programa, porque o estado deixa de depender de um endereço escolhido a mão. O exemplo da fonte está na Figura B.8.

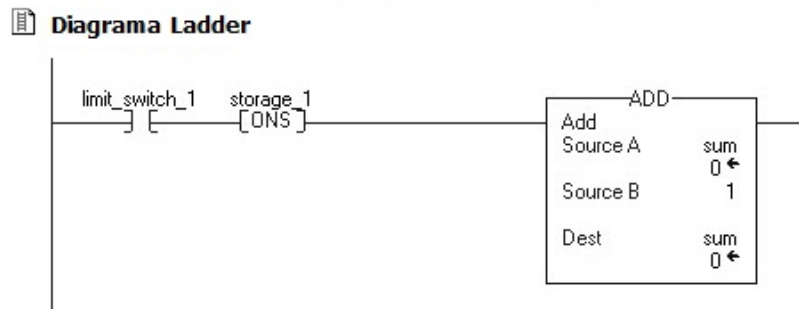


Figura B.8: Legado: ONS habilita ADD uma única vez por subida de `limit_switch_1`, usando `storage_1` como bit de armazenamento.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

OSR — One Shot Rising

Versão do pulso único que já devolve um bit de saída: energiza o bit de saída na transição de desenergizado para energizado e o desenergiza nas demais situações. O bit de saída pode ser usado em várias linhas depois. A fonte traz esse caso nas Figuras B.9 e B.10.

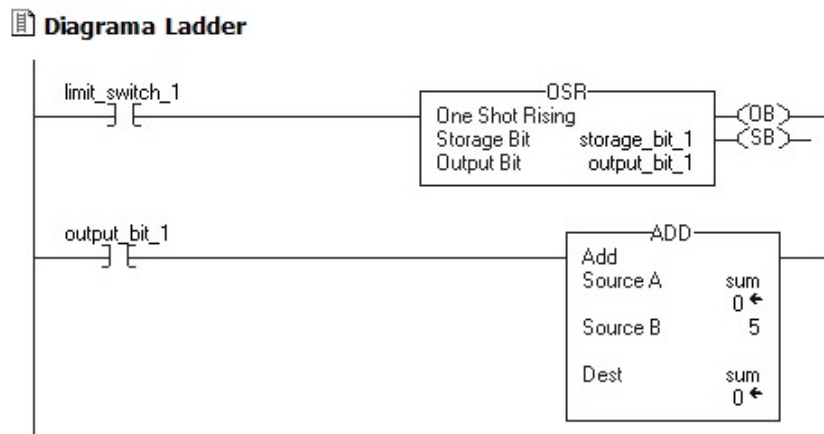


Figura B.9: Legado: OSR sobre `limit_switch_1` energiza `output_bit_1` uma vez por subida e soma 5 ao acumulado.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

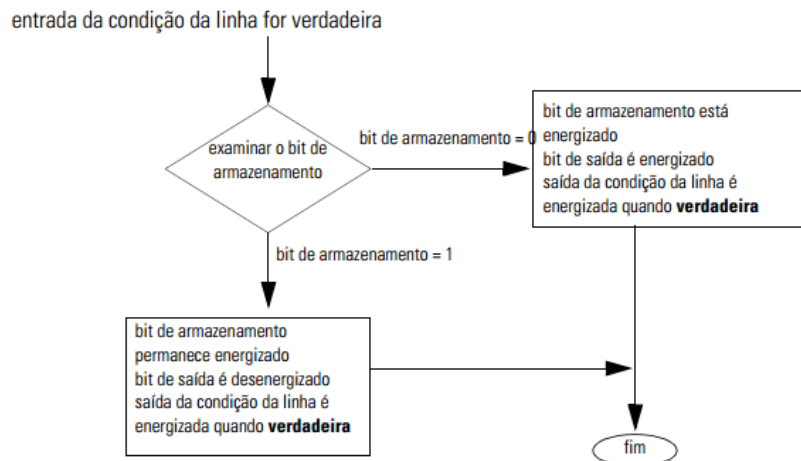


Figura B.10: Fluxo do OSR: o bit de saída depende do estado do bit de armazenamento e da condição da linha.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

OSF — One Shot Falling

O mesmo raciocínio do OSR, na transição inversa: energiza o bit de saída quando a entrada vai de energizada para desenergizada. Na norma, é o bloco F_TRIG. As Figuras B.11 e B.12 registram o caso como a fonte o apresenta.

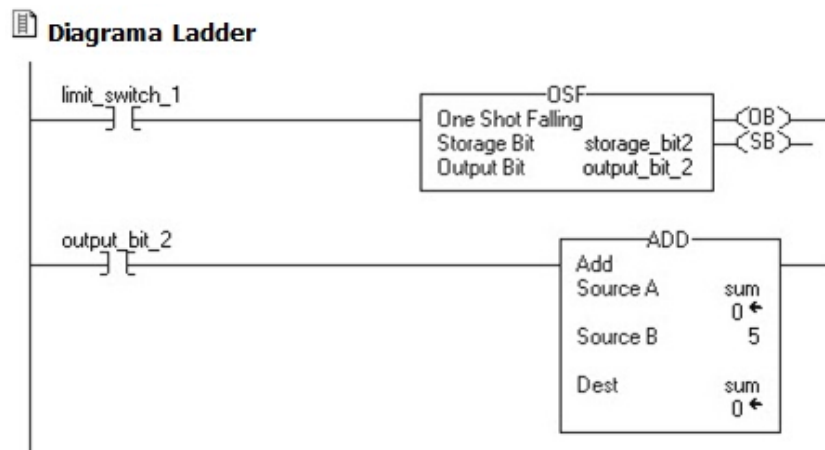


Figura B.11: Legado: OSF sobre limit_switch_1 energiza output_bit_2 uma vez por descida e soma 5 ao acumulado.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

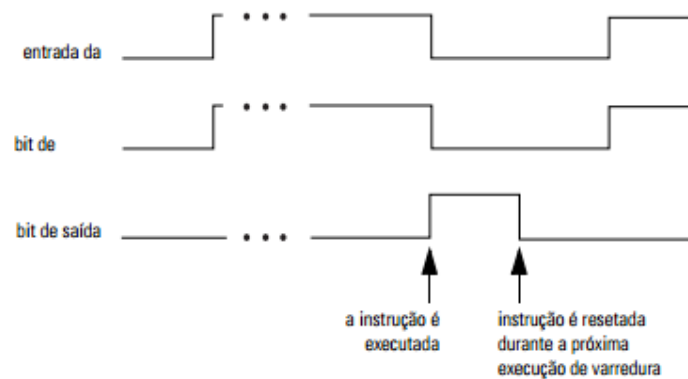


Figura B.12: Diagrama de tempo do OSF, que é o OSR visto pela transição de descida: a instrução é executada uma vez e só volta a operar na varredura seguinte.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

B.5.2 Temporizadores e contadores

TON — *Timer On Delay*

Temporizador não retentivo: acumula tempo enquanto a linha está verdadeira. *Preset* é o tempo de referência e *Accum* é o tempo já acumulado, em milissegundos. Enquanto conta, a saída EN fica ativa; quando o acumulado alcança o *Preset*, EN cai e DN sobe. Se a linha ficar falsa, o acumulado é zerado. Na norma, são as entradas IN e PT e as saídas Q e ET, com o tempo escrito por extenso (T#180ms) — sem base de tempo implícita. As Figuras B.13 e B.14 acompanham o exemplo da fonte.

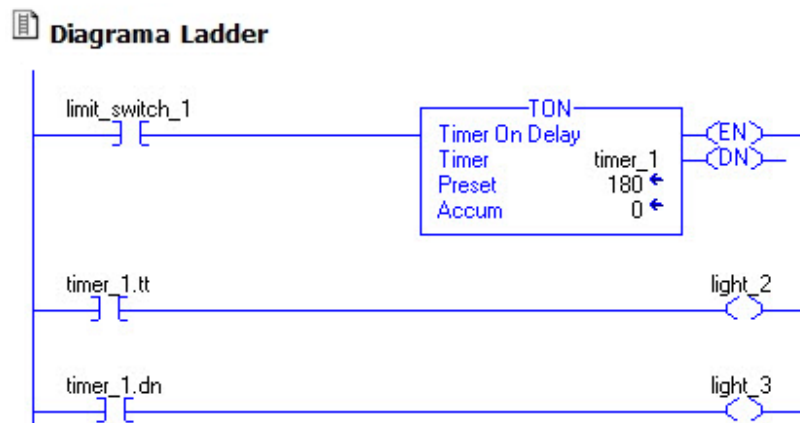


Figura B.13: Legado: TON com Timer `time_1`, *Preset* e *Accum*, acionado por `limit_switch_1`.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

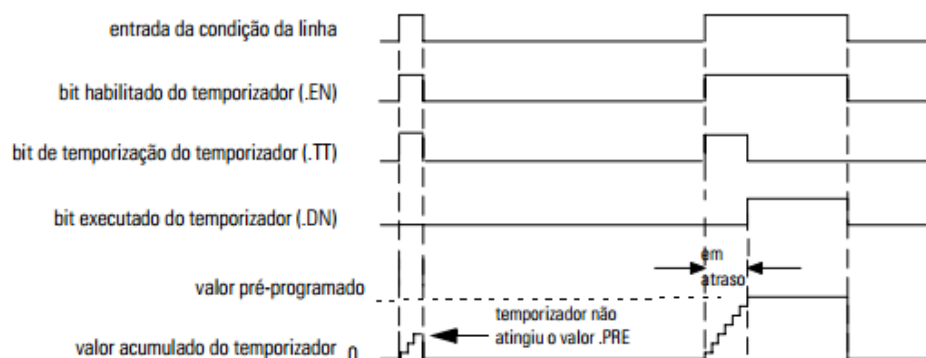


Figura B.14: Diagrama de tempo do TON: EN acompanha a entrada, TT indica a contagem e DN sobe ao atingir a referência.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

TOF — *Timer Off Delay*

Temporizador não retentivo com comportamento invertido: a saída DN é acionada assim que a instrução é energizada e permanece acionada até o acumulado atingir a referência, quando então cai. O tempo só começa a contar quando a entrada desliga — por isso é o temporizador usado em atrasos de desligamento. Os desenhos da fonte estão nas Figuras B.15 e B.16.

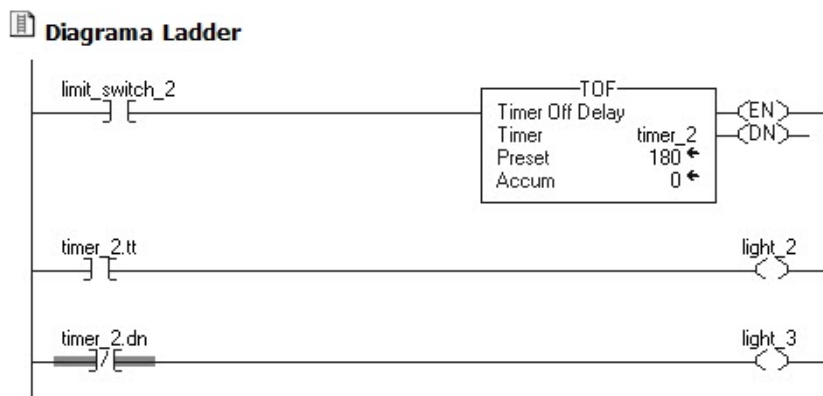


Figura B.15: Legado: TOF com Timer `time_2`; o atraso de 180 ms vale na desenergização.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

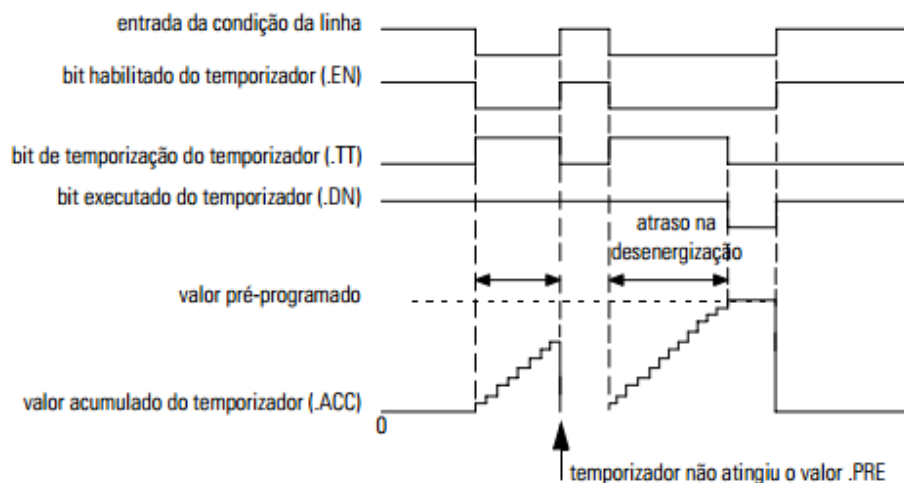


Figura B.16: Diagrama de tempo do TOF: a saída cai somente depois de transcorrido o tempo de referência após a entrada desligar.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

RTO — Retentive Timer On

Temporizador retentivo: acumula tempo enquanto habilitado, mas **retém** o valor acumulado quando a instrução é desabilitada. Para recomençar do zero é preciso reiniciá-lo com RES. Na norma, o reinício é a entrada RESET do bloco. A fonte traz esse caso nas Figuras B.17 e B.18.

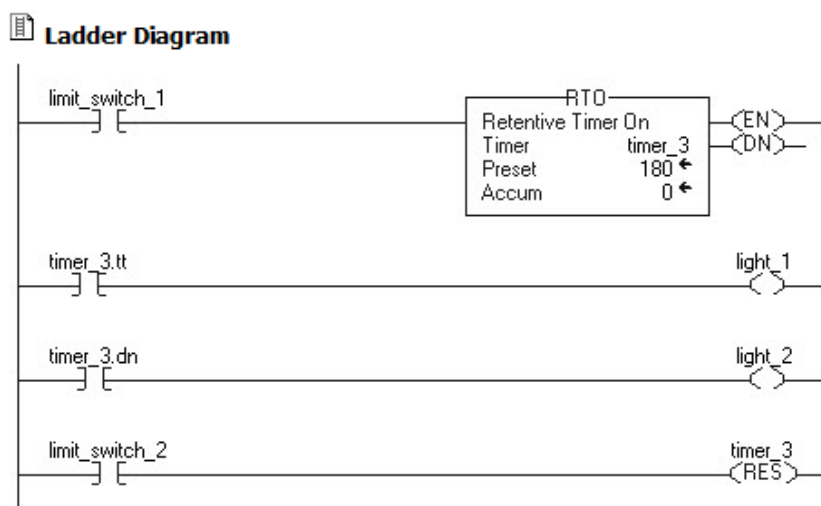


Figura B.17: Legado: RTO com Timer `time_3`; o acumulado é retido e só zera com RES.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

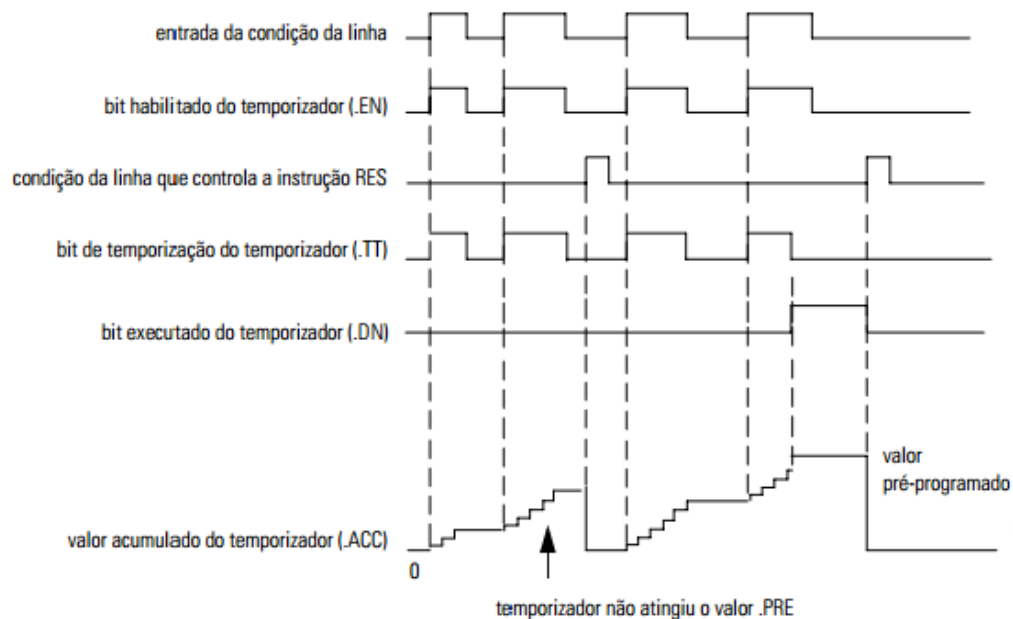


Figura B.18: Diagrama de tempo do RTO: o acumulado não é perdido quando a entrada desliga, ao contrário do TON.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

CTU — Count Up

Contador crescente: incrementa o acumulado em 1 a cada transição de desenergizado para energizado e mantém o valor quando desenergizado. O acumulado continua subindo mesmo depois de o bit de conclusão DN ser ativado, até que o contador seja reiniciado. As Figuras B.19 e B.20 registram o caso como a fonte o apresenta.

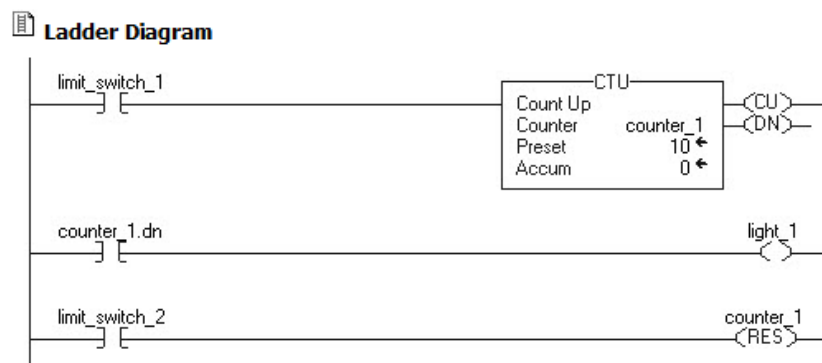


Figura B.19: Legado: CTU com referência 10; o bit DN energiza light_1 e RES zera o acumulado.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

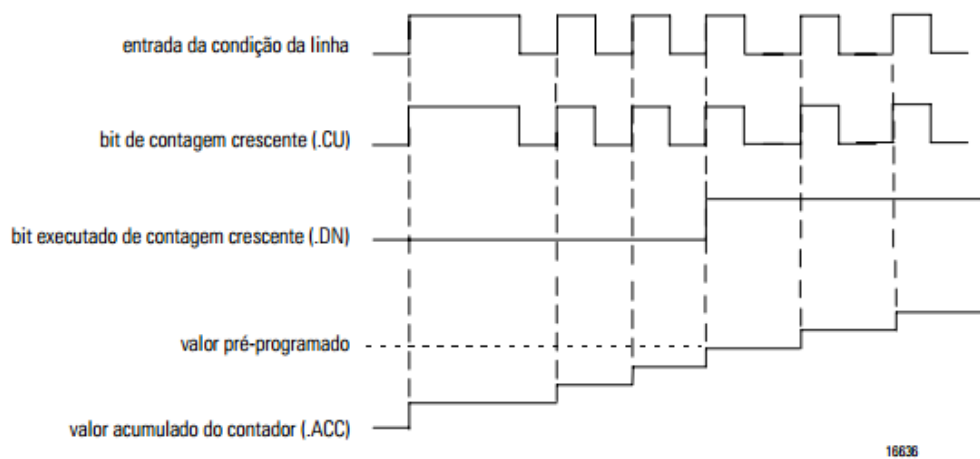


Figura B.20: Diagrama de tempo do CTU: a saída DN sobe ao alcançar o valor de referência e permanece até o reinício.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

CTD — *Count Down*

Contador decrescente: a estrutura é a mesma do CTU, mas cada transição *decrementa* o acumulado. Também precisa ser reiniciado para voltar ao valor inicial. A Figura B.21 reproduz o exemplo da fonte.

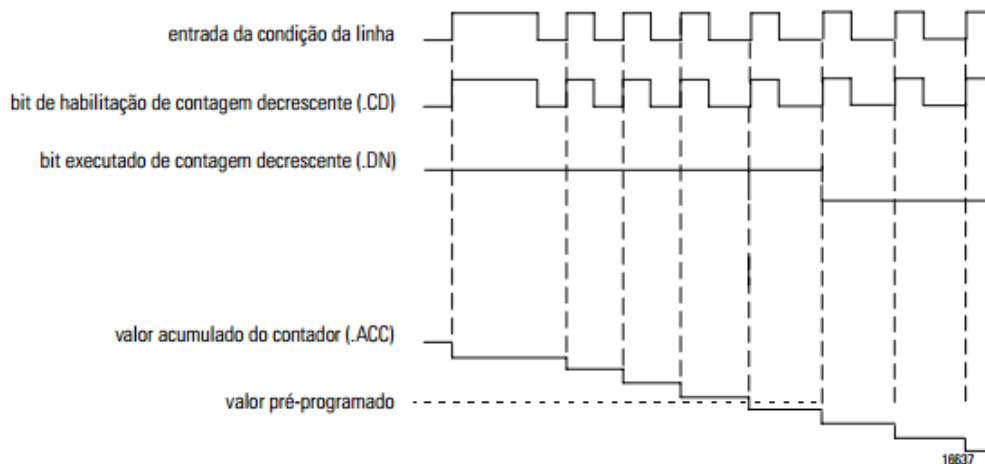


Figura B.21: Diagrama de tempo do CTD: o acumulado decresce a cada pulso de entrada.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

RES

Reinicia temporizadores, contadores e palavras: devolve o acumulado a zero. Na norma, é a entrada de reinício do próprio bloco ou uma atribuição de valor inicial. O exemplo da fonte está na Figura B.22.

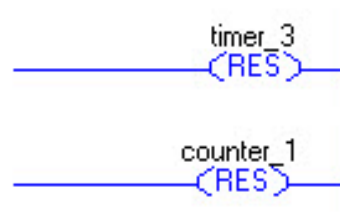


Figura B.22: Legado: RES zera counter_1 quando limit_switch_2 fecha.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

B.5.3 Comparadores

As sete instruções de comparação do legado trabalham do mesmo modo: avaliam a relação entre *Source A* e *Source B* e habilitam a linha quando a relação é verdadeira. A norma as substitui por operadores relacionais em expressão — $=$, $<>$, $<$, $>$, $<=$ e $>=$ —, que funcionam tanto em texto estruturado quanto na comparação dentro de blocos.

EQU — Equal

Verdadeira quando *Source A* é igual a *Source B*. Serve para números e também para comparar duas palavras, que serão iguais se seus caracteres coincidirem. A Figura B.23 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.



Figura B.23: Exemplo de EQU na fonte: se valor_1 for igual a valor_2, luz_a é energizada.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

NEQ — Not Equal To

Verdadeira quando os dois valores são diferentes. A Figura B.24 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.



Figura B.24: Exemplo de NEQ na fonte: com valores diferentes, luz_4 é energizada.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

LES — *Less Than*

Verdadeira quando **Source A** é menor que **Source B**. A Figura B.25 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

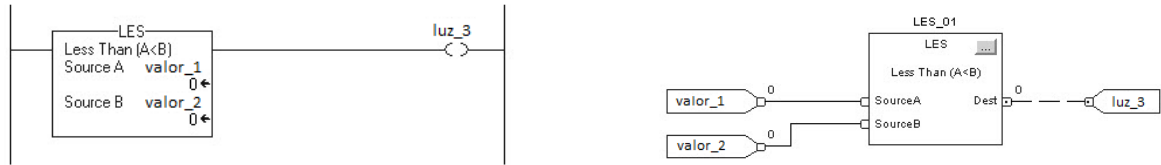


Figura B.25: Exemplo de LES na fonte: luz_3 é energizada quando valor_1 é menor que valor_2.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

GRT — *Greater Than*

Verdadeira quando **Source A** é maior que **Source B**. A Figura B.26 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

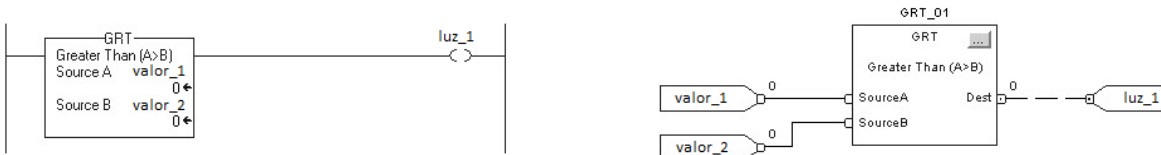


Figura B.26: Exemplo de GRT na fonte: luz_1 é energizada quando valor_1 é maior que valor_2.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

LEQ — *Less Than or Equal To*

Verdadeira quando **Source A** é menor ou igual a **Source B**. A Figura B.27 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

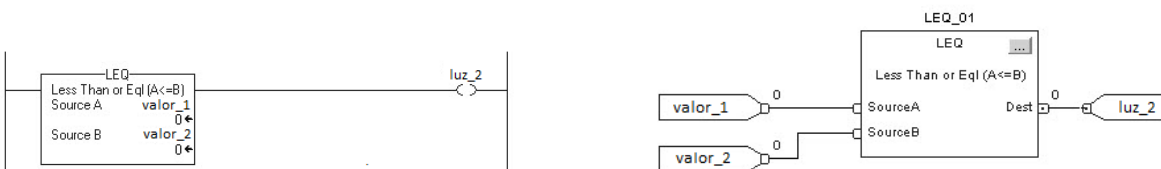


Figura B.27: Exemplo de LEQ na fonte: luz_2 é energizada quando valor_1 é menor ou igual a valor_2.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

GEQ — *Greater Than or Equal To*

Verdadeira quando **Source A** é maior ou igual a **Source B**. A Figura B.28 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

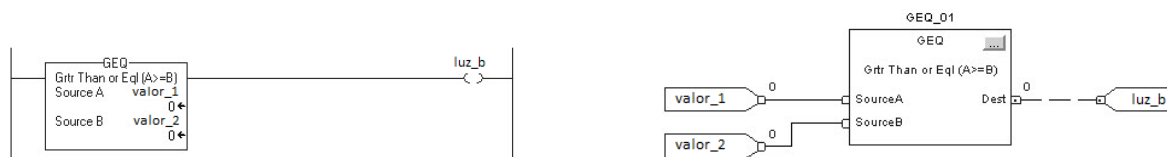


Figura B.28: Exemplo de GEQ na fonte: luz_b é energizada quando valor_1 é maior ou igual a valor_2.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

CMP — Compare

Compara expressões aritméticas escritas pelo programador em *Expression*, usando operadores, nomes de variáveis e valores imediatos. Exige parênteses nas expressões mais complexas; executa mais devagar e ocupa mais memória que as demais comparações, mas resolve uma expressão inteira em uma única instrução. Na norma não existe instrução equivalente: escreve-se a expressão direto, com os operadores relacionais. As Figuras B.29 e B.30 acompanham o exemplo da fonte.



Figura B.29: Legado: CMP avalia a expressão $(\text{valor}_1 * \text{valor}_2) - \text{valor}_3 < \text{valor}_4$.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

Operador:	Descrição:	Ótimo:
+	adição	DINT, REAL
-	subtração/negativo	DINT, REAL
*	multiplicação	DINT, REAL
/	divisão	DINT, REAL
=	igual	DINT, REAL
<	menor do que	DINT, REAL
<=	menor ou igual a	DINT, REAL
>	maior do que	DINT, REAL
>=	maior ou igual a	DINT, REAL
<>	diferente de	DINT, REAL
**	expoente (x elevado a y)	DINT, REAL
ABS	valor absoluto	DINT, REAL
ACS	arco co-seno	REAL
AND	bitwise AND	DINT
ASN	arco seno	REAL
ATN	arco tangente	REAL
COS	co-seno	REAL

Operador:	Descrição:	Ótimo:
DEG	radianos para graus	DINT, REAL
FRD	BCD para inteiro	DINT
LN	natural log	REAL
LOG	logaritmo na base de 10	REAL
MOD	módulo-divisão	DINT, REAL
NOT	bitwise complemento	DINT
OR	bitwise OR	DINT
RAD	graus para radianos	DINT, REAL
SIN	seno	REAL
SQR	raiz quadrada	DINT, REAL
TAN	tangente	REAL
TOD	inteiro para BCD	DINT
TRN	truncado	DINT, REAL
XOR	bitwise exclusivo OR	DINT

Figura B.30: Tabela de operadores aceitos na expressão de CMP e de CPT, com o tipo de dado exigido por cada um.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

B.5.4 Instruções matemáticas

As instruções desta família usam *Source A* e *Source B* como operandos e *Dest* como destino — a exceção é o CPT, que avalia uma expressão. Na norma, o destino é simplesmente o lado esquerdo de uma atribuição, e o tipo da variável de destino define o resultado.

ADD — Add

Soma *Source A* a *Source B* e guarda o resultado em *Dest*. A Figura B.31 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

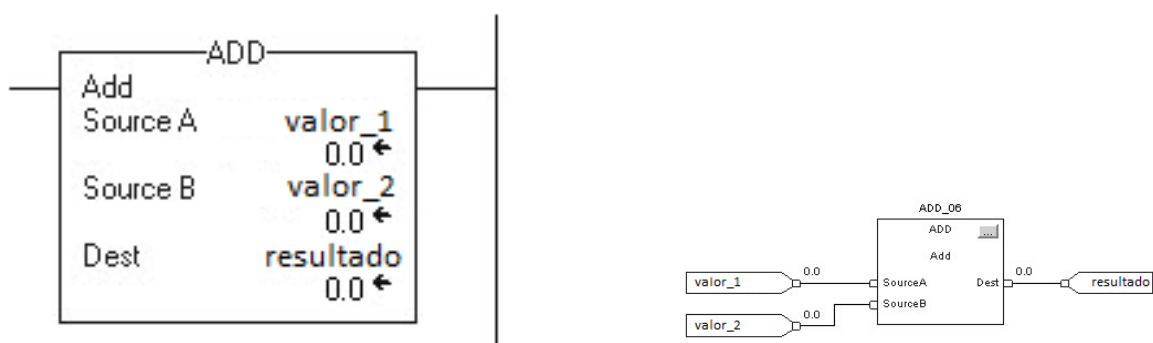


Figura B.31: Exemplo de ADD na fonte: a soma de **valor_1** e **valor_2** é armazenada em **resultado**.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

SUB — Subtract

Subtrai *Source B* de *Source A* e guarda o resultado em *Dest*. A Figura B.32 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

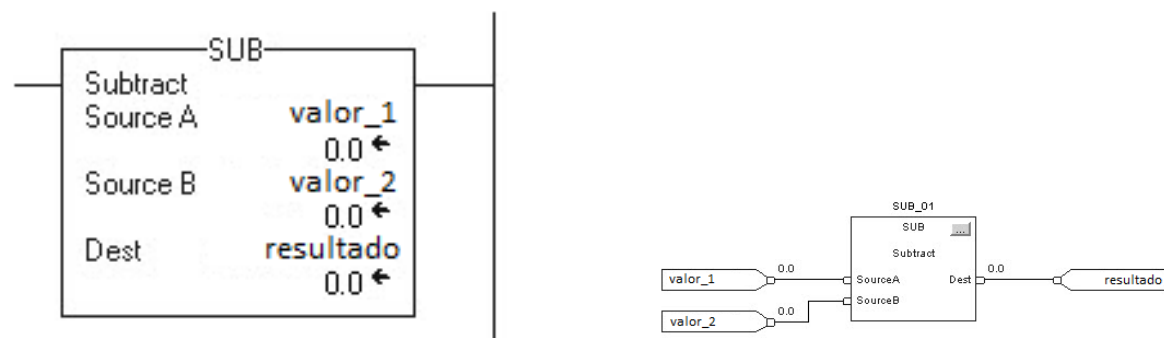


Figura B.32: Exemplo de SUB na fonte: **valor_1** menos **valor_2** é armazenado em **resultado**.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

MUL — Multiply

Multiplica Source A por Source B e guarda o resultado em Dest. A Figura B.33 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

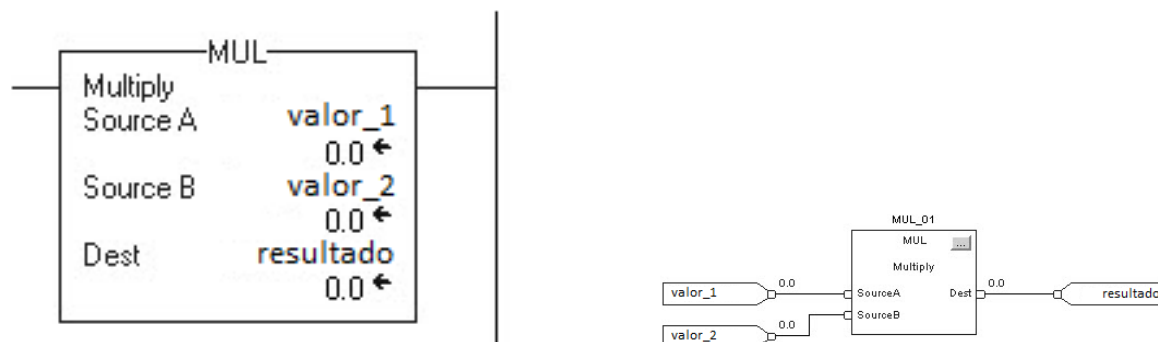


Figura B.33: Exemplo de MUL na fonte: o produto de **valor_1** e **valor_2** é armazenado em **resultado**.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

DIV — Divide

Divide Source A (dividendo) por Source B (divisor) e guarda o resultado em Dest. Atenção à tipagem: em texto estruturado, a divisão entre inteiros trunca o resultado. A Figura B.34 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

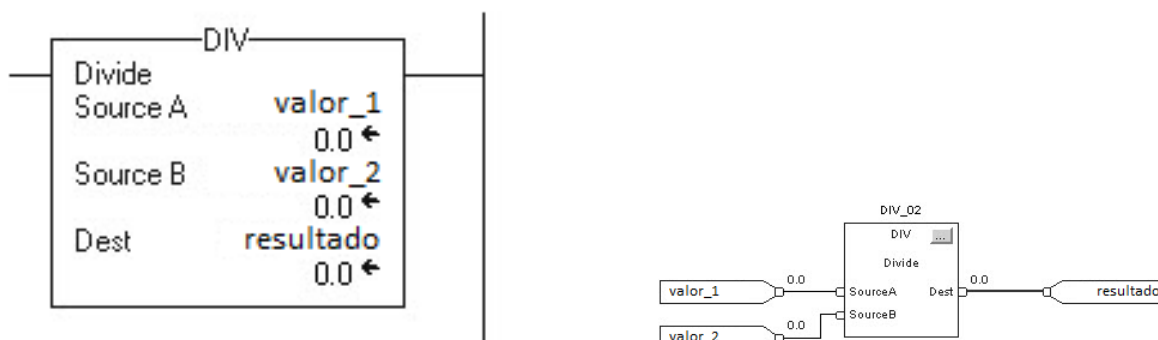


Figura B.34: Exemplo de DIV na fonte: **valor_1** dividido por **valor_2**, com o resultado em **resultado**.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

MOD — Modulo

Divide Source A por Source B e guarda em Dest apenas o **resto** da divisão. Na norma, é o operador MOD. A Figura B.35 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

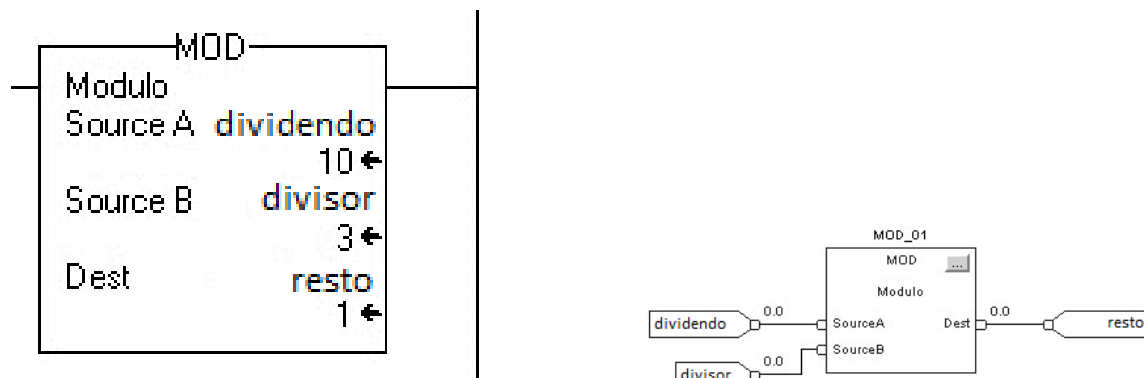


Figura B.35: Exemplo de MOD na fonte: o resto da divisão do dividendo 10 pelo divisor 3 é armazenado em **resto**.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

SQR — Square Root

Calcula a raiz quadrada do valor de **Source** e guarda o resultado em **Dest**. Se o valor for negativo, a instrução usa o valor absoluto. Na norma, é a função **SQRT**. A Figura B.36 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

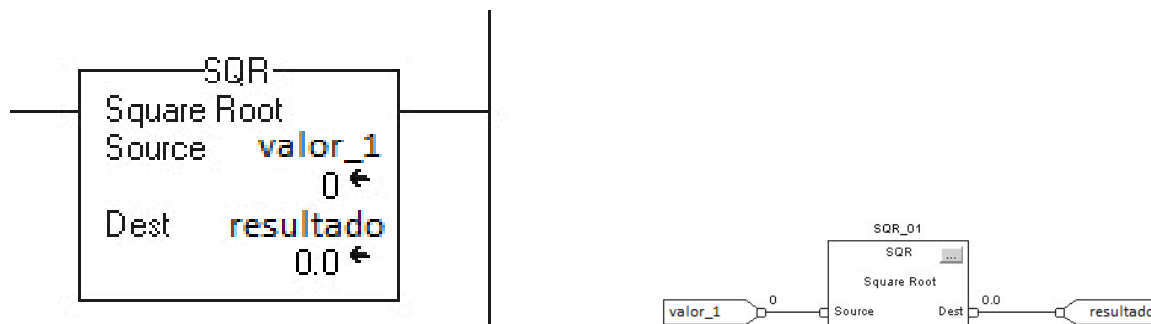


Figura B.36: Exemplo de SQR na fonte: a raiz quadrada de **valor_1** é armazenada em **resultado**.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

NEG — Negate

Inverte o sinal do valor de **Source**: a negação de um valor negativo resulta positiva, e vice-versa. A Figura B.37 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

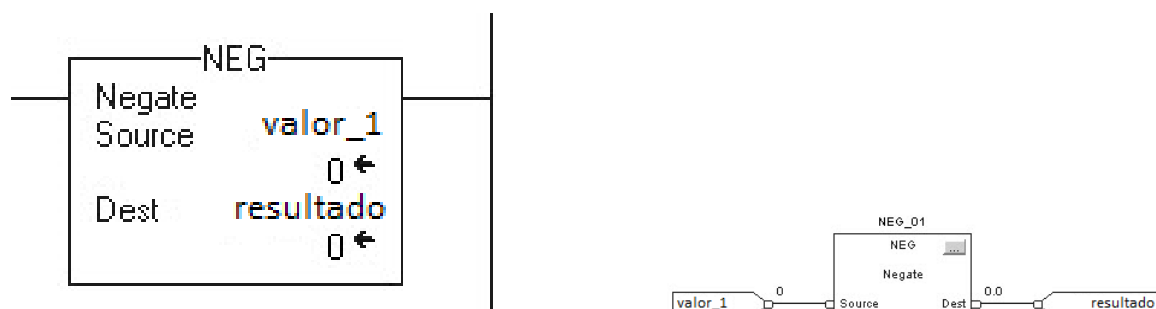


Figura B.37: Exemplo de NEG na fonte: a negação de **valor_1** é armazenada em **resultado**.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

ABS — *Absolute Value*

Obtém o módulo do valor de **Source** — o valor sem o sinal — e o guarda em **Dest**. A Figura B.38 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

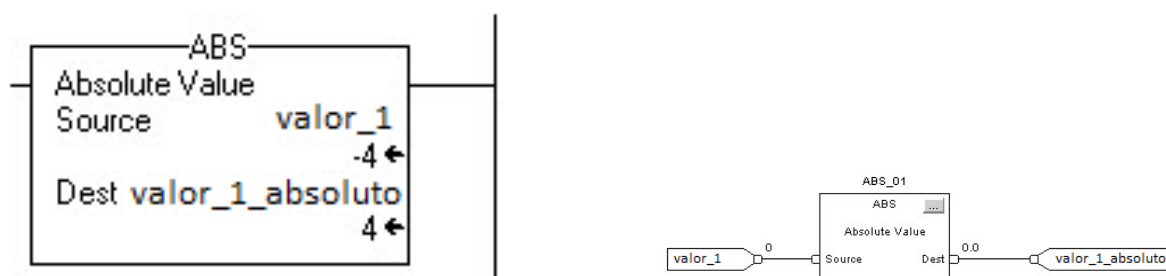


Figura B.38: Exemplo de ABS na fonte: o módulo de **valor_1** é guardado no destino.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

CPT — *Compute*

Avalia a expressão aritmética definida em *Expression* e guarda o resultado em **Dest**. Como o CMP, resolve expressões complexas em uma única instrução e admite a mesma tabela de operadores. O exemplo da fonte está na Figura B.39.

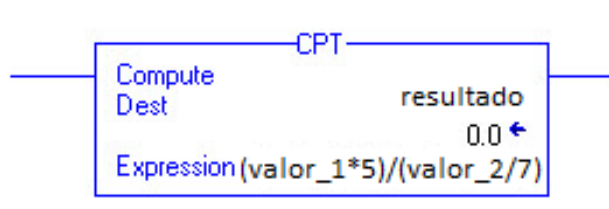


Figura B.39: Legado: CPT avalia $(\text{valor}_1 \cdot 5) / (\text{valor}_2 / 7)$ e guarda o resultado.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

B.5.5 Operadores lógicos e de movimentação

Estas instruções trabalham sobre *bits* da palavra, não sobre o valor numérico dela. É a mesma advertência que fecha a seção de conversão deste apêndice: o símbolo AND sobre variáveis booleanas é operação lógica; sobre palavras, é operação bit a bit.

MOV — *Move*

Copia a informação de **Source** para **Dest**; a informação continua armazenada em **Source**. Na norma, é uma atribuição. O caso da fonte aparece na Figura B.40.

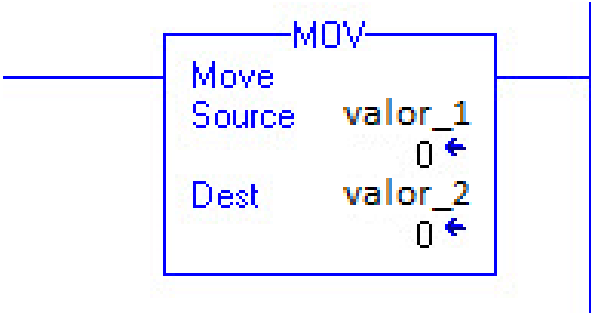


Figura B.40: Legado: MOV copia valor_1 para valor_2.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

AND — *Bitwise AND*

Faz a operação lógica E bit a bit entre **Source A** e **Source B** e guarda o resultado em **Dest**. A Figura B.41 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

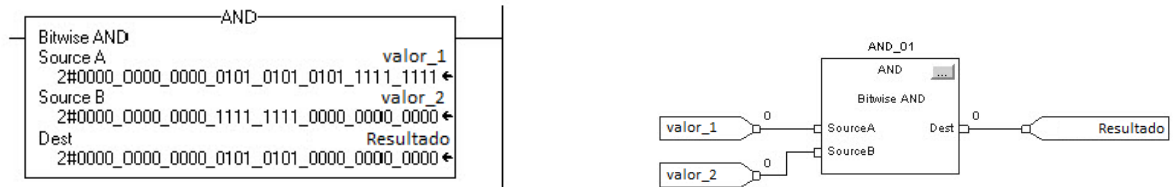


Figura B.41: Exemplo de AND na fonte: a operação bit a bit entre valor_1 e valor_2 resulta em resultado.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

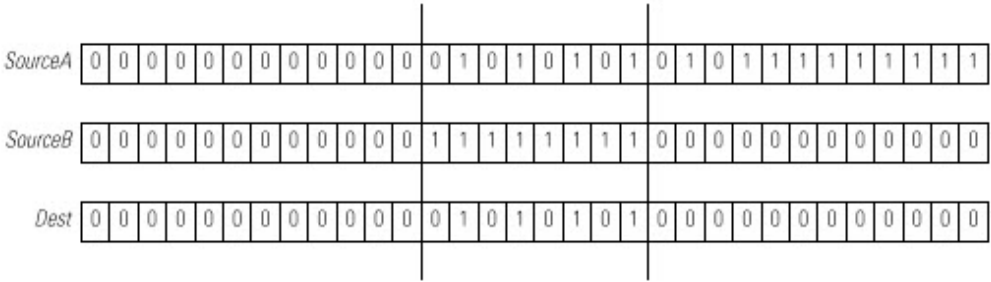


Figura B.42: Representação da operação AND sobre os bits: o resultado tem 1 apenas onde os dois operandos têm 1.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

OR — Bitwise Inclusive OR

Faz a operação lógica OU bit a bit entre os dois operandos e guarda o resultado em Dest. A Figura B.43 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

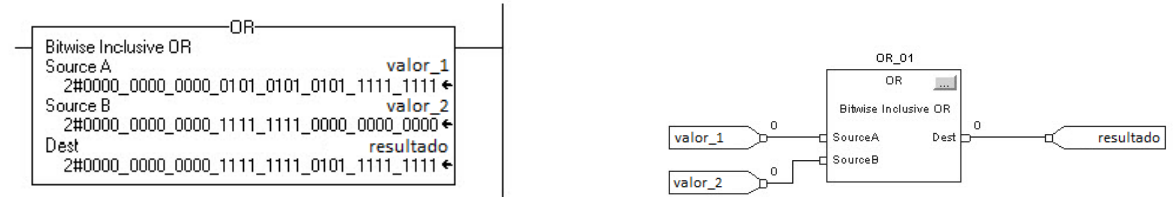


Figura B.43: Exemplo de OR na fonte: a operação bit a bit entre valor_1 e valor_2 resulta em resultado.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

Bit em Source A	Bit em Source B	Bit de Destino
0	0	0
0	1	1
1	0	1
1	1	1

Figura B.44: Tabela-verdade do OR bit a bit, como a fonte a apresenta.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

XOR — Bitwise Exclusive OR

Faz a operação OU exclusivo bit a bit: o resultado tem 1 onde os operandos *diferem*. A Figura B.45 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

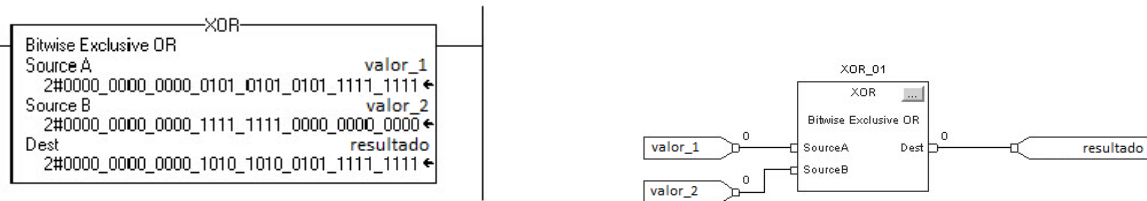


Figura B.45: Exemplo de XOR na fonte: a operação bit a bit entre `valor_1` e `valor_2` resulta em `resultado`.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

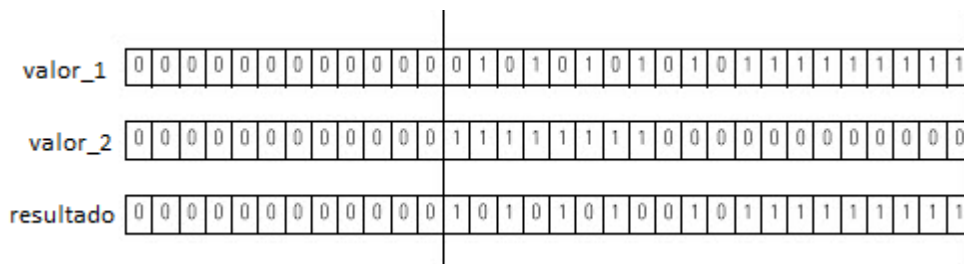


Figura B.46: Representação da operação XOR sobre os bits, com o resultado bit a bit.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

NOT — *Bitwise NOT*

Inverte todos os bits de **Source** e guarda o resultado em **Dest**: onde havia 1, passa a haver 0, e vice-versa. A Figura B.47 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.



Figura B.47: Exemplo de NOT na fonte: a inversão bit a bit de `valor_1` resulta em `resultado`.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.



Figura B.48: Representação da operação NOT sobre os bits: todos os bits são invertidos.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

CLR

Reseta todos os bits armazenados em **Dest**, igualando-os a zero. Na norma, é uma atribuição de valor zero — e, em Ladder, a bobina de desretenção faz o mesmo para uma saída retida. A fonte ilustra esse caso na Figura B.49.

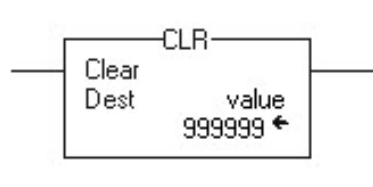


Figura B.49: Legado: CLR zera todos os bits da variável de destino.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

B.5.6 Movimentação de bits e desvio

BSL — Bit Shift Left

Desloca os bits de *Array* uma posição para a esquerda. O bit que sai do comprimento definido em *Length* vai para o bit *.UL*; os demais andam uma posição, e o bit indicado em *Source Bit* entra na posição 0. *.EN* indica que a instrução está habilitada e *.DN* que o deslocamento foi concluído. A norma não tem instrução equivalente: o deslocamento é montado com as funções de deslocamento sobre *bit strings* — e o bit que sai precisa ser lido *antes* do deslocamento. A Figura B.50 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.

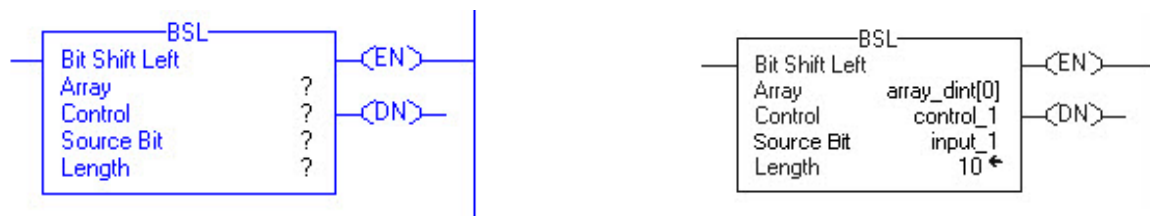


Figura B.50: Legado: BSL sobre `array_dint[0]`, com *Control*, *Source Bit*, *Length* e os bits EN e DN.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

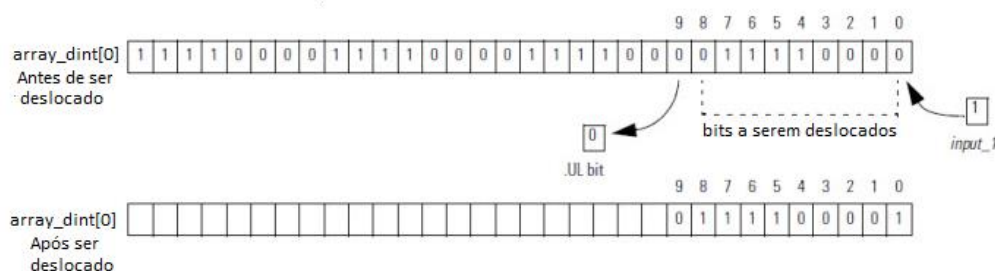


Figura B.51: Efeito do BSL sobre os bits: o bit 9 é descarregado em *.UL*, os demais andam para a esquerda e *input_1* entra no bit 0.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

BSR — Bit Shift Right

O mesmo que o BSL, no sentido oposto: desloca os bits uma posição para a direita, descarrega o último bit do comprimento em .UL e insere o bit de *Source Bit* na nova posição livre. A Figura B.52 mostra o exemplo nas duas representações, em Ladder e em diagrama de blocos.



Figura B.52: Legado: BSR sobre `array_dint[0]`, com *Control*, *Source Bit* e *Length*.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

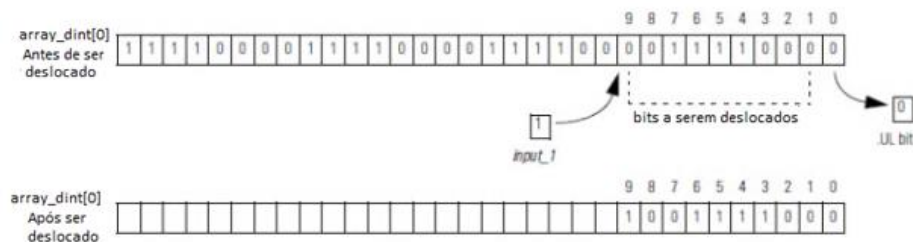


Figura B.53: Efeito do BSR sobre os bits: o bit 9 é descarregado em .UL e os demais andam para a direita.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

JSR — Jump to Subroutine

Salta da rotina principal para uma sub-rotina, executa-a e volta. O problema do legado não é o salto: é que a sub-rotina **não declara interface** — recebe e devolve dados por variáveis compartilhadas, o que torna impossível saber, olhando a chamada, de que ela depende. Na norma, chama-se uma POU com entradas e saídas declaradas (Seção 11.3). O caso da fonte aparece na Figura B.54.

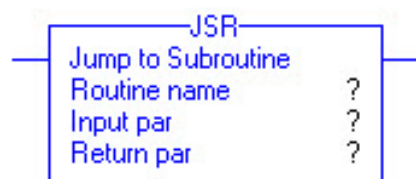


Figura B.54: Legado: JSR chama a rotina indicada em *Routine name*, com parâmetros de entrada e de retorno.

Fonte: TAVARES, L. O.; MONTEIRO, N. N. Controladores Lógicos Programáveis — apostila final (teórica e prática), Rev. 05D, 2021, item 8.2.

B.6 O que não se converte mecanicamente

A Tabela B.2 reúne os pontos em que a conversão exige decisão de projeto. Ela é o orçamento real de uma migração.

Tabela B.2: Pontos que exigem decisão na conversão, e o que verificar.

Ponto	Por que não converte sozinho	O que verificar
Base de tempo dos temporizadores	O valor numérico do legado depende de uma base implícita	Conferir cada temporizador e escrever o tempo por extenso, com unidade
Comportamento do contador após a re-ferência	O legado continua contando e mantém o bit de conclusão ativo	Confirmar no ambiente de destino o que ocorre com o acumulado e com a saída de conclusão
Detecção de borda com bit de armazenamento	O bit é um endereço compartilhado, escolhido pelo programador	Criar uma instância de bloco por uso e eliminar o endereço compartilhado
Precedência entre retenção e desretenção	Depende da ordem das instruções na varredura	Garantir exclusividade mútua ou usar bloco com precedência declarada
Deslocamento com comprimento e bit de saída	Não há equivalente direto nos operadores padronizados	Montar a função de registro de deslocamento de forma explícita e testá-la
Endereçamento físico	A norma não padroniza endereçamento de E/S	Refazer a ligação entre cada variável e o ponto físico, ponto por ponto
Blocos de sistema e de comunicação	São próprios de cada ambiente	Isolar em bloco de adaptação e testar antes de migrar o restante
Tipos e conversões	O legado converte implicitamente em vários casos; a norma pede tipo declarado	Declarar tipos e tornar explícitas as conversões
Primeira varredura e retomada	Lógica que depende de inicialização pode não ter equivalente óbvio	Tratar explicitamente a primeira varredura e a retomada após queda de energia (Capítulo 14)

B.7 Roteiro de conversão

A sequência abaixo evita o erro de começar pelo código:

1. **inventarie** o que existe: quais programas, quais rotinas, quais blocos proprietários e onde cada um é usado;
2. **identifique os blocos proprietários** e classifique-os em: tem equivalente padronizado, precisa ser montado, ou é específico de sistema e não tem equivalente;

3. **documente o comportamento antes de mudar o código:** para cada rotina, escreva em texto o que ela faz, com as condições de acionamento e de parada;
4. **reescreva em POU**, uma responsabilidade por unidade, com nome de processo;
5. **declare os tipos** e torne explícitas as conversões;
6. **teste em simulação** e depois em bancada, com o roteiro do Capítulo 16, antes de tocar a máquina;
7. **compare o comportamento** observado com o documentado no passo 3 — diferença encontrada aqui é conversão mal feita, não “melhoria”;
8. **registre o que ficou pendente**, com o motivo, e guarde o mapa de conversão junto do projeto.

Atenção

Conversão de programa em operação **não** é oportunidade de melhorar a lógica sem controle. Se o objetivo é migrar, o alvo é reproduzir o comportamento atual; qualquer melhoria entra depois, como alteração identificada, testada e registrada. Misturar as duas coisas produz um sistema que ninguém consegue comparar com o original.

Na prática

Escolha um trecho de programa em dialeto de fabricante — pode ser o do Capítulo 13 ou um da sua escola — e execute o roteiro do capítulo:

(a) documente em texto o que o trecho faz, antes de reescrever; (b) classifique cada instrução segundo a Tabela 13.2 do Capítulo 13; (c) reescreva em Ladder conforme a norma e em texto estruturado; (d) preencha a Tabela B.2 apenas com os pontos que apareceram no seu trecho; (e) escreva um parágrafo de avaliação: quanto do trecho foi conversão mecânica e quanto exigiu decisão.

Exercícios

- a) Explique o que significa cada um destes itens no código legado: .EN, .DN, .TT, .ACC, .PRE, .UL.
- b) Por que o bit de armazenamento da detecção de borda legada é uma fonte de defeito, e como a norma elimina esse problema?
- c) Reescreva em texto estruturado: `XIC chave1` em série com `XIO chave2`, acionando `OTE lampada1`.
- d) Explique por que trocar OTL/OTU por bobinas S/R não é suficiente em todos os casos.
- e) Um temporizador legado está configurado com o valor 50 e base de 100 ms. Quanto tempo isso representa, e como se escreve esse tempo na forma padronizada?
- f) O que pode acontecer com o acumulado de um contador convertido, se o comportamento após atingir a referência não for conferido?
- g) Descreva a conversão de CMP com a expressão $(valor_1 * valor_2) - valor_3 < valor_4$.

- h) Por que a conversão do deslocamento de bits com comprimento e bit de saída não é um para um, e o que é preciso montar?
- i) Cite duas diferenças entre a sub-rotina do legado, chamada por JSR, e a POU normatizada.
- j) Escolha três linhas da Tabela B.2 e explique, para cada uma, que tipo de teste detectaria o erro de conversão.
- k) Por que a etapa de documentar o comportamento, no roteiro, vem *antes* de reescrever o código?
- l) Um colega converteu o programa e aproveitou para “corrigir” um temporizador que considerou longo demais. Explique por que essa decisão é um problema e como deveria ter sido tratada.
- m) Escreva, em cinco linhas, a diferença entre o que este apêndice ensina e o que ele deliberadamente não ensina.

Capítulo C

Glossário

Objetivos do capítulo

Ao final deste apêndice você deve ser capaz de:

- usar o termo técnico correto para cada conceito da automação industrial;
- distinguir o termo que a norma padroniza do uso corrente de mercado;
- reconhecer termos que, confundidos, produzem erro de projeto;
- aplicar a regra “um conceito, um termo” na documentação do seu próprio projeto.

C.1 Como usar

A terceira coluna traz observações curtas e, quando existir, o capítulo em que o termo é desenvolvido. Onde a IEC 61131-3 padroniza um termo, a coluna em inglês registra a denominação normatizada; onde não padroniza, registra o uso corrente no mercado.

Uma obra escrita por muitas mãos acumula sinônimos. A regra que este glossário defende é a do Capítulo 14: **um conceito, um termo**. “Ciclo de varredura” é sempre ciclo de varredura, e não “tempo de scan”, “varredura” e “ciclo” alternados no mesmo texto.

Tabela C.1: Termos técnicos em português, o correspondente em inglês e a observação de uso.

Termo em português	Em inglês	Observação
Ação de controle	control action	Saída do controlador aplicada ao processo (Capítulo 9).
Acionamento por variador de frequência	variable frequency drive (VFD)	Controle da rotação do motor em regime; único modo de partida que atua no ponto de operação (Capítulo 23).
Alarme	alarm	Aviso ao operador. <i>Não</i> é proteção: proteção é intertravamento (Capítulo 17).
Arquitetura de automação	automation architecture	Organização dos níveis, dos equipamentos e das redes (Capítulo 3).
Atuador	actuator	Dispositivo que executa a ação sobre o processo.
Backup do projeto	project backup	Cópia versionada do projeto-fonte, guardada fora do equipamento (Capítulo 15).
Barramento de campo	fieldbus	Rede que interliga remotas de E/S e dispositivos de campo (Capítulo 17).
Bit de status	status bit	Indicador exposto por uma instrução legada (.EN, .DN); não existe na norma (Apêndice B).
Bloco de função	function block (FB)	POU com estado interno que persiste entre execuções.

Continua na página seguinte.

Continuação da Tabela C.1.

Termo em português	Em inglês	Observação
Bobina	coil	Elemento do Ladder que aciona o seu endereço; a norma distingue bobina simples, de retenção e de desretenção.
Cartão de entrada e saída	I/O module	Módulo que adapta o sinal de campo ao barramento interno (Capítulo 7).
Ciclo de varredura	scan cycle	Leitura das entradas, execução do programa e atualização das saídas; o tempo de ciclo é critério de escolha do controlador (Capítulo 9).
Cliente	client	Elemento que solicita dados ou serviços em uma comunicação.
Comissionamento	commissioning	Conjunto de testes e ajustes que coloca o sistema em operação (Capítulo 16).
Conduto	conduit	Canal de comunicação autorizado entre duas zonas, conforme a série IEC 62443 (Capítulo 22).
Configuração	configuration	Elemento de software da norma que reúne recursos e programas (Capítulo 11).
Contador	counter	Bloco que acumula eventos; exige reinício explícito quando é retentivo.
Contato	contact	Elemento do Ladder que conduz conforme o estado do endereço que identifica; pode ser normalmente aberto ou fechado.
Controlador de segurança	safety controller	Controlador dedicado à função de segurança, distinto do controlador de processo (Capítulo 22).
Controlador lógico programável	programmable logic controller (PLC)	Equipamento que lê entradas, executa lógica em memória e escreve saídas em tempo previsível (Capítulo 5).
Deslocamento de bits	bit shift	Operação sobre palavra; o registrador com comprimento variável do legado não tem equivalente direto (Apêndice B).
Deteção de borda	edge detection	Identifica a <i>transição</i> de uma variável, não o seu nível; indispensável em contagem de eventos.
Diagrama de blocos de função	function block diagram (FBD)	Linguagem gráfica padronizada.
Diagrama de contatos	ladder diagram (LD)	Linguagem gráfica padronizada; a mais próxima do comando elétrico.
Endereçamento de E/S	I/O addressing	Ligação entre variável e ponto físico. Não é padronizado pela norma (Capítulo 13).
Entrada analógica	analog input	Recebe grandeza contínua; a resolução obtida depende do cartão e da faixa (Capítulo 7).
Entrada digital	digital input	Recebe sinal de dois estados; exige atenção ao tipo de ligação (<i>sourcing</i> ou <i>sinking</i>).
Escala de sinal	scaling	Conversão do valor bruto em grandeza de engenharia.
Estado de operação	operating state	Condição do controlador (parado, executando, em falha) (Capítulo 9).

Continua na página seguinte.

Continuação da Tabela C.1.

Termo em português	Em inglês	Observação
Evento	event	Registro de algo que aconteceu, sem exigir ação imediata.
Exatidão	accuracy	Proximidade entre o valor medido e o valor verdadeiro; distinta de resolução, que é o menor passo representável.
Falha segura	fail-safe	Condição em que a perda de sinal leva o processo ao estado mais seguro (Capítulo 10).
Firmware	firmware	Software gravado no equipamento; a versão faz parte da identificação do sistema.
Força	forcing	Sobreposição manual do valor de um ponto pelo ambiente de programação; proibida em máquina em produção (Capítulo 24).
Gatilho	trigger	Condição que dispara a execução de uma tarefa (Capítulo 11).
Histórico (banco de dados)	historian	Armazenamento de longo prazo das variáveis, com carimbo de tempo (Capítulo 17).
Histerese	hysteresis	Faixa entre o valor de acionamento e o de desligamento de um alarme; evita oscilação (Apêndice A).
Interface homem-máquina	human-machine interface (HMI)	Dispositivo e software de operação local.
Intertravamento	interlock	Condição que impede um estado inseguro. <i>Executado pelo controle</i> , nunca pela supervisão.
Linguagem de programação	programming language	A norma define a forma de LD, FBD, ST e SFC; IL foi depreciada em 2013 e removida na 4ª edição.
Lista de instruções	instruction list (IL)	Linguagem textual depreciada; não deve ser ensinada como linguagem de partida (Capítulo 10).
Lista de tags	tag list	Relação de variáveis com tipo, unidade, faixa e descrição; parte da entrega técnica.
Malha de controle	control loop	Conjunto sensor, controlador e atuador que mantém uma grandeza.
Manutenção corretiva	corrective maintenance	Intervenção depois da falha.
Manutenção preditiva	predictive maintenance	Intervenção guiada por degradação medida e acompanhada (Capítulo 24).
Manutenção preventiva	preventive maintenance	Intervenção em intervalo programado.
Mapa de memória	memory map	Organização das áreas de memória do controlador (Capítulo 6).
Memória retentiva	retentive memory	Preserva o conteúdo após a queda de energia; depende de bateria ou de memória não volátil.
Nível de maturidade	maturity level	Avalia o <i>processo</i> de quem desenvolve ou integra, conforme a série IEC 62443.
Nível de segurança	security level (SL)	Resistência exigida de uma zona contra classes de atacante, conforme a série IEC 62443.
OEE	overall equipment effectiveness	Eficiência global do equipamento: disponibilidade $\times$ desempenho $\times$ qualidade (Capítulo 19).

Continua na página seguinte.

Continuação da Tabela C.1.

Termo em português	Em inglês	Observação
OPC UA	OPC Unified Architecture	Camada de informação e comunicação com modelo de dados e sessão segura (Capítulo 18).
Parada de emergência	emergency stop	Função de proteção; deve ser independente da lógica de processo comum.
Partida direta	direct-on-line starting	Partida com corrente elevada; adequada a motores pequenos e partidas pouco frequentes.
Partida suave	soft starter	Reduz a corrente e o esforço de partida sem controlar a rotação em regime.
Pirâmide da automação	automation pyramid	Representação dos níveis hierárquicos do campo à gestão (Capítulo 3).
Portabilidade	portability	Capacidade de o mesmo programa rodar em outro ambiente com esforço previsível (Capítulo 20).
Protocolo de comunicação	communication protocol	Regras de troca de dados entre equipamentos.
Publicação e assinatura	publish/subscribe	Modelo em que o produtor publica e o consumidor assina, desacoplados por um intermediário (Capítulo 18).
POU	program organization unit	Unidade de organização de programa: programa, bloco de função ou função (Capítulo 11).
Rede industrial	industrial network	Rede de campo, de controle ou de supervisão, com requisitos próprios.
Redundância	redundancy	Duplicação de elemento crítico para manter a operação em caso de falha (Capítulo 8).
Redundância em espera	hot standby	Unidade reserva pronta para assumir, mantida em espera energizada.
Registro de eventos	event log	Sequência com data e hora do que ocorreu; base da análise de causa.
Resolução	resolution	Menor variação representável; não se confunde com exatidão.
Retenção de saída	output latch	Saída que permanece no estado após o fim da condição; exige comando explícito de desretenção.
Segurança cibernética	cybersecurity (<i>security</i>)	Proteção contra ação humana intencional (Capítulo 22).
Segurança funcional	functional safety (<i>safety</i>)	Proteção contra falha do sistema que cause dano.
Servidor	server	Elemento que disponibiliza dados ou serviços em uma comunicação.
Sistema de controle distribuído	distributed control system (DCS)	Arquitetura de controle distribuído por malhas, típica de processo contínuo.
Sistema SCADA	supervisory control and data acquisition	Supervisão e aquisição de dados de processo (Capítulo 17).
Sinal discreto	discrete signal	Sinal de dois estados, também chamado digital.
Soft-PLC	soft PLC	Controlador implementado em software sobre hardware de propósito geral (Capítulo 21).
Sobressalente	spare part	Componente em estoque; a falta dele transforma falha simples em parada longa.

Continua na página seguinte.

Continuação da Tabela C.1.

Termo em português	Em inglês	Observação
Tarefa	task	Elemento que define <i>quando</i> uma parte do programa é executada (Capítulo 11).
Telemetria	telemetry	Transmissão de medidas a distância (Capítulo 19).
Temporizador	timer	Bloco que mede tempo; a norma distingue comportamentos na energização, na desenergização e por pulso.
Tipo de dado	data type	Define faixa e operações válidas sobre uma variável.
Tipo de dado derivado	derived data type	Construído a partir de tipos elementares: arranjo e estrutura (Capítulo 6).
Tipo de dado elementar	elementary type	Booleano, inteiro, real, tempo, cadeia de caracteres.
Transição	transition	Condição que habilita a passagem de um passo a outro em SFC.
Variável global	global variable	Declarada em nível de configuração ou recurso; aceita em todo o escopo.
Variável local	local variable	Declarada na POU; visível apenas dentro dela.
Variável retentiva	retentive variable	Preserva o valor após queda de energia.
Visão computacional	machine vision	Inspeção por imagem; roda em <i>hardware</i> externo ao controlador (Capítulo 21).
Zona	zone	Agrupamento de ativos com requisitos de segurança comuns, conforme a série IEC 62443.

Atenção

“Conforme a norma” e “compatível” não são sinônimos, e o glossário não os trata como tal. Sempre que uma afirmação de conformidade aparecer, ela precisa vir acompanhada da **edição e do ano** da norma — como no Capítulo 12.

Na prática

Monte o glossário do *seu* projeto. Pegue a lista de tags e a documentação que você produziu nas práticas do Apêndice A e:

- (a) liste os termos que aparecem com mais de um nome no mesmo documento; (b) escolha o termo padronizado para cada um e registre o porquê; (c) verifique se o termo escolhido é o que a IEC 61131-3 usa, quando existir; (d) aplique a substituição na documentação e na lista de tags; (e) escreva três linhas sobre o efeito da padronização na leitura do projeto por outra pessoa.

Exercícios

- a) Explique a diferença entre *exatidão* e *resolução*, com um exemplo numérico.
- b) Por que *safety* e *security* recebem termos distintos em inglês e um único termo em português? Que confusão prática isso pode causar?
- c) Cite três termos deste glossário cuja confusão produz erro de projeto, e explique cada um.

- d) Por que “endereço de E/S” aparece como não padronizado, enquanto “bobina” aparece como elemento da norma?
- e) Escolha cinco termos e escreva, para cada um, a frase em que ele seria usado incorretamente por um iniciante.
- f) O glossário registra que IL foi depreciada e removida. Por que manter o termo em um glossário de uma apostila atual?
- g) Explique a diferença entre *intertravamento* e *alarme*, e diga qual dos dois pode residir no sistema de supervisão.
- h) Por que o termo “redundância” precisa vir acompanhado da arquitetura (em espera, ativa, dupla) para significar algo?

Capítulo D

Referências normativas e bibliografia

Objetivos do capítulo

Ao final deste apêndice você deve ser capaz de:

- citar uma norma com edição e ano, e reconhecer uma citação incompleta;
- distinguir obra-fonte, bibliografia de apoio e documentação de fabricante;
- identificar qual referência sustenta cada afirmação técnica do texto;
- montar a lista de referências do seu próprio projeto.

D.1 Como as referências são citadas nesta obra

Três regras valem para todo o texto:

1. **toda afirmação normativa cita a edição e o ano.** “A norma define” não é referência: “a IEC 61131-3, 4ª edição, de maio de 2025” é;
2. **nada é reproduzido das normas.** Elas foram consultadas para conferência factual, e o texto desta apostila não transcreve trechos, não copia figuras e não inventa numeração de seção. Quem precisa do texto normativo o obtém junto ao organismo que o publica;
3. **figura ou tabela de terceiro entra com crédito.** Esquema de catálogo, manual de fabricante ou figura de outra obra traz a indicação de fonte — e, sempre que possível, é redesenhado em vez de copiado.

D.2 Obras de origem desta apostila

Esta apostila foi produzida a partir de duas obras do mesmo grupo de autores, incorporadas e atualizadas:

TAVARES, M.Sc. Leonardo de Oliveira; MONTEIRO, M.Sc. Natália Nogueira. *Controladores Lógicos Programáveis — apostila final (teórica e prática)*. 1ª edição, Rev. 0, 2017 (revisão de 2021, Rev. 05D). Instituto Federal Fluminense, Campus Campos dos Goytacazes. **Obra-fonte principal:** fundamentos, hardware, memória, cartões de E/S, princípio de funcionamento, linguagens, família IEC 61131 e a bateria de práticas.

VIANNA, D.Sc. William da Silva. *Controlador Lógico Programável*. Instituto Federal Fluminense, Campus Campos dos Goytacazes, 2023. **Obra-fonte complementar:** características, estrutura básica, mapa de memória, critérios de especificação e compra, e noções de sistema SCADA com uso de CLP.

D.3 Normas e documentos normativos citados

A Tabela D.1 reúne as normas mencionadas no texto. A coluna de edição e ano registra o que foi **efetivamente conferido em fonte pública**; onde a edição não pôde ser confirmada, a célula indica que ela deve ser conferida no catálogo do organismo, e o texto desta apostila não faz afirmação técnica que dependa dela.

Tabela D.1: Normas citadas, assunto e edição conferida.

Norma	Assunto	Edição e ano
IEC 61131-1	Informações gerais sobre controladores programáveis	<i>Conferir no catálogo</i> ; assunto citado no Capítulo 12
IEC 61131-2	Requisitos de equipamento e ensaios	<i>Conferir no catálogo</i> ; assunto citado nos Capítulos 7 e 12
IEC 61131-3	Linguagens de programação	1ª ed. 1993; 2ª ed. 2003; 3ª ed. 2013 (POUs orientadas a objeto; deprecia a Lista de Instruções); 4ª ed. maio de 2025 (vigente; removeu a Lista de Instruções)
IEC 61131-4	Orientações ao usuário	<i>Conferir no catálogo</i> ; assunto citado no Capítulo 12
IEC 61131-5	Comunicações	<i>Conferir no catálogo</i> ; assunto citado no Capítulo 12
IEC 61131-6	Segurança funcional	<i>Conferir no catálogo</i> ; assunto citado no Capítulo 12
IEC 61131-7	Programação de controle <i>fuzzy</i>	<i>Conferir no catálogo</i> ; assunto citado no Capítulo 12
IEC 61131-8	Implementação e aplicação das linguagens	<i>Conferir no catálogo</i> ; assunto citado no Capítulo 12
IEC 61131-9	Interface digital de comunicação ponto a ponto (SDCI, comercializada como IO-Link)	2013; assunto citado nos Capítulos 12 e 17
IEC 61131-10	Formato de intercâmbio XML de projetos IEC 61131-3, derivado dos esquemas da PLCopen	<i>Conferir no catálogo</i> ; assunto citado nos Capítulos 12 e 20
IEC 61499 (partes 1, 2 e 4)	Blocos de função para medição e controle de processo distribuído	Publicada em 2005; parte 1, 2ª ed. 2012; parte 2, 2ª ed. 2012; parte 4, 2ª ed. 2013. A parte 3 foi retirada em 2008
IEC 61508	Segurança funcional de sistemas elétricos, eletrônicos e eletrônicos programáveis (E/E/PE)	1ª ed. 1998–2000; 2ª ed. 2010
IEC 61511	Segurança funcional para a indústria de processo	<i>Conferir no catálogo</i> ; aplicação setorial da IEC 61508, citada no Capítulo 22
IEC 62443-1-1	Terminologia, conceitos e modelos da série de segurança para automação	2009 (publicação da série conjunta ISA/IEC)

Continua na página seguinte.

Continuação da Tabela D.1.

Norma	Assunto	Edição e ano
IEC 62443-2-1	Programa de segurança do sistema de automação e controle	2024
IEC 62443-2-4	Requisitos para prestadores de serviço de integração e manutenção	2023
IEC 62443-3-2	Avaliação de risco de segurança para projeto de sistema	2020
IEC 62443-3-3	Requisitos de segurança de sistema e níveis de segurança	2013
IEC 62443-4-1	Requisitos do ciclo de vida de desenvolvimento seguro	2018
IEC 62443-4-2	Requisitos técnicos de segurança para componentes	2019
ISO/IEC 27001	Sistemas de gestão da segurança da informação	3ª ed. outubro de 2022; emenda 1 de 2024
ISO/IEC/IEEE 60559	Aritmética de ponto flutuante	2020, conteúdo idêntico ao IEEE 754-2019
IEEE 754	Aritmética de ponto flutuante binária e decimal	2019 , substituindo a edição de 2008
NIST Cybersecurity Framework	Referência de programa de segurança cibernética	Versão 2.0, 2024
ISO/IEC 17065 e ISO/IEC 17025	Avaliação da conformidade e competência de laboratórios de ensaio	<i>Conferir no catálogo</i> ; citadas no contexto de certificação de produto (Capítulo 22)

Sobre as edições. As séries IEC são revisadas continuamente: uma parte pode ser confirmada, atualizada ou retirada. Toda edição indicada acima corresponde à publicação consultada e registrada em 2026, e **não** é garantia de que a versão vigente seja a mesma no futuro. Antes de citar uma norma em laudo, contrato ou trabalho acadêmico, confirme a edição atual no catálogo do organismo emissor.

Adoções nacionais. A ABNT publica adoções de normas IEC com a designação *NBR IEC*. Isso significa que alguns dos documentos acima podem existir em versão brasileira, com a mesma numeração de parte. Confirme no catálogo da ABNT qual parte está disponível e em que edição — e nunca utilize cópia não autorizada de norma.

D.4 Bibliografia consolidada

As obras abaixo reúnem as bibliografias das duas apostilas de origem, com duplicatas consolidadas e grafias evidentemente incorretas corrigidas. A marcação indica se a obra permanece uma referência útil ao estudante atual.

D.4.1 Livros e apostilas

FRANCHI, C. M.; CAMARGO, V. L. *Controladores Lógicos Programáveis — Sistemas Discretos*. 2ª ed. São Paulo: Érica, 2010. **Atual**. Referência de base, ainda alinhada à segunda edição da norma; confira a terminologia nas edições posteriores.

PRUDENTE, F. *Automação Industrial — PLC: Teoria e Aplicações*. 2ª ed. Rio de Janeiro: LTC, 2011. **Atual**. Tratamento didático de Ladder e de comandos, útil para quem parte do comando elétrico.

NATALE, F. *Automação Industrial*. São Paulo: Érica, 1992. **Histórica**. Cobre o período anterior à difusão das redes industriais; mantida por registro e para consulta a sistemas antigos.

OLIVEIRA, J. C. P. *Controlador Programável*. São Paulo: Makron Books, 1993. **Histórica**.

MORAES, C. M.; CASTRUCCI, P. L. *Engenharia de Automação Industrial*. 2ª ed. Rio de Janeiro: LTC, 2007. **Atual** para instrumentação, arquitetura de automação e sistemas industriais.

OIKO, R. K.; SARROUF, L. P. Controladores Programáveis — como comprar. *Instec*, São Paulo, n. 52, p. 32–45, março de 1992. **Histórica**. Mantida por ser registro do problema de especificação, tema retomado no Capítulo 8.

SOUZA, F. M. *Instrumentação — Automação Básica*. Espírito Santo, 1999. **Histórica**.

VIANNA, W. S. *Apostila de Controlador Lógico Programável*. Centro Federal de Educação Tecnológica de Campos, 1995. **Histórica**. Primeira versão da linha que deu origem à obra-fonte complementar.

VIANNA, W. d. S. *Controlador Lógico Programável*. Campos dos Goytacazes, 2008. **Histórica**.

D.4.2 Artigos, relatórios e documentos técnicos

CHOULIARAS, V. A.; NUNEZ-YANEZ, J. L. An IEEE 754 floating point engine designed with an electronic system level methodology. *IEEE*, 2007. **Pontual**. Sustenta a representação em ponto flutuante tratada no Capítulo 6.

HWANG, Y.-M.; KIM, M. G.; RHO, J.-J. Understanding Internet of Things (IoT) diffusion: focusing on value configuration of RFID and sensors in business cases (2008–2012). *Information Development*, v. 32, n. 4, 2016. **Pontual**.

LÜCKENHAUS, M. Machine Vision in IIoT. *Quality Magazine*, 2016. **Pontual**. Pano de fundo para a visão computacional tratada no Capítulo 21.

RAY, T.; SURESH, A.; DASH, P. S.; BANERJEE, P. K. Instrumentation and automation system at coal leaching pilot plant. *International Journal of Automation and Control*, v. 9, p. 158–171, 2015. **Pontual**.

SASAJIMA, H.; ISHIKUMA, T.; HAYASHI, H. Future IIoT in process automation — latest trends of standardization in industrial automation, IEC/TC65, 2015. **Pontual**. Origem da discussão de IIoT e padronização retomada nos Capítulos 18 e 21.

ROCKWELL AUTOMATION. *Controladores Logix5000 — Manual de Referência Geral do Conjunto de Instruções*. 2001. **Fonte do dialeto**. É a origem dos mnemônicos XIC, XIO, OTL e OTU documentados no Capítulo 13 e no Apêndice B. **Não é norma** e não deve ser usada como referência de linguagem padronizada.

EESC/USP. *Memórias*. Grupo de Sistemas Digitais, s.d. **Histórica**.

FEEC/UNICAMP. *Representação Numérica*. s.d. **Histórica**.

The father of invention: Dick Morley looks back on the 40th anniversary of the PLC. *Manufacturing Automation*, s.d. **Pontual**. Fonte do relato sobre a origem do controlador, usado no Capítulo 2.

D.4.3 Documentação técnica e fontes de organismos

OPC FOUNDATION. *OPC Unified Architecture*. Documentação técnica oficial, disponível publicamente. Consultada para os Capítulos 12 e 18.

OASIS. *MQTT Version 5.0* (e versão 3.1.1). Especificação padrão, disponível publicamente. Consultada para o Capítulo 18.

ECLIPSE FOUNDATION. *Eclipse Sparkplug Specification*, versão 3.0. Consultada para o Capítulo 18.

MODBUS ORGANIZATION. *Modbus Application Protocol Specification*. Consultada para o Capítulo 17.

PLCopen. *Biblioteca padronizada de blocos de função* e esquemas XML de intercâmbio. Consultada para os Capítulos 12 e 20.

NIST. *National Vulnerability Database* e *Cybersecurity Framework 2.0*. Consultados para o Capítulo 22.

CISA. *Known Exploited Vulnerabilities Catalog* e avisos ICS-CERT. Consultados para o Capítulo 22.

Documentação técnica dos fabricantes de controladores: manuais de instruções, de comunicação e de diagnóstico. São a fonte primária para endereçamento, blocos de sistema e comportamento específico de cada ambiente — e **não** devem ser citados como norma.

Atenção

Uma referência antiga não é uma referência errada: é uma referência *datada*. O erro está em citar como atual o que descreve outra geração de equipamento — por exemplo, apresentar a segunda edição da IEC 61131-3 como vigente, ou explicar endereçamento por área de memória como se fosse elemento da norma. Ao usar qualquer obra desta lista, confira em que edição da norma ela se apoia.

Na prática

Monte a lista de referências do projeto que você desenvolveu nas práticas do Apêndice A: (a) separe o que é norma, o que é bibliografia de apoio e o que é documentação de fabricante; (b) para cada norma citada, registre a edição e o ano que você efetivamente consultou; (c) identifique uma afirmação técnica do seu projeto que *não* tem referência e providencie uma; (d) substitua qualquer citação de manual de fabricante usada como se fosse norma; (e) escreva duas linhas explicando por que a data de consulta de uma norma faz diferença em um projeto que terá de ser mantido por dez anos.

Exercícios

- a) Explique por que a obra diz que “a norma define” não é uma referência aceitável.
- b) Qual é a diferença entre a obra-fonte principal e a obra-fonte complementar desta apostila?
- c) Por que o manual de instruções de um fabricante não pode ser usado como referência de linguagem padronizada?
- d) Escolha três normas da Tabela D.1 e diga em que capítulo cada uma é usada e para afirmar o quê.
- e) Explique por que a apostila marca algumas normas como “conferir no catálogo” em vez de informar uma edição.
- f) Por que a edição vigente da IEC 61131-3 importa para quem vai escrever um programa hoje?
- g) O que significa dizer que a ABNT publica uma norma como “NBR IEC”?
- h) Por que uma obra marcada como *histórica* nesta bibliografia continua listada?
- i) Cite duas obras desta bibliografia que permanecem úteis ao estudante atual, e justifique.
- j) Por que a apostila afirma que nada foi reproduzido das normas, e por que isso é uma questão de direito autoral além de rigor técnico?

Índice remissivo

- ambiente de programação
 - extensões, 139
- aplicações de CLP, 34
- automação industrial, 16
 - níveis, 25
 - objetivos, 16
- bibliografia, 219
- boas práticas de programação, 95
- broker MQTT, 121
- cabeamento
 - redução por rede, 111
- cartão de E/S, 49
- ciclo de varredura, 67
- CLP, 18
 - definição, 33
- complemento de dois, 45
- conversão de código, 181
- conversão de tipos, 43
- dialeto de fabricante, 90
- documentação
 - de projeto, 28
- documentação de programa, 101
- eficiência energética, 152
- entrada digital, 49
- especificação de compra, 60
- formação continuada, 164
- forte tipação, 41
- funções de conversão
 - IEC 61131-3, 43
- gateway de borda, 121
- glossário, 213
- IEC 61131-3
 - edições, 77, 86
- IEC 62443, 146
- IEEE-754, 45
- IIoT, 118
- indicador de desempenho, 125
- Internet das Coisas
 - industrial, 118
- interoperabilidade, 130
- ISO/IEC 27001, 146
- kWh por unidade produzida, 152
- legado (código), 90, 181
- linguagem de programação, 73
- Linux embarcado, 141
- Lista de Instruções, 77
- lógica fiada, 20
- manutenção
 - corretiva, 157
 - preditiva, 157
 - preventiva, 157
- memória
 - de CLP, 38
- memória imagem, 67
- microprocessador, 38
- migração de controlador, 134
- modelo de software, 80
- norma
 - IEC 61131-3, 18
- normas
 - ABNT NBR IEC, 168
- NPN, 53
- nuvem
 - limite de uso, 127
- OPC UA, 119
- optoacoplador, 53
- partida de motor, 153
- pirâmide da automação, 25
- plugin, 139
- PNP, 53
- ponto flutuante, 45
- portabilidade, 130
- porte de CLP, 60
- POU, 80
- processamento na borda, 127
- programação de CLP, 18
- projeto de automação, 28
- protocolo
 - EtherNet/IP, 115
 - Modbus, 115
 - PROFINET, 115
- prática de laboratório, 173
- rede industrial, 111
- referências normativas, 219

- relé
 - painel de comando, [20](#)
- representação numérica, [45](#)
- saída a relé, [55](#)
- saída a transistor, [55](#)
- saída a triac, [55](#)
- saída digital, [49](#)
- segurança cibernética, [145](#)
- segurança funcional, [145](#)
- simulação de programa, [105](#)
- tabela de alocação, [101](#)
- tarefa, [82](#)
- tecnologia de operação, [136](#)
- telemetria, [125](#)
- tempo de varredura, [67](#)
- teste de programa, [105](#)
- TI/TO, [136](#)
- tipos de dados, [41](#)
- variador de frequência, [153](#)
- watchdog, [82](#)
- Yocto, [141](#)
- zonas e condutos, [146](#)